Engineering Notebook



Team Number:	4613P
Team Name:	Barker Redbacks
School or Organization:	Barker College
Start and End Dates:	19/05/2026 – 16/10/2026
Book Number:	1

Introduction

Intro

This is 4613P's notebook. Every part of our design, strategy and robot will be listed here, as well as every part of how we function as a team.

Every ***design process*** is formatted around the 6 foundations of inquiry in robotics and engineering in general, which are;

IDENTIFY THE PROBLEM:

BRAINSTORM SOLUTIONS:

SELECT BEST SOLUTION:

BUILD & PROGRAM:

TEST SOLUTION:

REPEAT DESIGN PROCESS:

These key understandings are the core to engineering and robotics, and thus are what shaped our design process and our notebook contents.

Criteria Map

This page shows where every RoboDev Part 1 criterion is answered, so each one can be found quickly. Page numbers refer to the slide numbers in the footer.

Criterion (marks)	What earns full marks	Where to find it (pages)
A. Team Identification (5)	Members, roles, responsibilities, team name, robot name	5, 6–9
A. Project Timeline (5)	Comprehensive Gantt, T3W2–T4W1, planned AND actual	11–12, 13
A. Game and Strategy (5)	Skills overview + field diagram, rules, scoring, features, goals, strategy	15–27 (Skills focus: 20–23)
B. Planned Design (5)	Annotated drawings of all motors/sensors; ≥3 chassis + intake/outtake designs with pros/cons; citations	28, 30–44, 46–62, 64–75, 77–84, 194
C. Best Solution (5)	Multiple annotated views, decision matrices, week-by-week build plan with roles	85, 86, 87, 88, 89
C. Robot Components (5)	Specs, function/operation AND justification for every motor and sensor	90, 91
C. Wiring Diagram (5)	Accurate diagram of every component + electrical requirements	92
D. Autonomous Diagram (5)	Unambiguous field diagram of the move sequence	129
D. Autonomous Logic (5)	Unambiguous logical sequence + valid calculated score	130, 131
D. Lesson Entries (5)	3 entries per week, T3W2–T4W1, tasks + each member's contribution	163–164, 165–174
D. Programming Journal (5)	Weekly code record (≥ 8 weeks) with annotation	175–179; code pages 132–161
E. Testing the Solution (5)	Steps + results for every subsystem, with adjustments	180–182; refinements 184–191
F. Individual Commitment (5)	Each member's accountability list qualified with evidence	192–193

Contents

Section	Pages
Introduction and Criteria Map	2–3
Team identification and schedule	5–12
Gantt chart (planned and actual)	13
Game analysis and Robot Skills strategy	15–27
Planned design	28
Drivetrain / chassis	29–44
Intake and transfer	45–62
Lift (DR4B)	63–67
Innovate – gravity ratchet	68–75
End effector	76–83
Best solution: final views, matrices, build plan	85–89
Robot components and wiring diagram	90–92
Code development and localisation	93–127
Autonomous routine: diagram, logic, score	128–131
Robot code: driver control, lift, claw, autonomous, localiser	132–161
Build log and lesson entries	162–174
Programming journal	175–179
Testing the solution	180–182
Refinement (iteration 2) and robot reveal	183–191
Individual commitment	192–193
Sources and citations	194

Team Identification

Team name: Barker Redbacks
Team number: 4613P
Robot name: Switch
Name shown in our robot reveal video (YouTube, 24 Sep 2026).

Team schedule

- 3 timetabled lessons per week (Mon, Wed, Fri)
- After-school robotics: Tue + Thu, 3:30–5:30 pm
- ≈ 7 hours per week × 10 weeks ≈ 70 hours
- Project period: T3W2 (27 Jul) → T4W1 (16 Oct 2026)

Member	Role	Accountable for (project components)
Brandon	Lead Designer, Lead Programmer, Lead Mechanical, Team Captain	• lillet PROS library (odometry, AMCL, motion control) • Full robot CAD and manufacturing • Path-planning simulator + auton • Driver-control + autonomous code • Programming journal (weekly)
Ryan	Driver, Mechanical	• End effector prototypes 1 and 2 • Autonomous programming • Driving • Pneumatics and cable management • Transfer
Josh	First stage design lead, programming advisor, lead notebook	• Intake research, sketches, prototypes 1–4 • Transfer roller + chain power system • CAD drawings / planned design views • Intake + chassis decision matrices • Stand-off & dual-side drive reinforcement
Rio	Lead Mechanical, lead experimental design	• End Effector Concept • Building half the robot • Tuning • Intake • 4 Bar • Construction



Brandon

Role: Lead Designer, Lead Programmer, Lead Mechanical, Team Captain

Accountable for:

- lillet PROS library (odometry, AMCL, motion control)
- Full robot CAD and manufacturing
- Path-planning simulator + auton
- Driver-control + autonomous code
- Programming journal (weekly)



Ryan

Role: Driver, Mechanical, Son of Jesus Christ (William Qin)

Accountable for:

- End effector prototypes 1 and 2
- Autonomous programming
- Driving
- Pneumatics and cable management
- Transfer



Josh

Role: First stage design lead, programming advisor, lead notebook

Accountable for:

- Intake research, sketches, prototypes 1–4
- Transfer roller + chain power system
- CAD drawings / planned design views
- Intake + chassis decision matrices
- Stand-off & dual-side drive reinforcement



Rio Wang

Role: Lead Mechanical, lead experimental design

Accountable for:

- End Effector Concept
- Building half the robot
- Tuning
- Intake
- 4 Bar
- Construction


MOG OFF
MORE MOG
CLEAN LOOKS BETTER HUMAN
BRANDON BUILT DIFFERENT
BIGGER MOG ENERGY
VS
RYAN ALERT
BRANDON ALERT
MOG OFF
BETTER HAIR
HIGHER IQ
MORE MOG
CLEANER CUTS
BETTER SMILE
MORE MOG
ONLY ONE CAN WIN...

Schedule

MONTH: July **YEAR:** 2026

SUNDAY	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY
	Chassis + 4 Bar Start Production					
Chassis finished, focus on Dr4B						
Start on intake + combine DR4B and chassis						

MONTH: August **YEAR:**

SUNDAY	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY
			Gravity Gears + Motor integration + electronics			
Start on prog						
Start experimenting with end effector						

Schedule

MONTH: September **YEAR:** 2026

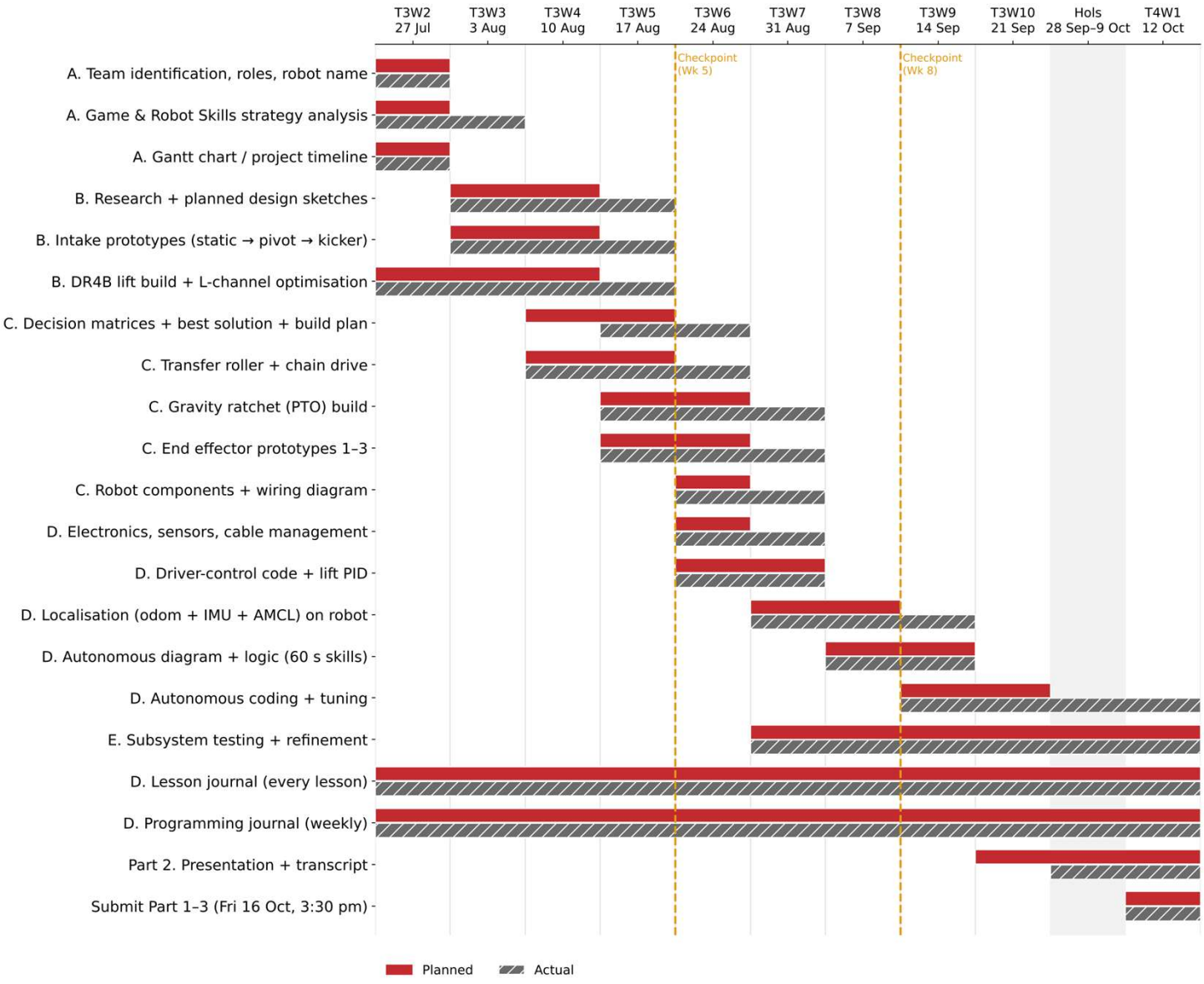
SUNDAY	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY

MONTH: October **YEAR:** 2026

SUNDAY	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY

Gantt Chart

Planned (red) versus actual (grey, hatched) timeframes for every task needed for the Project Plan and the robot. Amber lines are teacher checkpoints.



Variance notes (planned vs actual)

• Research + intake prototyping ran 1 week over: three intake iterations were needed before pickup reached 19/20 (Test T4).

• Gravity ratchet and end effector slipped 1 week because the ratchet guide had to be re-bent (Test T8). Decision matrices were finished after the Week 5 checkpoint.

• Localisation took an extra week (AMCL tuning in the simulator), so autonomous tuning continued into T4W1.

• Recovery action: testing ran in parallel with coding from T3W7 so the final deadline was still met.

BARKER REDBACKS 4613



Game Analysis

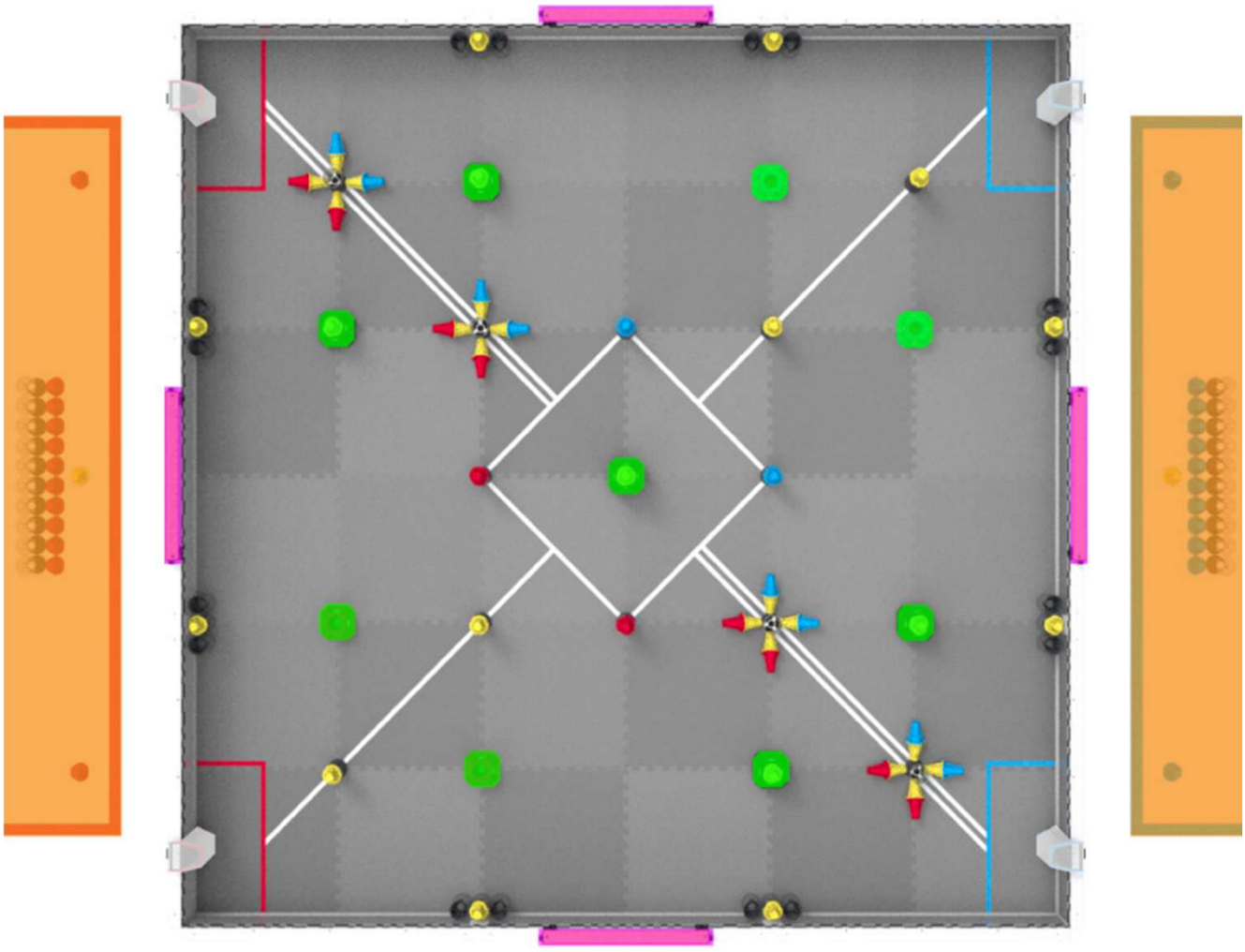
Introduction

Introduction

This sections is focused on taking apart the game and formulating certain foundational understandings (we call these axioms).

We identify potential problems in the game, delve into surface level observations, break down the game field, game pieces and positioning and complications regarding driving, Auton and autonomous in general. This is different to in depth specific problem identification in our cyclical design processes

This new game of *override* has introduced many new features compared to previous years. And thus, requires quite a bit of pre-season analysis to unpack.



**Override Game Field Layout – 18/05/25*

Identify the Problem

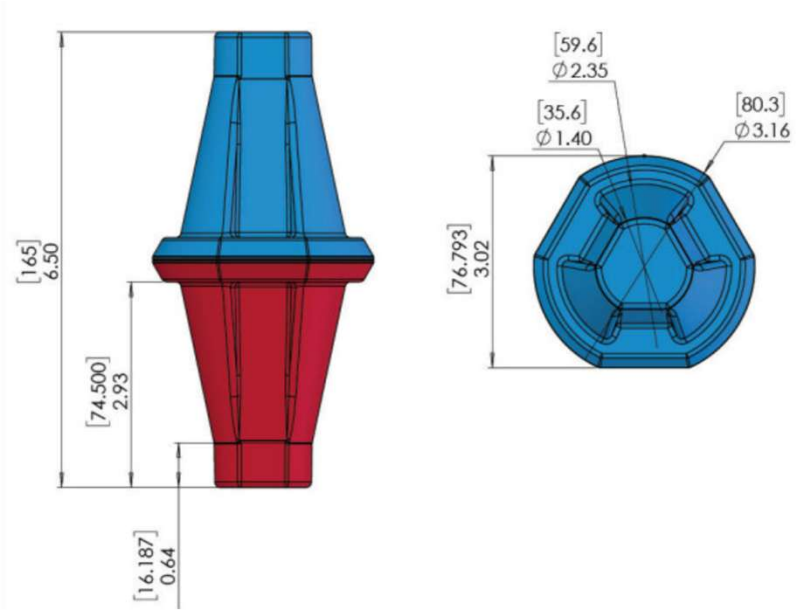
Field Inventory & Layout:

- **56 Cups:** 20 Match Loads (10 per alliance), 24 pre-placed (gray side up), 12 pre-placed (clear side up).
- **63 Pins:** Mixed distributions of Red/Blue, Red/Yellow, Blue/Yellow, and Yellow/Yellow.
- **9 Goals:** 4 Alliance-coloured (protected), 5 Neutral (4 short, 1 tall).
- **4 Toggles:** Control switches that dictate score modifiers/ownership.
- **4 Loaders:** Two adjacent to each Alliance Station for match-load introduction.
- **Possession Limit ():** Robots may only possess a maximum of **ONE Pin and ONE Cup** at any given time. *Analysis: This completely eliminates hoarding strategies. The game is inherently cycle-based. Cycle speed and route efficiency will dictate the elite tier.*
- **Endgame ():** The final 20 seconds transition into a "king of the hill" battle in the Midfield.

Looking at these game pieces, it is obvious that there are many many pins, but of course limited places to put them in. As well as this, the earlier placed pins in a stack will be much more achievable, and much easier to secure than ones that are high. As such, the difficulty level increases as the stack increases, while the opportunity that it brings value decreases.

Identify the Problem

Identifying Game Objects



*Pins in VEX OVERRIDE, Game Object – 23/05/25



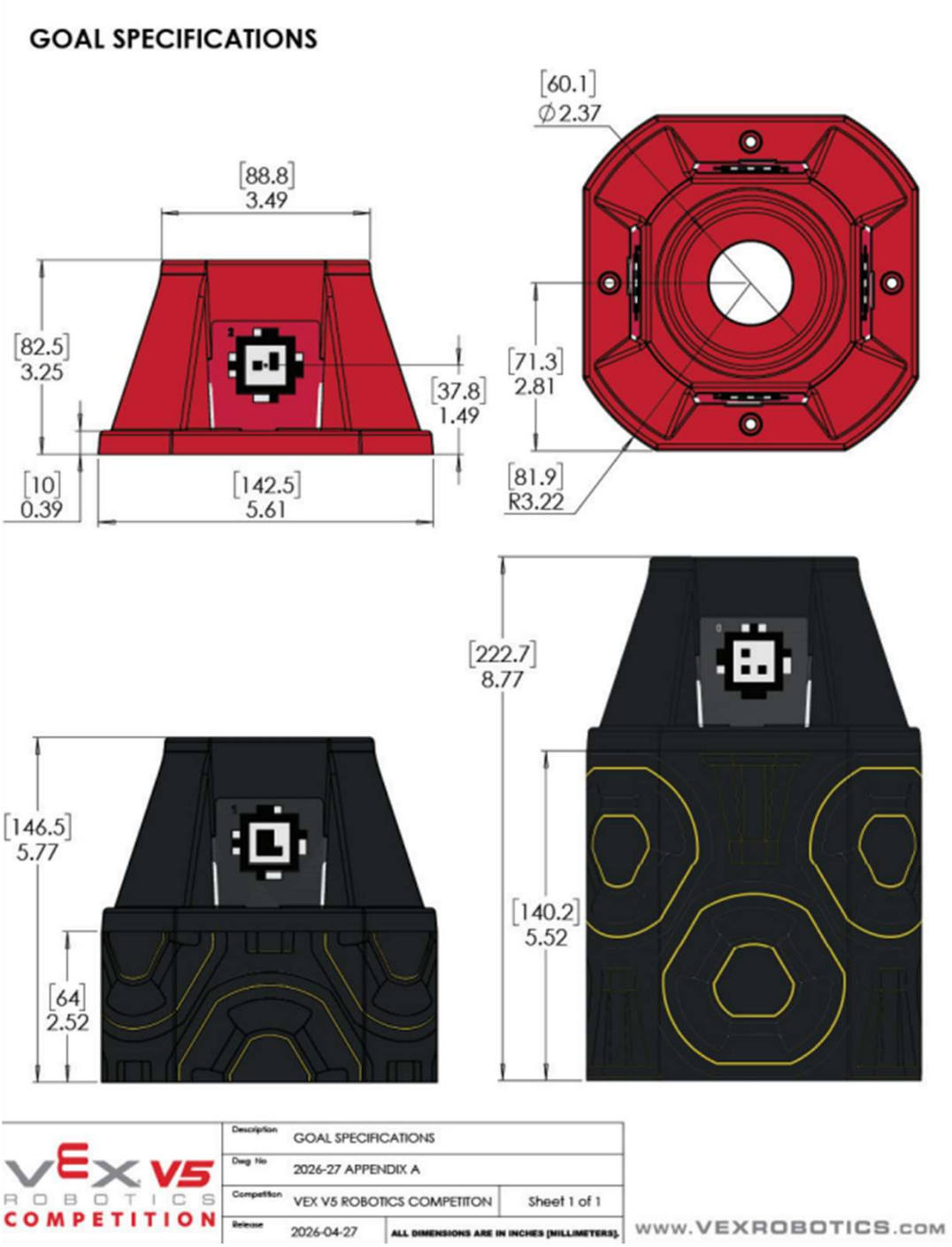
*Cups in VEX OVERRIDE, Game Object – 23/05/25

The extremely unique pin & cup design of this season will pose some build challenges. Like how do we design a ground intake if we do get one around this game piece? How can we pick this up omni-directionally from the ground with us scoring mechanism?

This unique design will force out unique engineering capabilities within our team, as well as mechanical strategy designed around this game object.

Identify the Problem

Identifying Game Objects



The unique design of the pin holder module and its symmetry makes it easy for us to build alignment tech onto our robot.

Where driving into the module with a certain shape at the front of the robot will cause it to align to it.

Theres not much else to initiate from looking at these schematics since they just hold the pins.

Identifying the Problem

Surface Observations

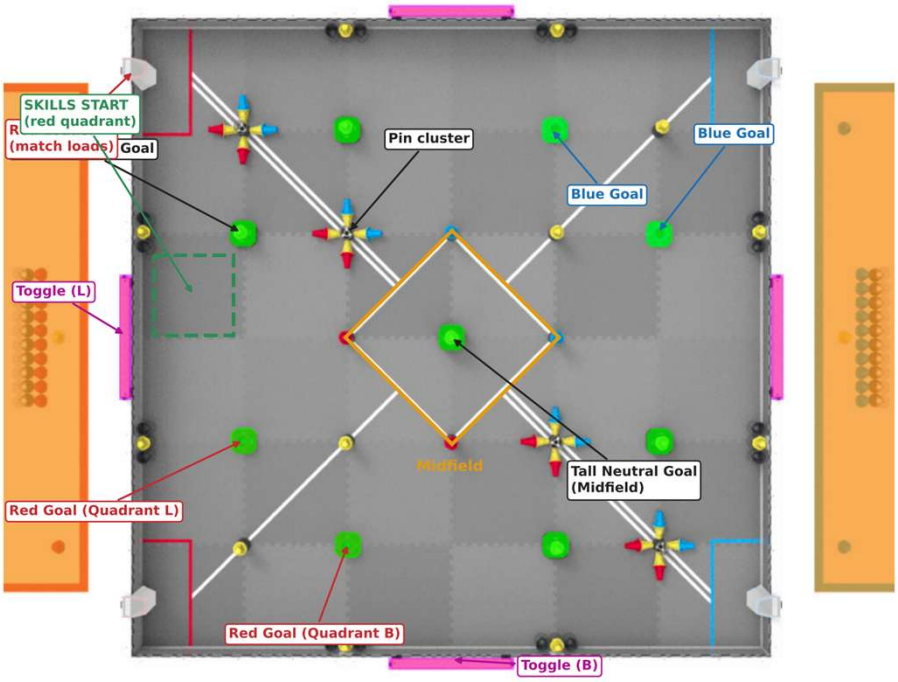
After watching the introduction video and reading the game manual (at the time of writing it was the 0.1.2 vers.) we unpacked and discussed our initial thoughts on the game and condensed those points and condensed them into things we knew we would need to address in the future. Like:

1) The apparent 55W drivetrain limit. This means we traverse the field slower, relying on the driver to perform well, which can be inconsistent due to human error. We could also opt for 5.5W motors and giving up power. Since standard vex motors are at 11W, other solutions may be:
   - Using an X – Drivetrain
   - Opting for a tank drive with Half Watt motors (3 powering each side)
   - Inventing our own drivetrain and kickstarting a new meta

2) The 1 cup + pin possession rule (SG6). This can be very technical as the term "possession" is not explicitly (as in extreme detail) defined. They do give a clarification where pushing game objects with a flat surface while they are on the ground is allowed. Thus, we need to introduce some sort of safety net to avoid accidently going into "possession" of two game objects triggers.

3) The amount of game objects + their unique designs in themselves. While the most previous games (over under, high stakes, pushback) have all been centred around 1 game object, this game is fundamentally built around 2. Meaning we would need a universal intake for both the cup and the pin. As well as figure out how to pick them both up extremely efficiently and from any angle, how to potentially combine the 2 inside the robot, as well as how to score quickly. We figure that even though this isn't necessarily a cycling game, compared to last season where the value of the last few seconds of the match matter way more than the start, this game has the beginning matter a lot more than the end (as the pins at the bottom of pillars once scored/descored with a cup are very hard to undo)

Skills Overview



How Robot Skills works

- One robot, 60-second matches on the 12 ft × 12 ft field.
- **Driving Skills**: 60 s fully driver-controlled.
- **Autonomous Coding Skills**: 60 s autonomous, limited human interaction.
- Ranking = best Driving + best Autonomous score combined.
- Robot starts in the quadrant next to the **red Alliance Station** (RSC2).

Field in Skills (RSC4)

- All 9 Goals start empty: 4 alliance Goals (2 red, 2 blue), 4 short neutral Goals and 1 tall neutral Goal in the Midfield.
- 4 Toggles (one per Quadrant wall) decide who Owns the yellow Pin Halves in that Quadrant.
- 1 red/yellow Pin is the Preload. Match Loads (3 red/yellow Pins, 4 blue/yellow Pins, 7 Cups) enter only through the red Loaders.
- Autonomous Coding Skills needs the VEX GPS code strip on the field.

Game objects

- Pins have two coloured halves (red/yellow, blue/yellow, yellow/yellow, red/blue). Each visible half scores separately.
- Cups nest on top of a Placed Pin so another Pin can be stacked on the Cup. A Pin half hidden inside the opaque half of a Cup does not score.
- A robot may possess only 1 Pin and 1 Cup at a time (SG6) – Skills is a cycle-speed game.

Source: V5RC Override Game Manual v2.1 (see References). Field render from the team's game-analysis page.

Skills Scoring

Scoring action (Robot Skills, RSC3)	Points	Condition
Red Pin Half Scored	5	Pin is Placed in a red Quadrant Goal or the Midfield
Blue Pin Half Scored	5	Pin is Placed in a blue Quadrant Goal or the Midfield
Yellow Pin Half Scored	10	Must be Owned: Quadrant Toggle set to an alliance colour, or (Midfield) robot ends in Midfield
Robot ends in the Midfield	8	Any part of robot inside Midfield volume at end (SC6)
Toggle set to yellow	0	Yellow Pin Halves in that Quadrant are not Owned
Pin Half inside opaque Cup half	0	Hidden halves do not score (SC3)

Worked values per Pin (red Quadrant, Toggle set red)

Pin type	Value	Working
Yellow / yellow	20	10 + 10 (both halves Owned)
Red / yellow	15	5 (red half, red Quadrant) + 10 (Owned yellow half)
Blue / yellow	10	0 (blue half in red Quadrant) + 10 — but 15 if placed in the Midfield Tall Goal
Red / blue	5	5 + 0 — but 10 in the Midfield Tall Goal

What this tells us

- A yellow/yellow Pin is worth 4× a red/blue Pin, so the intake and autonomous must target yellow Pins first.
- Toggles score nothing directly, but setting one to red doubles the value of every Pin in that Quadrant – it is the highest-value 3 seconds of a run.
- The Tall Goal in the Midfield counts BOTH colours, so blue/yellow Match Loads belong there, and parking in the Midfield adds +8 and Owns its yellow halves.

Rules & Features

Rule	Summary	Impact on our design / program
SG1	Start inside 18" × 18" × 18"	DR4B folds to < 18"; intake inside frame rails
SG2	Horizontal expansion ≤ 24" × 24"	End effector reach stays within 24" footprint
SG3	No part higher than 50"	DR4B max height 47" (measured 31 Jul)
SG6	Possess max 1 Pin + 1 Cup; plowing allowed	Top-plate stop + 'touch it, own it' intake (Test T5)
SC2–SC3	Placed Pins score per visible half	Cups nested clear-side over Pins when stacking
SC4–SC5	Toggle must be fully seated, not touched by robot at end	Robot backs away after pushing Toggle
RSC1	Standard rules apply; removing objects is not a violation	No penalty if an object leaves the field
RSC2	Start next to red station; 1 Preload; loads via red Loaders only	Auton start pose (-58, 9, 90°)
RSC3	Skills scoring (5 / 10 / 8)	Route prioritises yellow halves + Midfield
RSC4	Goals start empty; GPS strip for Auton Skills	Distance sensors + AMCL do not rely on GPS
RSC5	Skills Stop Time is a tiebreaker only	Optional – opt in with referee
R10 / R11	Motors limited; drivetrain capped at 55 W	6 drive motors = 55 W exactly
R25 / R26	Pneumatics limited; pressure gauge required	One air tank + gauge fitted

Key game features (Skills)

- Two different game objects (Pins + Cups) → a universal intake is needed.
- Vertical scoring: stacks grow upward on Goals, up to the 50" limit → a tall, stable lift (DR4B).
- Toggles control ownership of yellow halves → an early 'Toggle push' in every routine.
- Midfield + Tall Goal reward finishing in the centre → routines end there.
- Strict possession limit → speed of each cycle, not storage, decides the score.

Goals & Strategy

Objective	Target	How we will achieve it
Autonomous Coding Skills	103 pts max 83 pts safe	9-step route: 2 red Toggles, 5 yellow-half Pins, end in Midfield (see Autonomous Logic)
Driving Skills	≥ 150 pts	≈ 9 cycles of ~6 s; Match Loads into Tall Goal; park in Midfield
Cycle time	≤ 6 s per Pin	Touch-it-own-it intake + gravity-ratchet transfer
Reliability	3 runs in a row ≥ 83	AMCL localisation; test log every lesson

Driving Skills strategy (60 s)

1. Push the left Toggle red on the way out of the start Quadrant (0–4 s).
2. Score yellow/yellow Pins into the two left-Quadrant Goals, nesting clear-side Cups to stack (4–25 s).
3. Push the bottom Toggle red and fill the two bottom-Quadrant Goals (25–40 s).
4. Collect Match Loads from the red Loader; place blue/yellow Pins on the Tall Goal where both halves count (40–55 s).
5. Finish inside the Midfield (+8, and Owns the Tall Goal yellow halves) (55–60 s).

Autonomous strategy

• Same priorities, but only uses field Pins near our two red Quadrants so travel is short and localisation stays accurate.

• A time guard abandons any step if < 9 s remain so the +8 Midfield points are always banked.

Why not the blue Quadrants? Red Pin Halves only score in red Quadrants, and the blue Quadrants are far from the red Loaders, so they cost travel time for fewer points.

Strategic Ideas

Mechanical Strategies

To address the elevation of the problem, that needs to take inspiration from previous seasons that we competed in, as well as adapt them to this new unique game.
There is a vertical expansion limit of 50 inches, which needs to consider.
The main theory is to use 4 bar mechs, but there are some variations. These are some alternate ideas.

- Double Reverse 4 Bar intake directly off the ground + omnidirectional claw design to pick up cups + pins
  - Ditching ground intake means more weight saving, which helps our limited powered drivetrain to function quicker than other robots and allow us to out-drive them therefore allowing us to score more points.

- Double Reverse 4 Bar intake + ground intake w/ flex wheels --> pushes directly into aligner for 4 bar.
  - Which ground intake means guaranteed scoring. Pushing our robot into a game piece, will get picked up by ground intake, which gives our 4-bar perfect alignment. Cycling relatively quickly

  - A bad thing is because it is a reverse 4 bar, the ground intake and initial 4 bar is congested, which might lead to being a bit back heavy (not optimal)

Strategic Ideas

Mechanical Strategies

Intake & Manipulation (Active vs. Passive):

- *Claw bots:* Simple, high precision for placing a single Pin/Cup, but to slow on alignment.

- *Active Roller Intakes:* We require an active roller system that can instantly index a Pin or Cup upon contact, regardless of robot orientation. Top-down gripping mechanisms will dominate over traditional front-facing claws.

Centre of Gravity (CG) & Anti-Tip: The Midfield endgame will involve heavy robot-to-robot interaction. Stacking on the Tall Neutral Goal raises our CG.

- *Engineering Priority:* The chassis must feature a heavy, low-slung base(steel components strategically placed low) and an active anti-tip deployment system (pneumatic or passive drop-down wheels) to prevent tipping when contesting the Midfield.

Strategic Ideas

Auton Strategies

Solo AWP is the main goal for our team. One aim for this game is not achieving S-AWP but rather using April tag sensors for perfect localisation, and there is no doubt they would help create an Auton that would hit 100% of the time, So the big question, how do we integrate into our design?

We could use 1, but that leaves multiple liabilities, like:

- What if our 1 sensor overheats or gets blocked. Reliability on 1 device is not optimal

We could use 2, which makes sense. Having 2 sensors means 2 different reference points which gives us opportunity to perform localisation mathematics. (This will be explained further in our notebook)

We also could use 3, which serves us extremely high accuracy in localisations, but that also means how they are spaced out must be extremely accurate. Also, we might not have enough space for 3 in the build.

Besides the AI vision sensors, there are other ideas floating like:

Do we count on alliances having reliable autonomous, or do we go for S-AWP early season even though it might not work?

We could use a burner Auton for early comps, then work for S-AWP for later ones, but that means constricting our entire season into 1 robot just to tune Autonomous. If we want to rebuild, there would be no point going for S-AWP early.

But going for S-AWP early could secure easy wins at comps for us, giving us credibility in Alliance Selections in High Stakes environments (Elims in Regionals, States, Nationals, Worlds etc)

Strategic Ideas

Match Strategies

Every VEX game is generally divided into two key phases: the early game, where scoring is the primary objective, and the endgame, where additional strategic elements focused on field control become important. These endgame objectives often include features such as climbs, parking zones, or other mechanisms that reward positioning and control. In this year's game, this role is filled by the **King of the Hill** endgame.

Several important strategic questions arise when considering how matches may be played:

- Is it worthwhile to actively contest King of the Hill, and if so, how should this be incorporated into an overall match strategy?
- Should priority be given to scoring and securing pins with cups, or is it more beneficial to spend time de-scoring opponents?
- Is it more effective to defend key switch locations to protect points, or focus on securing the relatively accessible points available in the centre of the field while contesting the hill?

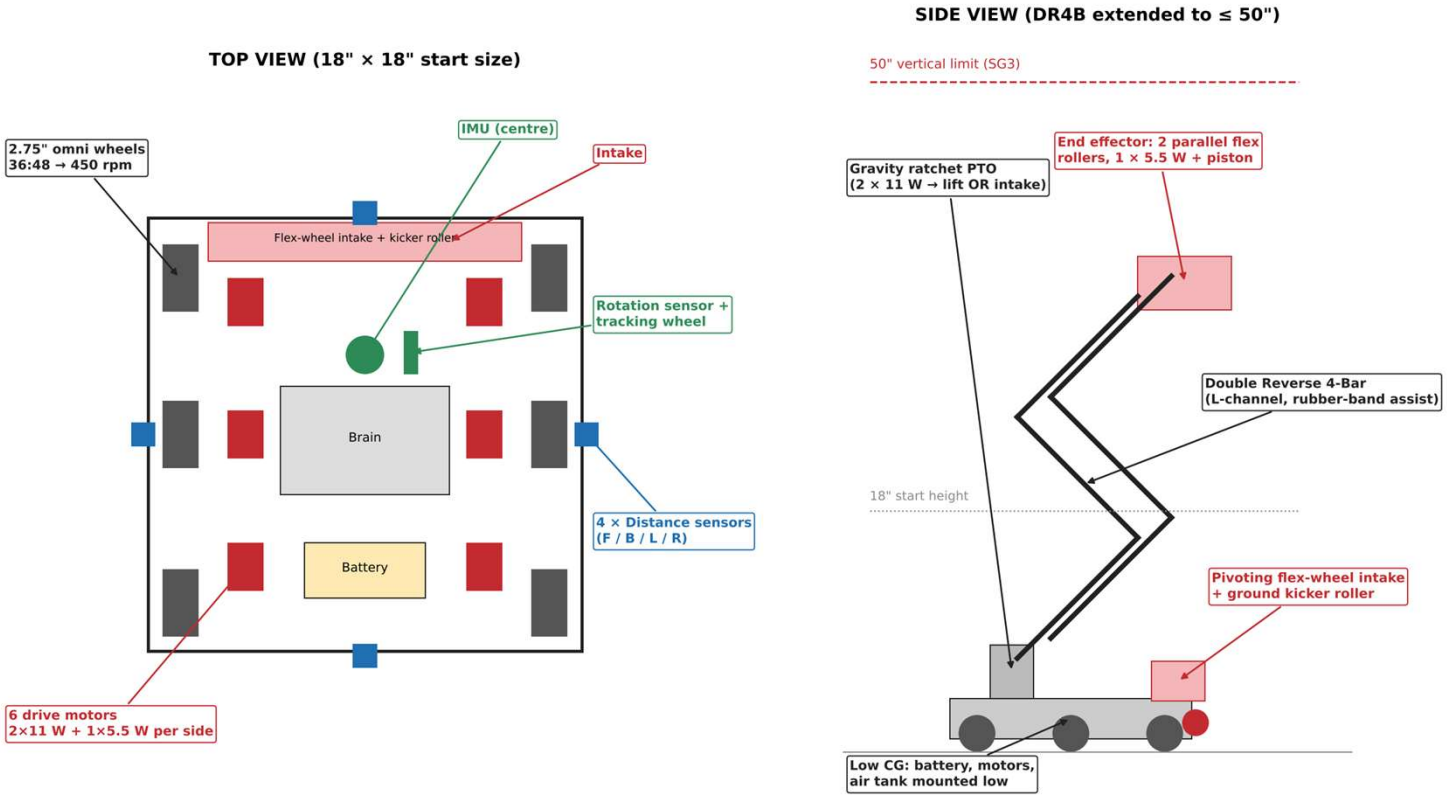
The game presents a significant level of strategic depth, with many factors influencing decision-making throughout a match. At present, the most effective approach appears to be a flexible strategy that adapts to the opposition. Against teams that focus primarily on scoring, the objective may be to either outscore them directly or, where that is unlikely, slow their progress through defensive play.

Scouting is expected to play a major role in strategic planning. Understanding the strengths, weaknesses, and capabilities of opponents can help determine the best approach before a match begins. For example, teams with highly effective switch-manipulation mechanisms may require a stronger defensive response, while exceptionally strong switch-focused teams may influence which scoring opportunities are prioritised.

As a result, detailed scouting of opponents, alliance partners, and potential threats will be essential. Accurate information can improve adaptability, strengthen match strategy, and in some cases provide a significant competitive advantage before a match has even started.

Planned Design

Plan drawing used to guide the build. Every motor, sensor and key design feature is labelled. Dimensions in inches.



Engineering considerations shown on the plan

- **Centre of gravity:** battery, drive motors and air tank sit as low as possible so the robot does not tip when the DR4B is at 47".
- **Footprint:** everything starts inside 18" × 18" × 18" (SG1); the DR4B folds flat over the drive.
- **Power budget:** 55 W drive (R11) + 33 W mechanisms = 88 W; the gravity ratchet lets 2 motors run both lift and intake.
- **Sensing:** IMU at the turning centre (least vibration); one forward tracking wheel (tank drive cannot strafe, so lateral motion = 0); 4 distance sensors facing walls for AMCL.
- **Object path:** ground kicker → pivoting flex-wheel intake → transfer roller → end effector on the DR4B.
- **Serviceability:** intake and ratchet are bolted-on modules (4 bolts per side / 8 mounting points) for fast repair.

Research and alternative designs: chassis pages 30–44, intake pages 46–55, end effector (outtake) pages 77–84. Sources on page 194.

Drivetrain

Regionals

Identifying the Problem

Introduction

The drivetrain, or chassis, is one of the most important components of any robot. It serves as the foundation that supports and connects all other mechanisms while also providing the robot with the ability to move around the field. Because of its importance, several key factors must be considered during the design and construction process.

Build Quality

As the structural backbone of the robot, the drivetrain must be built to an exceptionally high standard. It should be strong, reliable, and capable of withstanding the stresses of competition without loosening or failing. Under normal match conditions, the chassis should remain intact and only be disassembled intentionally by team members when required.

Movement and Performance

The drivetrain must be designed with the 55W power limit in mind while still maintaining fast and responsive movement. Appropriate gearing should be used to maximise motor performance, allowing the robot to achieve a high and stable RPM without sacrificing reliability or control. Efficient movement is essential for both scoring and defensive play throughout a match.

Electrical Components

Sufficient space must be allocated within the chassis for all required electrical components, including the brain, battery, motors, wiring, and pneumatic tanks if used. These components should be positioned in a way that allows easy access while also ensuring they do not interfere with the operation of other mechanisms such as intakes or a 4-bar lift.

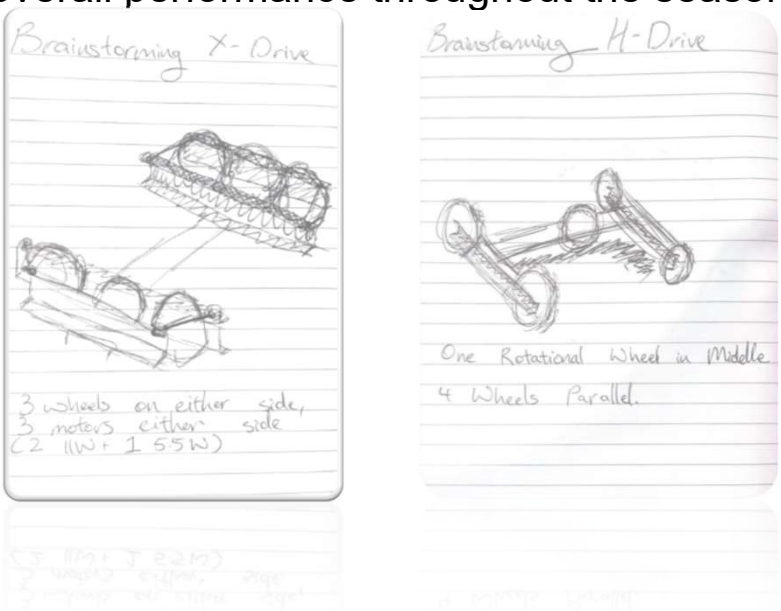
Brainstorming Solutions

Drivetrain Design Types

There are several common drivetrain designs used in VEX Robotics, each with its own advantages and disadvantages depending on the game and robot requirements. The most common designs include:

- H-Drive Chassis
- X-Drive Chassis
- Tank Drive
- Mecanum Drive

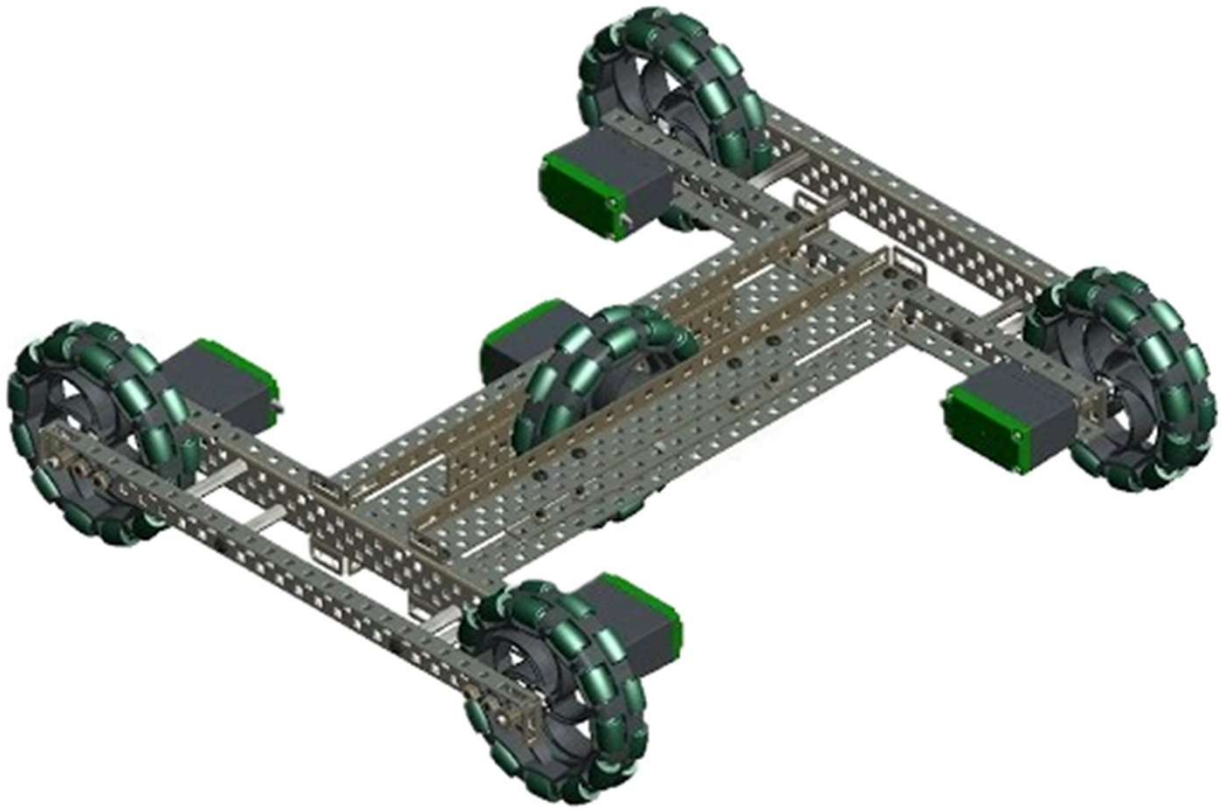
Selecting the most suitable drivetrain design is an important decision, as it can significantly influence the robot's speed, manoeuvrability, reliability, and overall performance throughout the season.



We started by sketching different drivetrain designs to help us clearly visualise how the robot could move and how each layout would fit within the frame. These drawings let us compare options like tank, mecanum, and omni drive systems in a simple, visual way before committing to one. By mapping out wheel placement and overall chassis shape, we were able to better understand how each configuration would affect movement, turning ability, and space for other components. This step made it easier to identify the strengths and weaknesses of each design and helped guide our decision-making before moving into more detailed planning.

Brainstorming Solutions

H-Drive



H-Drive

Since most VEX motors are 11W, it makes sense that we want 5 motors for a drivetrain since the limit is 55W (55 ÷ 11 = 5). This is why an H-drive configuration was considered. Using an H-drive configuration allows us to maintain a powerful drivetrain while also enabling sideways and diagonal movement, which can be especially useful for quickly aligning to the goal. However, we ultimately decided against this design because the centre wheel occupies valuable space in the robot, reducing room for other mechanisms. Additionally, the centre wheel can create extra friction when turning, making the drivetrain less efficient and potentially reducing manoeuvrability compared to alternative designs.

Brainstorming Solutions

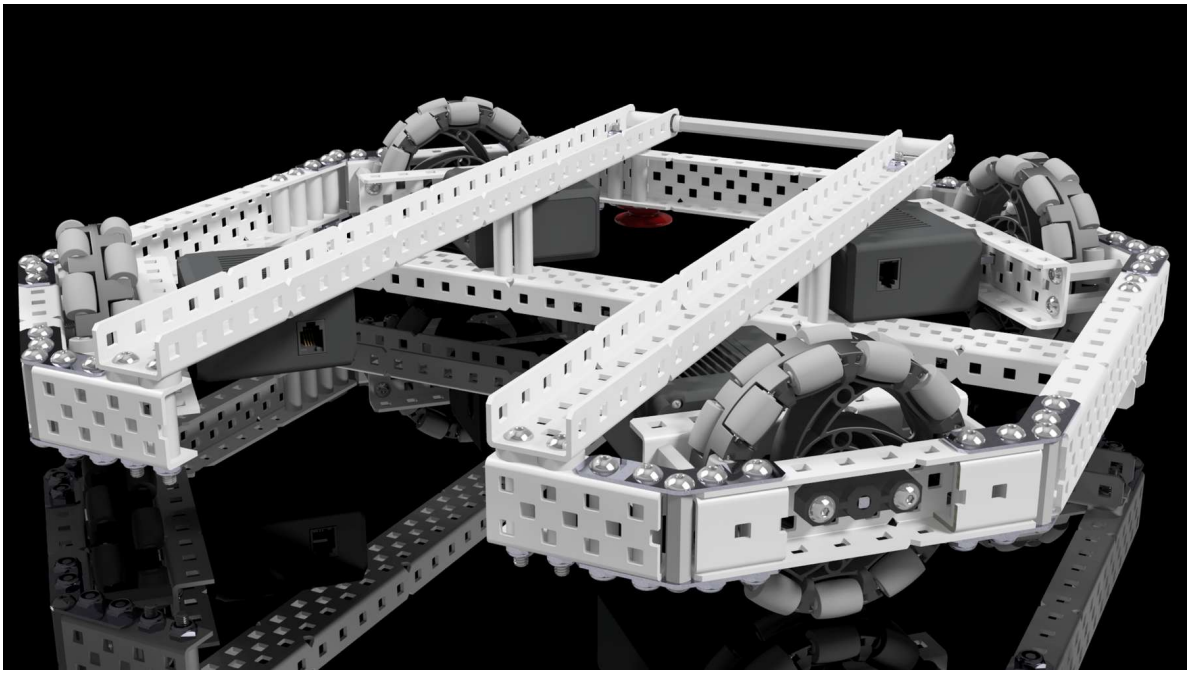
Positives	Negatives
Omni-directional movement The H-drive allows the robot to move in any direction regardless of its orientation, enabling more advanced autonomous paths and giving the driver greater freedom of movement during tele-operated periods. This can be particularly useful for precise alignment with scoring elements	**Ineffective usage of space** Although the front of the robot is clear, the perpendicular wheel in the centre takes up space that may be ideal for odometry encoders, brain, battery and pneumatic tanks.
Simplicity With only one additional wheel compared to a standard tank drive, the H-drive retains much of the simplicity of a conventional drivetrain. It allows for flexibility in motor allocation, wheel spacing, and drive tuning without introducing excessive mechanical complexity.	**Vulnerability to defence** The centre omni wheel generally produces less traction than the main drive wheels. Under defensive pressure, the robot may struggle to maintain precise lateral movement, reducing one of the drivetrain's primary advantages.

Conclusion

Overall, the H-drive does not appear to be the most promising option for our robot. While its omni-directional movement and mechanical simplicity are attractive, the inefficient use of internal space and reduced effectiveness under defensive pressure outweigh these benefits. Additionally, if the robot is unable to consistently align with the scoring pillars while being defended, the advantages of lateral movement become significantly less valuable. As a result, alternative drivetrain designs may provide a better balance of performance, pace efficiency, and reliability.

Brainstorming Solutions

X-Drive



Positive	Negative
Omni-directional movement The X-drive allows the robot to move in any direction without changing its orientation. This can make positioning and aligning to scoring elements quicker and more precise.	**Reduced pushing power** Because the wheels are mounted at 45° angles, not all of the motor's force contributes to forward movement. This results in lower pushing power compared to a tank drive.
Precise turning The drivetrain can be tuned for very accurate rotational movement, which is useful when lining up with goals, field elements, or vision targets.	**Space and complexity** The drivetrain requires four omni wheels and more complex programming than a standard tank drive. It also limits wheel and gearbox configurations compared to simpler drivetrain designs.

Conclusion

Overall, the X-drive offers excellent manoeuvrability and precise movement, making alignment and positioning easier. However, its reduced pushing power and greater complexity make it less effective in defensive situations. As a result, we decided that other drivetrain options would provide a better balance between mobility, strength, and simplicity for our robot.

Brainstorming Solutions

Tank drive

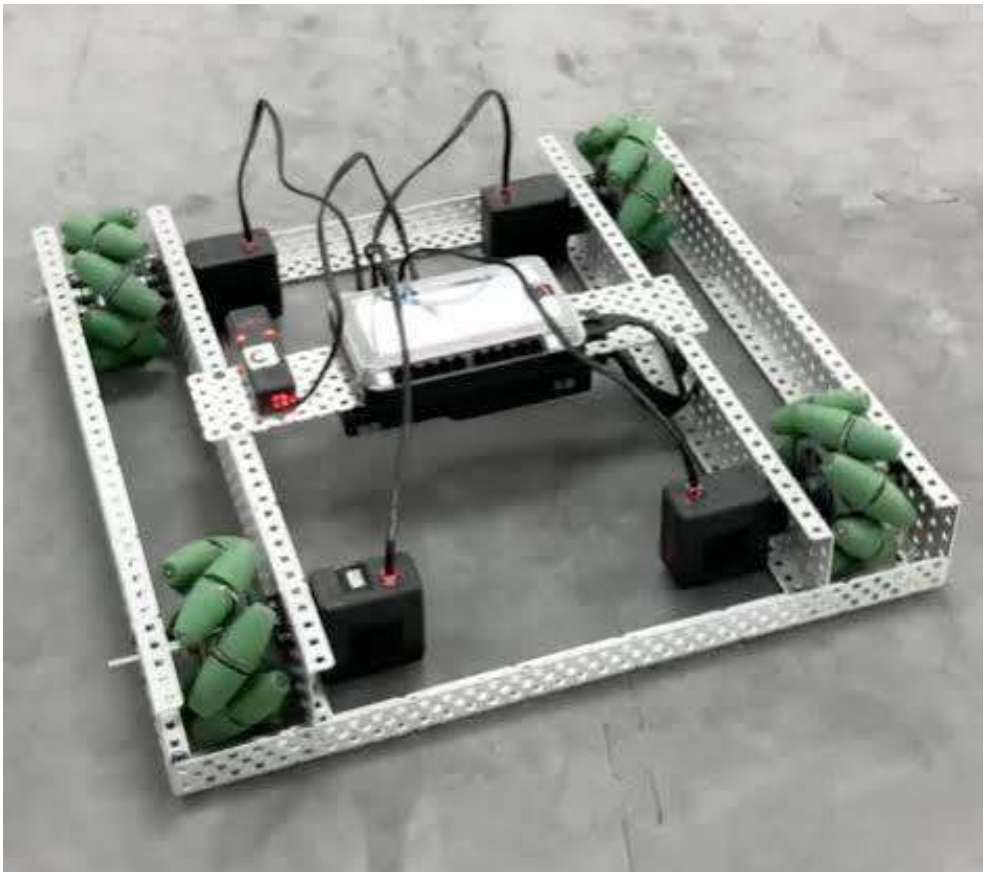
Positive	Negative
High pushing power With all wheels contributing directly to forward movement, tank drive provides excellent pushing power and acceleration, making it effective in defensive situations	**Less manoeuvrable** Tank drive cannot strafe sideways, limiting movement options in tight spaces and during precise positioning tasks.
Simple and reliable Tank drive is mechanically simple, requiring fewer specialised components and making it easier to build, maintain, and program.	**Predictable movement** Because the robot can only move forwards, backwards, and turn, its movement is more predictable and can be easier for opponents to defend against.

Conclusion

Overall, tank drive provides the best balance of power, simplicity, and reliability. Its high pushing force, stable traction, and efficient use of space make it well-suited for defensive play and consistent performance. While it lacks the manoeuvrability of holonomic drivetrains, its strengths in control and robustness make it the most practical choice for our robot.

Brainstorming Solutions

Mecanum drive

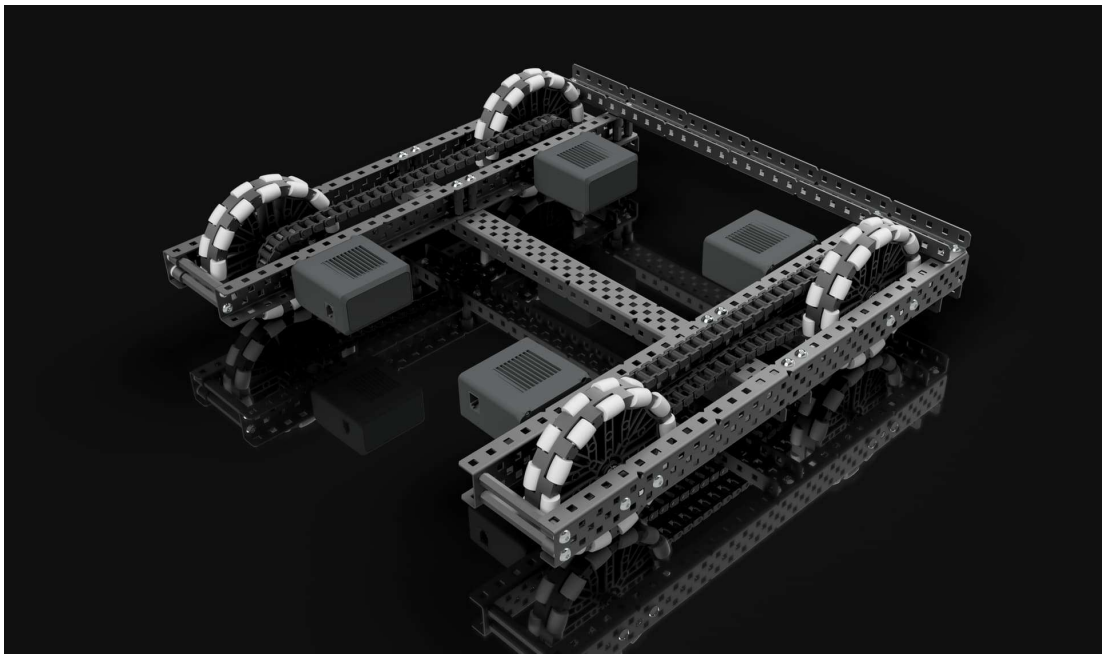


Mecanum Drive

Mecanum drive was also considered as a drivetrain option. It enables omnidirectional movement, allowing the robot to translate in any direction (forward, backward, sideways, and diagonally) while independently controlling its rotational orientation. This is achieved through four Mecanum wheels with rollers angled at approximately 45 degrees, which convert wheel rotation into combined force vectors along both the longitudinal and lateral axes. By controlling the relative speed and direction of each wheel, the system produces a resultant motion vector that can be precisely tuned for complex positioning tasks. This drivetrain is particularly well-suited to applications requiring accurate alignment and repeatable positioning, especially when paired with vision-based systems such as AI Vision Sensors and AprilTags, and it also provides more flexible internal layout options for mounting mechanisms and electronics within the chassis.

Brainstorming Solutions

Tank drive



Tank Drive
Tank drive was one of our top contenders when deciding on a drivetrain. By using three wheels on each side powered by an 11W + 5.5W + 11W motor configuration, the drivetrain can provide high torque and strong pushing power. While it lacks the omni-directional movement of the H-drive and X-drive, its simplicity, reliability, and efficient use of space make it a strong option for our robot.

Brainstorming Solutions

Positive	Negative
Omni-directional movement. Allowing for more complex autonomous and tele-op movements. The ability to move around better means better accuracy, and more reliance on the robot movement, rather than Odom turning.	**Ineffective usage of space** The VEX Mecanum wheels are extremely thick and large, which increase the width of our drivetrain.
Better turning and maneuverability Because of its unique traits, it holds a lot of agility traits that make it a strong choice.	**Lack of speed and torque** There is a lack of traction which results in a much lower acceleration rate and max speed. Also they each require one motor each which is ineffective
	Susceptibility to defence Similar to the previous point, the slipping nature of the Mecanum wheels result in the chassis being able to be pushed.

Conclusion

In conclusion, we decided not to go with mecanum drive since it would be hard to integrate field oriented driving into Vex, as well as get the right wheels for this drivetrain.

Brainstorming Solutions

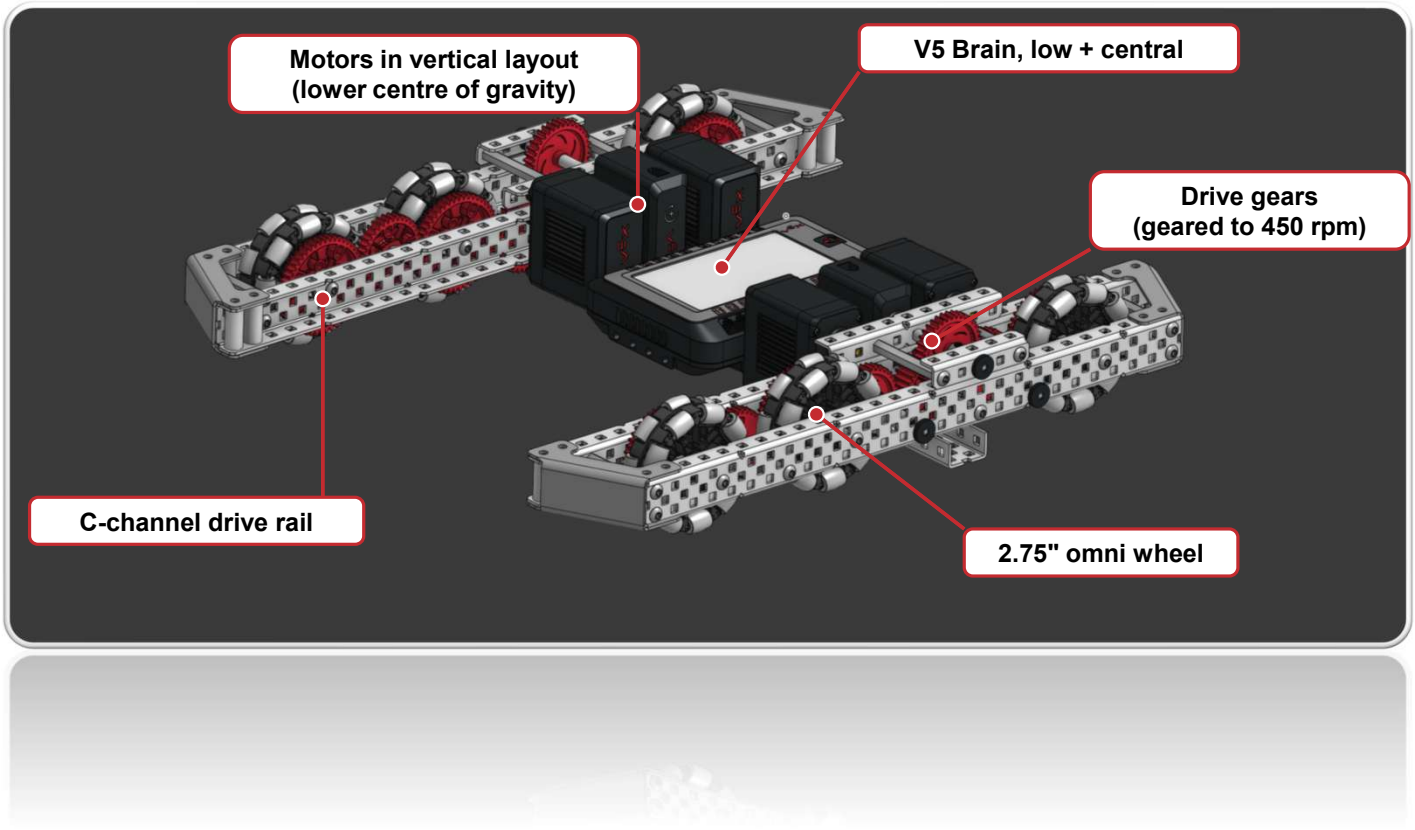
Tank drive

Positive	Negative
High pushing power With all wheels contributing directly to forward movement, tank drive provides excellent pushing power and acceleration, making it effective in defensive situations	**Less manoeuvrable** Tank drive cannot strafe sideways, limiting movement options in tight spaces and during precise positioning tasks.
Simple and reliable Tank drive is mechanically simple, requiring fewer specialised components and making it easier to build, maintain, and program.	**Predictable movement** Because the robot can only move forwards, backwards, and turn, its movement is more predictable and can be easier for opponents to defend against.

Conclusion

Overall, tank drive provides the best balance of power, simplicity, and reliability. Its high pushing force, stable traction, and efficient use of space make it well-suited for defensive play and consistent performance. While it lacks the manoeuvrability of holonomic drivetrains, its strengths in control and robustness make it the most practical choice for our robot.

Selecting Best Solution



CADDING

Our setup uses four 2.75-inch omni wheels mounted directly along the sides of the robot.

We use a vertical motor layout with two 11-watt motors and one 5.5-wattmotor. This configuration is more space-efficient than a stacked motor layout and also lowers the robot's centre of gravity, which we identified as essential given the heights the robot will need to reach.

We paired our motors with gearing to achieve a wheel speed of 450 RPM. Speed Calculation: To determine the robot's speed, we calculate the circumference of the 2.75-inch wheels using the formula C = πd. This gives a wheel circumference of approximately 8.64 inches. Multiplying this by 450wheel rotations per minute and dividing by 60 seconds results in a speed of about 65 inches per second (5.4 feet per second). At this speed, the robot can cross the entire game field in under three seconds.

Test Best Solution

Tank Drive

To fix this without using up any motor ports or air from our tanks, we took thin sheets of smooth plastic (polycarbonate) and bent them to form an **Outward-Angled Shield** around the drive base.

- **How the Physics Work Simply:** Instead of a flat wall, defenders hit a slippery slope that leans outward toward them. When they try to ram or pin our side, their forward force hits the angle and gets directed *upward*. Their front wheels get lifted slightly off the foam tiles, losing their floor grip. Since they can no longer push effectively, our fast omni drive can simply slide sideways and escape the pin.

- **No Snagging:** The plastic wraps completely around the corners. There are no sharp metal edges for an opponent's intake or arms to hook onto. As a bonus, the smooth curved front acts like a funnel, guiding our robot smoothly centred onto the scoring goals even if the driver approaches at a messy angle.

The rules state you can only hold **two game pieces at a time**. (One pin and one cup) If you build a giant vacuum intake that sucks up everything in its path, you will accidentally grab a third piece and get a penalty. We needed an intake that is highly precise, grabbing exactly what we want and leaving the rest behind.

Drive Speed Analysis

Design Objective

After selecting tank drive, we needed to confirm that our chosen gear ratio and wheel size would produce a drivetrain fast enough to move scoring pieces quickly across the field, while remaining controllable enough for precise positioning. We calculated maximum theoretical speed before committing to a final motor configuration.

Wheel Speed Calculation

Our drivetrain uses **2.75-inch wheels** running at **450 RPM** at the output shaft. We computed wheel circumference and then projected maximum field speed:

$C = \pi \times D = 3.1416 \times 2.75 \approx 8.64$ in | $v = 450$ RPM $\times 8.64$ in $\div 60$ s ≈ 64.8 in/s ≈ 1.65 m/s

At 64.8 in/s, the robot can cross the entire 144-inch field in about 2.2 seconds (144 ÷ 64.8). This is fast enough to reach any game piece in the first few seconds of autonomous, yet the gear ratio keeps the motor running well below stall, protecting motor temperature across a full match.

Motor Efficiency

The drivetrain uses six V5 motors – two 11 W and one 5.5 W per side (55 W, the R11 drivetrain cap). Distributing the load across six motors means each motor draws less current at any given moment — motors are most efficient near their designed RPM and produce the most heat when approaching stall. With six sharing the load, no single motor ever works near its torque limit during normal driving, reducing heat build-up and keeping battery voltage stable through the end of a match.

Localization Implication

The 450 RPM output also matches well with our autonomous control system. The motion controller reads the localiser at **100 Hz** and issues velocity commands via **move_velocity()**. At maximum speed the robot travels approximately **0.65 inches per 10 ms** — small enough that each controller cycle can make smooth, incremental corrections without overshooting.

Chassis Engineering

Weight Reduction Strategy

Every gram added to the robot increases the force required for the same acceleration (F = ma). Higher mass means higher motor loads, greater battery consumption, and reduced acceleration performance. Our chassis design was driven by a deliberate **weight minimisation** strategy: we used four primary C-channels plus two cross supports, adding additional structure only where load analysis identified genuine stress paths.

Centre of Gravity

A low centre of gravity was a primary design constraint. Heavy components — **battery, ratchets, pneumatics, and drive motors** — are all positioned as near to the base as possible. This lowers the tipping threshold at high speed, improves stability when defending, and keeps the centre of mass close to the midpoint of the wheelbase so all four drive wheels maintain consistent contact pressure with the field tiles.

Material Selection

Contact surfaces — the funnel and outer guards — use **polycarbonate** rather than aluminium. Polycarbonate offers significantly greater impact toughness per gram than aluminium sheet: it deforms under load and springs back, absorbing collision energy without cracking. Aluminium is rigid and strong in static loading but brittle under sharp impact — exactly the conditions of a defensive VEX match. Polycarbonate panels can also be replaced in minutes if damaged between rounds, whereas bent aluminium requires re-machining.

Funnel Geometry

The front funnel uses **angled polycarbonate panels** rather than straight side rails. When an opposing robot contacts the funnel, the angled surface resolves the collision force into a component that pushes the opponent sideways, away from our robot's path — effectively converting defensive pressure into a self-clearing force. Straight rails create catch points; angled panels deflect.

Sensor Integration

Sensor Suite

The robot carries five sensors that feed directly into the localisation and motion systems. One **V5 Inertial Sensor (IMU)** provides gyroscope-based heading, mounted at the robot's geometric centre to minimise vibration coupling from the drivetrain. One **V5 Rotation Sensor** drives a forward tracking wheel, measuring cumulative centidegrees of wheel rotation. Four **V5 Distance Sensors** face forward, back, left, and right from the robot's faces, providing absolute wall-distance readings that the particle filter uses to correct odometry drift.

Calibration Procedure

Accurate localisation depends entirely on accurate physical constants. We follow a four-step hardware calibration before every competition. **Tracking wheel radius**: roll the robot exactly 48 inches forward, read the encoder in centidegrees, and compute radius = 48 × 36000 / (cdeg × 2π). **Wheel offset**: spin the robot 10 full rotations in place; any centidegree reading from the forward encoder reveals the offset of the tracking wheel from the turning centre. Both values are entered into a single configuration header — no other file in the codebase needs to change.

Why IMU + Encoder, Not Encoder Alone

Early testing showed that computing heading from the difference between left and right drive encoder counts drifts severely over long runs — tiny encoder noise compounds into a spiralling position error. The IMU measures rotation directly and doesn't accumulate this error. We therefore use the **IMU for heading** and the **tracking wheel for distance**. Together they give a reliable short-term dead-reckoning pose that the particle filter corrects every 8 inches of travel — or immediately if the filter's confidence score (Neff) drops below 0.30, indicating that the sensor readings and the current position estimate have diverged.

AMCL Filter Tuning

Particle filter parameters — noise alphas, sensor sigma, resample threshold — are not guessed. We run **30 automated trials** in our browser-based field simulator, which drives the robot on random paths and measures mean position error for each candidate parameter set using hill-climbing optimisation. The best-performing set is copied directly into the robot's configuration header. This closed-loop tuning process is what gives our autonomous consistent sub-inch localisation accuracy across different field conditions and battery levels.

Intake + Transfer

Regionals

Identify the Problem

Introduction

The intake is one of the most important mechanisms on the robot. Without an effective intake system, the robot cannot reliably collect game objects, making consistent scoring impossible. As a result, the design of the intake plays a major role in the overall success of the robot.
Several key design requirements have been identified during the brainstorming and development process.

Efficiency

The intake must be highly effective at collecting game objects from the ground. It should be able to acquire objects quickly and consistently while minimising the risk of jams. At the same time, sufficient contact and tension must be maintained between the intake and the game piece to ensure reliable collection.

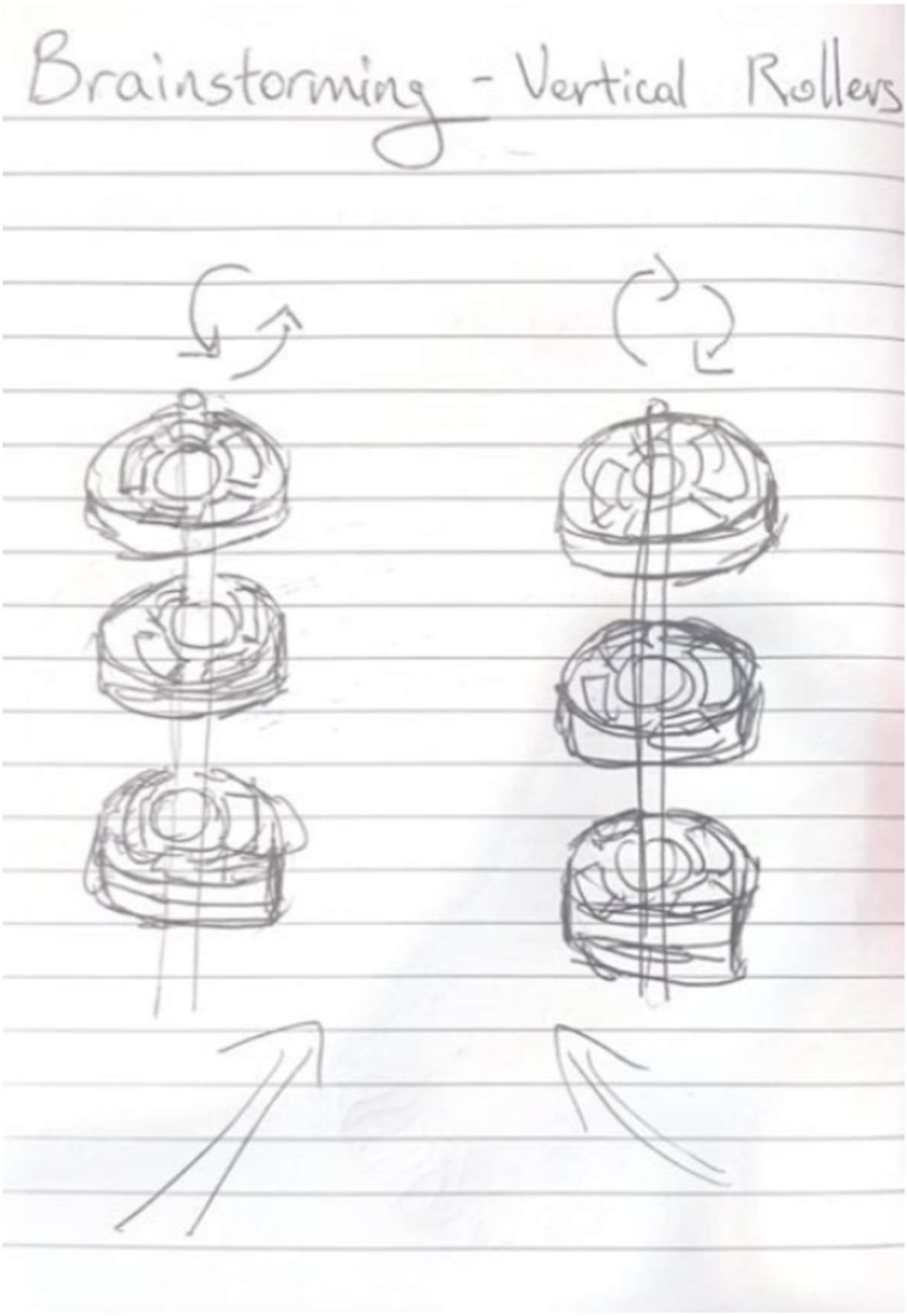
Multiple Orientations

Ideally, the intake should be capable of collecting game objects from a wide range of angles. The unusual shape and orientation of the game pieces mean that an omni-directional intake could significantly improve collection speed and reduce the time required to align the robot. This would ultimately improve cycling efficiency and overall scoring potential.

Speed and efficiency

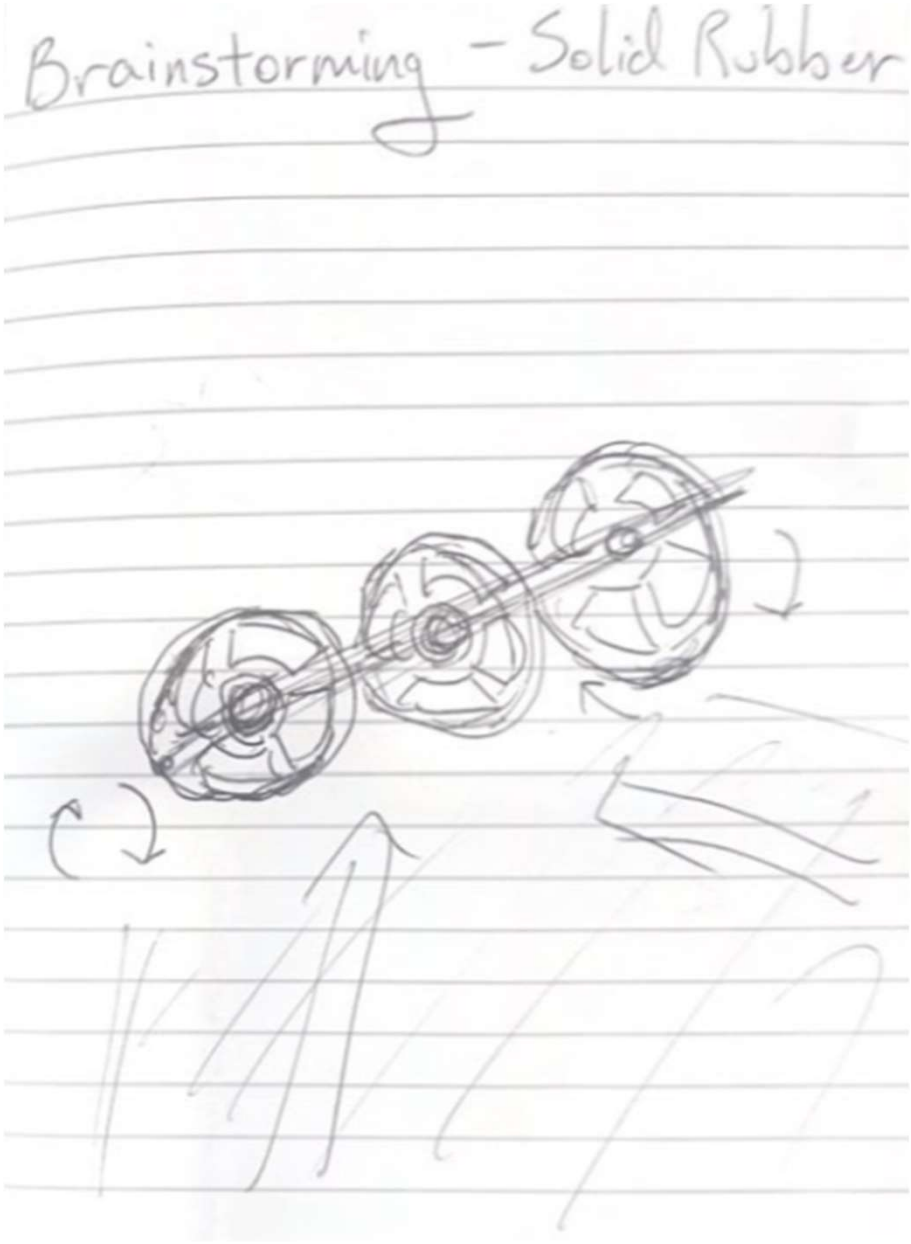
The intake must have enough power to quickly transfer game objects into the robot while maintaining precise control over the number of objects being collected. The design should ensure compliance with rule by preventing the accidental intake of multiple pins or cups at the same time.

Brainstorming Solutions



Vertical rollers are a potential intake. They spin in opposite directions to each other and push the game object into the robot in the upright position, which is perfect for the 4 bar to receive. This can be optimal for space organisation as putting all the motors at the top can be convenient for leaving space open at the bottom for other electrical components.

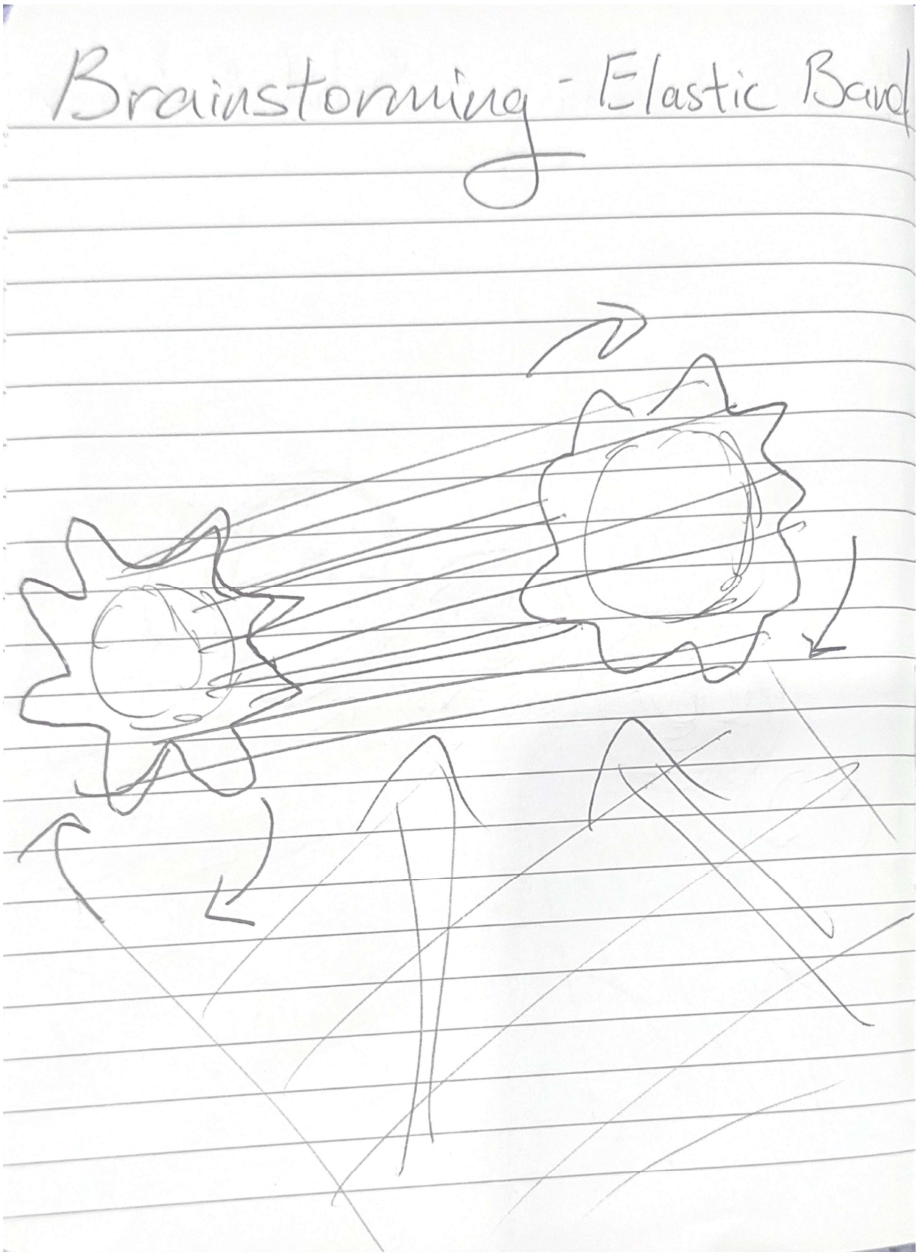
Brainstorming Solutions



Horizontal bar of rubber wheels was last season's meta; thus this intake naturally came to mind. It has great efficiency, and by applying force both downwards and into the machine, it can be very effective and grabbing game objects from the floor directly.

As these game objects are extremely prone to tipping and lying on the ground in hard to reach orientations, having horizontal rollers close to the ground allow us to capitalise off game objects that other teams maybe not be able to reach.

Brainstorming Solutions



The elastic band intake was also what was used last season to push blocks upwards. Thus, is also came to mind. The rubber bands serve as incredibly lightweight alternatives to traditional intakes of wheels, and as such looks very promising.

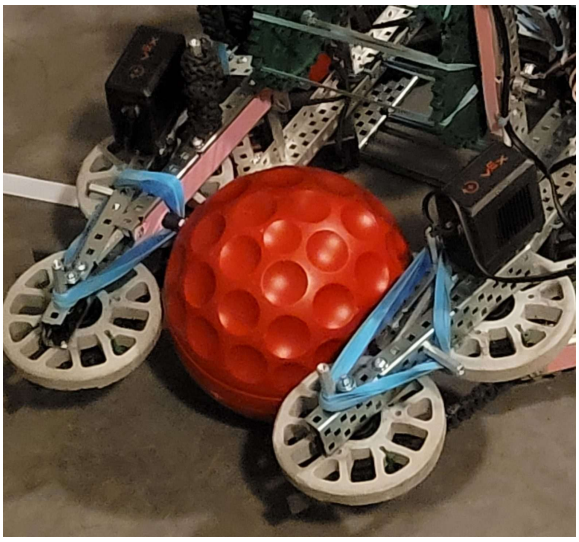
The rubber band gaps can grasp onto the extruding edges of the pins and cups, which makes it consistent and lightweight at the same time.

Brainstorming

Horizontal vs Vertical Rollers

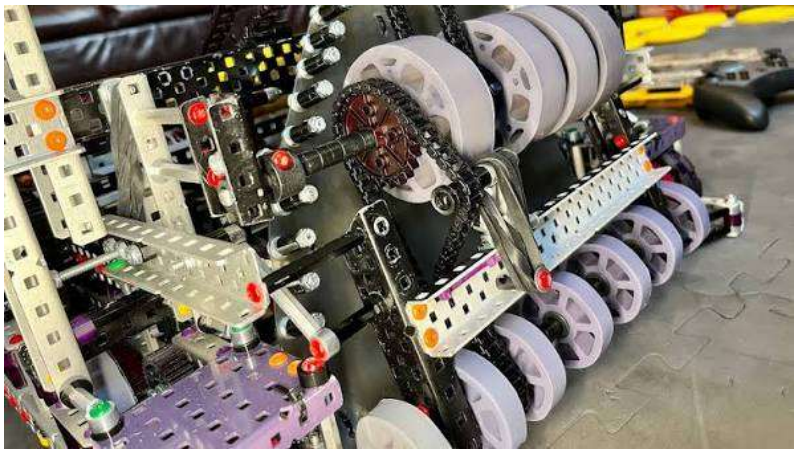
We evaluated both horizontal and vertical intake configurations before finalising the design.

Vertical Rollers



Team - 94999E

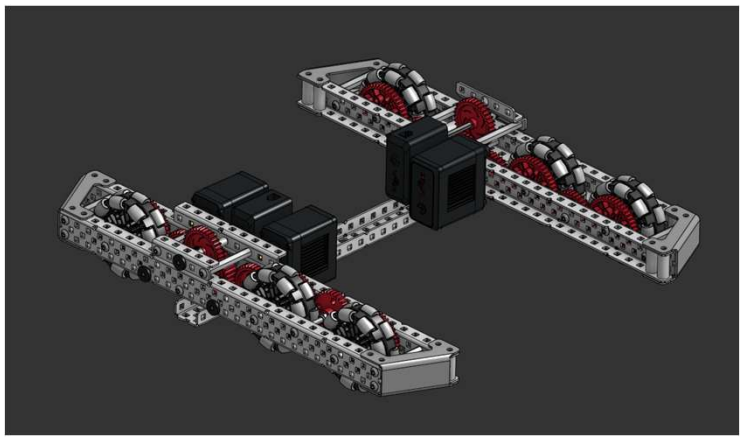
Horizontal Rollers



Team - 606X

A vertical intake initially appeared attractive because it naturally guides objects upward through gravity-assisted motion.

However, detailed CAD analysis showed several structural and packaging disadvantages. Our drivetrain is based on horizontal cross-bracing, which provides the primary structural rigidity of the robot.
A vertical intake would require additional C-channel structures, increasing both weight and complexity.



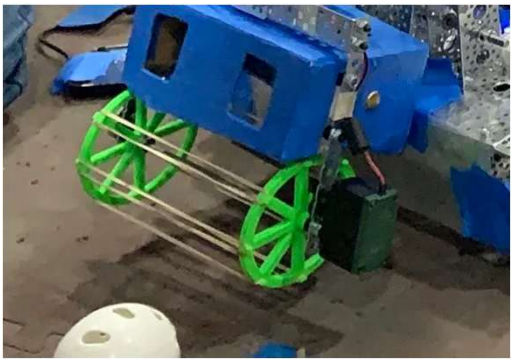
Current Chassis Design

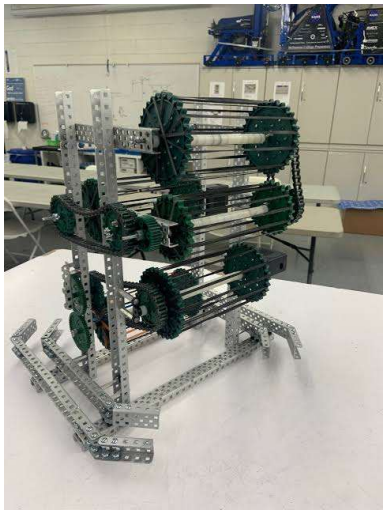
Brainstorming

Fixed point roller

The first intake concept used a fixed front roller with large rubber bands feeding into an intake ramp.

The idea was that the compliance of the rubber bands would allow the intake to adapt to different object sizes while maintaining a simple, low-part-count system. The ramp would guide objects into the robot after initial contact.





This design was selected initially because it was mechanically simple and easy to prototype. It also required minimal actuation, which reduced potential failure points.

At this stage, we prioritised simplicity and fast iteration to establish a baseline intake performance. However, we did not yet fully understand how the game objects would interact with the intake under load, particularly when compression forces and friction against the field surface were considered.

Brainstorming

Ideating

The intake system was designed to meet several core requirements that defined every later iteration.

The primary goal was to reliably collect both pins and cups reliably from a wide variety of angles. This was important because it reduces the need for precise driver alignment and increases consistency during fast gameplay.

The system also needed to integrate cleanly with the rest of the robot, particularly the drivetrain and internal scoring mechanisms.

Additional requirements included keeping the subsystem lightweight, compact, and easy to manufacture. We also prioritised modularity so the intake could be removed quickly for repairs or replacement during competition.

Early in the design process, we decided that the intake would not exist as an isolated mechanism but instead act as part of a larger object transfer pipeline into the centre of the robot.

These requirements were used as constraints during CAD development and helped eliminate designs that were either too tall, too complex, or too heavy. Every subsequent design decision was evaluated against these criteria.

Brainstorming

Static Roller Analysis

During analysis, we identified several fundamental issues with the static roller design.

The most significant problem was that the roller primarily applied downward force rather than lifting objects. Instead of guiding objects upward, the rubber bands compressed them into the field surface, increasing friction and resistance.

This effect was especially problematic for pins and cups, which have sharp edges that could easily catch between the ramp and the ground. Once caught, the applied force from the roller caused the objects to jam before entering the intake.



Another issue was packaging efficiency. The static roller occupied more space than expected because of the size of the rollers. This was due to the amount of compression needed to meet the different sized game pieces.

Which then restricted internal layout options for downstream mechanisms. It also limited our ability to optimise the centre of the robot for object transfer.

Based on these issues, we concluded that the static roller did not meet our core requirement of reliable ground interaction and required a new approach.

Selecting Best Solution

Solution

We condensed picking the intake into 4 factors, which are listed as size/space, weight, handling and complexity. We don't want to prioritise complexity as there is no need for it. As well as this, we are considering the weight factor as it can be good for balancing the relatively heavy 4-bar mechanism. As well as this, the size and space the intake requires should be as little as possible for optimal weight saving.
Following these realisations, here is a table of our recorded intake designs:

Feature	Option 1: Vertical Rollers	Option 2: Solid Rubber Wheels	Option 3: Elastic Band Roller (Picked)
Size/Space	Poor; forces the front wheels to be too wide apart.	Medium; requires a tall space to clear the wheels.	**Excellent; fits entirely inside our flat frame rails.**
Weight	High; requires two heavy, matching gear setups.	Medium; solid rubber wheels add rotational weight.	**Ultra-Low; lightweight plastic sprockets.**
Handling	Poor; parts must be perfectly sized or they jam.	Medium; requires a heavy swinging hinge with springs.	**Excellent; bands stretch and flex over bumps naturally.**
Complexity	High; lots of spinning shafts and brackets.	High; moving hinges can break or pinch wires.	**Very Low; static axles with zero extra moving links.**

We rejected solid rubber wheels because the game pieces in *Override* have uneven shapes and bumpy surfaces. To make a solid wheel grip a bumpy object, you must mount the wheel on a swinging arm pulled down by springs so it can bob up and down over the bumps. This adds extra weight, hinges, and points of failure.

In conclusion, we picked the Elastic Band Roller, since it was incredibly light weight, was malleable space wise while also being relatively small, can handle the pieces very well since the naturally spiky bits of the game pieces can fit through the rubber bands and be brought in easily.

Selecting Best Solution

Solution

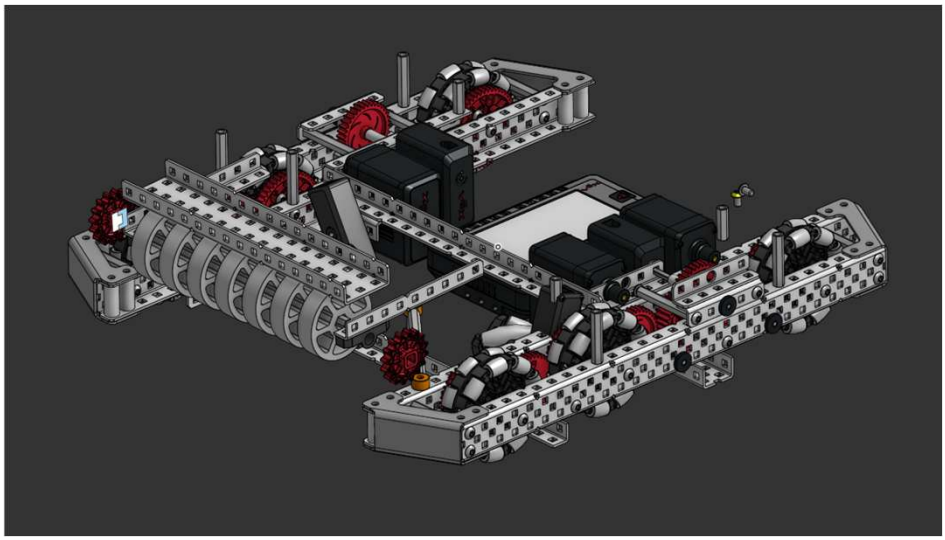
In conclusion, we went with a hybrid intake, Using both flex wheels for the first stage, then rubber band rollers for the second stage. This allowed us to achieve the best of both options.
The flex wheels on the first stage allow us to maintain “Touch it, own it” With a quick pickup and enough torque. The rubber band roller than allows for the different shapes of the cup and pin to be smoothly transferred into the end effector. Removing the instability and improving consistency over just flex wheels.

Prototyping

Pivoting Intake

To address the limitations of the static system, we redesigned the intake as a pivoting mechanism using 2-inch Flex Wheels.

The key change was allowing the intake to rotate slightly when interacting with game objects, rather than remaining fixed. This pivoting action reduced the effective downward force on objects and allowed the intake to adapt to variations in object size and position.



The use of Flex Wheels improved grip while maintaining compliance, which helped reduce jamming compared to the rubber band roller.

This iteration significantly improved reliability during early testing and represented a major improvement in control over intake force.

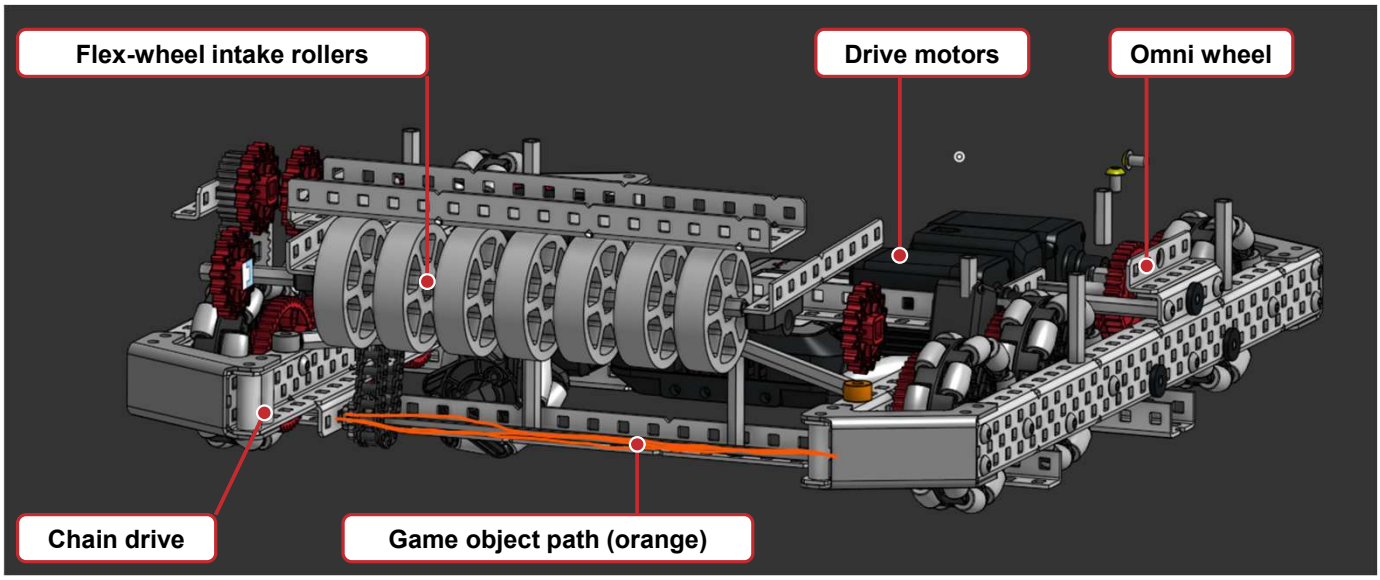
However, while performance improved, we still observed inconsistent entry of objects into the intake, which led to the next design iteration involving a ground-level assist mechanism.

Prototyping

Kicker Roller Addition

Although the pivoting intake improved interaction with the field, it still struggled to consistently lift objects from flat contact.

In many cases, objects would slide in front of the intake rather than being pulled in. To solve this, we added a small ground roller positioned approximately 1 cm below the main intake ramp.
This roller acts as the first point of contact and slightly lifts the object before it reaches the main intake wheels.
This change altered the initial interaction geometry, ensuring that the object is directed upward rather than being pushed forward.



It effectively reduces the required friction force from the main intake system and improves consistency in edge cases where objects are flat or partially obstructed.

This addition significantly improved intake reliability without increasing system complexity or weight.

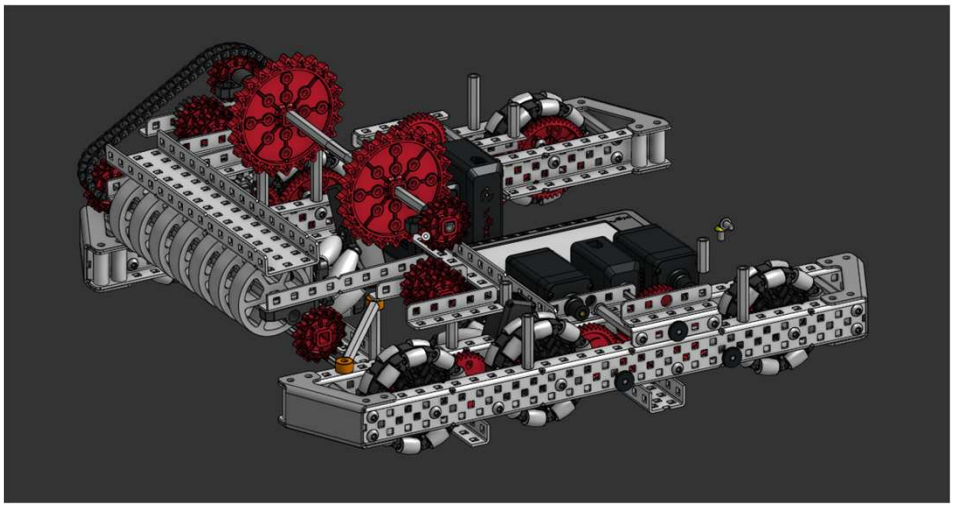
Brainstorming

Transfer Roller

After testing the initial intake design, we found that game pieces were not consistently positioned correctly for loading into the claw.

To solve this issue, we introduced a second powered transfer roller positioned behind and above the pivoting intake.

This roller actively guides game pieces from the intake into the centre of the robot, ensuring they are correctly aligned before reaching the claw. By controlling the movement of the game pieces throughout the transfer process, the additional roller improved reliability, reduced the likelihood of jams, and increased the consistency of the intake system during operation.



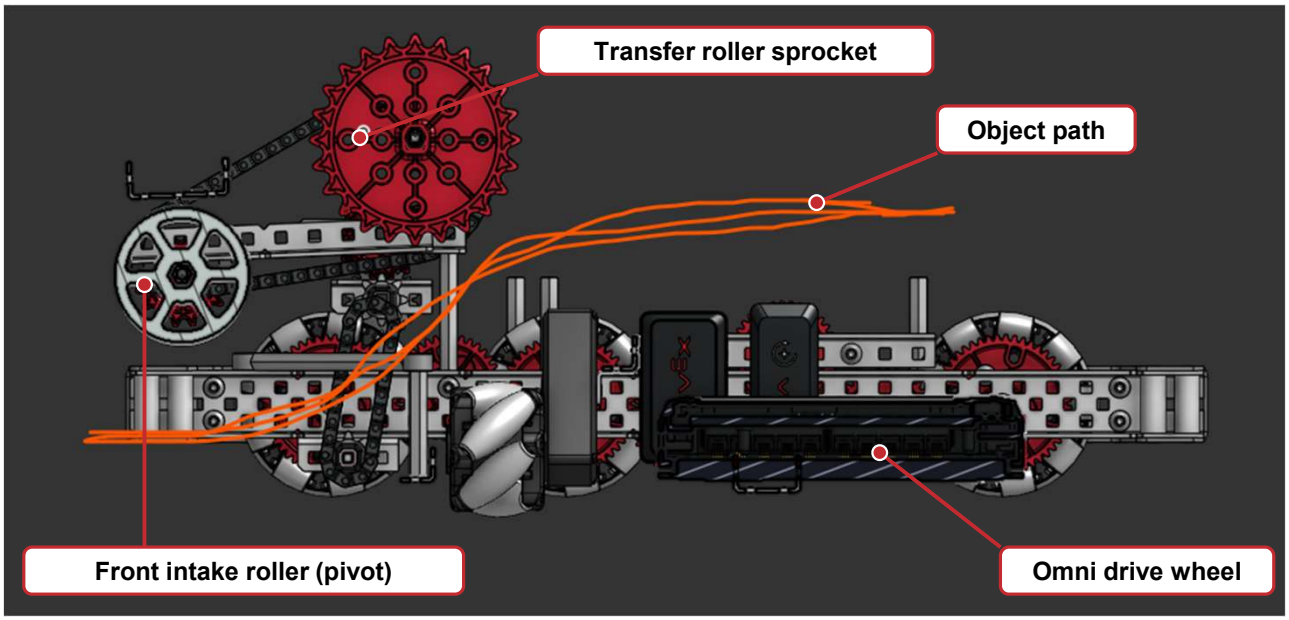
Brainstorming

Transfer Roller

Initially, we intended to mount this roller directly on the pivot axis of the intake. This would have simplified power transmission because the roller position would remain fixed relative to the intake motion.

However, CAD analysis showed that this approach would require raising the pivot height.
Raising the pivot would significantly increase the minimum intake height from the ground, which directly conflicted with our primary design requirement of low pickup geometry.

As a result, we decoupled the transfer roller from the pivot and instead designed a partially elevated transfer section while keeping the main intake as low as possible.
This compromise maintained performance while preserving ground interaction efficiency.



Intake Powering

Transfer Roller

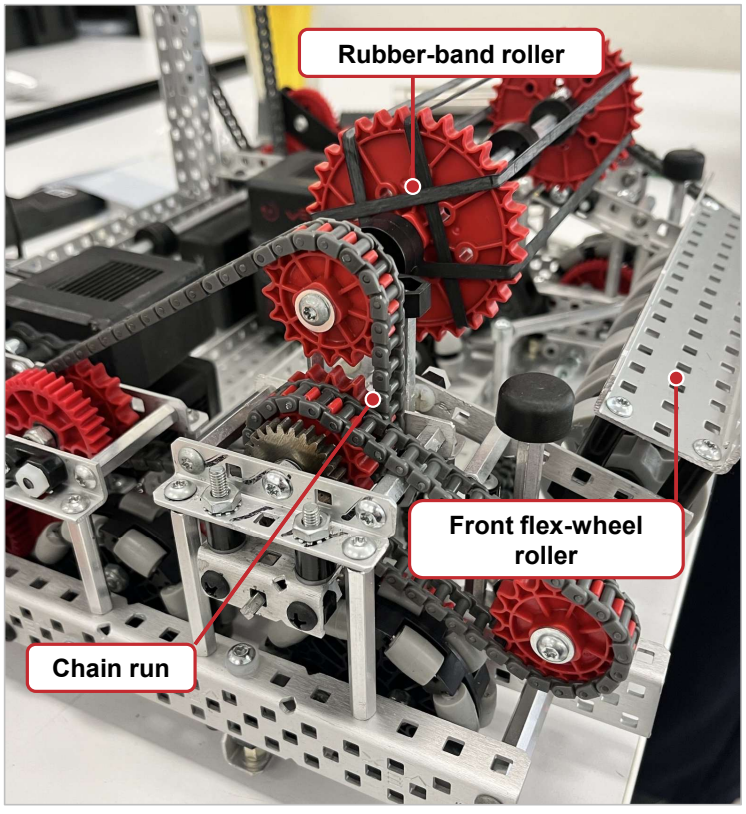
The intake system uses a chain-driven power transmission system to synchronise all rollers.

This includes the main intake wheels, transfer roller, and kicker mechanism. Chain was selected due to its reliability under load and ease of maintenance compared to belt systems.

A key design feature is the inclusion of a simple tensioning system using a banded bar.

This allows chain tension to be adjusted quickly without requiring complex mechanisms or additional components.

The power transmission system was designed to ensure consistent motion across all rollers, preventing desynchronisation which could lead to jams or uneven feeding of game objects.



What the photo shows

- One chain run links the front roller, kicker and rubber-band roller so they always turn together.
- Idler sprockets route the chain around the drive rail.
- Photo taken on the finished robot.

Intake Structure

Four changes that made the intake light, strong and easy to service. Each panel is dated with when the change was made.

Design choices · 29/07/26

- Stand-off shafts used as the main frame members instead of C-channel where possible – rigid enough, much lighter.
- High-strength shafts only where there is direct load or bearing support.

Interchangeable design · 3/08/26

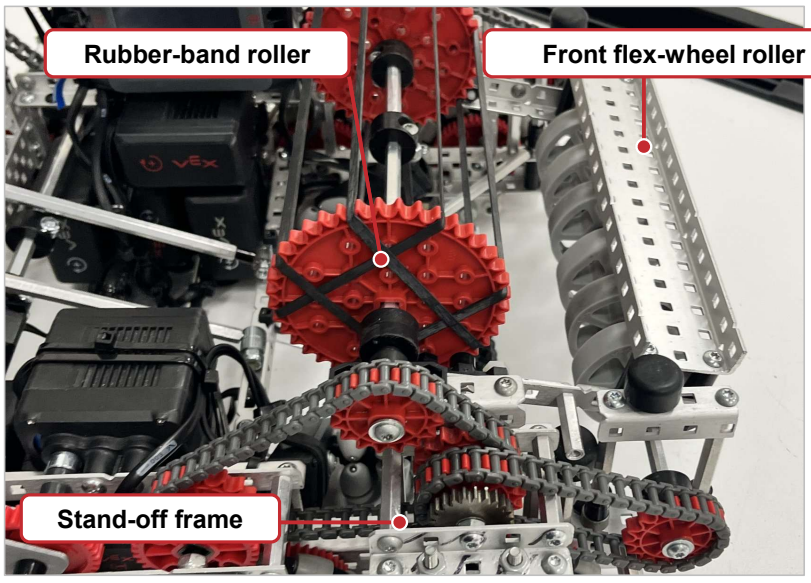
- The intake is a separate module held on by four bolts per side.
- It can be removed for repair, swapped at competition and tested on its own.

Stand-off reinforcement · 31/08/26

- Testing showed stand-offs loosening under repeated loading in one direction.
- High-strength collars now clamp the flat faces, so intake vibration cannot unscrew them.

Dual-sided drive · 31/08/26

- Driving from one side twisted the intake during high-resistance pickups.
- Driving both sides spreads the torque, lowers shaft stress and keeps the rollers aligned.



Result

- Lighter intake module with no loose stand-offs after the collar fix.
- Whole module can be unbolted in minutes.

Intake Integration

How the intake was fitted around the lift, battery air tank and drivetrain, and what we learned from the intake design cycle.

Integration with lift · 10/08/26

- The compact intake sits close to the DR4B without hitting it.
- Geometry lets pieces transfer straight into the scoring mechanism – the two systems had to be designed together.

Weight optimisation · 12/08/26

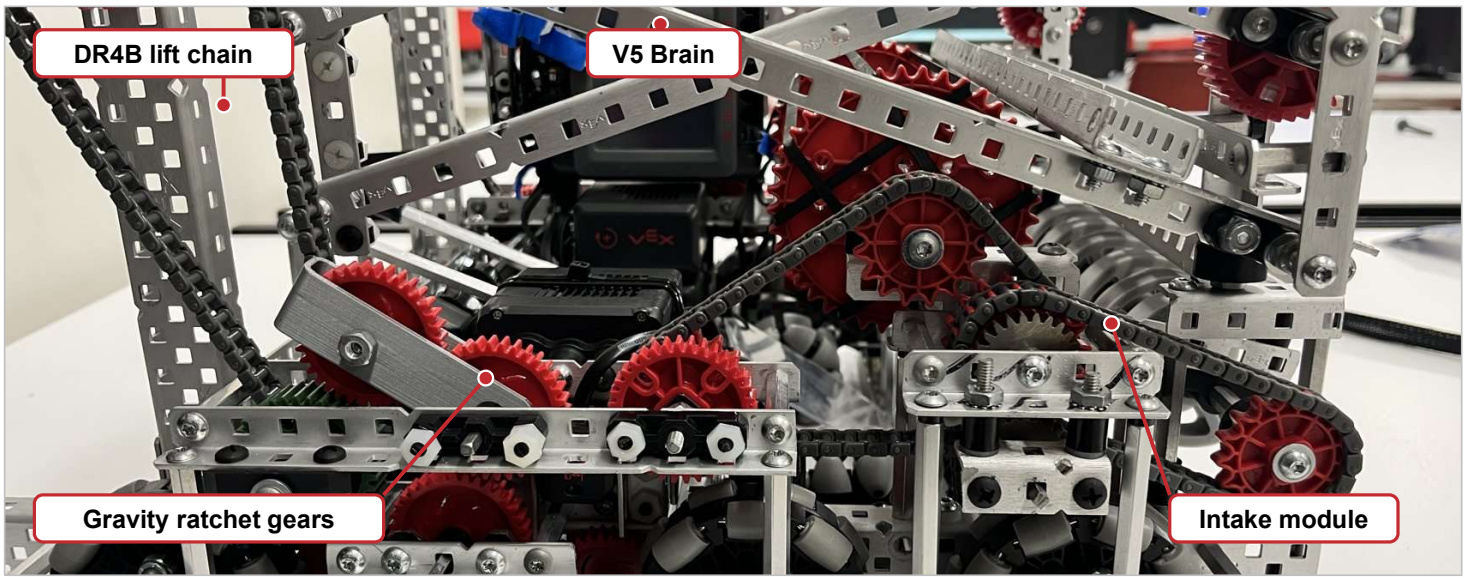
- Stand-offs replaced heavier parts and unneeded parts were removed every iteration.
- Less mass means less strain on the 55 W drivetrain.

Final design reflection · 14/08/26

- Each iteration fixed one failure mode from the version before.
- Only the final design was manufactured; it balances low pickup, reliable collection, compact packaging and modular build.

Pneumatic tank placement · 28/08/26

- Under the chassis the tank hit the drivetrain and intake.
- Moved onto the intake: it fits, and its weight pushes the intake down for better ground contact.



Lift

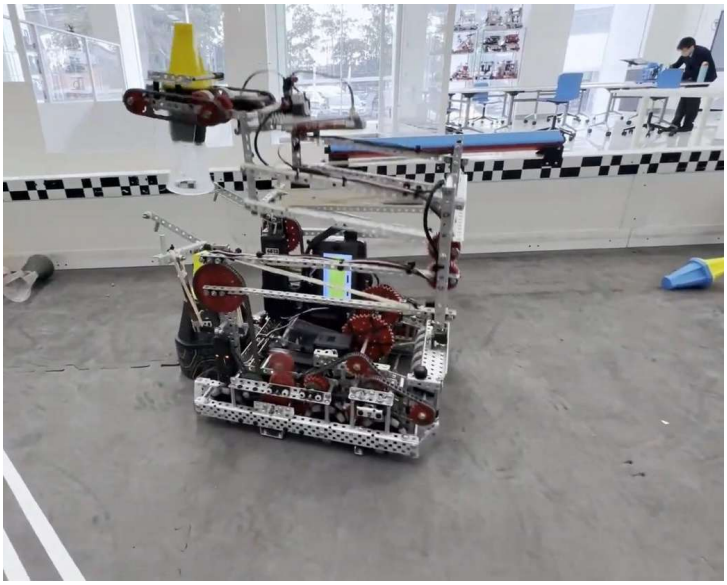
Regionals

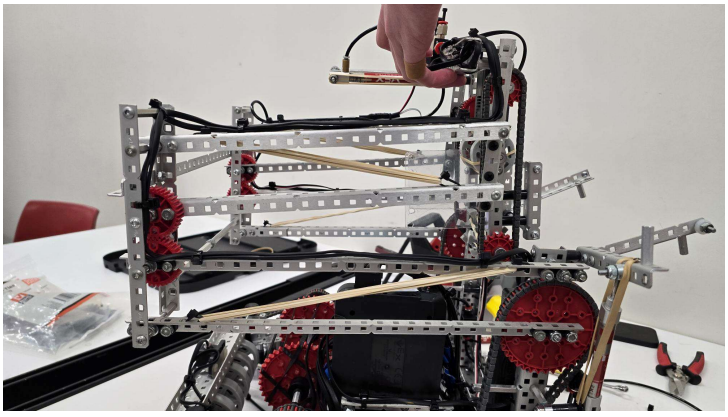
Lift Mechanism Selection

Lift Mechanism Selection

We selected a Double Reverse Four Bar (DR4B) lift over alternatives such as a linkage lift, a single four bar, and a cascade lift after considering game requirements and design trade-offs.

- *Cascade lift:* Ruled out — added unnecessary complexity with no meaningful competitive advantage.
- *Single four bar:* Insufficient vertical travel. The DR4B offers large extension with a compact folded height, ideal for our constraints.
- *Weight advantage:* Fewer parts and lower mass, improving acceleration, reducing motor load, and increasing efficiency.





Selecting Best Solution

Scoring Game Objects:

Early on, we built an arm with a motorized wrist joint that reached all the way down to the foam tiles to grab pieces directly off the floor, avoiding an internal intake completely. It failed for three reasons:

- *Driver Latency:* It took too long for the driver to center the claw exactly over a loose piece on the floor, slowing down our scoring speed.

- *Motor Strain:* Lifting a heavy claw and game piece from way out in front of the robot created massive leverage, making the motors run hot and draw too much battery current.

- *Pinch Points:* Running electrical wires along multiple moving hinges caused the wires to get caught, pinched, and cut during fast movements.

We abandoned that prototype and chose to pair our internal horizontal intake with a clean **Double Reverse Four Bar (DR4B)** lift instead.

Why DR4B:

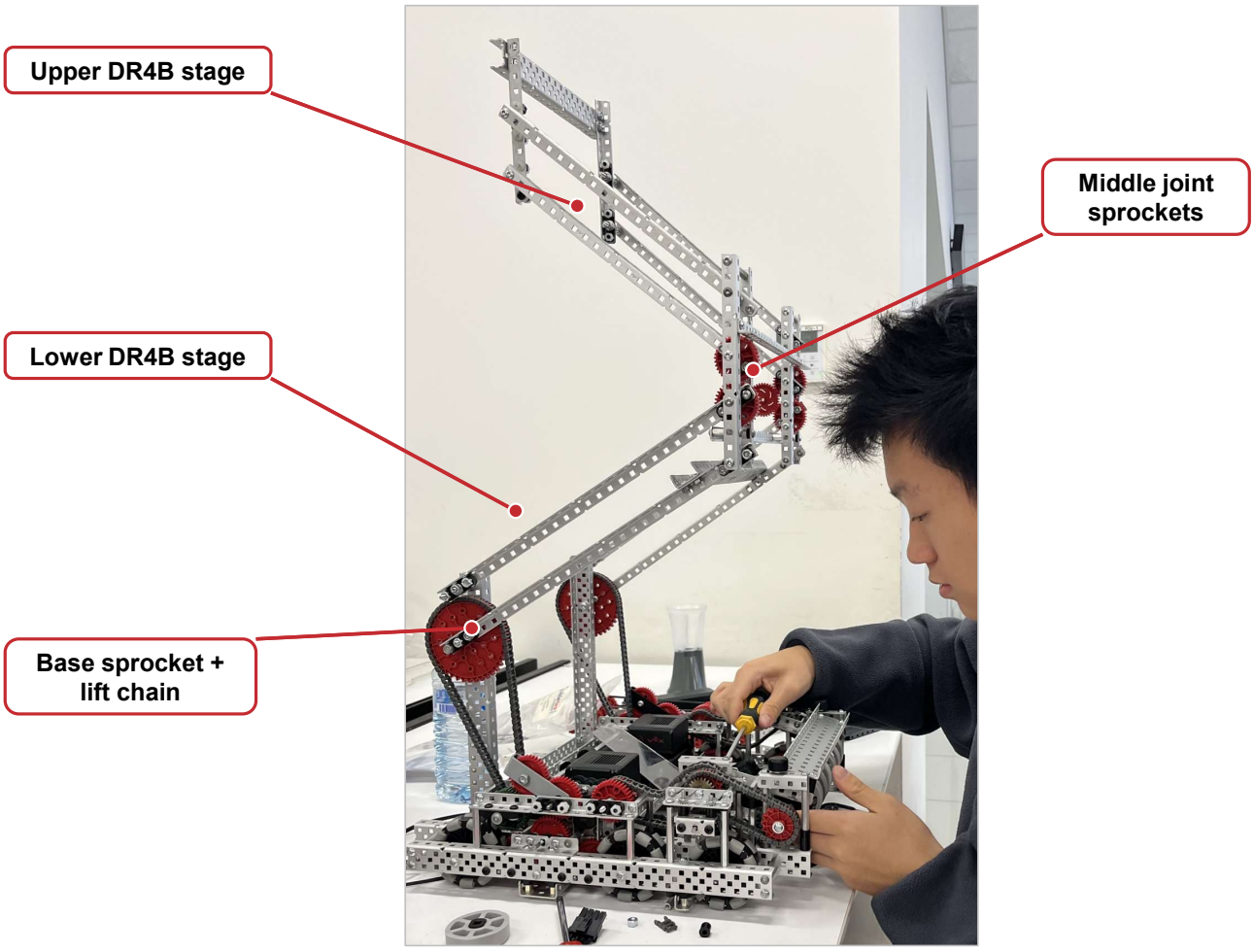
Unlike a single arm that swings in a wide arc, the DR4B expands straight up and down. This keeps the weight of the game pieces centred right over our middle wheels, preventing the robot from becoming top-heavy or tipping over when fully extended. It also matches our gravity ratchet because the arm falls back down smoothly to the floor under its own weight when the motor power is cut.

DR4B Optimisation

DR4B Prototype Development

The entire DR4B was initially constructed using C-channels to quickly validate geometry and motion characteristics. Once proven, the structure was refined to use L-channels wherever possible.

- *C-channel prototype:* Rapid validation of geometry and motion. Reliable baseline for iteration.
- *L-channel optimisation:* Significantly reduced overall weight while maintaining rigidity and strength for competition use.
- Mass saved = m_prototype − m_final. Reducing moving mass improves acceleration, reduces motor load, and increases overall efficiency.



Assembling the L-channel DR4B on the chassis.

DR4B Torque Analysis

Why a Double Reverse Four-Bar

The robot requires significant vertical reach to score above field obstacles while keeping its collapsed footprint small enough to pass under gate structures. We evaluated three lift concepts. A **single arm** is simple but the moment arm grows rapidly as the arm extends, creating very high torque requirements on the motor. A **cascade lift** gives excellent reach but requires complex rigging and a large volume when collapsed. The **DR4B** keeps its end-effector level throughout the stroke and provides a compact collapsed profile — ideal for our packaging constraints.

Critical Load Point

The most demanding condition for any four-bar lift occurs at the **beginning of the stroke, at the lowest angle**. At low angles the effective moment arm on the driven link is shortest, meaning the same gravity load requires the highest motor torque to overcome. We sized our gear reduction so that available motor torque at this worst-case point exceeds the required lifting torque by a **comfortable safety margin**. This reduces stall risk, prevents overheating mid-match, and extends gear life.

Rubber Band Assist

Rubber bands are routed across the four-bar links to provide a **passive lifting force** that approximates the gravity load. When correctly tensioned, the bands nearly counterbalance the arm weight at all points in the stroke, so the motors only need to supply the net force required for acceleration and deceleration — not to support the arm's weight. This reduces average current draw significantly and means the arm can accelerate quickly even with a reduced motor count, freeing motors for other subsystems.

Intake Compliance

The intake uses a hybrid: compliant flex wheels on a pivot at the front and a rubber band transfer roller behind. Game pieces have irregular geometry that creates uneven contact and jamming with rigid surfaces. The rubber band system introduces passive compliance — the bands deform around each piece, increasing effective contact area, adapting to shape variation, and dramatically reducing jam probability during rapid collection cycles.

BARKER REDBACKS 4613

Innovate

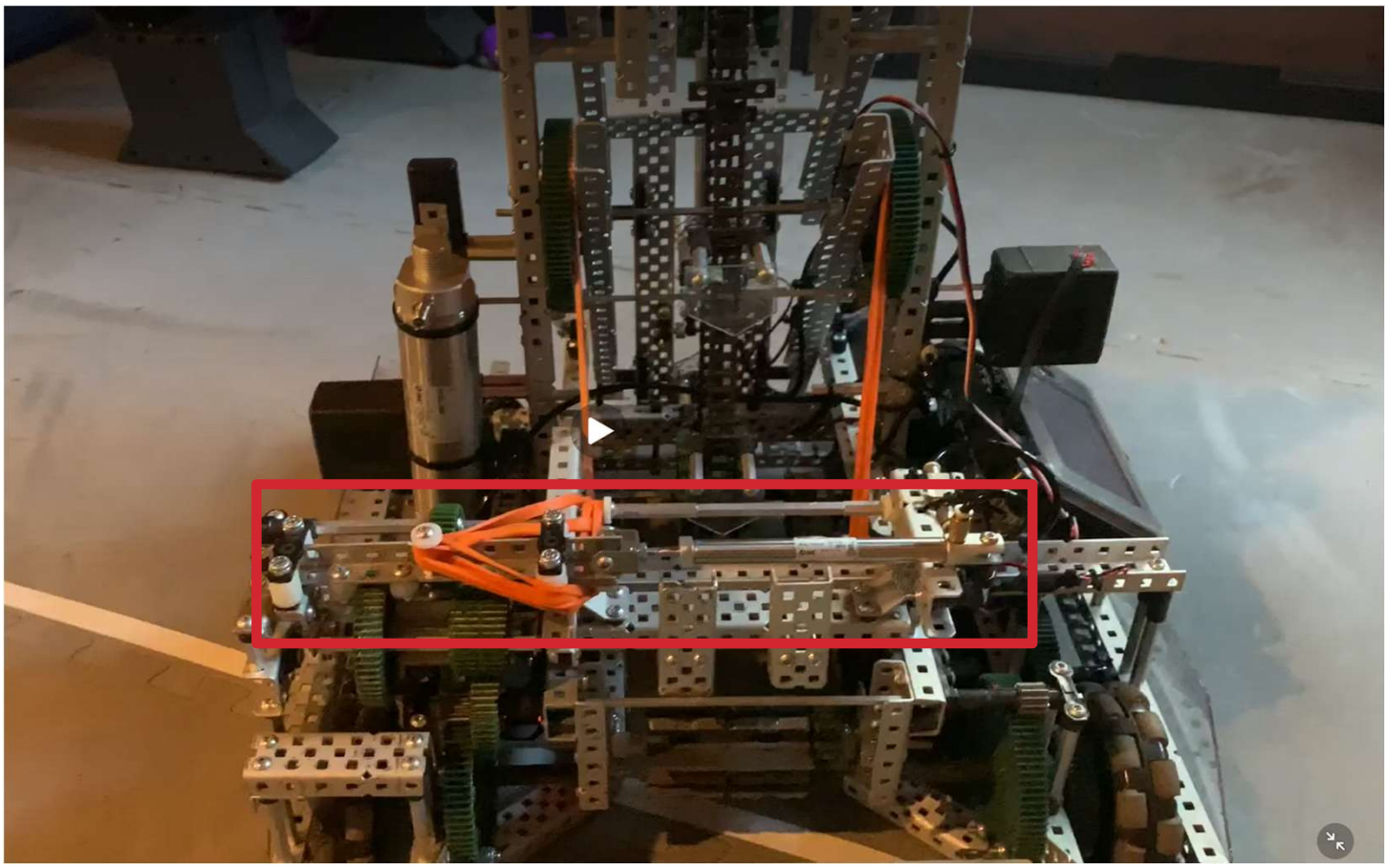
Gravity Ratchet

Identify + Brainstorm

Introduction

One of the key goals of the robot design was to minimise weight while making the most efficient use of the available motors. In a typical VEX robot, separate motors are dedicated to the intake and lift mechanisms. While this approach is effective, it increases both weight and motor usage. To overcome this issue, many teams implement a **Power Take-Off (PTO)** system, which allows a single set of motors to power multiple mechanisms.

Traditional PTO systems often rely on pneumatic pistons to slide gears between different drive paths. Although effective, these systems require additional space and consume compressed air, both of which are valuable resources that could be used elsewhere on the robot.



473Z – PTO system

Gravity Ratchet

PTO & Air Distribution

Because our gear-shifter works automatically through the motors, we don't need any air pistons to swap functions. This allowed us to carry only **one single air tank** on the whole robot to save weight.

To prove one tank was enough for a full 2-minute match, we ran a simple air usage model based on how much air our tiny claw pistons gulp down on each push:

In the real world, air systems lose pressure due to tiny leaks in the plastic tubing and friction in the linkages. We cut our theoretical guess down by 25% to be safe, leaving us with about **120 reliable pushes**. Since our claw mechanism uses 5 tiny pistons for its movements, this gives us exactly **24 clean scoring runs** before the tank drops below 35 PSI and becomes too weak. This perfectly matches our match strategy while keeping the robot light.

$$\text{Piston Volume} = \underbrace{\pi r^2}_{\text{bore area}} \times \underbrace{L}_{\text{stroke length}}$$

$$\text{Sips} = \frac{\text{Tank Volume}}{\text{Piston Volume}} \approx 160 \text{ pushes}$$

Gravity Ratchet

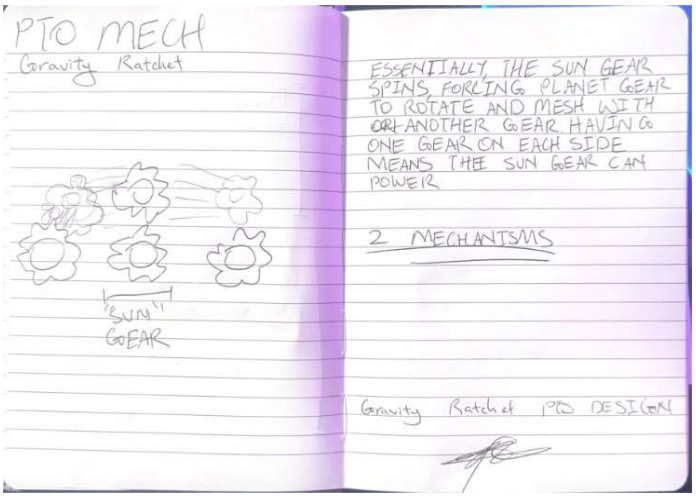
Power system ideation

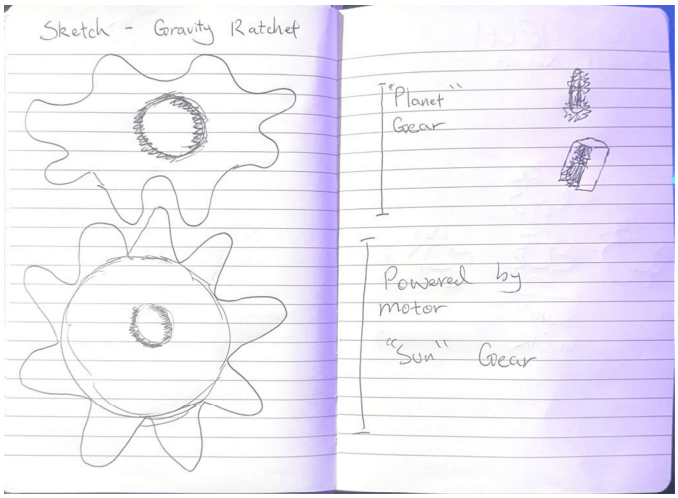
As a result, an alternative PTO solution was explored. Any alternative design would need to meet several key requirements:

- **Space Efficient** – The mechanism should occupy as little space as possible, allowing more room for scoring and intake systems.
- **Reliable and Sustainable** – The design should not rely heavily on consumable resources or place excessive strain on motors, reducing the risk of performance loss throughout a match.
- **Consistent** – Since the system is responsible for powering two critical mechanisms, it must operate reliably under all match conditions.

With these requirements in mind, the design process focused on developing a more practical and efficient alternative to traditional pneumatic PTO systems.

We proceeded to brainstorm alternative solutions that could power multiple subsystems without the use of pneumatics

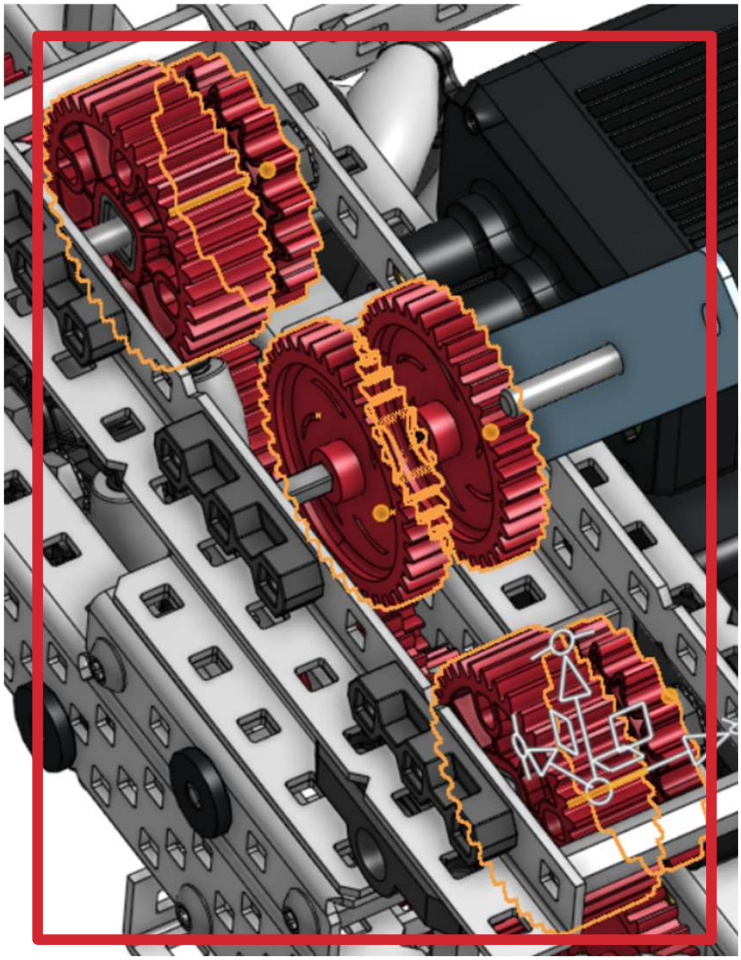




Gravity Ratchet

Gravity Ratchet — Analysis & Trade-offs

The friction between the gears does introduce some power loss, reducing the overall efficiency of the system. However, we found this effect to be minimal in testing. The free-speed current draw displayed on the V5 Brain remained effectively unchanged with the gravity ratchet engaged, suggesting that the added friction had little measurable effect on the motor at free speed.



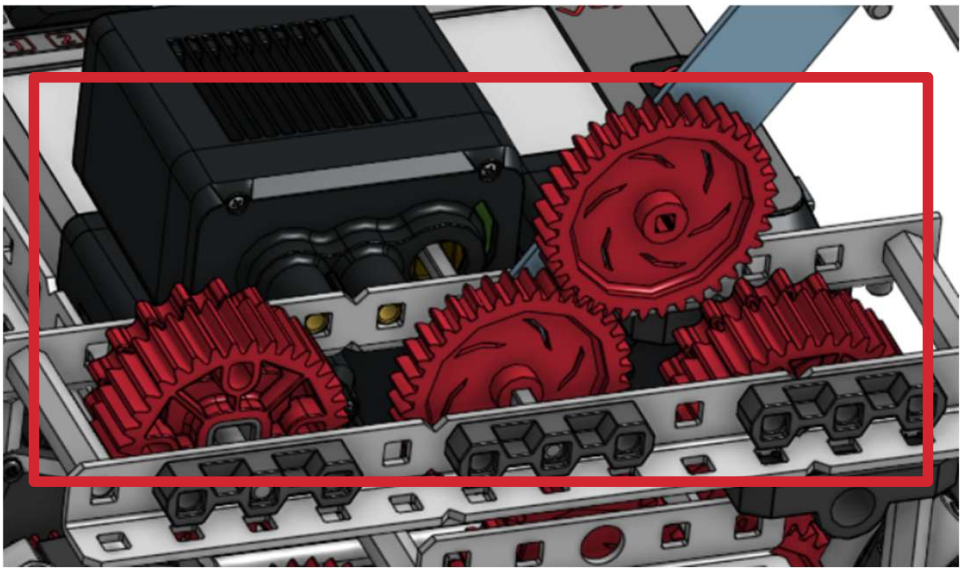
This made the small efficiency loss an acceptable trade-off for the simpler power-switching system. One of the larger limitations of this system is that each subsystem can only run in one direction through the ratchet. This is acceptable for the lift, which only needs to move upwards during operation, and the intake, which mainly runs forwards. When the intake needs to reverse, we can use one of our remaining motors as a dedicated reverse motor. This is possible because the ratchet allows us to use fewer motors overall, leaving extra motor capacity available.

Gravity Ratchet

Designing

We proceeded to conceptualise a novel mechanism called the “gravity ratchet” which provided several advantages. The resulting solution is the, a mechanism that automatically transfers power between the intake and lift systems based solely on the direction of motor rotation.

By eliminating the need for pneumatic actuators and sliding gear assemblies, this design reduces complexity, saves space, and improves overall efficiency while still allowing two separate functions to be powered by the same motors. We used the idea of inherent friction of gears, by placing the sun gear and a freely moving planet gear on a bent polycarbonate guide we can alter which system the motor powers based off its direction. The gear friction moves the planet gear around the sun, engaging one of two output gears on each side. Reversing the motor shifts the planet gear in the opposite direction, automatically engaging the other output.



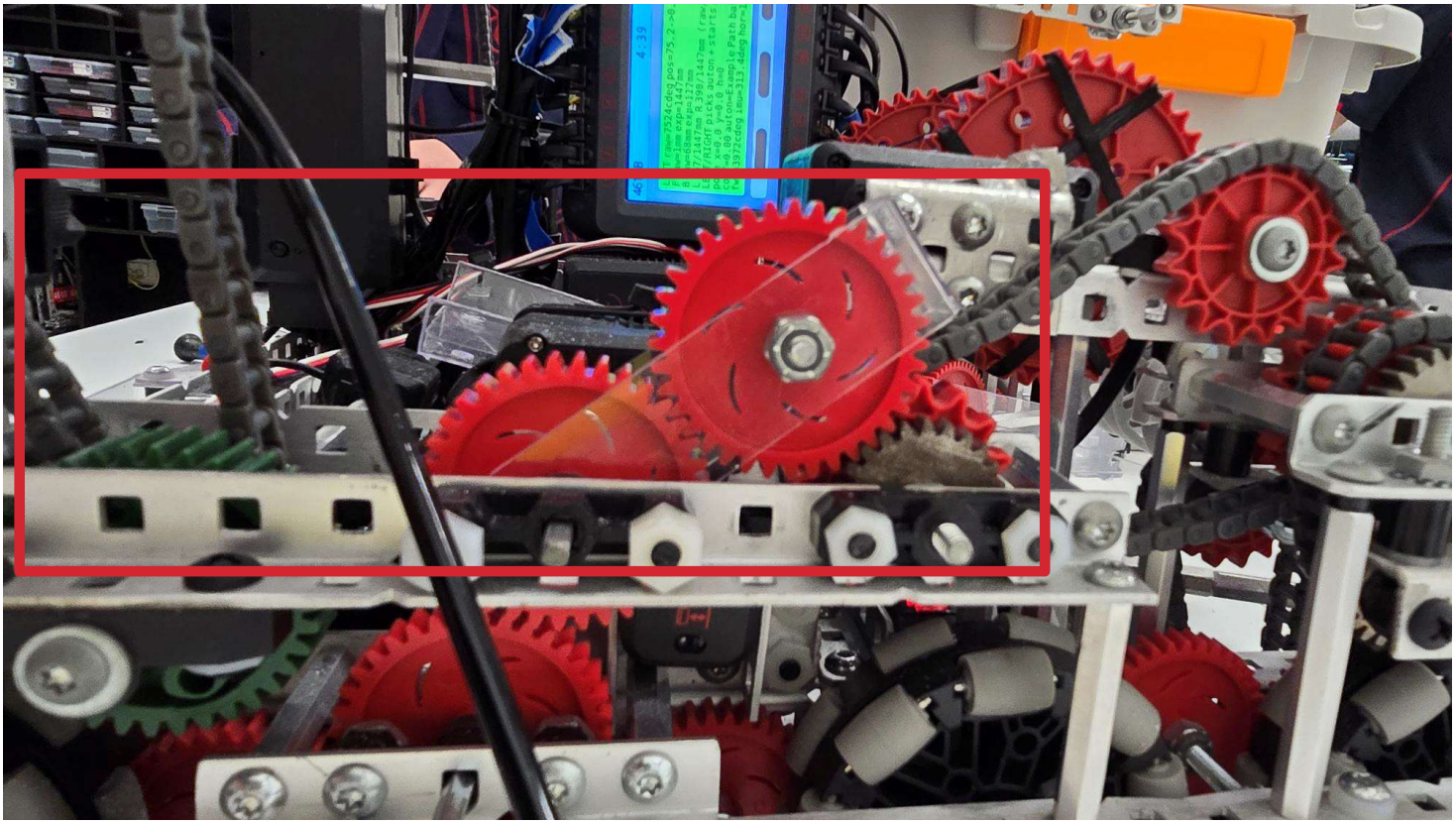
Motor ports are our most constrained resource. A conventional robot design would allocate **two motors to the intake and two to the lift** — four motors total for these two subsystems, leaving only four for the drivetrain. The gravity ratchet system allows us to only use 2 motors for both the intake and lift increasing our power for each subsystem overall providing a more effective robot.

Gravity Ratchet

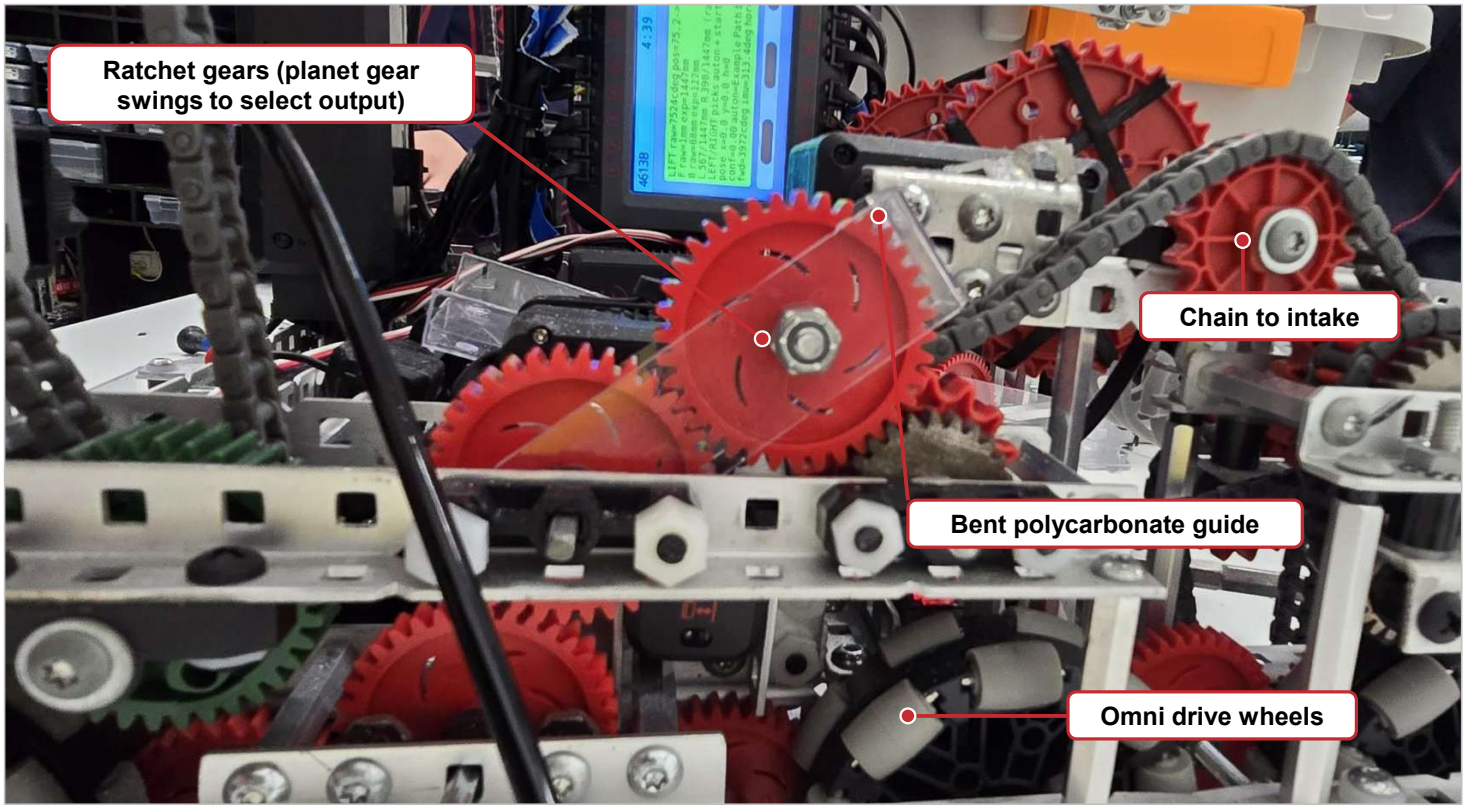
The implementation of our gravity ratchet was done relatively prematurely compared to our other mechanisms.

The implementation first starts by elevating L channels through standoffs above the side of the drive train to align them perfectly with the sprockets that connect to the 4 bar, and the set of sprockets and gear modules that power all attributes of the intake (which are the big elastic band roller, the kick bar and the front ground intake flex wheels).

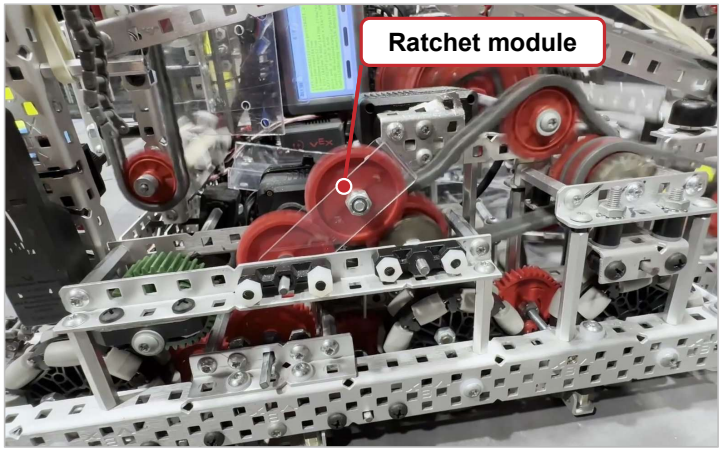
The implementation was designed in a way which allowed the entire gravity ratchet system to be hot swapped, by mounting it securely through just 8 screws on 8 mounting points, it allows for us to switch it out extremely quickly in case something catastrophic happens to our gravity ratchet system, that would disrupt our robot entirely.



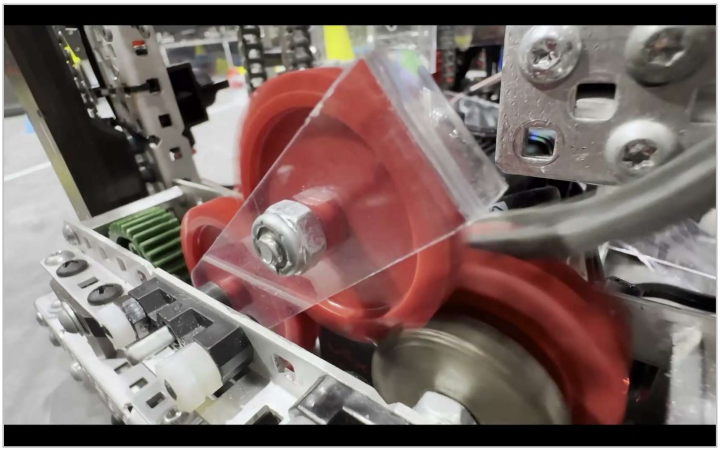
Gravity Ratchet Built



Close-up of the gravity ratchet on the finished robot.



Side view from our robot reveal video.



Reveal video: the guide sits over the gears.

What the photos show

- The whole ratchet module sits above the drive rail on L-channel and stand-offs, lined up with the lift and intake sprockets.
- It is held by 8 screws on 8 mounting points, so it can be swapped quickly if it is damaged.
- Only 2 motors drive both the lift and the intake – the direction of the motor picks which one.

End Effector

Regionals

End Effector Concepts

End Effector Concepts Prototyped

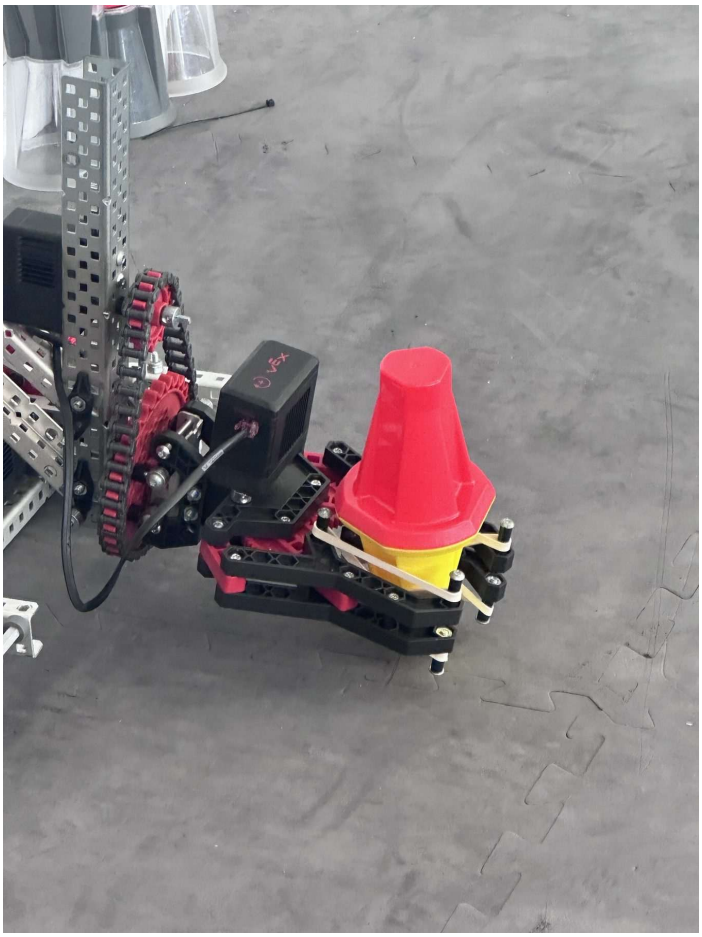
Several end effector concepts were prototyped before arriving at the final design.

- **Standard VEX claw**: Functional but could only reliably hold a single game object at a time, limiting scoring efficiency and multi-object manipulation.
- **Roller-based intake:** Improved acquisition but required a motor at the end effector, significantly increasing rotational inertia ($I = mr^2$) at maximum radius from the pivot.
- **Inertia impact:** Motor positioned at maximum r from the pivot makes its contribution to inertia particularly significant, directly reducing lift responsiveness.
- **Pneumatic Rotating Claw:** A claw capable of both clamping and rotating around It's axis, allowing the capability for flipping game objects after possessing them. Important for pins being the correct way for scoring

Brainstorming

Prototype 1 – Standard Vex Claw

Positive	Negative
Easy to Implement The design is very basic, it can be attached almost anywhere, its not very big as well	It doesn't fit the transfer we have leading into the DR4B lift design that we are going with. This makes it especially hard to put in for our specific robot archetype
	Because of its basic functionality, it has trouble retaining both pins and cups, and can suffer from its basic design.



Select Best Solution

Prototype 2:

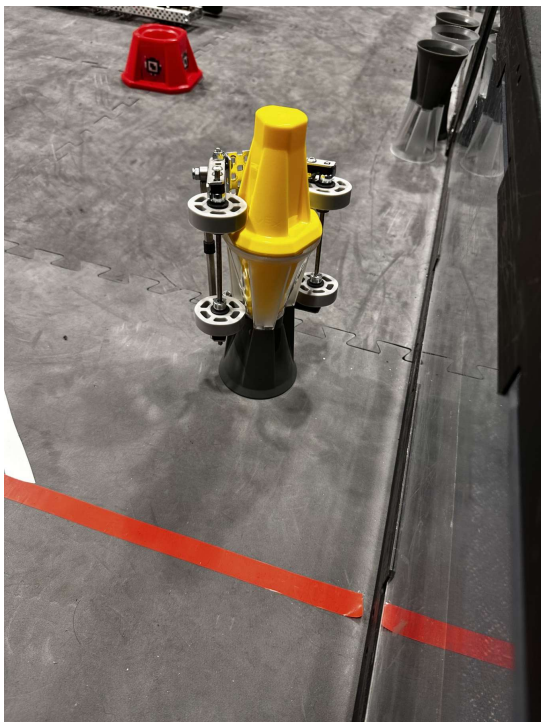
For the second design of an end effector, after the basic vex claw, we attempted a powered horizontal roller.

Design:
This end effector will use horizontal flex wheels to grip the game objects from the side. This design allows for both quick intaking directly from the match loader and intaking within our robot from the ground. This reduces cycle times and the powered wheels greatly improve reliability compared to a claw.

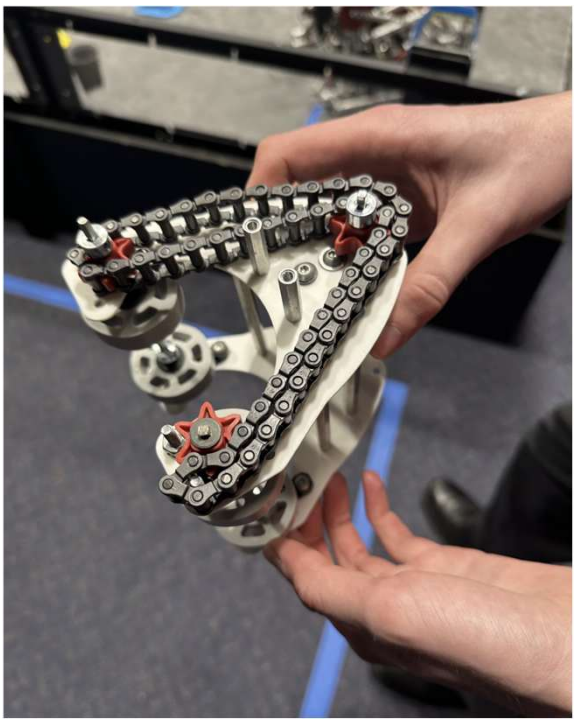


FRC *Team 2056*, 2025 - End effector

Trade-offs:
We chose to not replicate the flipping mechanism on this version of end effector, as we decided that it's durability and construction issues were not worth the minor inefficiency. Additionally, this design consumed a 5.5w motor to spin its wheels, which could have been used elsewhere.



Iteration 1 of **Prototype 2**

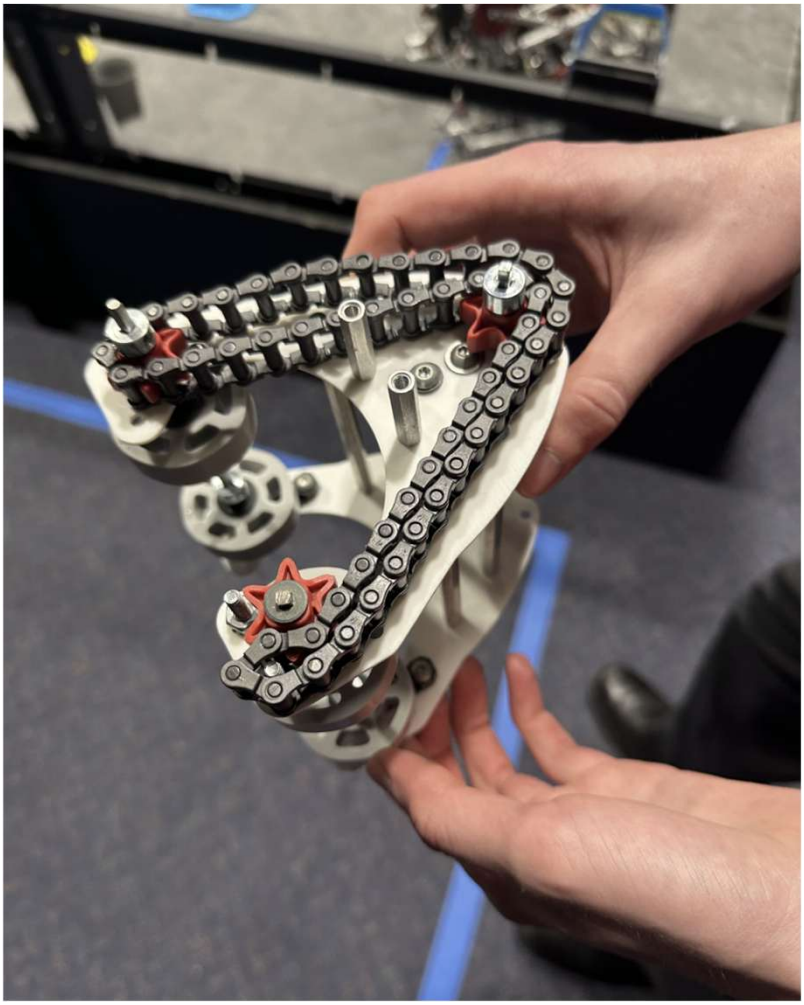


Iteration 2 of **Prototype 2**

Brainstorming

Idea 2 – Powered Horizontal Roller

Positive	Negative
Powering it is easy, it is extremely good at picking up both pin and cup, and the 2 game pieces together	Extremely inefficient space wise, it is very massive volume wise, and powering it with one singular circuit of chain, and the friction from the chain itself makes it very unreliable
Fits well with the archetype of our transfer and lift	Because of its bulky nature, attaching it to our DR4B means that every time we move the pin from the transfer into scoring mode, the 4 bar has to lift slightly which is not clean or efficient at all.



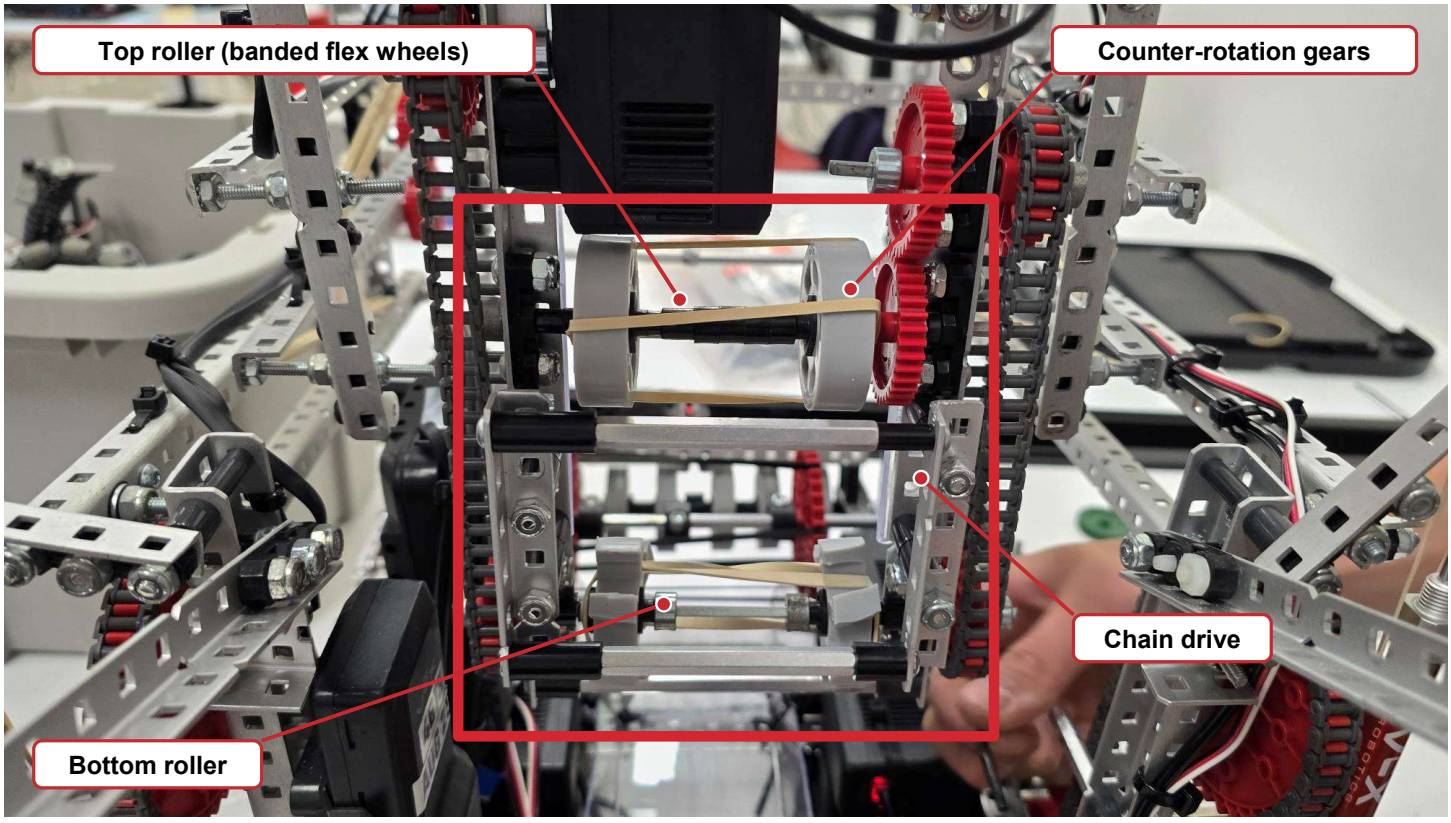
Select Best Solution

Prototype 3:

For the final and third design of our end effector, we decided to finish with using 2 parallel flex wheel rollers to continuously push our game pieces into a certain position, and then powering those flex wheels the opposite way can drop the pin and cup to score extremely effectively.

Design:

The design requires 2 parallel flex wheels, as well as L channels and cross bracing to retain a strong optimal shape, with the space between the flex wheels have tuned compression to optimise the control we have over the pin. As well as this, spacings and hard stops were also all tuned to a very find degree to ensure the pin and cup always end up in the same spot when scoring.



Power wise, it works through one vex half watt motor, with a series of sprockets and gears to ensure the 2 flex rollers spin in opposite directions at all times.

Select Best Solution

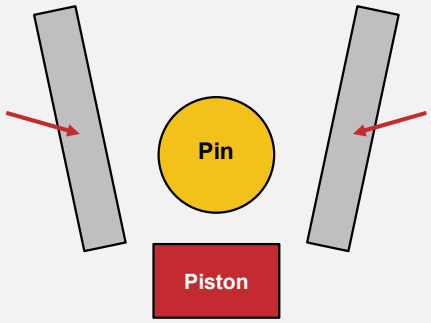
Idea 3 – Powered Horizontal Roller

Positive	Negative
Powering it easy, as just using a simple set of chain, sprockets and gears ensure the rollers spin opposite directions of each other	Needs its own 5.5 W motor at the top of the lift, adding some mass high on the robot
It is also incredibly space efficient, as it is basically nothing width wise, making the transfer directly into the end effector incredibly easy	Roller compression and hard-stops had to be tuned carefully so both a Pin and a Cup are gripped

Iteration Summary

Our end effector went through three iterations. Each change fixed a problem we found once the previous version was fitted inside the robot.

Piston claw – top view (diagram)

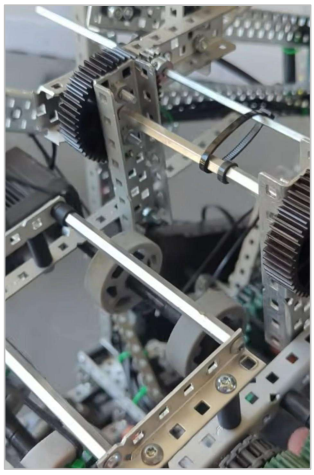


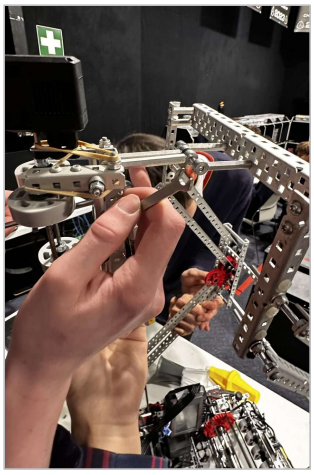
Jaws clamp in – piece must be lined up by hand

1 Piston claw

- A pneumatic piston clamped two jaws around the game piece.
- Too large for the limited space inside the robot.
- Inaccurate during match loading – the piece had to be lined up with the claw by hand, with little guidance.

Outcome: rejected





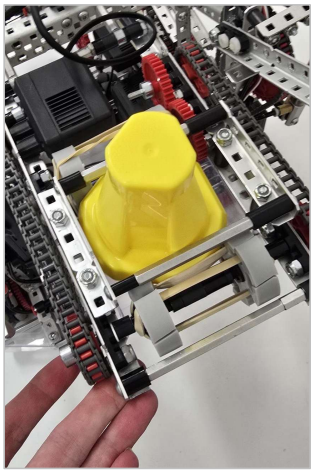
Powered claw prototype and fitting it to the lift

2 Powered claw

- A motor-driven claw that enclosed the game piece.
- It fitted the robot and gave better control than the piston claw.
- Still large and bulky, so intaking pieces in the confined space inside the robot was difficult.

Outcome: rejected





Rollers holding a Pin; final version in the robot

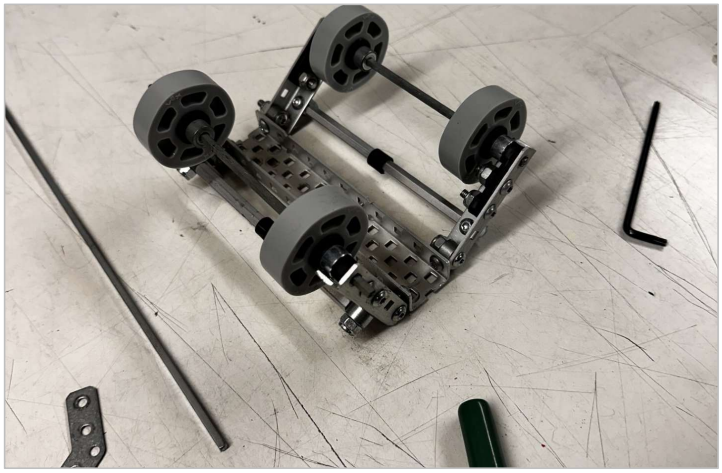
3 Vertical rollers – final

- Two vertical rollers enclose the game piece and pull it in from both sides.
- Much more compact than either claw.
- Far more reliable intake – no precise manual alignment needed.

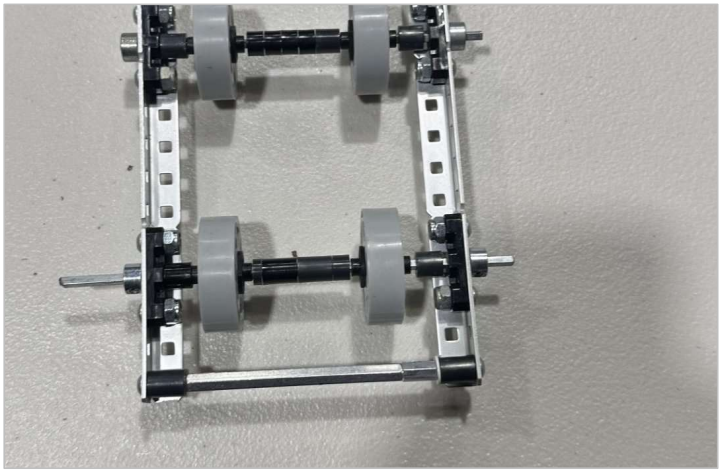
Outcome: selected

Final End Effector Build

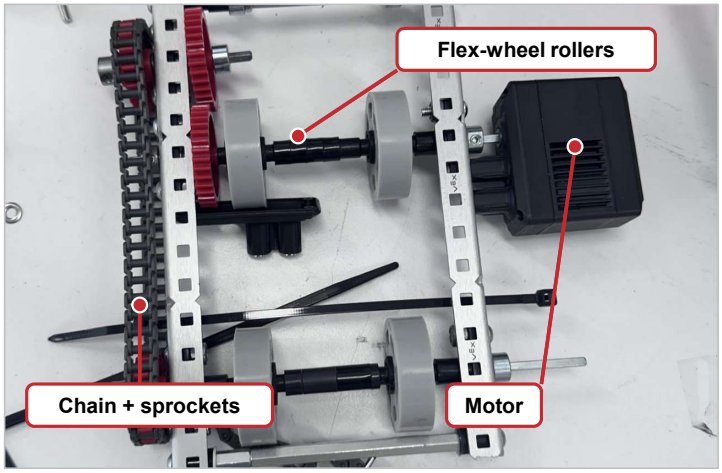
How the vertical-roller end effector was built, from first mock-up to grip test.



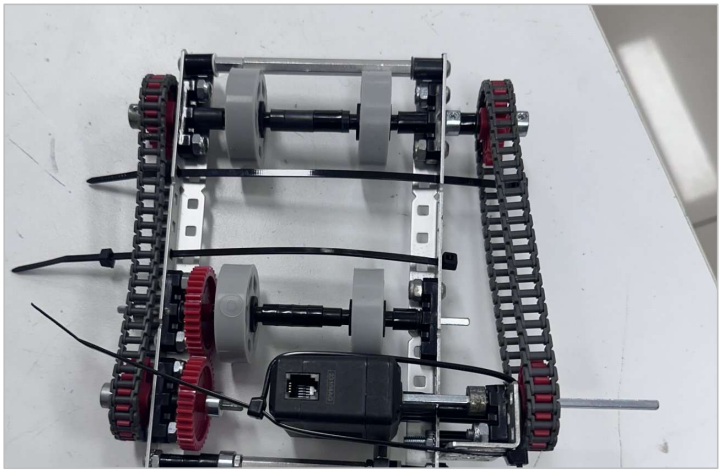
1 Early compact roller-pair mock-up



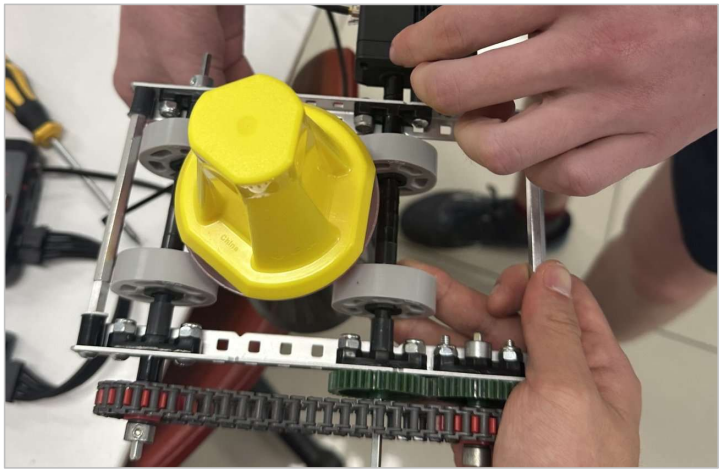
2 Frame with two flex-wheel roller shafts



3 Fitting the chain and gears



4 Module assembled with motor

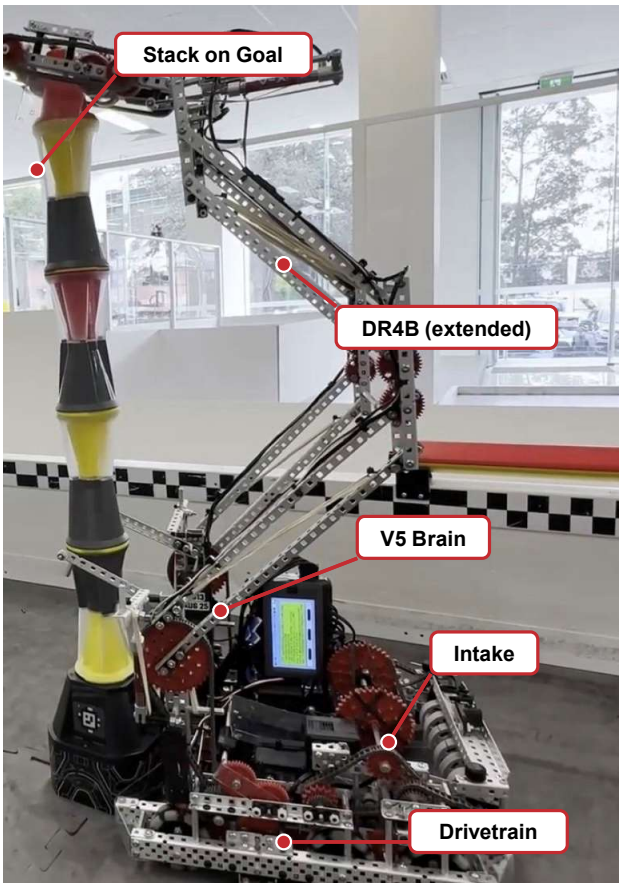


5 Grip test – rollers holding a Pin

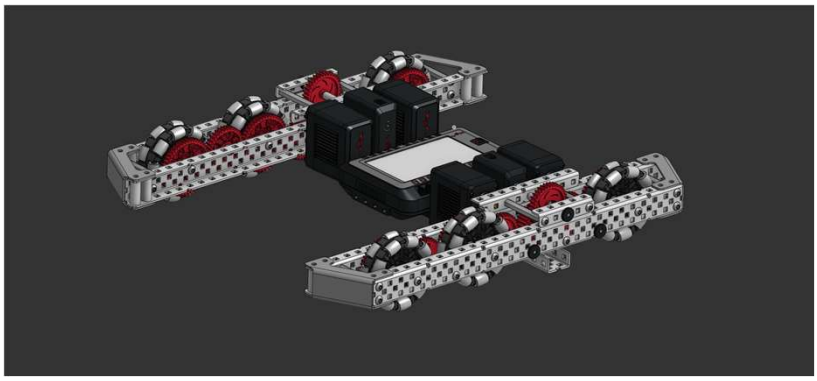
Why it works

- Gears and chain make the two rollers spin in opposite directions, pulling the piece in from both sides.
- Running the rollers backwards releases the piece onto the Goal.
- Roller spacing and hard-stops were tuned so pieces end up in the same place every time.

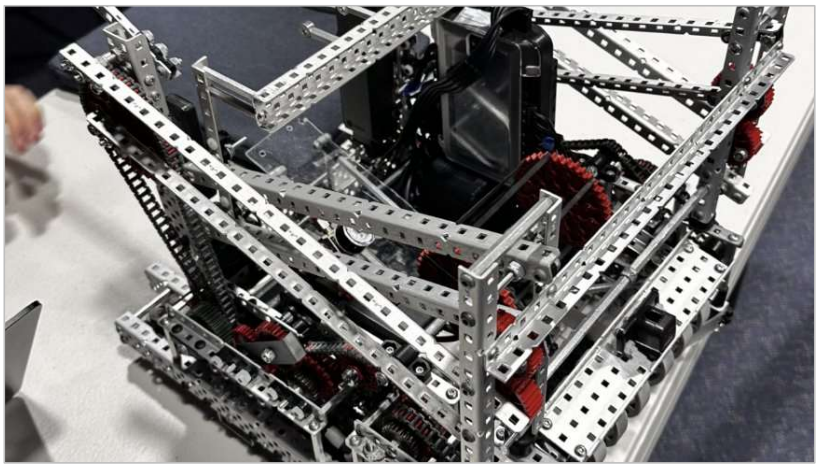
Final Robot Views



Built robot – side view, DR4B extended



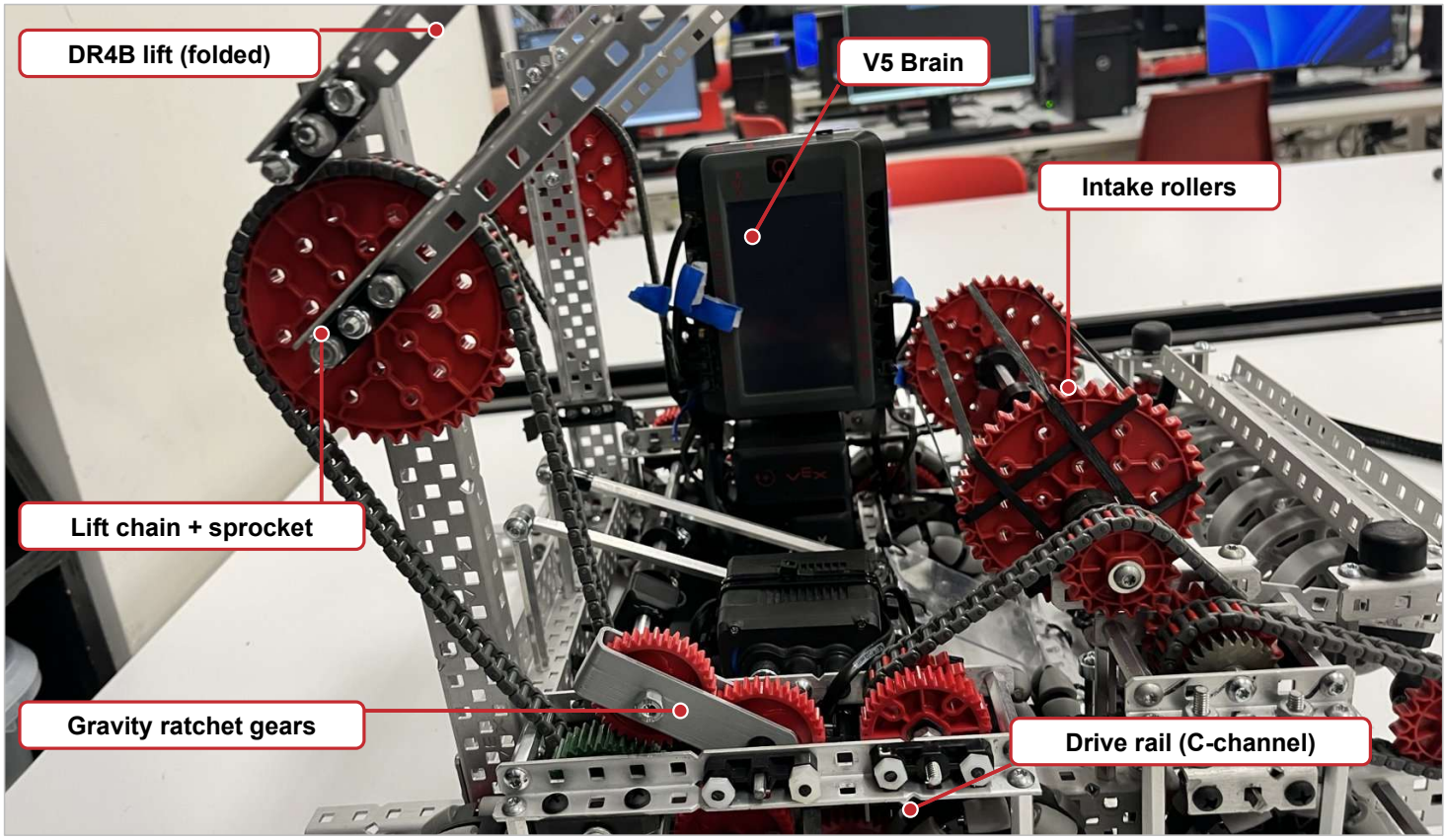
CAD – drivetrain (isometric)



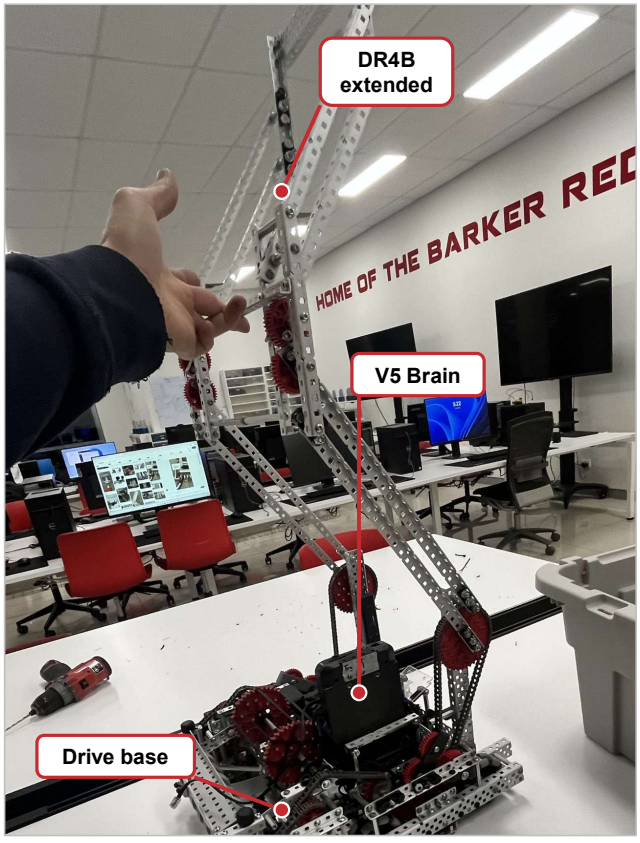
Built robot – top view, lift folded

#	Component	Annotation
1	Tank drive, 2.75" omni	6 motors (2×11 W + 1×5.5 W per side) geared to 450 rpm → 64.8 in/s
2	Pivot intake + kicker	Flex wheels on a pivot + ground roller 1 cm below ramp; chain-driven
3	Transfer roller	Elevated, decoupled from pivot; aligns Pin/Cup for the end effector
4	Gravity ratchet	2 × 11 W motors; motor direction selects intake (+) or lift (−)
5	DR4B lift	L-channel, rubber-band assist; ≤ 47" reach; keeps effector level
6	End effector	Two parallel flex rollers, 1 × 5.5 W; hard-stops for repeatable placing
7	Sensors	IMU (centre), rotation sensor + tracking wheel, 4 distance sensors F/B/L/R
8	Electronics	Brain + battery low and central; one air tank on intake for down-force

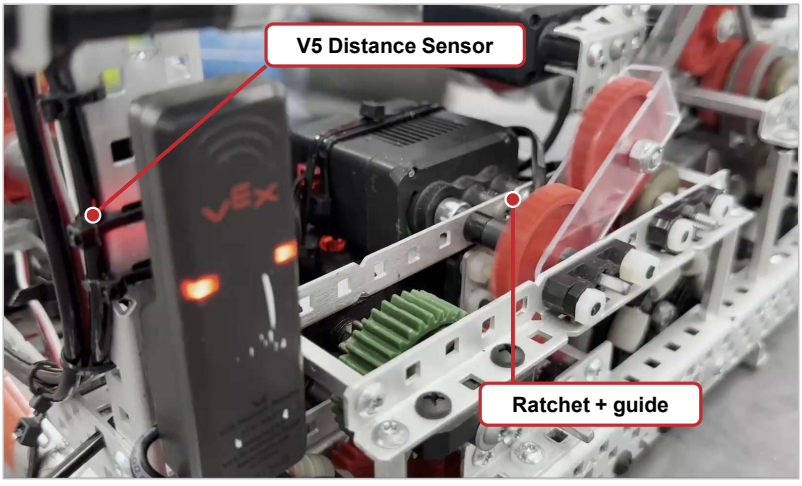
Annotated Robot Views



Side view – lift folded inside the 18" starting size.



Lift fully extended.



Close-up from our reveal video.

Three views of the build

- Side, extended and close-up views show every subsystem.
- Full component list and CAD views: previous page.

Chassis & Lift Matrix

Scores 1 (poor) – 5 (excellent) from our brainstorming pages and tests. Weights reflect Robot Skills: speed, accuracy and space matter most because there is no defence in Skills.

Chassis options

Criterion (weight)	H-drive	X-drive	Tank (chosen)	Mecanum
Speed / acceleration in 55 W (×3)	4 → 12	4 → 12	5 → 15	2 → 6
Autonomous accuracy (odometry fit) (×3)	3 → 9	3 → 9	4 → 12	3 → 9
Space for intake, ratchet, electronics (×3)	2 → 6	3 → 9	5 → 15	2 → 6
Build simplicity + reliability (×2)	4 → 8	2 → 4	5 → 10	3 → 6
Motor efficiency (×2)	3 → 6	3 → 6	5 → 10	2 → 4
TOTAL (max 65)	41	40	62	31

Justification: tank drive scores 62/65. All wheels push forward so no power is lost to 45° rollers (X/mecanum); no centre wheel stealing space (H-drive); and because it cannot strafe, sideways motion is always zero, so one forward tracking wheel + IMU gives accurate odometry.

Lift options

Criterion (weight)	Single 4-bar	DR4B (chosen)	Cascade	Arm + wrist
Vertical reach to 47" (×3)	2 → 6	5 → 15	5 → 15	3 → 9
Compact folded height (×2)	3 → 6	4 → 8	3 → 6	3 → 6
Keeps effector level (×2)	4 → 8	5 → 10	5 → 10	2 → 4
Mass / motor load (×2)	4 → 8	4 → 8	2 → 4	2 → 4
Build simplicity (×1)	5 → 5	3 → 3	1 → 1	3 → 3
TOTAL (max 50)	33	44	36	26

Justification: DR4B 44/50 – straight-up motion keeps load over the middle wheels and falls back under gravity, which suits the gravity ratchet.

Intake & Outtake Matrix

Intake options (sketches on pages 46–55)

Criterion (weight)	Vertical rollers	Solid wheels	Band roller	Hybrid (chosen)
Pickup reliability (Test T4) (×3)	2 → 6	3 → 9	3 → 9	5 → 15
Size / space (×2)	1 → 2	3 → 6	5 → 10	4 → 8
Weight (×2)	2 → 4	3 → 6	5 → 10	4 → 8
Complexity (×1)	2 → 2	2 → 2	5 → 5	3 → 3
Possession-rule control (×2)	3 → 6	3 → 6	3 → 6	5 → 10
TOTAL (max 50)	20	29	40	44

Justification: the hybrid (pivoting flex-wheel first stage + ground kicker + rubber-band transfer) scores 44/50. The band roller was lighter, but tests showed only the pivot + kicker reached 19/20 pickups and controlled the 1-Pin limit.

Outtake / end effector options (pages 77–84)

Criterion (weight)	VEX claw	Horizontal roller	Parallel flex (chosen)	Pneumatic rotating claw
Grips both Pin and Cup (×3)	2 → 6	5 → 15	4 → 12	3 → 9
Cycle speed + fit to transfer (×3)	1 → 3	3 → 9	5 → 15	2 → 6
Mass at arm tip (inertia $I = mr^2$) (×2)	4 → 8	2 → 4	4 → 8	3 → 6
Width / space (×2)	4 → 8	1 → 2	5 → 10	3 → 6
Reliability (×2)	4 → 8	2 → 4	4 → 8	2 → 4
TOTAL (max 60)	33	34	53	31

Justification: parallel flex rollers score 53/60 – almost no width, so the transfer feeds straight in; one light 5.5 W motor keeps inertia at the top of the DR4B low; reversing the rollers places the Pin and Cup in the same spot every time.

Build Plan

Week	Build steps	Lead (role)	Done when…
T3W2	DR4B C-channel prototype; static intake test; port map	Ryan (lift), Josh (intake), Brandon (config)	DR4B moves full stroke ≤ 50"
T3W3	Pivot flex-wheel intake; kicker roller; L-channel DR4B	Josh, Ryan	Pickup ≥ 18/20
T3W4	Mount DR4B on chassis; transfer roller; chain drive	Ryan, Josh	Object reaches effector
T3W5	Gravity ratchet PTO; end effector prototypes 1–3	Ryan, Josh (ratchet), Rio (effector)	One motor pair runs lift + intake
T3W6	All 10 motors, sensors, pneumatics; wiring	Ryan, Brandon	Robot passes 88 W / 55 W check
T3W7	Reinforcement (collars, dual-side drive); subsystem tests	Josh, Ryan, Rio	All T1–T10 tests run
T3W8	Localisation on robot; auton steps 1–5	Brandon, Rio	AMCL error ≤ 2"
T3W9	Auton steps 6–9; tuning; driver practice	Brandon, Rio	First full 60 s run
T3W10	Refinement from test results; presentation plan	All	≥ 83 pts × 3 runs
T4W1	Final checks, inspection, submission	All	Submit Fri 16 Oct 3:30 pm

Build order logic

1. Chassis first (already built in July) because every subsystem mounts to it.
2. Lift and intake next – they set the object path and the space left for electronics.
3. Gravity ratchet only after lift and intake gear positions were fixed.
4. Electronics and sensors last, then programming and testing on the finished robot.

Components – Motors

Component	Qty / port	Specification	Function / operation	Justification
V5 Smart Motor 11 W – drive	4 / 1,3,4,6	Blue cartridge 600 rpm; 36:48 → 450 rpm	Front + back drive wheels each side	Most power per port; 11 W × 4 + 5.5 W × 2 = 55 W, exactly the R11 cap
V5 Smart Motor 5.5 W – drive	2 / 2,5	Geared to the same 450 rpm	Middle wheel each side	Uses the last 11 W of the 55 W drive cap; vertical layout keeps CG low
V5 Smart Motor 11 W – gravity ratchet	2 / 7,8	Green 200 rpm (torque)	+ direction drives intake, − direction drives DR4B up	One motor pair powers two subsystems → frees motors (PTO without air)
V5 Smart Motor 5.5 W – end effector	1 / 9	Chain/sprockets so rollers counter-rotate	Grips Pin + Cup; reverses to place	Light motor at the top of the lift keeps inertia ($I = mr^2$) low
V5 Smart Motor 5.5 W – intake reverse/kicker	1 / 10	Drives kicker + allows reverse	Ejects extra objects; spins kicker roller	Ratchet can only drive intake forwards; needed for SG6 compliance
TOTAL	10	4×11 + 2×5.5 + 2×11 + 2×5.5		= 88 W (55 W drive + 33 W mechanisms)

Calculations

- Drive speed: $v = \pi d \times rpm \div 60 = \pi \times 2.75 \times 450 \div 60 = 64.8$ in/s.
- Lift torque needed: effector + Pin + Cup + upper links ≈ 1.2 kg acting at ≈ 0.30 m → $\tau = m\,g\,r = 1.2 \times 9.81 \times 0.30 \approx 3.5$ N·m.
- Lift torque available: 2 motors × ≈1.05 N·m stall (200 rpm) × 5:1 (12T:60T) ≈ 10.5 N·m stall. Running at ≈ ⅓ of stall (3.5 N·m) keeps motors cool (T7: 48 °C). Rubber bands reduce the load further.
- Air budget (from Gravity Ratchet page): ≈ 160 theoretical pushes, −25 % for leaks ≈ 120 usable; measured 128 (Test T12).

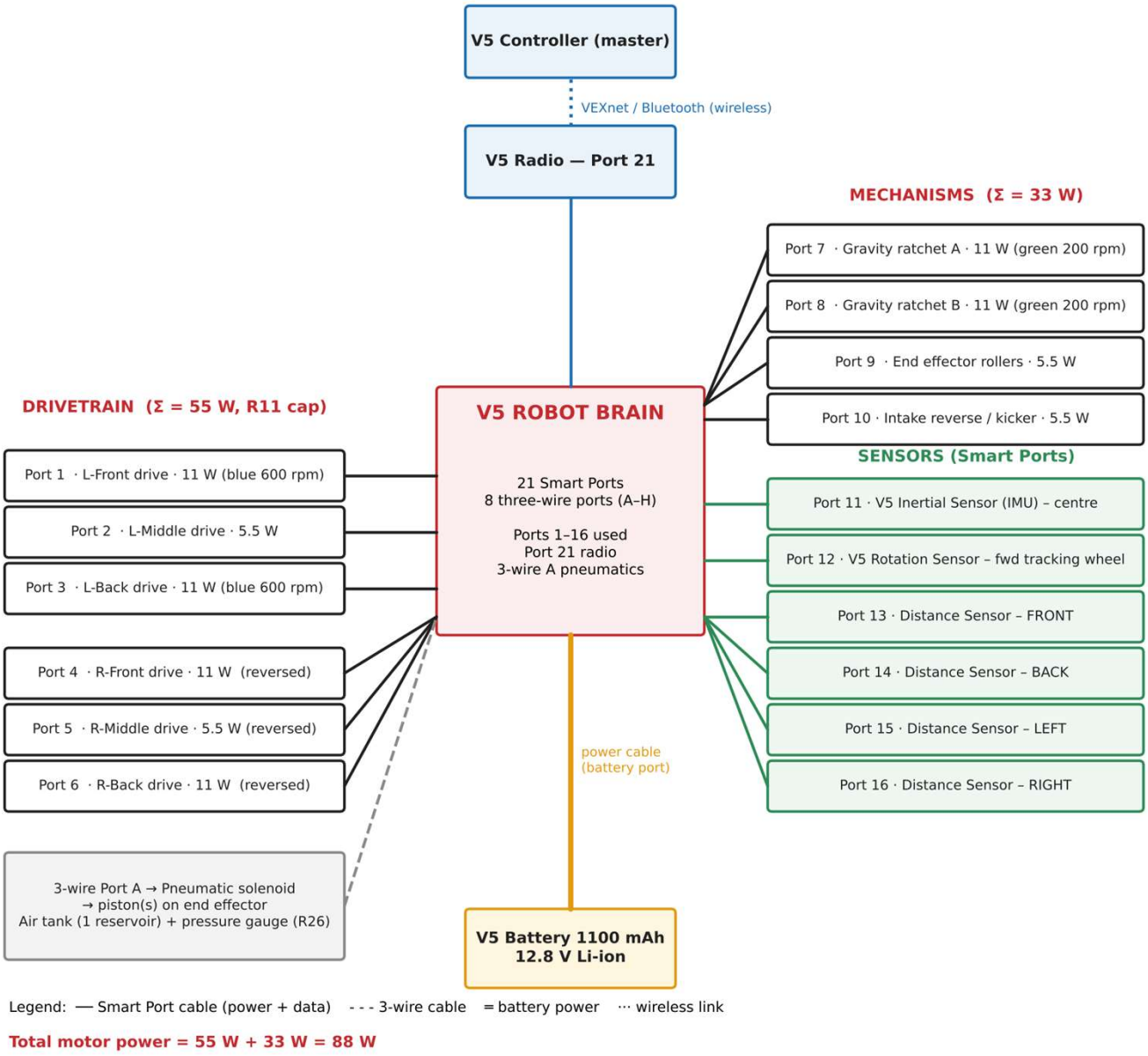
Components – Sensors

Component	Qty / port	Specification	Function / operation	Justification
V5 Inertial Sensor (IMU)	1 / 11	3-axis gyro + accelerometer; heading 0–360°	Gives absolute heading every 10 ms	Heading from wheel differences drifts badly (tested); IMU does not slip. Mounted at centre to reduce vibration
V5 Rotation Sensor + tracking wheel	1 / 12	Centidegree resolution, unpowered wheel	Measures forward distance travelled	Not affected by drive-wheel slip; one wheel is enough because tank drive cannot strafe
V5 Distance Sensor	4 / 13–16	Laser time-of-flight, ≈ 20–2000 mm	Measure distance to walls/Goals F, B, L, R	Gives AMCL absolute position fixes that remove odometry drift (T11: 3.1" → 0.9")
V5 Robot Brain	1	21 Smart Ports, 8 three-wire ports, touchscreen	Runs PROS + lillet library; auton selector	Required controller; 16 ports used leaves spares
V5 Battery	1	1100 mAh Li-ion, 12.8 V	Powers brain + all motors	Mounted low for CG; strap added (T3W10)
V5 Radio + Controller	1 + 1 / 21	VEXnet link	Driver control + match control	Required for Driving Skills
Pneumatics	1 tank, 1 solenoid / A	Pressure gauge (R26)	Actuates end-effector piston	No air used for the PTO, so one tank is enough (T12)

Why distance sensors instead of GPS or AI vision?

GPS strips can be blocked by Goals and objects and stop updating; AI vision/AprilTag ideas from early brainstorming needed tags we could not rely on in Skills. Four distance sensors always see at least two walls, so the particle filter can correct position even when one beam is blocked (occlusion gate, Programming Journal T3W7).

Wiring Diagram



Electrical requirements considered

- **Power budget:** 88 W of motors on one 1100 mAh battery – drive motors share load so none run near stall; motor temperature checked in T7.
- **Port plan:** drive 1–6 (right side reversed in code), mechanisms 7–10, sensors 11–16, radio 21 – matches robot_config.hpp so wiring and code never disagree.
- **Cable routing:** Smart cables run along the C-channel and are zip-tied away from DR4B pivots (a wire was pinched in T3W8 – rerouted).
- **Brownout protection:** battery and brain cables strain-relieved; spare ports 17–20 available if a port fails.

Code Development

Regionals

Programming Overview

Objectives

For VRC Override, we decided early that standard odometry wouldn't give us the position accuracy we needed for consistent autonomous routines. Our solution was creating a **lillet** a custom PROS library we designed, built, and tuned ourselves from scratch.

At its core, lillet does two things: it knows where the robot is, and it moves there precisely.

For localisation, we implemented **AMCL (Adaptive Monte Carlo Localisation)** — the same algorithm used in full-scale autonomous vehicles. It runs a 200-particle filter that continuously fuses our tracking wheel, IMU heading, and four distance sensors against a ray-cast model of the field to produce a fused pose estimate far more robust than odometry alone. If the robot gets bumped or shoved, the filter recovers. If a sensor is blocked by another robot, the filter ignores it.

For motion, we built a **boomerang-carrot controller** with a LemLib-compatible API, so autonomous paths designed in our browser simulator compile and run unchanged on the robot. All tuning — from sensor noise to motion gains — lives in a single file (robot_config.hpp) and was validated in the simulator before ever touching the robot.

The entire library was developed test-first: a JavaScript simulator and an automated test suite (sim_tests.mjs) served as our behavioural oracle, meaning we could tune and validate the algorithms on a laptop before a single wheel turned.

These are our crafted websites that showcase our work:

Custom Montecarlo LIbrary
https://beel487.github.io/lillet/docs.html

Custom Robot Simulator and path planner
https://beel487.github.io/lillet/Sim.html

Localisation Strategy

Brainstorming

Localisation is the process of determining a robot's position and orientation on the field. Accurate localisation is essential for autonomous navigation, allowing the robot to understand where it is relative to its environment and follow planned paths reliably. Because localisation errors can accumulate over time and reduce autonomous performance, several methods were investigated and evaluated to identify the most accurate and competition-compliant solution for our robot.

Method	
Dead Reckoning	Rejected — unbounded drift
2-Wheel Odometry	Rejected — no absolute heading
3-Wheel Odometry	Strong candidate — heading still drifts
IMU + Odometry	**Baseline used (odometry layer)**
Vision / QR Codes	Rejected — markers not permitted
GPS + Monte Carlo + IMU + Odom	GPS rejected as sole method; MCL + IMU + Odom SELECTED ✓

Testing

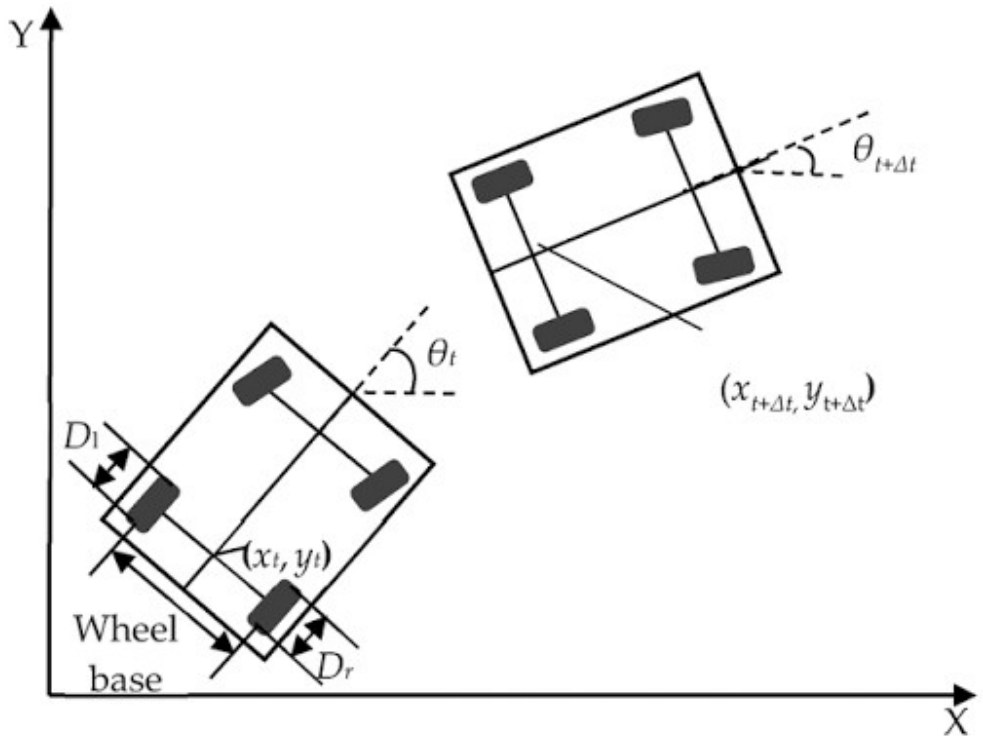
To ensure accurate localisation we evaluated seven methods. Each was assessed on position accuracy, heading drift, hardware requirements and VEX V5 compatibility. The following slides present each approach with pros and cons, leading to our final selection: AMCL particle filter fused with IMU + tracking-wheel odometry.

Localisation Strategy

Dead Reckoning

Positive	Negative
Zero hardware cost Uses only built-in VEX V5 motor encoders — no extra sensors needed.	**Unbounded error accumulation** Wheel slip and IME drift compound with every movement — position error grows continuously and cannot self-correct.
Trivial to implement Simple integration loop; fully compatible with VEX V5 and PROS/VEXcode.	**No heading reference** Cannot distinguish rotation from translation — any heading error directly corrupts X/Y position estimates.

Verdict
Rejected. Position error grows too rapidly to trust path planner outputs beyond the first movement. Sim-to-real path deviation makes this unsuitable for accurate VRC autonomous routines.

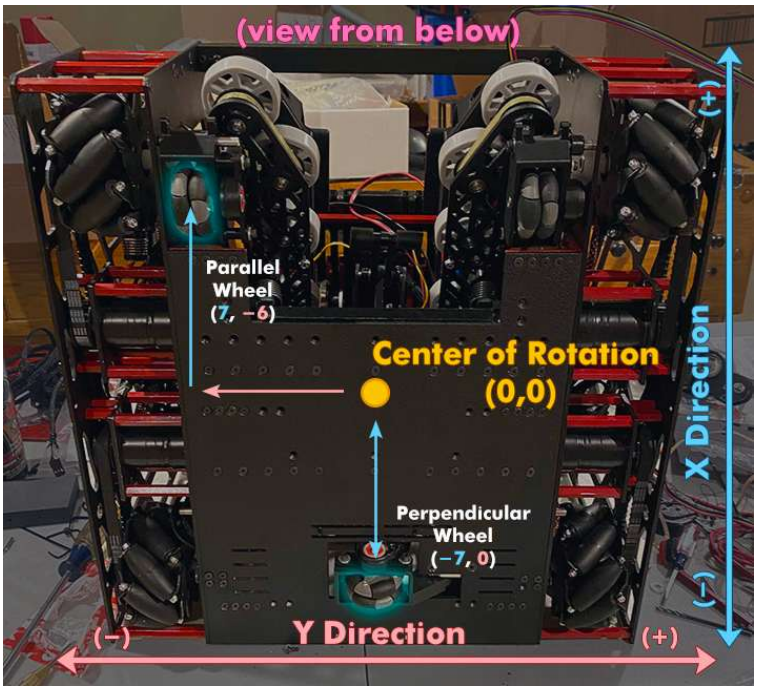


Localisation Strategy

2-wheel Odom

Positive	Negative
Eliminates drive-wheel slip errors Passive tracking wheels are decoupled from the drivetrain — slip during aggressive acceleration no longer corrupts position.	**Heading computed from encoder difference** Heading relies on the differential of two wheels — sensitive to tyre diameter mismatch and encoder noise; no absolute heading reference.
Significant accuracy improvement Much more accurate than dead reckoning on the smooth VRC tile floor over short routines.	**Cannot resolve lateral drift** A perpendicular pair of wheels cannot detect side-slip; heading error still accumulates over a full 15-second routine.

Verdict
Rejected. Without an absolute heading reference, heading drift still accumulates over a full autonomous routine, leading to meaningful sim-to-real path deviation at the endpoint.

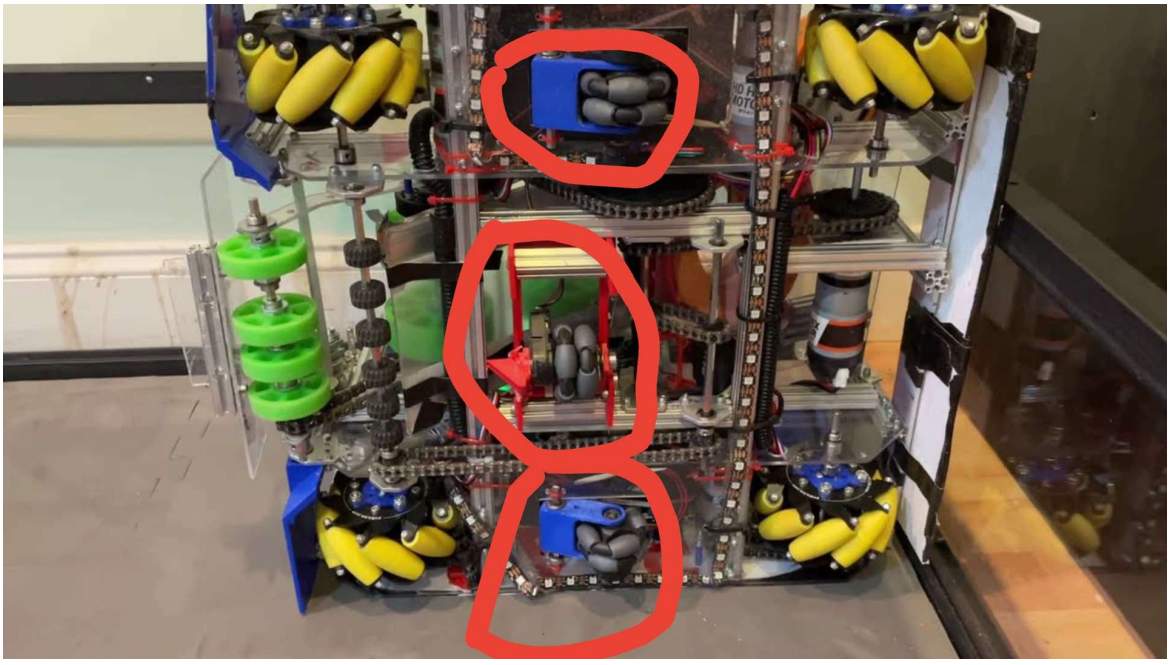


Localisation Strategy

3-wheel Odom

Positive	Negative
Full 2D localisation The perpendicular (strafing) wheel independently measures lateral displacement — X, Y, and heading are all tracked.	**Heading still drifts over time** Heading is computed from the differential of two parallel encoders — small calibration errors accumulate across long routines without an absolute reference.
Industry standard for VRC Widely proven at high levels of competition; well-documented with ARMS, LemLib, and similar frameworks.	**Mechanical complexity** Requires a precise tracking-wheel pod with spring tensioning; tight tolerances are critical — misalignment skews all position data.

Verdict
Strong candidate. Provides full X/Y/heading tracking with proven accuracy in VRC. However, heading drift over a full routine without an absolute sensor remains a limitation — addressed in the next method.



Localisation Strategy

Odom + IMU tracking

Positive	Negative
Absolute heading referenceThe V5 Inertial Sensor (IMU) provides a reliable heading that prevents drift accumulation from encoder noise — the single biggest improvement over 3-wheel odom alone.	**IMU gyro drift under vibration** The V5 inertial sensor can drift under sustained motor vibration; requires periodic recalibration or fusion weighting to manage.
Consistent sim-to-real fidelity Heading accuracy is largely preserved between simulation and physical robot — path planner outputs closely match real-world trajectories.	**Additional calibration overhead** IMU must be calibrated at startup; robot must remain stationary during calibration — adds pre-match setup time.

Verdict

Selected as our odometry baseline (later fused with MCL – see Monte Carlo pages). Fusing the V5 inertial sensor with tracking-wheel odometry delivers precise, drift-corrected X/Y/heading estimation — maximising path planner accuracy and minimising sim-to-real effects across a full autonomous routine.



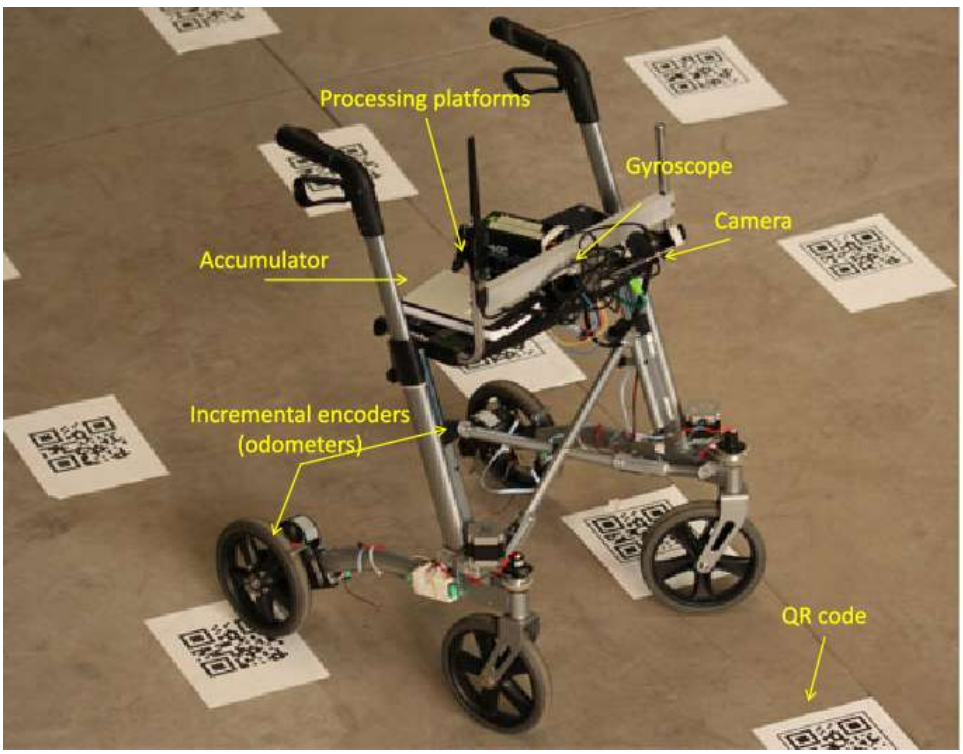
Localisation Strategy

Vision based system using QR codes

Positive	Negative
Absolute position updates Each QR detection provides an absolute X/Y fix — position error is bounded and does not accumulate between detections.	**Intermittent and unreliable in VRC** VEX Vision Sensor is not designed for QR decoding; field lighting, marker occlusion by game elements, and sensor processing delays make detection unreliable mid-match.
No drift between detections When a marker is visible, position resets to ground truth — ideal in principle for correcting encoder drift.	**Not permitted / not practical** Adding custom QR markers to the competition field is not allowed under RECF rules; the VEX Vision Sensor cannot natively decode QR codes in real-time.

Verdict

Rejected. Custom field markers are not permitted under RECF competition rules, and the VEX Vision Sensor lacks native QR decoding capability — making this approach infeasible in a real match environment.

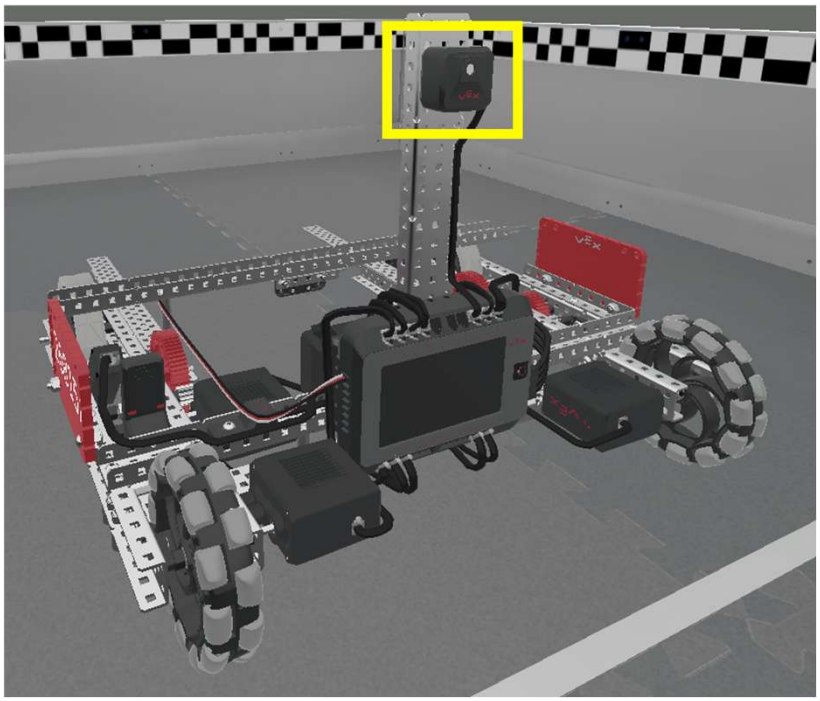


Localisation Strategy

GPS sensor

Positive	Negative
Continuous absolute position Provides real-time X, Y, and heading from field-perimeter strips — no drift accumulation and no reliance on encoder integration.	**Low update rate and occlusion** The GPS sensor updates at approximately 50 Hz and loses line-of-sight to the strip when game elements or field walls block the view — position freezes during occlusion.
No encoder fusion needed Eliminates the need for tracking wheel pods — reduces mechanical build complexity and potential failure points.	**Field-dependent and not always available** Requires official VEX GPS strips mounted on the field perimeter — not available at all competition venues; cannot be relied upon as a sole localisation source.

Verdict
Rejected as sole method. Occlusion failures and field availability make it unreliable as a standalone solution. Could supplement IMU + Odom when the strip is visible, but cannot replace it.



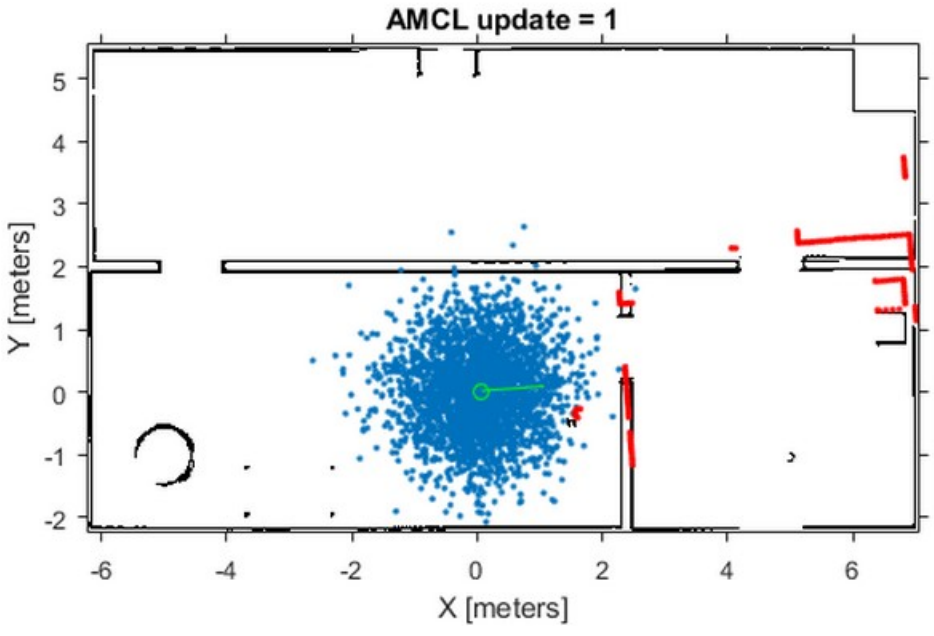
Localisation Strategy

Monte-Carlo

Positive	Negative
High Accuracy Uses many position hypotheses (particles) to estimate the robot's location, reducing the effect of sensor noise and odometry errors.	**Requires external references** To achieve high accuracy, Monte Carlo localisation typically relies on vision sensors, field landmarks, or other reference measurements that may not always be available.
Can recover from errors If the robot's estimated position becomes incorrect, the algorithm can use new sensor data to re-localise and find its correct position again.	**Computationally intensive** Continuously updating and resampling hundreds of particles requires significant processing power, making it challenging to run efficiently on the VEX V5 Brain.

Conclusion

Monte Carlo Localisation fused with 2-wheel odometry and the V5 IMU was selected as our localisation system. This combination provides the best balance of accuracy, drift correction, and real-time performance available on the VEX V5 platform — and minimises the sim-to-real gap to mechanical tolerances alone.



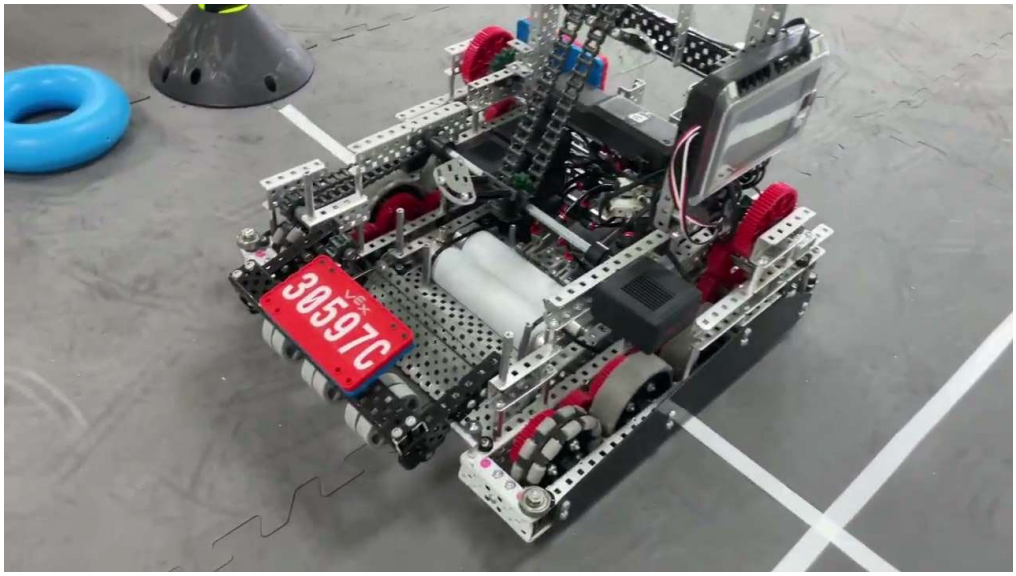
Monte Carlo (MCL)

Monte-Carlo localisation using distance sensors

Positive	Negative
Absolute position correction4 V5 Distance Sensors (laser time-of-flight) (front, back, left, right) measure wall distances. The particle filter continuously matches these to a known field map, correcting encoder drift in real time.	**Computationally expensive** Maintaining hundreds of weighted particles requires significant CPU cycles on the V5 brain — particle count must be tuned carefully to balance accuracy against loop latency.
Bounds position error Unlike pure odometry, MCL error does not grow unboundedly — sensor updates pull the estimate back to ground truth whenever a reliable wall reading is available.	**Symmetry ambiguity** The VRC field is largely symmetric — identical sensor readings in two positions can cause the particle filter to maintain two competing hypotheses. IMU heading and 2-wheel odom resolve this ambiguity.

Verdict: Our Final Choice

Selected. Monte Carlo Localisation fused with IMU heading and 2-wheel odometry delivers the most precise and consistent localisation of all evaluated methods. Odometry provides smooth motion prediction between sensor updates; the IMU eliminates heading ambiguity; and the particle filter corrects accumulated drift using 4 distance sensors against the field map — minimising the sim-to-real gap across the full autonomous routine.



Software Architecture

Design Goal

Our biggest weakness in previous seasons was autonomous consistency. Timed and dead-reckoned routines were highly sensitive to wheel slip, battery voltage changes, and field conditions, causing paths to drift over time. Our goal was to develop a localisation and motion control system capable of continuously estimating the robot's position and accurately driving to any coordinate and heading with repeatable performance.

Why We Wrote Our Own Library

Three software approaches were considered. VEXcode offered simplicity but limited access to advanced localisation and multitasking capabilities. Community libraries such as LemLib provided a proven framework, but would not allow us to fully develop and understand the underlying control systems. We therefore chose to build our own C++17 library using PROS, enabling complete control over localisation, motion control, and system architecture while providing valuable experience with robotics software engineering.

System Architecture

The software is divided into three primary components. A localisation task continuously fuses sensor data to estimate the robot's pose. A chassis controller uses this pose estimate to execute autonomous path-following and navigation routines. Finally, the driver control layer processes joystick inputs and translates them into motor commands. Centralising hardware configuration and tuning parameters ensures the system remains maintainable and adaptable throughout the season.

Control Algorithms

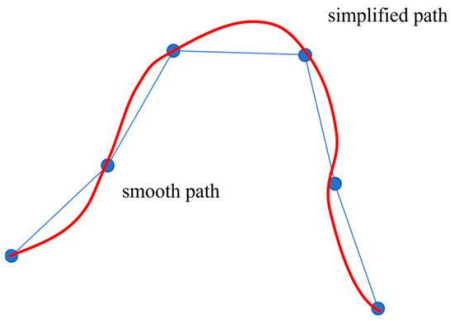
Control Design

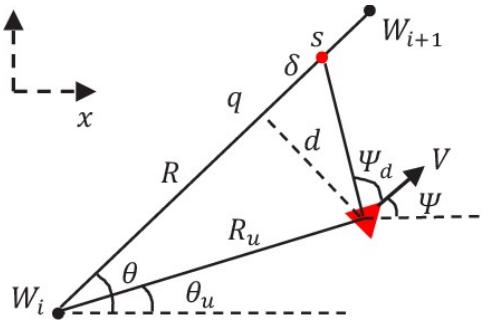
The robot's software architecture was developed around a series of control and localisation algorithms designed to improve autonomous accuracy and consistency.

PID controllers were used to minimise error between the robot's current and target states, allowing precise control of both linear movement and turning. Extensive PID tuning was conducted to optimise responsiveness, reduce overshoot, and ensure reliable performance across different battery voltages and field conditions.

$$Output = (K_p \cdot Error) + \left(K_i \cdot \int Error\, dt\right) + \left(K_d \cdot \frac{d(Error)}{dt}\right)$$

To generate smooth and efficient paths, a Boomerang (Carrot Chasing) algorithm was implemented. Rather than driving directly to a target point, the robot continuously follows a moving look-ahead point along the desired path, resulting in smoother trajectories and more stable path following.





Accurate localisation was achieved using an odometry tracking system that continuously calculates the robot's X-coordinate, Y-coordinate, and heading in real time. This provides a complete estimate of the robot's pose on the field and forms the foundation for autonomous navigation.

(Coordinate system diagram: see Coordinate System page.)

To further improve localisation accuracy, a Monte Carlo Localisation (Particle Filter) algorithm was explored. This approach uses multiple position hypotheses, known as particles, and updates them using measurements from four distance sensors to reduce accumulated odometry drift and improve confidence in the robot's estimated position.

(Particle distribution: see Inside the Particle Filter page.)

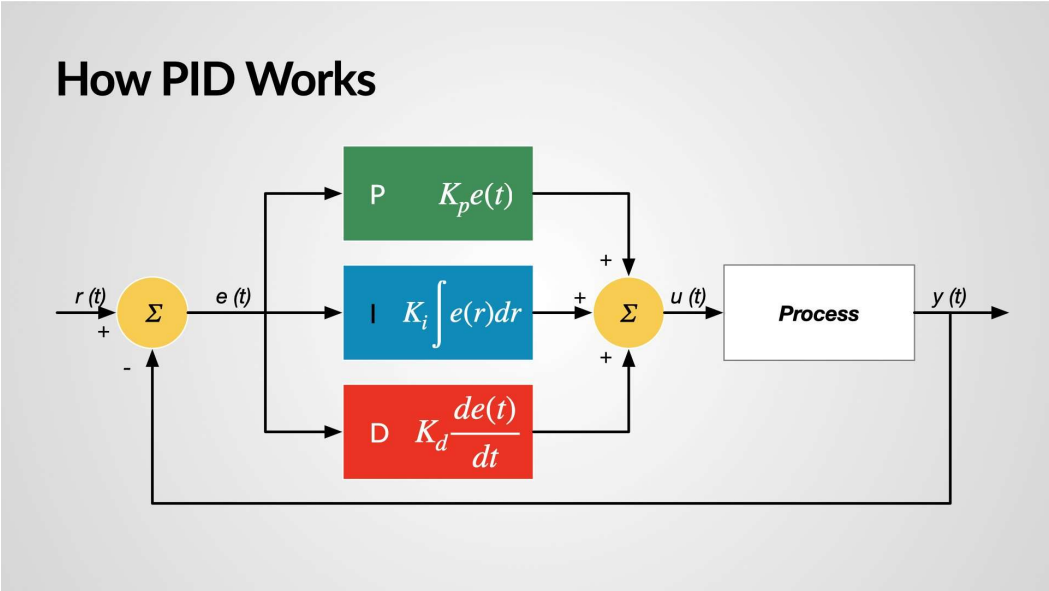
PID Controller

PID

Component	Plain English Explanation
P — Proportional **I — Integral** **D — Derivative**	How far off are we right now? Go harder if further away Have we been off for a long time? Add a correction for that Are we approaching too fast? Slow down to avoid overshooting
The equation: **u(t) = Kp·e(t) + Ki·∫e dt + Kd·(de/dt)** e(t) = error = target − current position	**In code:** output = Kp * error + Ki * integral + Kd * derivative ***Code: Programming Journal T3W5.***

How to think about PID

Imagine trying to park a car perfectly on a line. If you are far away you press hard (P). If you have been off for a while you add a little more pressure (I). As you get close you ease off before you overshoot (D). PID does exactly this for every motor on our robot — every 10 ms. We use it to control straight-line distance, turning angle, and lateral alignment independently.



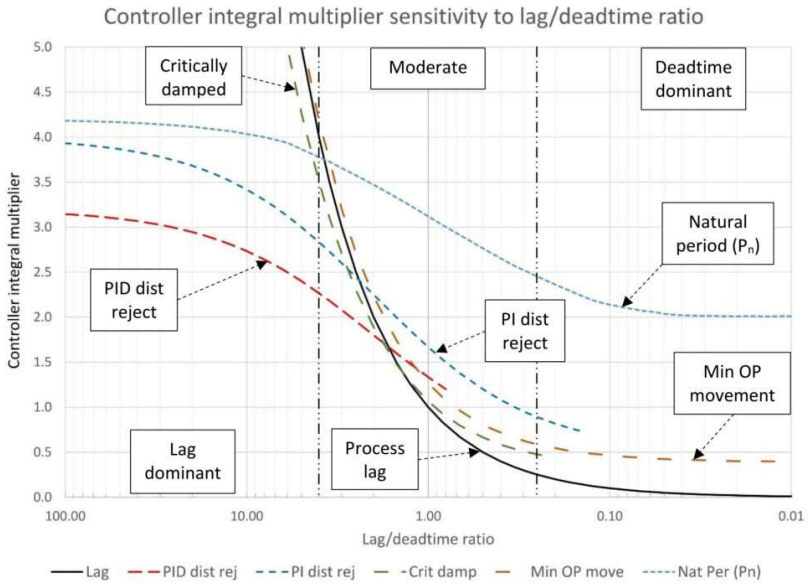
PID Tuning

PID

Gain	Tuning Steps
Step 1: Tune Kp **Step 2: Add Kd** **Step 3: Add Ki**	Set Ki=Kd=0. Raise Kp until robot reaches target but oscillates. Then halve it. Raise Kd until oscillations damp out quickly. If Kd is too high, it becomes jerky. Add small Ki only if robot stops short of target. Too much causes windup.
Integral windup fix: Clamp integral: if |integral| > maxIntegral then integral = maxIntegral ***Results: Test T3 + T7.***	**Symptoms to watch for:** Oscillates = Kp too high or Kd too low Stops short = Ki too low or Kp too low Overshoots badly = Kd too low

Why tuning matters

A poorly tuned PID makes the robot behave completely differently to the simulation — it might overshoot a waypoint, stop short, or oscillate before settling. Good tuning is what makes our simulated paths actually match what the robot does on the field. We log error vs. time graphs to verify that tuning matches the simulated response.

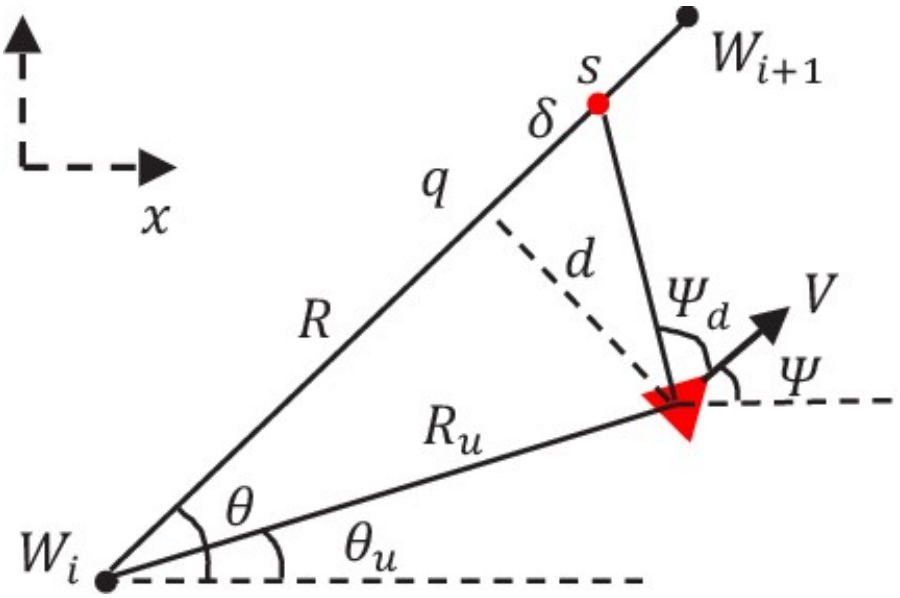


Boomerang Chasing

Carrot pathing

Algorithm	Key Equations
Carrot Point A moving target point placed a lookahead distance L ahead on the path. The robot always steers toward this "carrot" — never toward the final goal directly.	**$d = \sqrt{((x_{n+1}-x)^2 + (y_{n+1}-y)^2)}$** Find the point on the path exactly L metres ahead of the robot's current position
Boomerang Controller A carrot-chasing variant where the final pose (x, y, θ) is approached smoothly using a curved arc. The lead point is placed behind the target so the robot curves into the correct exit heading.	**$\alpha = \text{atan2}(y^c - y, x^c - x) - \theta\alpha$ = heading error to carrot. PID drives $\alpha \rightarrow 0$ while throttle drives distance $\rightarrow$ 0Code: Programming Journal T3W9.**

Why not just drive straight to each waypoint?
If the robot turns and drives in straight segments it wastes time and creates jerky movements. Boomerang chasing lets the robot arc smoothly between waypoints at full speed, arriving at the exact correct exit angle for the next movement. This is how we achieve fast, smooth autonomous routines that match the simulator output.



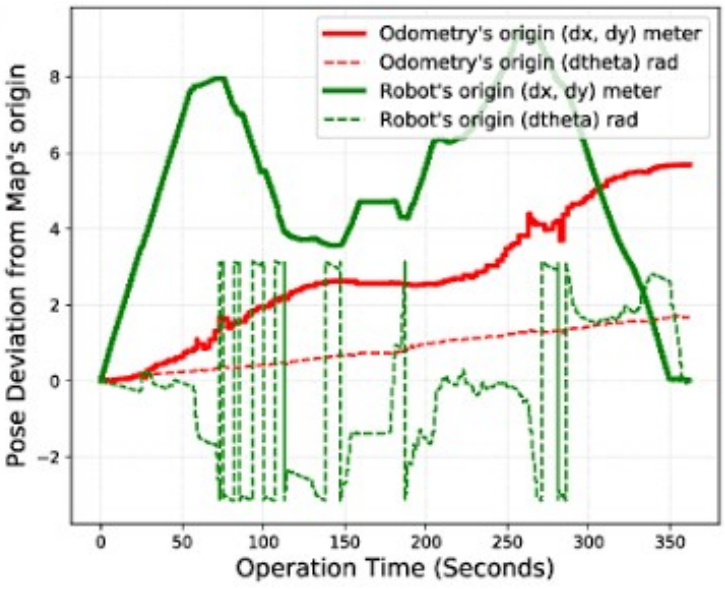
Odometry Tracking

Odom tracking

What we track	Equations (run every 10 ms)
Heading θ (from IMU) **Δs_l and Δs_r (encoder ticks → metres)**	**θ = IMU.heading()Δs = (Δs_l + Δs_r) / 2Δx = Δs · sin(θ)Δy = Δs · cos(θ) (θ clockwise from north)**
State update (each tick): **x += Δx** **y += Δy** ***Code: Programming Journal T3W4.***	**Encoder ticks to metres:** **dist = ticks / 360 × π × wheel_diameter** Runs on a dedicated RTOS task at 10 ms intervals, feeding both the MCL filter and the path controller simultaneously.

Plain English: how does the robot know where it is?
Think of counting steps while walking blindfolded. Each tick of our tracking wheels is one tiny step. We use the IMU to know which direction those steps went, then add them to our last known position. Done 100 times per second, this gives us a smooth, accurate estimate of X and Y on the field — which the MCL filter then refines using distance sensor readings.



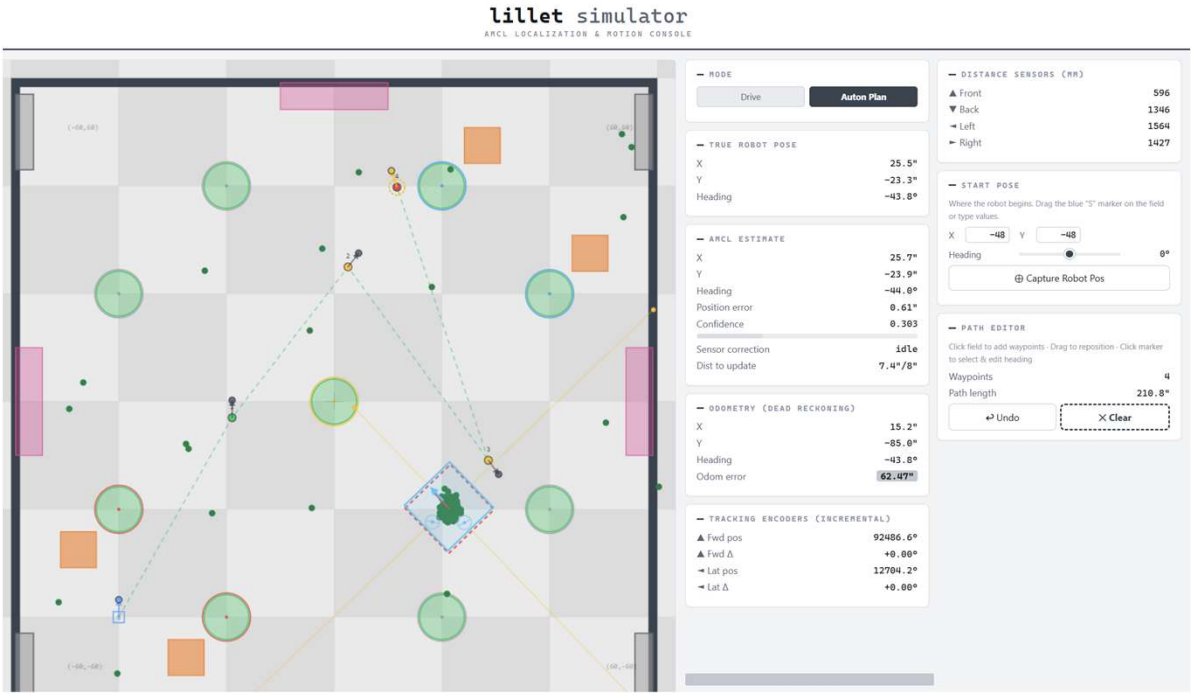


Monte Carlo Filter

Robot Sim

Path Planning and Simulator
One of our primary software goals this season was to eliminate the traditional "sim-to-real gap" that exists between autonomous routines developed in simulation and their performance on the physical robot. In previous seasons, autonomous routines often relied on dead reckoning and timed movements, causing errors to accumulate due to wheel slip, battery voltage variation, drivetrain friction, and mechanical tolerances. Even small inaccuracies could result in missed scoring opportunities during competition.

To address this, we developed a browser-based path planning and simulation environment that allowed autonomous routines to be designed, tested, and validated before being deployed to the robot. The simulator was designed to replicate the behaviour of the real robot as closely as possible, with the ultimate goal of reducing end-of-path position error to less than 2 cm — comparable to normal mechanical build tolerances.



The Simulator

Features

Feature	Purpose
Browser-based path planner Live localisation error monitor Non-coder autonomous builder	Draw and edit autonomous paths visually on the field Watch X/Y error grow in real time as the auton runs Anyone on the team can create and export autons without coding
Worst-case deviation prediction Friction + drift simulation Auton verification	Simulate worst-case path and widen tolerances accordingly Model real-world drivetrain effects to close sim-to-real gap Confirm paths score before taking them to competition

Why we built it

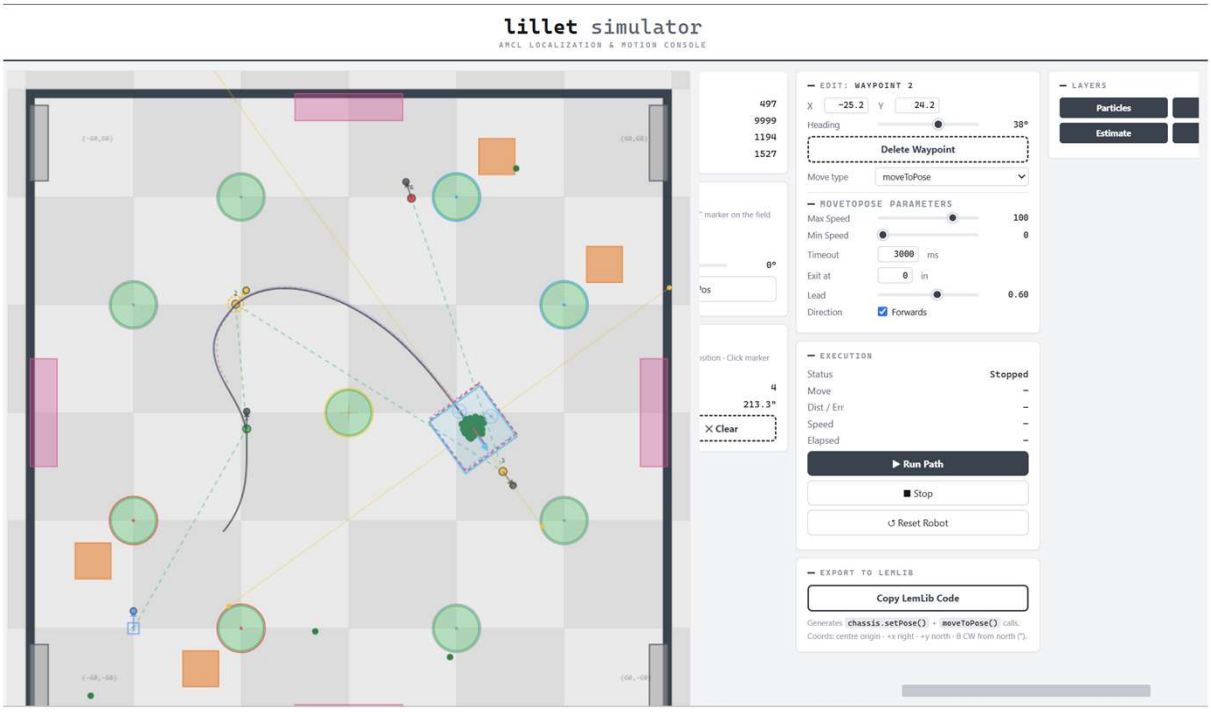
We built a path planning simulator directly into a website so the whole team could work on autonomous routines — not just programmers. Our goal was to reduce the sim-to-real gap so that a path drawn in the browser runs identically on the robot. The following slides explain what we built, how we verified it, and what the friction and drift experiments showed us.

Path Planner Website

Custom Website

Positive	Negative
Visual autonomous builder Draw waypoints by clicking on a scaled field image. The site generates the boomerang path, exports coordinates as C++ constants, and lets non-coders contribute autons without touching the codebase.	**Simulator is idealised** The base simulation assumes perfect traction and zero sensor noise. Without calibration against the real robot, paths may look correct on screen but drift on field — addressed by our friction and drift experiments.
Live localisation error monitor As the simulated auton runs, the site plots X error, Y error, and heading error over time — we can see exactly where a path diverges and by how much.	**Requires real-robot data** Accurate friction and inertia constants must be measured from the physical robot and entered into the simulator — the models are only as good as the data fed into them.

Key goal: predict worst-case path deviation before competition
By running the same autonomous 100 times in simulation with randomised noise, we can measure the maximum possible deviation at each scoring location. We then design path waypoints to score even under worst-case conditions — meaning a path that scores in sim will always score on the real field.



Sim-to-Real Gap

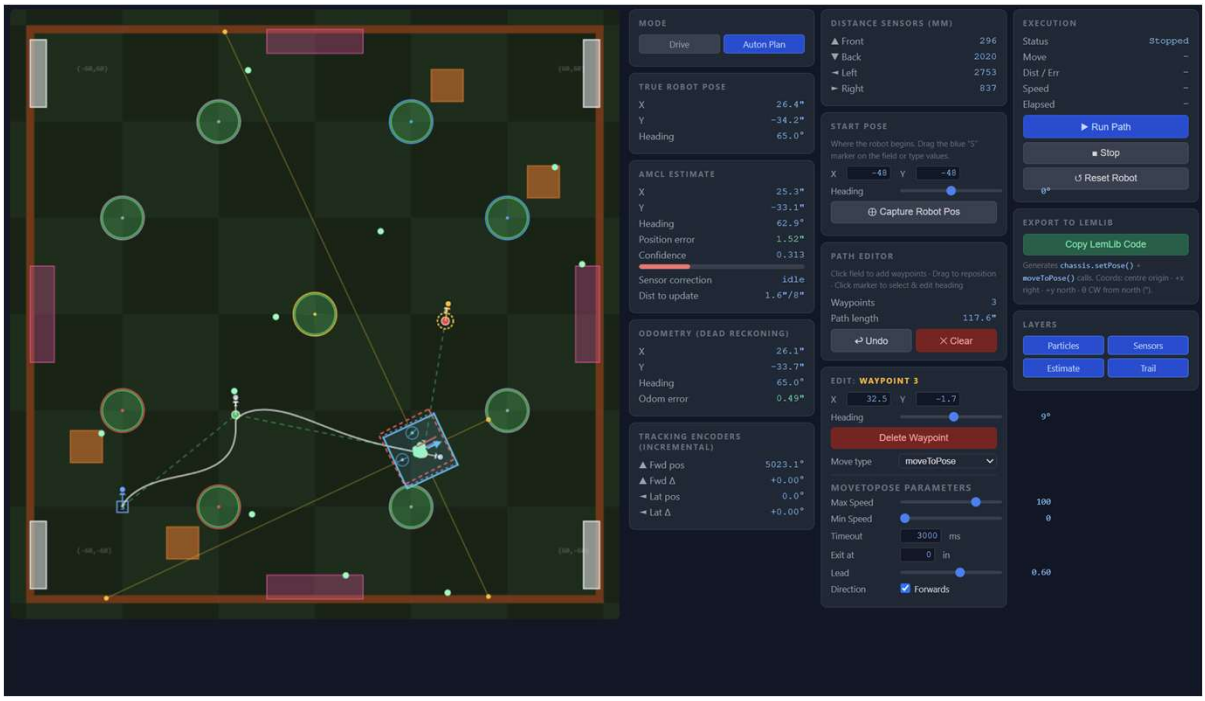
Localisation error

Source of Gap	How we address it
Localisation drift **PID mismatch** **Mechanical tolerances**	MCL particle filter corrects accumulated odom error using 4 distance sensors Tune PID on real robot; log step response and match to simulator model Measure wheel diameter and track width precisely; update sim constants
Drivetrain friction **Horizontal drift (drift drive)**	Measured deceleration profile; modelled as viscous friction in simulator Quantified lateral displacement per metre; added as a systematic offset in sim (experiment results on next slides)

Our target: sim and real are the same

Every source of divergence between simulation and reality has a physical cause. By measuring each one and encoding it in the simulator, we make the sim progressively more accurate. Our goal was to reduce end-of-path position error to under 2 cm — comparable to mechanical build tolerances — so that the sim-to-real gap is essentially zero for practical purposes.

Simulation Conclusions



Friction Model

To better evaluate our autonomous paths in simulation, we introduced a friction model to more closely replicate the robot's real-world deceleration behaviour.

$$v_{t+1} = \begin{cases} \max(0, v_t - d) & v_t > 0 \\ \min(0, v_t + d) & v_t < 0 \end{cases}$$

However, testing showed that the effect on overall autonomous performance was minimal, while the additional parameters introduced unnecessary complexity and tuning requirements. As a result, the friction model was removed from the final implementation to keep the simulator simpler and more maintainable.

Coordinate System

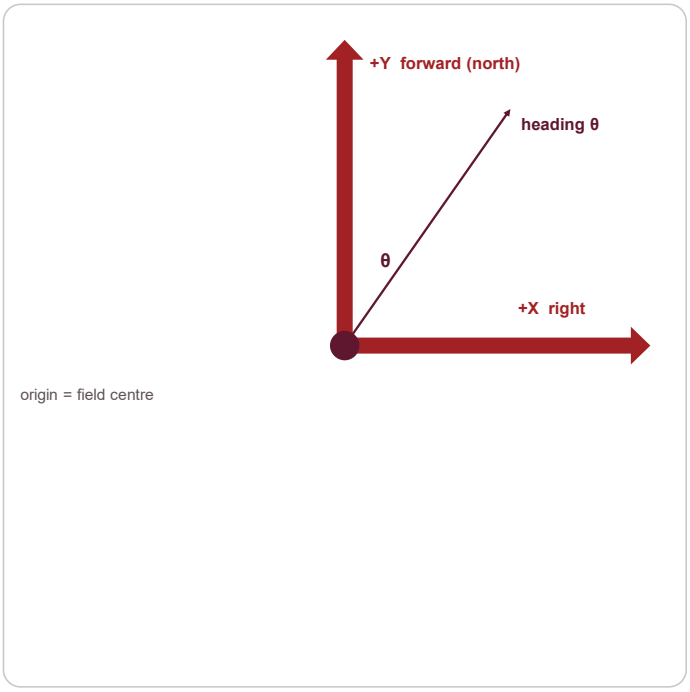
One Shared Frame

Every subsystem shares the same coordinates, so a position is never ambiguous between odometry, localisation and motion. The origin is the **field centre**; +X is right and +Y is forward, both in inches; and heading **θ** is measured clockwise from north and wrapped to the range (-π, π].

Why Clockwise-From-North

This convention matches a compass and the field drawing, so the forward direction is simply the unit vector below. Turn maths never flips sign except at the ±π wrap, which a single **normalizeAngle()** helper handles in one place, keeping the rest of the code free of edge cases.

forward = (sinθ, cosθ) **θ ∈ (-π, π]**



θ measured clockwise from +Y

Localisation System

Functional Role

The localisation system continuously estimates where the robot is on the field and how confident that estimate is. Autonomous movement cannot rely on time-based dead-reckoning: wheel slip, tile seams and impacts accumulate error with every movement, so the motion controller needs a **fresh field pose throughout the routine**. We built this as a reliability layer, not just a coordinate calculator.

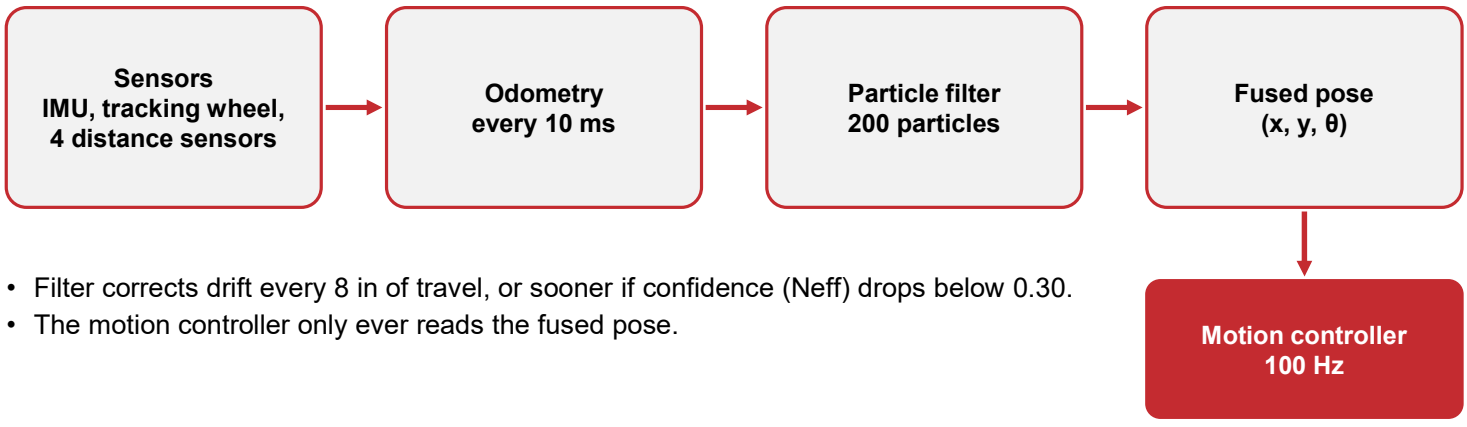
Performance Targets

- Position error below **2 in** across a full autonomous routine
- Heading error below **2°** at all times
- A fresh pose available for every **10 ms** control-loop tick
- CPU use low enough to never disrupt motor commands

How We Solved It

Our answer is an **Adaptive Monte Carlo Localisation (AMCL) particle filter fused with odometry**. Odometry supplies a continuous, low-latency baseline; the particle filter holds 200 simultaneous pose hypotheses and uses the distance sensors to periodically correct odometry drift. The following pages explain the odometry, the particle filter, and the way the two are fused together.

Localisation pipeline (runs on its own task)



- Filter corrects drift every 8 in of travel, or sooner if confidence (Neff) drops below 0.30.
- The motion controller only ever reads the fused pose.

The Odometry Baseline

Why We Need Odometry

The distance sensors only give a useful correction now and then — every 8 in of travel, or when confidence drops. Between those corrections the robot is still moving fast, and the motion controller needs a new pose every 10 ms. Odometry fills that gap: it gives a **continuous, low-latency position estimate**. The trade-off is that it drifts — every tiny encoder error or wheel slip adds up — which is exactly the drift the particle filter later corrects.

IMU for Heading, Tracking Wheel for Distance

We deliberately use two different sensors for the two different jobs. The **inertial sensor (IMU)** measures heading directly and does not drift from wheel slip, so it is trusted for rotation. A **single forward tracking wheel** measures distance travelled. Because our tank drive cannot strafe, lateral movement is always zero — that is a fact, not an approximation, so no sideways tracking wheel is needed.

Each cycle we read the IMU heading and the tracking-wheel encoder, which reports cumulative hundredths of a degree. We convert the encoder change into inches of travel:

$$\Delta fwd = \Delta cdeg \times (2\pi / 36000) \times rwheel$$

and apply a small **arc-to-chord correction** so that movement during a turn is measured as the straight-line chord, $\Delta = 2r \cdot \sin(\Delta\theta/2)$, rather than the longer arc.

This produces a clean per-cycle motion estimate in the robot's own frame — the input the particle filter then spreads across its 200 hypotheses.

Sensor Fusion

Why a Particle Filter, Not a Kalman Filter

A Kalman filter assumes one Gaussian blob of uncertainty. That breaks down at the start of a match, when we genuinely do not know if the robot was placed perfectly, and after a collision, when our true position is no longer a tidy bell curve. A **particle filter** makes no such assumption: it holds **200 separate pose guesses (particles)** at once and lets the sensor data decide which ones survive.

The Fusion: Odometry Drives, Sensors Correct

This is the heart of our design — odometry and the particle filter are not separate systems, they are fused in a three-step cycle that repeats every 40 ms.

- **Predict (every cycle).** The odometry motion is applied to all 200 particles, each with its own random noise drawn from Thrun's motion model. This is the fusion point: odometry moves the whole cloud, and the noise honestly represents how much we trust that movement.
- **Update (when a correction fires).** Each particle is re-weighted by how well its predicted sensor readings, found by ray-casting against a model of the field, match the real distance readings.
- **Resample.** High-weight particles are copied, low-weight ones dropped, and 5% are re-spawned at random to guard against getting permanently stuck on a wrong answer.

The noise added in the predict step uses two standard deviations — one for heading, one for translation — each built from four tuned α constants:

$$\sigma\theta = \sqrt{(\alpha_1 \cdot \Delta\theta^2 + \alpha_2 \cdot \Delta d^2)} \qquad \sigma d = \sqrt{(\alpha_3 \cdot \Delta d^2 + \alpha_4 \cdot \Delta\theta^2)}$$

Sensor Fusion Cycle

Odom-Primary, Sensor-Corrected

Because predict runs every cycle but update only fires after **8 in of travel or when confidence falls below 0.30**, the cloud tracks odometry tightly between corrections and snaps to the sensor-corrected position when one fires. The final pose we report is the weighted mean of all particles, using a circular mean for heading so it stays correct across the ±180° wrap.

One filter cycle

1 Predict

Move all 200 particles by the odometry change + random noise (every cycle)

2 Check trigger

≥ 8 in travelled since last fix, or Neff < 0.30?

No → skip to step 5

3 Update

Ray-cast each particle; weight it by how well its 4 predicted readings match the real ones

4 Resample

Copy likely particles, drop unlikely ones, re-spawn 5 % at random

5 Read out pose

Weighted mean of x, y; circular mean of heading

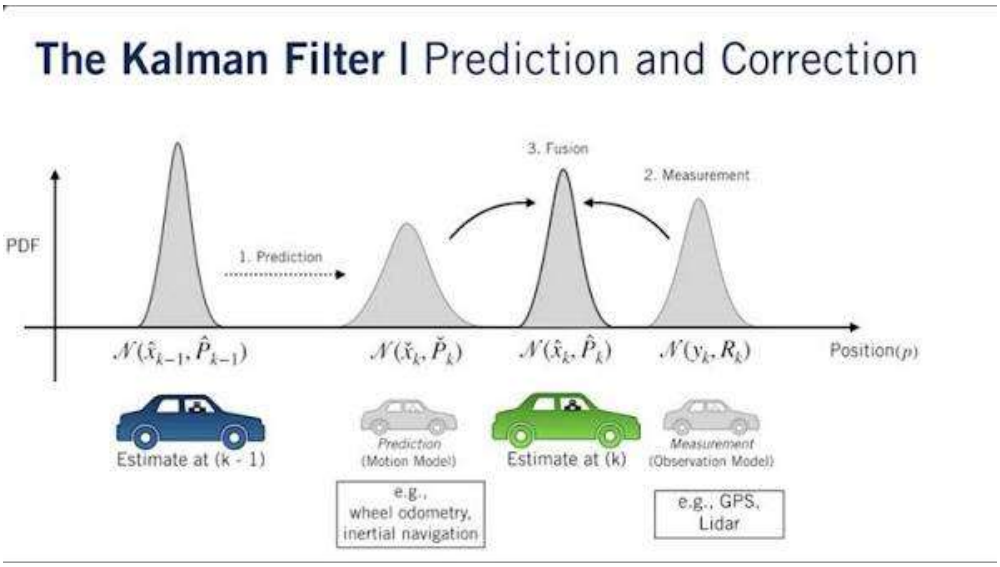
Why AMCL?

Our Filter

Positive	Negative
Survives contact and bad starts Holding 200 hypotheses means a bump or an imperfect placement never erases our estimate — unlikely particles simply lose weight while the cloud recovers.	**Heavier to compute** Ray-casting 200 particles against four sensors is far more work than updating one Gaussian, so we gate sensor corrections to keep the V5 brain comfortable.
No drift, full match Odometry gives a smooth pose every 10 ms while periodic sensor corrections wipe out accumulated drift, so accuracy holds across a whole 15-second routine.	**More tuning required** Four motion-model alphas plus sensor-noise terms must be tuned. We solved this with an auto-tuner in our simulator, but it is genuinely more setup than a PID.

Conclusion

A single-estimate filter such as a Kalman filter is lighter and simpler, but it assumes one tidy bell-curve of uncertainty and breaks down exactly when we need it most — at the start of a match when we are unsure of our placement, or after an opponent shoves us. Our AMCL-plus-odometry stack costs more computation and more tuning, but it is the only approach that recovers from those disruptions on its own. For a robot whose entire autonomous depends on trusting its position, we judged that robustness was worth the extra cost.



Inside the Particle Filter

What a Particle Is

Our filter holds **200 particles**, and each one is a complete guess of where the robot might be: an **(x, y, heading)** pose with a weight describing how believable that guess currently is. Instead of trusting one position, the robot reasons about a whole cloud of possibilities at once. This is what lets it stay confident even when any single estimate would be wrong.

How the Cloud Converges

At the start of a match the particles are spread out, because we are not perfectly certain of our placement. As the robot drives near walls and goals, each particle predicts what its four distance sensors **should** read from that imagined position. Particles whose predictions match the real readings gain weight; particles that disagree lose it. Within a second or two the cloud collapses onto the true position, and the robot's confidence climbs.

Reading Out a Single Pose

The motion controller still needs one pose, not a cloud, so we collapse the particles into a weighted mean. Position is a straightforward weighted average, but heading is averaged on the circle so that angles either side of the **±180° wrap** combine correctly:

$$\theta = \operatorname{atan2}\left(\Sigma\, w_i \cdot \sin\theta_i \,,\quad \Sigma\, w_i \cdot \cos\theta_i \right)$$

Correcting Drift

Predicting What Each Sensor Should See

To score a particle we need to know what its sensors would read if the robot really were at that pose. We built a **map of the field** in code — the four walls plus the goals, loaders and toggles — and **ray-cast** from each particle: we shoot a line out of every sensor and measure the distance to the first surface it hits. Across 200 particles and four sensors this is roughly 17,000 intersection tests per correction, which is why corrections are carefully gated rather than run every cycle.

Scoring the Match

Each particle's weight is multiplied by how closely its predicted readings match the real ones. We use a **hybrid likelihood**: a sharp Gaussian rewards particles near the right spot, while a small uniform term keeps a particle alive even if one reading is freakish. This blend stops a single noisy measurement from wiping out an otherwise good hypothesis.

$$w \mathrel{\times}= 0.75 \cdot \mathrm{Gauss}(\Delta;\ \sigma = 35\,\mathrm{mm}) + 0.25 \cdot (1/9000)$$

The Occlusion Gate

An opponent parked in front of a sensor reads far too short. If a reading comes back more than **300 mm shorter** than expected, we assume the beam is blocked and stop trusting it, rather than punishing every well-placed particle for something that is not really there.

Staying Reliable

Resampling Keeps the Cloud Useful

After each correction the cloud is rebuilt by **systematic resampling**: high-weight particles are copied, low-weight ones are dropped, so computing power follows the most likely poses. To avoid every particle collapsing onto one guess with no way to recover, we re-spawn **5% of particles (10 of 200)** at random each cycle. This diversity injection is our insurance against being "stuck" in a wrong answer.

Knowing When It Is Confident

The filter measures its own health with an effective-sample metric. When confidence falls below a threshold, it forces an immediate sensor correction instead of waiting for the usual distance trigger.

$$N_{_aa} = 1 / (N \cdot \Sigma w_i^2) \qquad \text{correct early if } N_{_aa} < 0.30$$

Surviving the Worst Case

We stress-tested this in our simulator with a **kidnap button** that teleports the robot without warning. A well-tuned filter scatters its particles, gathers fresh sensor evidence, and **reconverges within 5–10 seconds** — the same recovery that protects us when an opponent rams the robot mid-routine.

The Motion Controller

Turning a Pose Into Movement

Localisation tells us where we are; the motion controller decides what the wheels do about it. It runs at **100 Hz**, reading the fused pose every 10 ms and issuing fresh motor commands so that drift never has time to build up between corrections.

Why Not Just PID?

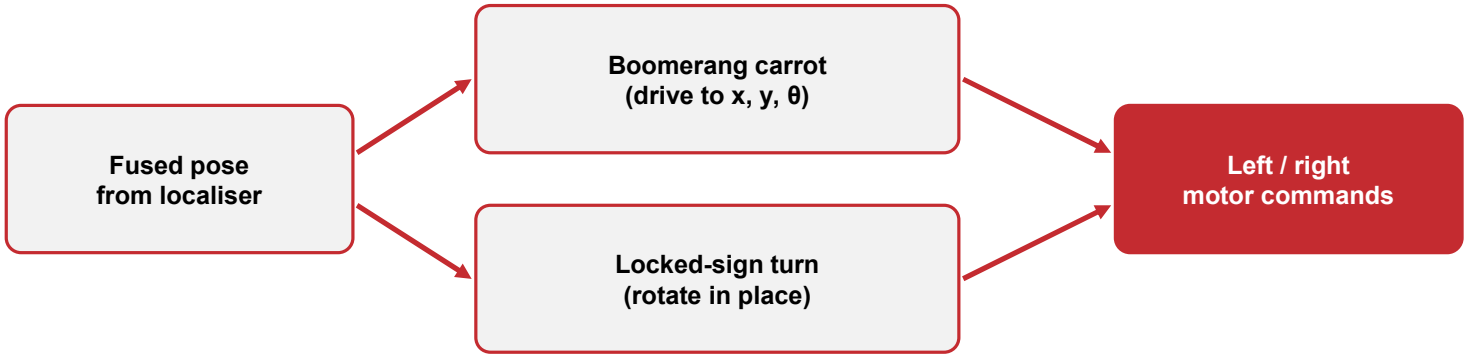
A plain PID-to-target works for straight lines, but for moving to a position and a heading at once it has two faults. First, the heading error can flip from +π to −π with a tiny move, making the output snap back and forth and the robot wobble. Second, it drives straight at the target then spins in place to face the right way — slow and jerky. We solved each with a purpose-built method.

Two Tools, One Controller

- **Boomerang carrot** shapes the path, so the robot curves into the target from the correct direction.
- **Locked-sign turn profile** handles in-place rotations with a smooth, oscillation-free accelerate–decelerate ramp.

Both were designed and validated in our simulator before being ported to the robot, so the on-field behaviour matched what we tuned on screen. The next two pages explain each in turn.

How a move command is executed



Repeats every 10 ms (100 Hz) until the localiser confirms the target pose.

Boomerang Pathing

Chasing a Moving Target

Instead of driving straight at the destination, the robot chases a **carrot point** that sits behind the target, offset along the target heading. The carrot pulls the robot around so it arrives facing the right way, tracing a smooth curve rather than a straight line and a spin. The path bends back like a boomerang — hence the name.

$$\textbf{carrot} = \textbf{target} - (\sin\theta, \cos\theta) \cdot d_{\text{tahlet}} \cdot \textbf{lead}$$

With a **lead of 0.6**, a robot 30″ out aims at a carrot 18″ behind the target. As it closes in, the distance shrinks and the carrot slides onto the target itself, so the approach tightens naturally.

The Near-Lock Phase

Within the last **6″**, the carrot offset becomes too small to point at reliably, so the controller switches to the true target heading and hands the final rotation to the locked-sign turn profile. Forward speed is also ramped down over the last 20″ so the robot settles onto the point instead of slamming into it.

Locked-Sign Turning

The Wraparound Problem

In-place turns are where a naive controller falls apart. If we drive the turn from a live error term, **omega = Kp x error**, the angle error is wrapped into the range **plus or minus 180 degrees**. Near that boundary a tiny change in heading flips the error from +180 to -180, so the motor command suddenly reverses direction. The robot sits at the target and **oscillates back and forth**, never settling.

Locking the Direction Once

Our fix is to decide the turn direction **exactly once**. On the first call we measure the total angle to turn and capture its sign — clockwise or anticlockwise — and we never recompute it. From then on the turn runs as an open-loop **kinematic velocity profile**: speed ramps up at a fixed angular acceleration, holds at a capped maximum, then ramps down. Crucially, the braking speed is limited by the kinematic stopping equation so the robot decelerates exactly onto its target:

$v_{max} = \sqrt{(2 \cdot a \cdot s_{remaining})} \quad \Rightarrow \quad$ decelerate to stop on the target

Open-Loop Shape, Closed-Loop Exit

The velocity shape is open-loop, but the **exit condition stays closed-loop**: we only end the turn when the localiser reports the real heading has arrived. If the robot is bumped or a motor stalls mid-turn, the move simply waits until the measured heading is correct rather than declaring victory early. We get the smoothness of a planned profile with the honesty of a sensor-checked finish.

Building Autonomous

Designing Paths in the Simulator

We do not write autonomous routines blind. We built a **browser-based field simulator** that runs the exact same maths as the robot — the same particle filter, the same boomerang and turn controllers, ported line for line. We click the field to place waypoints, drag each one to set its target heading, and press **Run Path** to watch the full run play out, including how confident localisation stays at every point along the route.

From Simulator to Robot

Once a path looks good, one button copies it out as ready-to-compile movement calls that paste straight into our autonomous code with no edits:

moveToPose(x, y, θ, t_out, {maxSpeed})

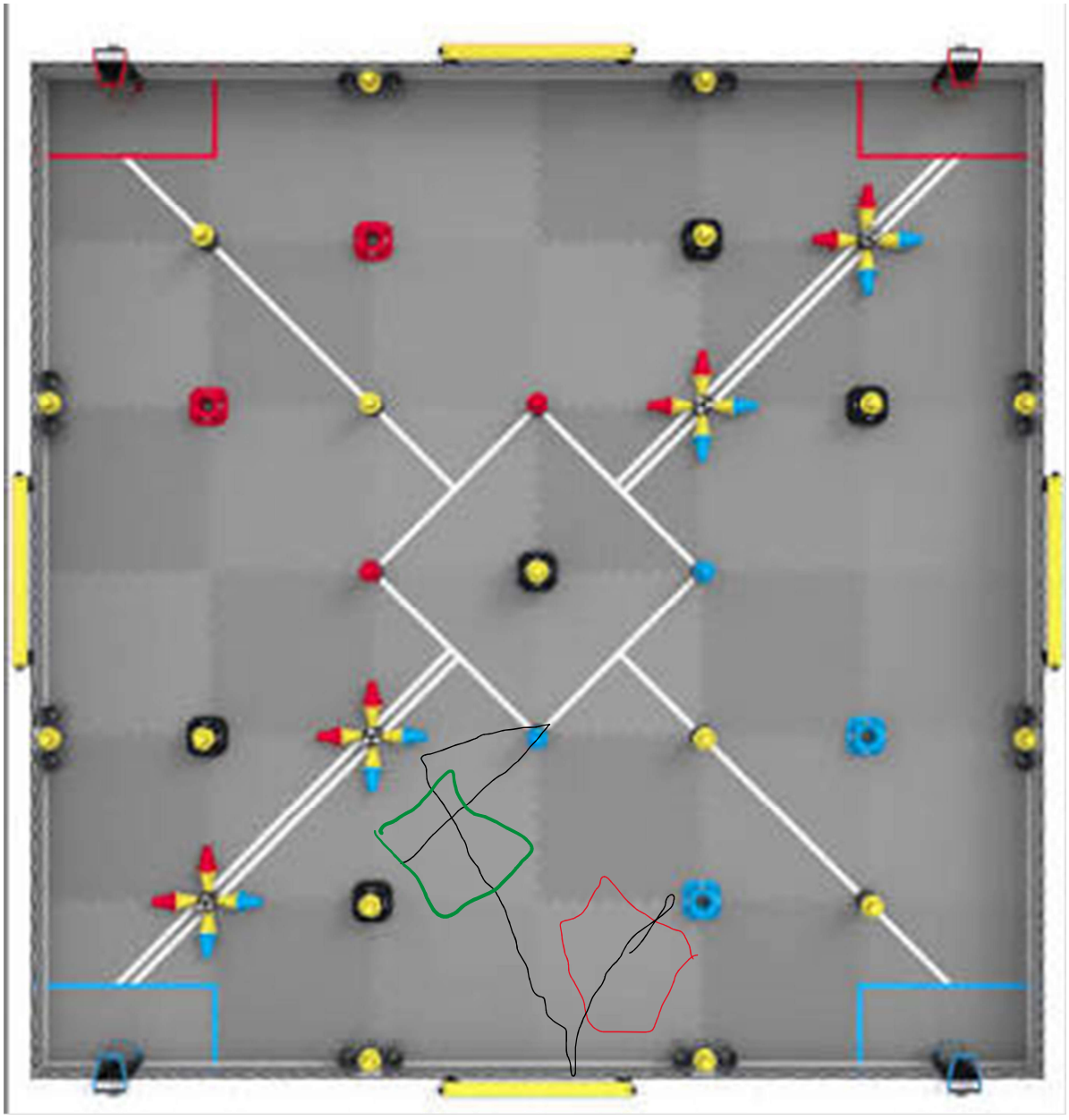
Because the simulator and the robot share identical control code, a path that works on screen behaves the same way on the field. This shortens our tuning loop dramatically — most of an autonomous routine is designed and validated before the robot ever touches a tile.

Why This Matters

Together, these systems answer the weakness we set out to fix. Localisation tells us **where we are**, the motion controller takes us **where we want to go**, and the simulator lets us **design and trust the route in advance**. The result is an autonomous that scores the same way every single match.

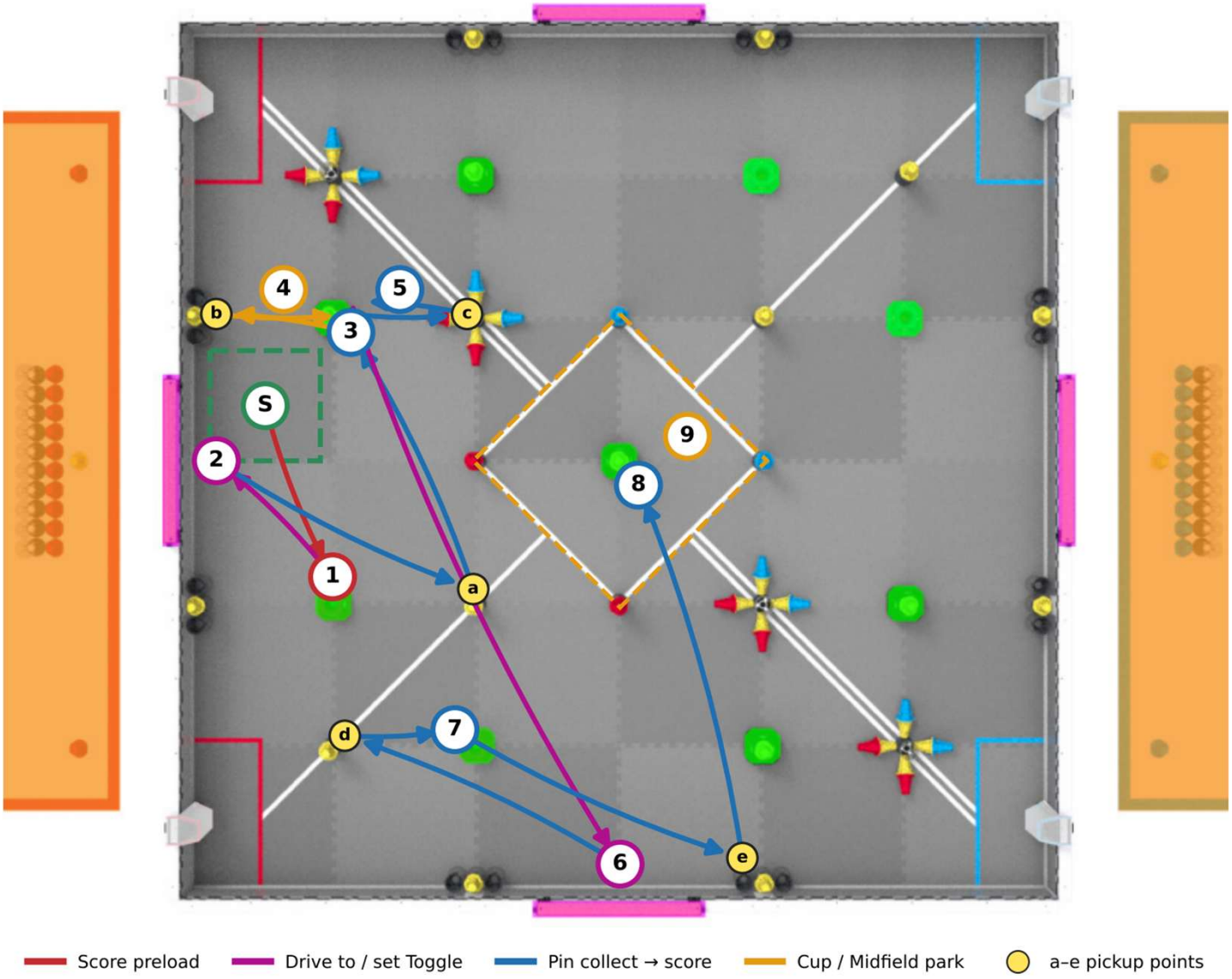
Autonomous routine

Goals:
Through reviewing the game manual, we concluded that autonomous would become vital in placing high during qualifications and in Robot Skills, where Autonomous Coding Skills is half of the combined ranking score. The field template below is used for our 60-second Autonomous Diagram, Logic and score calculation on the following pages.



Autonomous Diagram

Field template (red Alliance Station on the left). Numbers = scoring/Toggle actions in order; yellow letters = pickup points. Coordinates in inches from field centre.



#	Action	#	Action
S	Start (-58, 9) facing 90°, preload R/Y Pin	5	Pin c (cluster) stacked on the Cup – Short Goal (L)
1	Preload → Red Goal, left Quadrant	6	Push bottom Toggle to red
2	Push left Toggle to red	7	Pin d → Red Goal, bottom Quadrant
3	Pin a → Short neutral Goal (L)	8	Pin e (wall) → Tall Goal in Midfield
4	Cup b → nest clear-side on that Pin	9	Lower lift, stay in Midfield at 60 s

Autonomous Logic

Numbered sequence (unambiguous order)

0. initialise(): calibrate IMU (robot still ~3 s); seed AMCL at (-58, 9, 90°).

1. moveToPose(-47, -19, 180°) → effector.score() — preload R/Y Pin on Red Goal L. [≈ 4 s]

2. moveToPose(-66, 0, 270°) → push Toggle L to red → reverse 4" so robot is not touching it (SC4). [≈ 4 s]

3. intake ON → drive to Pin a (-24, -21) until Pin detected → moveToPose(-44, 21, 0°) → lift SHORT → score. [≈ 8 s]

4. drive to Cup b (-66, 24) → hold Cup clear-side down → nest on Pin at Short Goal L. [≈ 7 s]

5. intake Pin c (-25, 24) → lift SHORT_2 → stack on Cup. [≈ 7 s]

6. moveToPose(0, -66, 180°) → push Toggle B to red → reverse. [≈ 6 s]

7. intake Pin d (-45, -45) → moveToPose(-27, -44, 90°) → score Red Goal B. [≈ 8 s]

8. IF time left ≥ 9 s: intake Pin e (20, -65) → moveToPose(3, -4, 0°) → lift TALL → score. ELSE skip to 9. [≈ 10 s]

9. lower lift; remain inside Midfield until 60 s. [≈ 2 s]

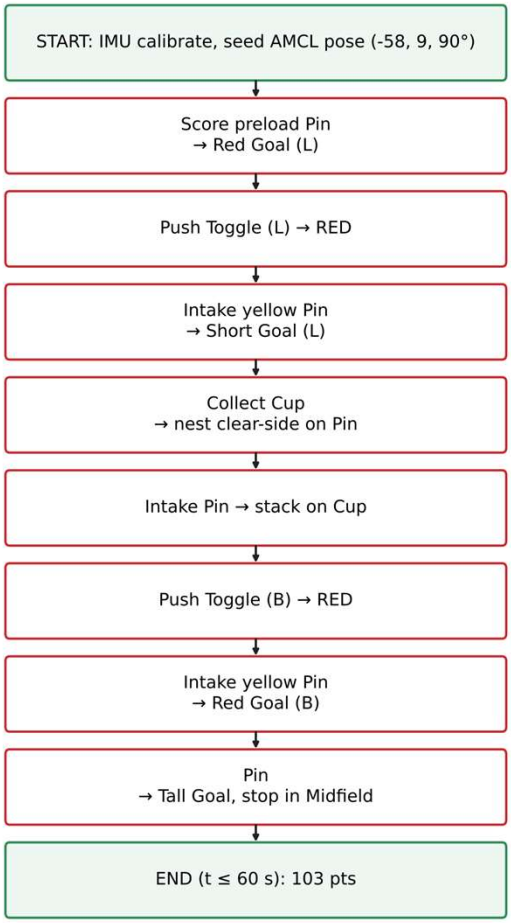
Rules built into the logic

• Every move has a timeout, so one failed move never stalls the routine.

• Intake runs asynchronously while driving (.async = true) to save time.

• Only 1 Pin + 1 Cup ever held (SG6).

• Total planned time ≈ 56 s → 4 s margin.



Autonomous Score

Step	Scoring element	Working	Points	Running
1	R/Y preload in Red Goal L (left Toggle red)	red half 5 + yellow half 10	15	15
2	Toggle L set red	enables yellow ownership in Quadrant L	0	15
3	Y/Y Pin on Short Goal L	10 + 10	20	35
4	Cup nested clear-side	keeps lower Pin halves visible	0	35
5	Y/Y Pin stacked on Cup	10 + 10	20	55
6	Toggle B set red	enables yellow ownership in Quadrant B	0	55
7	Y/Y Pin in Red Goal B	10 + 10	20	75
8	Y/Y Pin on Tall Goal (Midfield)	10 + 10, Owned because robot ends in Midfield	20	95
9	Robot ends in Midfield	+8	8	103

Maximum expected score

103 points

15 + 20 + 20 + 20 + 20 + 8 = 103

Safe score if stacking fails

83 points

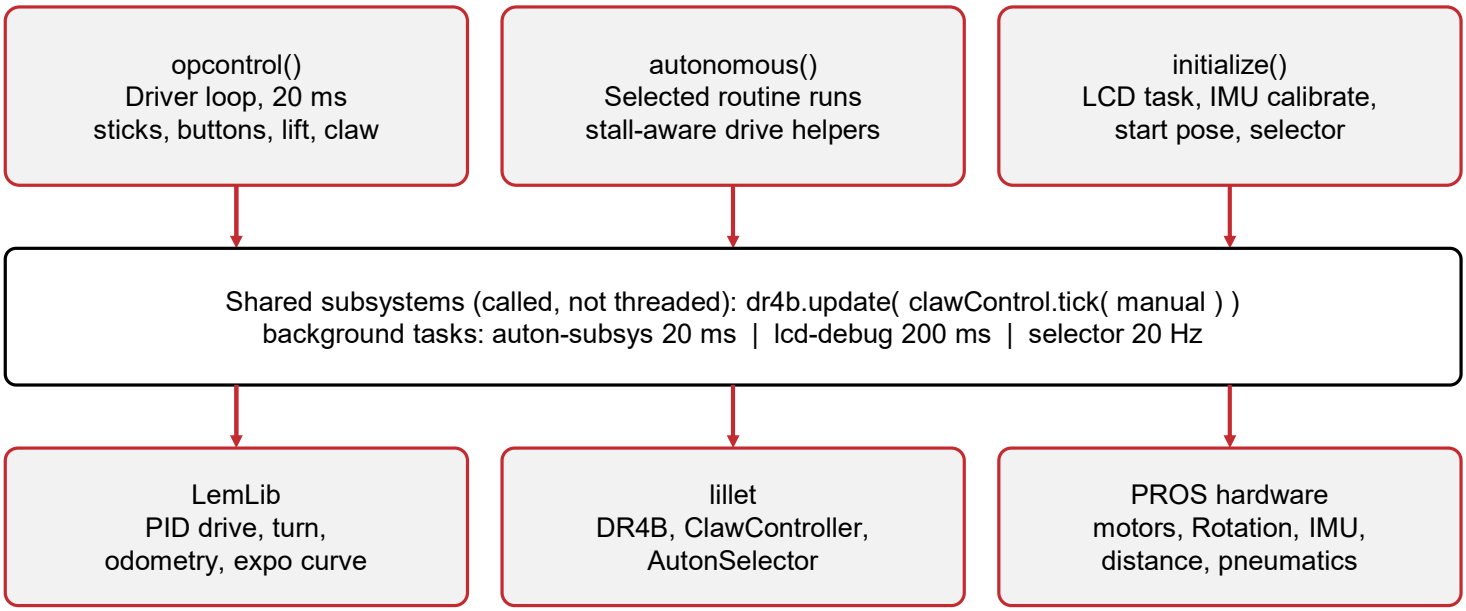
Skip steps 4–5: 103 − 20 = 83

Assumptions behind the calculation

- Each visible Pin Half on a Placed Pin scores separately (SC3); the Cup is nested clear-side so the halves below stay visible.
- In Skills, yellow halves in a Quadrant are Owned when that Quadrant's Toggle is set to an alliance colour (RSC3) – we set red.
- Toggles must be fully seated and not touching the robot at the end (SC4), so the robot reverses after each push.
- Tested result: best 103, mean of last 5 runs 96 (Test T13).

Robot Code Overview

The robot program is written in C++ on PROS. LemLib provides the PID drive and odometry, and our own library lillet provides the lift, claw and autonomous-selection logic plus the Monte Carlo localiser. Everything below is taken from the repository 4613B (src/main.cpp and include/).



File	Lines	Job
src/main.cpp	656	Hardware, initialize, selector, autonomous, opcontrol
include/robot_config.hpp	702	Every port, gain, speed and preset in one file
include/dr4b/dr4b.hpp	220	Slew-limited DR4B lift ramp
include/claw/claw_controller.hpp	377	Claw state machine, jam stop, piston writer
src/localization/*.cpp	741	Particle filter, field model, localiser task
src/motion/*.cpp	408	lillet chassis, PID, exit conditions

How to read the next pages

- Hardware map, set-up and controller map (pages 133–137).
- Driver loop, lift controller and claw controller.
- Autonomous: helpers, subsystem task and the routine line by line.
- Localiser (Monte Carlo): task loop, motion model, sensor model, resampling and the field model.
- Last page: what is running on the robot today and what is built but not enabled.

Hardware Declarations

src/main.cpp — hardware objects + LemLib chassis construction (what the robot actually runs)

```
17  1  pros::Rotation fwd_enc(cfg::PORT_FWD_ENCODER);
18     pros::Rotation hor_enc(cfg::PORT_HOR_ENCODER);
19  2  pros::Imu      imu    (cfg::PORT_IMU);
20
21  3  pros::Distance dist_front(cfg::PORT_DIST_FRONT);
22     pros::Distance dist_back (cfg::PORT_DIST_BACK);
23     pros::Distance dist_left (cfg::PORT_DIST_LEFT);
24     pros::Distance dist_right(cfg::PORT_DIST_RIGHT);
25
26     constexpr pros::MotorGears DRIVE_MOTOR_GEARSET = pros::MotorGears::blue;
27  4  pros::MotorGroup drive_left ({cfg::PORT_DRIVE_LF, cfg::PORT_DRIVE_LB, cfg::PORT_DRIVE_L_AUX}, DRIVE_MOTOR_GEARSET);
28     pros::MotorGroup drive_right({cfg::PORT_DRIVE_RF, cfg::PORT_DRIVE_RB, cfg::PORT_DRIVE_R_AUX}, DRIVE_MOTOR_GEARSET);
29
30     pros::Controller master(pros::E_CONTROLLER_MASTER);
31
32  5  pros::Motor intakeLeft(6);
33     pros::Motor intakeRight(-8);
34
35     pros::Motor claw(cfg::PORT_CLAW);
36     pros::Motor clawEnd(cfg::PORT_CLAW_END);
37
38  6  pros::adi::Pneumatics clawPneumatic(cfg::PORT_CLAW_PNEUMATIC, false);
39
40     pros::adi::Pneumatics leftSidePneumatic (cfg::PORT_LEFT_SIDE_PNEUMATIC,  false);
41     pros::adi::Pneumatics rightSidePneumatic(cfg::PORT_RIGHT_SIDE_PNEUMATIC, false);
42
43     pros::Rotation dr4bEncoder(cfg::PORT_DR4B_ENCODER);
44  7  lillet::DR4B dr4b(
45         {&intakeLeft, &intakeRight}, dr4bEncoder,
46         {cfg::DR4B_MAX_SPEED, cfg::DR4B_ACCEL_STEP, cfg::DR4B_DECEL_START_DEG, cfg::DR4B_TOLERANCE_DEG,
47          cfg::DR4B_RAISE_SIGN},
48         cfg::DR4B_ENCODER_GEAR_RATIO
49     );
50
51     lillet::LiftController liftController(dr4b, clawPneumatic, {&claw, &clawEnd});
52
53  8  lillet::ClawController clawControl(dr4b, clawPneumatic, claw, clawEnd,
54                                        cfg::makeClawConfig());
55
56  9  lemlib::TrackingWheel verticalWheel  (&fwd_enc, 2.0f * cfg::FWD_WHEEL_RADIUS_IN,
57                                          -cfg::FWD_OFFSET_IN, cfg::FWD_GEAR_RATIO);
```

Smart ports (negative = reversed)

- Drive left: ports -2, -10, -3; drive right: 5, 4, 7 (2 x 11 W + 1 x 5.5 W per side)
- Intake motors 6 and -8 also raise the DR4B through the ratchet
- Claw rollers -20 and -16; DR4B encoder -1; IMU 14
- Tracking wheels: forward -19, horizontal 11

Why the signs matter

- A negative port reverses a device in PROS, so reversal is written once in the declaration
- A positive command must drive the robot forward on all six drive motors; earlier code negated the sticks, which felt right in teleop but sent autonomous moves backwards
- The forward wheel is reversed so pushing forward increases the reading; the DR4B encoder is reversed so position increases while raising

Distance sensors + pneumatics

- Distance sensors: front 18, back 9, left 17, right 15
- ADI: claw piston C, left side piston A, right side piston B
- Pistons start retracted (false)

Source: 4613B/src/main.cpp lines 1-60, documentation/02-hardware-and-ports.md

LemLib Set-up

src/main.cpp — hardware objects + LemLib chassis construction (what the robot actually runs)

```
58      lemlib::TrackingWheel horizontalWheel(&hor_enc, 2.0f * cfg::HOR_WHEEL_RADIUS_IN,
59                                            -cfg::HOR_OFFSET_IN, cfg::HOR_GEAR_RATIO);
60
61  10  lemlib::Drivetrain drivetrain(&drive_left, &drive_right, cfg::DRIVE_TRACK_WIDTH_IN,
62                                    cfg::DRIVE_WHEEL_DIAM_IN, cfg::DRIVE_RPM,
63                                    cfg::LL_HORIZONTAL_DRIFT);
64
65  11  lemlib::ControllerSettings lateralController(
66          cfg::LL_LAT_KP, cfg::LL_LAT_KI, cfg::LL_LAT_KD, cfg::LL_LAT_WINDUP,
67          cfg::LL_LAT_SMALL_ERR_IN, cfg::LL_LAT_SMALL_ERR_MS,
68          cfg::LL_LAT_LARGE_ERR_IN, cfg::LL_LAT_LARGE_ERR_MS, cfg::LL_LAT_SLEW);
69
70  12  lemlib::ControllerSettings angularController(
71          cfg::LL_ANG_KP, cfg::LL_ANG_KI, cfg::LL_ANG_KD, cfg::LL_ANG_WINDUP,
72          cfg::LL_ANG_SMALL_ERR_DEG, cfg::LL_ANG_SMALL_ERR_MS,
73          cfg::LL_ANG_LARGE_ERR_DEG, cfg::LL_ANG_LARGE_ERR_MS, cfg::LL_ANG_SLEW);
74
75      lemlib::OdomSensors sensors(&verticalWheel, nullptr, &horizontalWheel, nullptr, &imu);
76
77      lemlib::ExpoDriveCurve throttleCurve(cfg::MTN_DRIVE_CURVE_DEADBAND,
78                                           cfg::MTN_DRIVE_CURVE_MIN_OUTPUT,
79                                           cfg::LL_DRIVE_CURVE_GAIN);
80      lemlib::ExpoDriveCurve steerCurve   (cfg::MTN_DRIVE_CURVE_DEADBAND,
81                                           cfg::MTN_DRIVE_CURVE_MIN_OUTPUT,
82                                           cfg::LL_DRIVE_CURVE_GAIN);
83
84  13  lemlib::Chassis chassis(drivetrain, lateralController, angularController, sensors,
```

1. **line 17** — Forward tracking wheel — V5 Rotation sensor (port -19, negative = reversed). Feeds LemLib odometry.
2. **line 19** — Inertial sensor (port 14) — heading source for odometry.
3. **line 21** — Four V5 distance sensors (front/back/left/right). On the current firmware they are read for the LCD + wall-calibration screen; the Monte Carlo filter that consumes them is not started by main.cpp.
4. **line 27** — Left drive: 3 motors (2 x 11W blue + 1 x 5.5W aux). Right side is identical. Ports/signs in robot_config.hpp section 1.
5. **line 32** — Shared intake motors (ports 6, -8). The same motors also raise the DR4B lift through a one-way ratchet.
6. **line 38** — Pneumatics: claw piston (ADI C) + left/right side pistons (ADI A/B).
7. **line 44** — lillet::DR4B — slew-limited, one-direction lift controller using the intake motors + a Rotation encoder.
8. **line 53** — lillet::ClawController — tick-driven R1/R2/L1/L2 claw behaviours with stall (jam) detection.
9. **line 56** — LemLib tracking wheels (radius/offset/gear ratio all come from robot_config.hpp section 2).
10. **line 61** — LemLib drivetrain model: track width, wheel diameter, 450 rpm, horizontal drift.
11. **line 65** — Lateral PID (kP 6.25, kD 5.25 ...) — drives straight-line distance.
12. **line 70** — Angular PID (kP 2.0, kD 12.5 ...) — drives turns.
13. **line 84** — The LemLib chassis object every autonomous move and arcade() call goes through.

What is configured

- Drivetrain: track width 12.0 in, 2.75 in wheels, 450 rpm, horizontal drift 2 (omni)
- Two ControllerSettings (lateral and angular) hold kP, kI, kD, windup, small/large error ranges and slew
- OdomSensors take the forward wheel, horizontal wheel and the IMU
- Two ExpoDriveCurves shape the stick input (deadband 3, gain 1.0 = linear)

Numbers that follow from it

- Free drive speed = (450 / 60) x pi x 2.75 = about 64.8 in/s
- Wheel radius was measured as 1.0 in; an earlier value of 0.5 made odometry read half the real distance
- All gains live in robot_config.hpp so tuning never edits main.cpp

Source: 4613B/src/main.cpp, include/robot_config.hpp, documentation/02-hardware-and-ports.md

Initialise Routine

src/main.cpp — auton selector, LCD status, initialize()

```
87  1  static void seedPose(const lillet::Pose& p) {
88         chassis.setPose(p.x, p.y, p.theta * (180.0f / lillet::PI));
89     }
90
91  2  lillet::AutonSelector autonSelector(
92         {
93             {"Red Preload",  {-48.0f, -48.0f,   0.0f}},
94             {"Example Path", {-48.0f, -48.0f,   0.0f}},
95             {"Do Nothing",   {-48.0f, -48.0f,   0.0f}},
96         },
97         5,
98         [](const lillet::Pose& p) {
99  3          seedPose(p);
100        },
101        false
102    );
103
104    static pros::Mutex statusMutex;
105    static char        statusLine[48] = "";
106
107    static void setStatus(const char* text) {
108        std::lock_guard<pros::Mutex> guard(statusMutex);
109        std::snprintf(statusLine, sizeof(statusLine), "%s", text);
110    }
111
112    template <typename... Devices>
113    static double avgTemperatureC(Devices&... devices) {
114        double sum = 0.0;
115        int n = 0;
116        auto scan = [&](auto& dev) {
117            for (double t : dev.get_temperature_all()) {
118                if (t >= 0.0 && t < 200.0) { sum += t; ++n; }
119            }
120        };
121        (scan(devices), ...);
122        return n ? sum / n : 0.0;
123    }
124
125    static void printPose() {
126        lemlib::Pose p = chassis.getPose();
127
128        pros::lcd::print(0, "LIFT raw=%ldcdeg pos=%.1f->%.1fdeg",
129                         (long)dr4b.getPositionCdeg(),
130                         dr4b.getPositionDeg(), dr4b.getTargetDeg());
131
132        pros::lcd::print(1, "F raw=%dmm  B raw=%dmm",
133                         (int)dist_front.get(), (int)dist_back.get());
134        pros::lcd::print(2, "L raw=%dmm  R raw=%dmm",
135                         (int)dist_left.get(), (int)dist_right.get());
136        {
137            std::lock_guard<pros::Mutex> guard(statusMutex);
138            pros::lcd::set_text(4, statusLine);
139        }
140
141        pros::lcd::print(5, "pose  x=%.1f  y=%.1f  h=%.0f", p.x, p.y, p.theta);
142        static uint32_t debugTick = 0;
143        pros::lcd::print(6, "auton=%s batt=%d%% t=%lu",
144                         autonSelector.getSelectedName().c_str(),
145                         (int)pros::battery::get_capacity(), (unsigned long)(++debugTick));
146        pros::lcd::print(7, "fwd=%ldcdeg imu=%.1fdeg hor=%ldcdeg",
147                         (long)fwd_enc.get_position(), imu.get_heading(),
148                         (long)hor_enc.get_position());
149    }
150
151    static void runPidTuneTest(bool angular) {
```

What happens at power-up

- Unbuffered stdout so every printed line reaches pros terminal immediately
- A low-priority lcd-debug task prints pose, lift and sensor values every 200 ms
- Drive aux motors are geared up to match wheel speed; dr4b.start() zeroes the lift
- chassis.calibrate() blocks while the IMU settles; a fault shows on the status line

Why it is written this way

- The IMU must finish calibrating before any motion, so it is done once in initialize()
- The start pose is seeded from the highlighted routine so odometry begins in the right place
- Telemetry runs in its own task so printing never slows the control loop

Source: 4613B/src/main.cpp initialize(), documentation/06-autonomous.md

Autonomous Selector

src/main.cpp — auton selector, LCD status, initialize()

```
152         chassis.setPose(0.0f, 0.0f, 0.0f);
153         if (angular) {
154             chassis.turnToHeading(cfg::PID_TUNE_ANGULAR_DEG, cfg::PID_TUNE_TIMEOUT_MS, {}, true);
155         } else {
156             chassis.moveToPoint(0.0f, cfg::PID_TUNE_LATERAL_IN, cfg::PID_TUNE_TIMEOUT_MS, {}, true);
157         }
158         const uint32_t t0 = pros::millis();
159         while (chassis.isInMotion()) {
160             const lemlib::Pose p = chassis.getPose();
161             std::printf("TUNE %lu,%.2f,%.2f,%.2f\n", (unsigned long)(pros::millis() - t0),
162                         p.x, p.y, p.theta);
163             static bool cancelHeld = true;
164             const bool btn = master.get_digital(DIGITAL_UP) || master.get_digital(DIGITAL_DOWN);
165             if (btn && !cancelHeld) chassis.cancelAllMotions();
166             cancelHeld = btn;
167             pros::delay(20);
168         }
169         const lemlib::Pose p = chassis.getPose();
170         const float err = angular ? (cfg::PID_TUNE_ANGULAR_DEG - p.theta)
171                                   : (cfg::PID_TUNE_LATERAL_IN - p.y);
172         std::printf("TUNE done t=%lums err=%.2f %s\n", (unsigned long)(pros::millis() - t0), err,
173                     angular ? "deg" : "in");
174         char buf[48];
175         std::snprintf(buf, sizeof(buf), "Tune %s err=%.2f", angular ? "ANG" : "LAT", err);
176         setStatus(buf);
177     }
178
179     static void lcdDebugTaskFn(void*) {
180         while (true) {
181             if (!cfg::PID_TUNE_ENABLED && master.get_digital(DIGITAL_DOWN)) {
182                 lillet::printDistCalibration(dist_front, dist_back, dist_left, dist_right);
183             } else {
184                 printPose();
185             }
186             pros::delay(200);
187         }
188     }
189
190 4   void initialize() {
191         std::setvbuf(stdout, nullptr, _IONBF, 0);
192
193         pros::lcd::initialize();
194
195 5       new pros::Task(lcdDebugTaskFn, nullptr, TASK_PRIORITY_MIN,
196                        TASK_STACK_DEPTH_DEFAULT, "lcd-debug");
197
198         drive_left.set_gearing(pros::MotorGears::green, 2);
199         drive_right.set_gearing(pros::MotorGears::green, 2);
200
201 6       dr4b.start();
202
203         setStatus("Calibrating IMU...");
204 7       chassis.calibrate();
205         if (imu.get_status() == pros::ImuStatus::error) {
206             setStatus("IMU FAULT - check PORT_IMU / wiring!");
207         } else {
208             setStatus("LEFT/RIGHT picks auton + start pose");
209         }
210
211         seedPose(autonSelector.getSelectedStartPose());
212 8       autonSelector.start();
213     }
214
215     void disabled() {}
216
217     void competition_initialize() {}
```

1. **line 87** — Seeds the LemLib pose from a start pose (radians to degrees).
2. **line 91** — AutonSelector: LEFT/RIGHT on the Brain LCD cycles routines; each carries its own start pose.
3. **line 99** — onSelect callback — re-seeds the chassis pose each time the selection changes.
4. **line 190** — Runs once at power-up.
5. **line 195** — Low-priority task paints lift/sensor/pose/battery debug lines on the Brain LCD every 200 ms.
6. **line 201** — Zeroes the lift encoder — the arm MUST be folded down at boot.
7. **line 204** — Calibrates the IMU (~3 s) and starts LemLib odometry. Robot must be still.
8. **line 212** — Starts the selector background task.

How a routine is chosen

- AutonSelector holds three routines, each with a name and start pose: Red Preload, Example Path, Do Nothing
- LEFT and RIGHT on the Brain screen cycle through them (polled at 20 Hz with rising-edge detection)
- Each change re-seeds the chassis pose, so odometry always starts from the highlighted start pose

Behaviour to know

- There is no confirm step: whatever is highlighted when autonomous() starts is run
- All three routines share the start pose (-48, -48, 0) today
- The chosen name is shown on LCD row 6

Source: 4613B/src/main.cpp, src/selector/selector.cpp, documentation/06-autonomous.md

Controller Map

Input	Action	How it works
Left stick	Drive (single-stick arcade)	chassis.arcade(fwd, turn) through the expo curve
Right stick Y	Manual lift jog	Deadzone 15; stick up raises (reverses the shared motors)
R1 (hold)	Intake	Piston retracts; intake and claw rollers forward; rollers stop on a jam
R2 (hold)	Intake reverse	Drives the 5.5 W intake motor backwards
L1 (hold)	Score	Piston extends; claw rollers reverse (-127)
L2 (hold)	Matchload	Piston extends; claw rollers forward (127)
X / Y / A / B	Lift level 4 / 3 / 2 / 1	300 / 220 / 160 / 100 degrees; piston extends once after a 10 degree rise
D-pad LEFT / RIGHT	Side pistons	Each side has its own on/off flag
D-pad UP / DOWN	PID test / calibration	PID tune tests only when PID_TUNE_ENABLED; DOWN shows distance calibration

Design choices behind the map

- Both drive axes are on the left stick so the right thumb stays free for the lift and claw buttons
- Hold buttons (R1, L1, L2) are read directly; one-shot actions use our own edge detector (next pages)
- The right stick always wins over a lift preset, so the driver can cancel any move by grabbing the stick
- Controller row 0 shows average drive and intake temperatures plus a heartbeat character every 250 ms

Source: 4613B/documentation/05-teleop-driver-control.md, src/main.cpp opcontrol()

Driver Control Loop 1/2

src/main.cpp — opcontrol(): driving, manual lift, side pistons, level buttons

```
478     void opcontrol() {
479         uint32_t lastControllerPrint = 0;
480
481         chassis.cancelAllMotions();
482
483         while (true) {
484             const int fwd  = master.get_analog(ANALOG_LEFT_Y);
485             const int turn = master.get_analog(ANALOG_LEFT_X);
486 (1)         chassis.arcade(fwd, turn);
487
488             constexpr int LIFT_STICK_DEADZONE = 15;
489             const int rightLiftRaw = master.get_analog(ANALOG_RIGHT_Y);
490 (2)         const int liftManualSpeed =
491                 (std::abs(rightLiftRaw) > LIFT_STICK_DEADZONE) ? -rightLiftRaw : 0;
492
493             if (liftManualSpeed != 0 && clawControl.busy()) {
494 (3)             clawControl.reset();
495             }
496
497             // Each side keeps its OWN explicit flag and every press writes that new
498             // state to exactly that piston: LEFT only touches the left piston, RIGHT
499             // only the right, independent of R1/R2/lift state.
500             static bool leftSideOut = false, rightSideOut = false;
501 (4)         if (newPress(DIGITAL_LEFT)) {
502                 leftSideOut = !leftSideOut;
503                 if (leftSideOut) leftSidePneumatic.extend();
504                 else             leftSidePneumatic.retract();
505             }
506 (5)         if (newPress(DIGITAL_RIGHT)) {
507                 rightSideOut = !rightSideOut;
508                 if (rightSideOut) rightSidePneumatic.extend();
509                 else              rightSidePneumatic.retract();
510             }
511
512             static const pros::controller_digital_e_t heightBtn[4] = {
513                 DIGITAL_X, DIGITAL_Y, DIGITAL_A, DIGITAL_B};
514             bool anyClawPress = false;
515             // X/Y/A/B move the lift. They never retract the claw piston. If it isn't
516             // already extended, it is extended exactly ONCE, after the lift has risen
517             // CLAW_PISTON_EXTEND_RISE_DEG off wherever it started (so it clears the
518             // bottom hardstop) — then it just stays out. Changing level while it's
519             // already extended only changes the height: no in/out cycling, no air waste.
520             static bool  pistonExtendPending = false;
521             static float pistonRiseStartDeg  = 0.0f;
522             for (int i = 0; i < 4; ++i) {
523                 if (newPress(heightBtn[i])) {
524                     anyClawPress = true;
525                     if (!clawControl.pistonOut() && !pistonExtendPending) {
526 (7)                     pistonExtendPending = true;
527                         pistonRiseStartDeg  = dr4b.getPositionDeg();
528                     }
529 (6)                 dr4b.move(cfg::clawHeightForButton(i));
530                     // Diagnostic (pros terminal): identical for X/Y/A/B — only the target differs.
531                     std::printf("LEVEL btn=%c target=%.0f pos=%.1f piston=%d pending=%d\n",
532                                 "XYAB"[i], cfg::clawHeightForButton(i), dr4b.getPositionDeg(),
533                                 (int)clawControl.pistonOut(), (int)pistonExtendPending);
534                 }
535             }
536             if (newPress(DIGITAL_R2)) {
537                 clawControl.requestMoveOut();
538                 pistonExtendPending = false;   // move-out extends the piston itself
```

(1) **line 486** — Left stick only: Y = throttle, X = turn, via LemLib arcade() + expo curve.
(2) **line 490** — Right stick Y = manual lift jog (deadzone 15, inverted sign).
(3) **line 494** — Moving the stick cancels any running claw sequence.
(4) **line 501** — D-pad LEFT toggles the left side piston; each side has its own state flag.
(5) **line 506** — D-pad RIGHT toggles the right side piston.
(6) **line 529** — X/Y/A/B send the lift to level 4/3/2/1 (300/220/160/100 deg).
(7) **line 526** — First level press arms a one-shot piston extend once the lift has risen 10 deg.

Each 20 ms pass, in order

- Read the left stick and drive with chassis.arcade()
- Read the right stick; a jog above the deadzone cancels any running claw sequence
- D-pad LEFT and RIGHT toggle their own piston flag and write only that piston
- X, Y, A, B set a lift level with dr4b.move()

Why it is structured like this

- Manual input always wins so the driver can never be locked out by a macro
- Every piston has its own flag, so one press can never change the other side
- The piston extends once per level change instead of every press, saving air

Source: 4613B/src/main.cpp opcontrol() lines 539-600

Driver Control Loop 2/2

src/main.cpp — opcontrol(): claw buttons, lift update, tuning mode, controller screen

```
536 (1)         if (newPress(DIGITAL_R2)) {
537                 clawControl.requestMoveOut();
538                 pistonExtendPending = false;   // move-out extends the piston itself
539                 anyClawPress = true;
540             }
541 (2)         clawControl.setIntake(master.get_digital(DIGITAL_R1));
542 (3)         clawControl.setScore(master.get_digital(DIGITAL_L1));
543 (4)         clawControl.setMatchload(master.get_digital(DIGITAL_L2));
544             if (master.get_digital(DIGITAL_R1) || master.get_digital(DIGITAL_L1) ||
545                 master.get_digital(DIGITAL_L2)) {
546                 anyClawPress = true;
547             }
548
549             if (anyClawPress && !dr4b.isEncoderConnected()) {
550                 setStatus("DR4B ENCODER FAULT - check port/wiring");
551             }
552
553 (5)         dr4b.update(clawControl.tick(liftManualSpeed));
554
555             // R1 retracts the piston, so cancel a pending extend while it is held.
556             if (master.get_digital(DIGITAL_R1)) pistonExtendPending = false;
557             if (pistonExtendPending && dr4b.isEncoderConnected()) {
558                 const float rise =
559                     (dr4b.getPositionDeg() - pistonRiseStartDeg) * cfg::DR4B_RAISE_SIGN;
560                 if (rise >= cfg::CLAW_PISTON_EXTEND_RISE_DEG) {
561                     pistonExtendPending = false;
562 (6)                 clawControl.setPistonOut(true);   // once
563                 }
564             }
565
566 (7)         if (cfg::PID_TUNE_ENABLED) {
567                 static bool upWas = false, downWas = false;
568                 const bool up = master.get_digital(DIGITAL_UP);
569                 const bool down = master.get_digital(DIGITAL_DOWN);
570                 const bool upEdge = up && !upWas, downEdge = down && !downWas;
571                 upWas = up; downWas = down;
572                 if (upEdge)        runPidTuneTest(true);
573                 else if (downEdge) runPidTuneTest(false);
574             }
575
576             const uint32_t now = pros::millis();
577             if (now - lastControllerPrint >= 250) {
578                 lastControllerPrint = now;
579                 static bool beat = false;
580                 beat = !beat;
581 (8)             master.print(0, 0, "D%.1f I%.1f %c   ",
582                              avgTemperatureC(drive_left, drive_right),
583                              avgTemperatureC(intakeLeft, intakeRight),
584                              beat ? '*' : '-');
585             }
586
587             pros::delay(20);
```

(1) **line 536** — R2 (press) = one-shot "move claw out" sequence.
(2) **line 541** — R1 (hold) = intake: shared intake + claw forward, stops claw on stall.
(3) **line 542** — L1 (hold) = score: claw reverse.
(4) **line 543** — L2 (hold) = matchload: piston out + claw forward.
(5) **line 553** — The ONE place the shared intake/lift motors are written in teleop: claw logic returns a speed, DR4B arbitrates it with any lift ramp.
(6) **line 562** — Piston extends exactly once after the lift clears the hardstop.
(7) **line 566** — Optional PID tuning mode: UP = 180 deg turn test, DOWN = 24 in drive test (streams TUNE lines).
(8) **line 581** — Controller screen: drive/intake motor temps + heartbeat every 250 ms.

What this half does

- R1, L1 and L2 are passed to the claw controller as held states
- dr4b.update(clawControl.tick(manual)) is the only place the shared intake/lift motors are written in teleop
- Once the lift has risen 10 degrees, the claw piston extends exactly once
- Optional PID tune mode and the 250 ms controller print follow

Problem this solved

- The intake and the lift use the same motors; two pieces of code writing them in one tick made them fight
- Routing every command through one call means a single owner per tick
- The encoder fault message appears on the status line if a lift button is pressed with the sensor unplugged

Source: 4613B/src/main.cpp opcontrol() lines 536-587

Button Edge Detection

src/main.cpp - newPress()

```
static bool newPress(pros::controller_digital_e_t button) {
    static bool last[32] = {};
    const int   i    = static_cast<int>(button) & 31;
    const bool  now  = master.get_digital(button);
    const bool  edge = now && !last[i];
    last[i] = now;
    return edge;
}
```

What it does

- Remembers the last state of every button in its own table
- Returns true only on the loop where a button goes from released to pressed
- Held actions (R1, L1, L2) use get_digital() directly; one-shot actions (D-pad, X/Y/A/B) use newPress()

The bug it fixed

- We first used the library call get_digital_new_press()
- R1, R2 and lift-height presses were swallowing the D-pad events, so LEFT and RIGHT stopped toggling the side pistons
- A single private table means no other code can consume or misattribute a press

Why this is good engineering

- It is a few lines, easy to read and test by pressing buttons
- It removes a hidden dependency on shared library state
- The same approach is used for the PID tune buttons

Source: 4613B/src/main.cpp lines 530-537, documentation/05-teleop-driver-control.md section 5.4

DR4B Lift Controller

include/dr4b/dr4b.hpp — DR4B one-direction slew-limited lift ramp

```
115         void update(int manualSpeed) {
116 (1)        if (manualSpeed != 0) {
117             raising_   = false;
118             holding_   = false;
119             rampSpeed_ = 0.0f;
120             applySpeed(manualSpeed);
121             return;
122         }
123
124         if (!raising_ && !holding_) { applySpeed(0); return; }
125
126 (2)     if (!isEncoderConnected()) {
127             // Lost the encoder mid-raise — stop rather than drive off a
128             // bogus reading. Manual R1/R2 jogging still works via the
129             // manualSpeed branch above even with the encoder dead.
130             raising_   = false;
131             holding_   = false;
132             rampSpeed_ = 0.0f;
133             applySpeed(0);
134             return;
135         }
136
137         // >0 while raising still has further to go, regardless of whether
138         // getPositionDeg() counts up or down while it happens (raiseSign).
139 (3)     const float error = (targetDeg_ - getPositionDeg()) * slew_.raiseSign;
140         if (error <= slew_.toleranceDeg) {
141             // Keep the target armed in a hold state instead of fully switching
142             // off. Gravity can pull the arm back down by a couple degrees, and
143             // we want the next tick to re-engage the reverse raise command as
144             // soon as the position leaves the target tolerance band.
145             raising_   = false;
146 (4)         holding_   = true;
147             rampSpeed_ = 0.0f;
148             applySpeed(0);
149             return;
150         }
151
152         // Once the arm drifts outside the tolerance window the target is no
153         // longer "held" — start the normal one-direction raise ramp again.
154         raising_ = true;
155         holding_ = false;
156
157         // Ease off approaching the target so it doesn't slam into the
158         // mechanical stop at full speed.
159         float targetMag = slew_.maxSpeed;
160 (5)     if (error < slew_.decelStartDeg) {
161             targetMag = slew_.maxSpeed * (error / slew_.decelStartDeg);
162         }
163
164         // Slew-rate limit — ramp the applied speed toward targetMag by at
165         // most accelStep per tick. This IS the "speed smoothing": no
166         // feedback correction, just a bounded rate of change.
167 (6)     if (rampSpeed_ < targetMag) rampSpeed_ = std::min(targetMag, rampSpeed_ + slew_.accelStep);
168         else                        rampSpeed_ = std::max(targetMag, rampSpeed_ - slew_.accelStep);
169
170         // One-direction rule enforced right here: raising is ALWAYS reverse.
171 (7)     applySpeed(-static_cast<int>(rampSpeed_));
172     }
```

(1) **line 116** — Any manual command always wins and cancels the ramp.
(2) **line 126** — Encoder lost mid-raise: stop rather than drive off a bogus reading.
(3) **line 139** — Error is signed by raiseSign so it works whichever way the sensor counts.
(4) **line 146** — Arrived: motors off, ratchet holds, target stays armed in case gravity drifts.
(5) **line 160** — Ease off in the last 20 deg so the arm does not slam the stop.
(6) **line 167** — Slew limit — the "smoothing". No PID (a forward correction would disengage the ratchet).
(7) **line 171** — Raising is ALWAYS reverse.

Settings (cfg::DR4B_*)

- maxSpeed 100 (0-127); accelStep 8 per tick; decelStart 20 degrees; tolerance 2 degrees; raiseSign +1

Each update() call

- Manual speed non-zero: cancel the ramp and write it straight to the motors
- Not raising and not holding: write 0
- Encoder disconnected: stop and drop the ramp
- Error within 2 degrees: motors off, holding = true (the ratchet holds the arm)
- Otherwise ramp the speed up by at most 8 per tick and ease off inside 20 degrees; always written as reverse

Safety checks

- moveToPosition() refuses to arm if the Rotation sensor reports PROS_ERR (wrong port or unplugged)
- A target already behind the arm cannot be commanded
- resetZero() re-zeros the encoder and waits up to 50 ms for it to read about 0

Source: 4613B/include/dr4b/dr4b.hpp, documentation/07-lift-and-claw-subsystems.md

Why The Lift Has No PID



Reverse = ratchet engages = arm RAISES

Forward = ratchet releases = pure intake, arm FALLS

The constraint

- One pair of motors does two jobs: intake and lift
- There is no way to drive the arm down, only to let it fall by spinning forward
- The same motors are also written by the claw logic and the driver

Why PID is wrong here

- A PID loop corrects overshoot with a forward command, which would disengage the ratchet and drop the arm
- So the controller only ever commands reverse or zero
- A slew-limited ramp (max speed, accel step, ease-in) gives smooth motion with no overshoot correction needed

The design that followed

- Synchronous, single owner: DR4B and ClawController have no tasks of their own
- The caller invokes dr4b.update(clawControl.tick(manual)) once per loop (teleop) or from the auton-subsys task (autonomous)
- Holding stays armed, so if gravity sags the arm out of the 2 degree band the ramp re-engages

Trade-off we accepted

- Levels can only be raised to; lowering means running the intake motors forward
- In exchange the lift needs no extra motors, which saves weight and motor budget

Source: 4613B/documentation/07-lift-and-claw-subsystems.md section 7.1, include/dr4b/dr4b.hpp

Claw Controller 1/2

include/claw/claw_controller.hpp — ClawController::tick() state machine

```
192         int tick(int manualIntakeSpeed) {
193             const uint32_t now       = pros::millis();
194             int            intakeCmd = manualIntakeSpeed;
195
196             switch (phase_) {
197                 case Phase::Idle:
198                     break;
199
200 1               case Phase::OutRaise:
201                     if (!lift_.isRaising() || now - phaseStart_ >= cfg_.outLiftTimeoutMs) {
202                         extendPiston();
203                         enter(Phase::OutSettle, now);
204                     }
205                     break;
206
207 2               case Phase::IntakeLand:
208                     retractPiston();
209                     // Shared intake forward lets the arm fall under gravity.
210                     intakeCmd = cfg_.intakeSharedSpeed;
211                     if (!lift_.isEncoderConnected() ||
212                         lift_.getPositionDeg() <= cfg_.intakeLandTolDeg ||
213                         now - phaseStart_ >= cfg_.intakeLandTimeoutMs) {
214                         startIntakeHop(now);
215                     }
216                     break;
217
218                 case Phase::IntakeHop:
219                     retractPiston();
220                     // Raising is the shared motors' REVERSE direction. Drive them
221                     // directly (not via dr4b's slow ramp, which barely moved the arm
222                     // for a small target) until the arm is up to intakeClearDeg.
223                     intakeCmd = -cfg_.intakeHopSpeed;
224                     if (!lift_.isEncoderConnected() ||
225                         lift_.getPositionDeg() >= cfg_.intakeClearDeg ||
226                         now - phaseStart_ >= cfg_.intakeClearTimeoutMs) {
227                         enter(Phase::Idle, now);
228                         // Restart the jam grace so spin-up after the hop isn't read as a stall.
229                         intakeHoldStart_ = now;
230                         intakeJamSince_  = 0;
231                     }
232                     break;
233
234                 case Phase::OutSettle:
235                     if (now - phaseStart_ >= cfg_.outPneuSettleMs) enter(Phase::OutIntake, now);
236                     break;
237
238                 case Phase::OutIntake:
239                     intakeCmd = cfg_.outIntakeSpeed;
240                     if (now - phaseStart_ >= cfg_.outIntakeMs) {
241                         intakeCmd = 0;
242                         enter(Phase::Idle, now);
243                     }
244                     break;
245
246                 case Phase::MatchloadRetract:
247                     retractPiston();
```

Precedence while idle

- MATCHLOAD (L2) > INTAKE (R1) > SCORE (L1)
- A running one-shot (R2 move-out, or the optional land-and-hop) owns the claw, piston and shared intake until it finishes

State machine phases

- Idle, OutRaise, OutSettle, OutIntake, MatchloadRetract, MatchloadExtend, IntakeLand, IntakeHop
- The last four are optional paths: land-and-hop is switched off and matchload release was replaced by a plain hold

Source: 4613B/include/claw/claw_controller.hpp tick()

Claw Controller 2/2

include/claw/claw_controller.hpp — ClawController::tick() state machine

```
248                     claw_.move(0);
249                     clawEnd_.move(0);
250                     if (now - phaseStart_ >= cfg_.matchloadRetractMs) enter(Phase::MatchloadExtend, now);
251                     break;
252
253                 case Phase::MatchloadExtend:
254                     extendPiston();
255                     if (now - phaseStart_ >= cfg_.matchloadExtendMs) {
256                         claw_.move(0);
257                         clawEnd_.move(0);
258                         enter(Phase::Idle, now);
259                     }
260                     break;
261             }
262
263             // Held-button behaviours only drive the claw while no one-shot
264             // sequence owns it. Priority: MATCHLOAD > INTAKE > SCORE.
265             bool r1DrivesClaw = false;   // R1's own claw logic (with jam stop) ran
266             if (phase_ == Phase::Idle) {
267 (3)             if (matchloadHeld_) {
268                     extendPiston();
269                     // Matchload is a continuous intake hold while the driver moves
270                     // the arm over the bar / through the up/down loading motion.
271                     // Do NOT stall-stop the wheels here — the operator is actively
272                     // managing the arm position with the button held.
273                     claw_.move(cfg_.matchloadClawSpeed);
274                     clawEnd_.move(cfg_.matchloadClawSpeed);
275                 } else if (intakeHeld_) {
276                     retractPiston();
277                     intakeCmd = cfg_.intakeSharedSpeed;
278                     r1DrivesClaw = true;
279 (4)                 driveClawWithStall(intakeHoldStart_, intakeJamSince_,
280                                        cfg_.intakeClawSpeed, now);
281                 } else if (scoreHeld_) {
282                     if (scoringPistonReady_) extendPiston();
283 (5)                 claw_.move(cfg_.scoreClawSpeed);
284                     clawEnd_.move(cfg_.scoreClawSpeed);
285                 } else {
286                     claw_.move(0);
287                     clawEnd_.move(0);
288                 }
289             }
290
291             // Whenever the shared intake is spinning FORWARD (intaking) the end effector
292             // spins with it, overriding whatever else the claw was doing (score,
293             // matchload, one-shot sequences, idle). Covers the intake pulse in R2's
294             // move-out, the lift's drop-to-bottom in R1's land step, and a manual
295             // stick-down intake. R1's normal hold is excluded — it has its own claw
296             // logic with the jam stop, so a seated piece isn't driven into the claw.
297             // Speed follows the shared command, so a gentle stick gives a gentle claw.
298             if (intakeCmd > 0 && !r1DrivesClaw) {
299                 const int spd = cfg_.intakeClawSpeed * std::min(intakeCmd, 127) / 127;
300                 claw_.move(spd);
301                 clawEnd_.move(spd);
302             }
303
304             return intakeCmd;
```

1. **line 200** — R2 sequence: raise, settle, pulse intake, idle.
2. **line 207** — Optional land-and-hop on R1 (disabled by config).
3. **line 267** — Priority: MATCHLOAD > INTAKE > SCORE.
4. **line 279** — R1 runs the claw but stops it when claw RPM stays below 10 for 150 ms (piece seated).
5. **line 283** — L1 reverses the claw.

Jam (stall) detection

- While R1 is held the claw rollers run forward
- After a 200 ms spin-up grace the controller watches |claw velocity|
- If it stays under 10 rpm for 150 ms the piece is seated, so the claw rollers stop while the shared intake keeps running
- Releasing and re-pressing R1 re-arms the check

Single piston writer

- setPistonOut(bool) stores the wanted state and writes the pneumatic only when it changes
- Before this, the 20 ms tick re-sent extend() 50 times a second while a button was held
- Each real change is logged with the phase and held buttons, so stray cycling can be diagnosed from pros terminal

Source: 4613B/include/claw/claw_controller.hpp, documentation/07-lift-and-claw-subsystems.md section 7.3

Claw Behaviour Table

Behaviour	Piston	Shared intake motors	Claw rollers	Stops when
R1 intake	retract	+127 (arm falls)	+127 until stalled	R1 released or jam
R2 move out	extend after raise	+127 for 500 ms	follow shared intake	sequence ends
L1 score	extend	manual	-127	L1 released
L2 matchload	extend	manual	+127 continuous	L2 released

Timing constants (robot_config.hpp)

- Intake speeds 127 / 127; jam rpm 10, hold 150 ms, grace 200 ms
- Move-out: raise to 100 degrees (2 s limit), piston settle 250 ms, intake pulse 500 ms
- Score roller speed -127; matchload roller speed 127
- Lift levels: 100, 160, 220, 300 degrees on B, A, Y, X; piston extend rise 10 degrees

Extra rule

- Whenever the shared intake is commanded forward and R1 jam logic is not in charge, the claw rollers spin in proportion (clawSpeed x command / 127), so the end effector always intakes with the shared intake

Cancelling

- Moving the right stick while a one-shot is running calls clawControl.reset(), so the driver can always take back control

Source: 4613B/documentation/05-teleop-driver-control.md sections 5.6, 07-lift-and-claw-subsystems.md section 7.3

Claw Piston Logic

1. **Level button pressed with piston retracted**
   set pistonExtendPending and remember the lift angle

2. **Every loop**
   rise = (angle now - start angle) x raise sign

3. **rise >= 10 degrees**
   setPistonOut(true) exactly once, then clear pending

4. **Level changed while piston already out**
   only the height changes - no in/out cycling

5. **R1 held**
   cancels a pending extend (R1 retracts the piston)

Why "extend once"

- Pressing X, Y, A or B repeatedly must not cycle the piston every time you change level
- Extending only after a 10 degree rise keeps the claw clear of the bottom hard-stop
- Air is limited, so avoiding wasted cycles keeps pressure for the whole match

Single writer rule

- All piston writes go through ClawController::setPistonOut(), which tracks the wanted state and writes only on change
- It also prints PISTON extend/retract with the phase and held buttons

Source: 4613B/src/main.cpp lines 560-565, documentation/05-teleop-driver-control.md section 5.5

Auton Subsystem Task

src/main.cpp — background "auton-subsys" task (lift/claw run while driving)

```
250     static bool             autonTaskCreated = false;
251     static std::atomic<bool> autonTaskActive{false};
252     static std::atomic<bool> intakeTaskRun{false};
253
254 1   static void ensureAutonTask() {
255         if (autonTaskCreated) return;
256         autonTaskCreated = true;
257         new pros::Task([]() {
258             while (true) {
259                 if (autonTaskActive) {
260 2                   if (intakeTaskRun)                          dr4b.update(clawControl.tick(0));
261                     else if (dr4b.isRaising() || dr4b.isHolding())  dr4b.update(0);
262                 }
263 3               pros::delay(20);
264             }
265         }, TASK_PRIORITY_DEFAULT, TASK_STACK_DEPTH_DEFAULT, "auton-subsys");
266     }
267
268     static void intakeState(bool on) {
```

1. **line 254** — Creates the helper task once.
2. **line 260** — Every 20 ms: same tick + update pair teleop uses, so intake logic behaves identically in auton.
3. **line 263** — 50 Hz subsystem loop, independent of the drive loop.

Why a separate task

- DR4B and ClawController are synchronous: someone has to call update() and tick() regularly
- During autonomous the main thread is blocked inside a drive move, so a 50 Hz auton-subsys task does it
- If the intake logic is on it runs dr4b.update(clawControl.tick(0)); otherwise it keeps an armed lift ramp running

autonomous() lifecycle

- Create the task once; autonTaskActive = true
- Switch on the selected index and run the routine (blocking)
- intakeState(false); autonTaskActive = false; wait 40 ms
- dr4b.release(), stop intake motors, set the "Auton done" status line

Result

- intakeState(true) in autonomous behaves exactly like holding R1 in teleop, including the jam stop
- The lift can rise while the robot is driving, which saves time

Source: 4613B/src/main.cpp lines 254-272 and 504-525, documentation/06-autonomous.md section 6.2

Autonomous Helpers

src/main.cpp — autonomous helper moves with stall detection

```
345     static constexpr int   AUTON_MOVE_STALL_MS = 500;
346     static constexpr int   AUTON_TURN_STALL_MS = 500;
347     static constexpr int   AUTON_HARD_CAP_MS   = 10000;
348 (1) static constexpr float AUTON_STILL_POS_IN  = 0.05f;
349     static constexpr float AUTON_STILL_DEG     = 0.2f;
350
351 (2) static void waitUnlessStalled(int stallMs) {
352         lemlib::Pose last = chassis.getPose();
353         uint32_t stillSince = pros::millis();
354         while (chassis.isInMotion()) {
355             pros::delay(20);
356             const lemlib::Pose now = chassis.getPose();
357             const float dPos = std::hypot(now.x - last.x, now.y - last.y);
358             const float dTh  = std::fabs(now.theta - last.theta);
359             if (dPos > AUTON_STILL_POS_IN || dTh > AUTON_STILL_DEG) {
360                 stillSince = pros::millis();
361             } else if (pros::millis() - stillSince >= static_cast<uint32_t>(stallMs)) {
362 (3)             chassis.cancelAllMotions();
363                 break;
364             }
365             last = now;
366         }
367     }
368
369 (4) static void driveForward(float inches, int maxSpeed = 80, int stallMs = AUTON_MOVE_STALL_MS) {
370         const lemlib::Pose p = chassis.getPose();
371         const float th = p.theta * (3.14159265f / 180.0f);
372         const float dir = (inches >= 0.0f) ? 1.0f : -1.0f;
373         chassis.moveToPoint(p.x + inches * std::sin(th), p.y + inches * std::cos(th),
374                             AUTON_HARD_CAP_MS,
375                             {.forwards = dir > 0.0f, .maxSpeed = static_cast<float>(maxSpeed)}, true);
376         waitUnlessStalled(stallMs);
377     }
378
379 (5) static void driveTurn(float degrees, int maxSpeed = 70, int stallMs = AUTON_TURN_STALL_MS) {
380         const float target = chassis.getPose().theta + degrees;
381         chassis.turnToHeading(target, AUTON_HARD_CAP_MS, {.maxSpeed = maxSpeed}, true);
382         waitUnlessStalled(stallMs);
383     }
384
385     static void driveCurve(float forwardIn, float rightIn, float turnDeg,
```

1. **line 348** — "Not moving" thresholds: less than 0.05 in and 0.2 deg counts as still.
2. **line 351** — Waits for a LemLib motion to finish, but cancels it if the robot has been still for stallMs (so contact with a wall/goal ends the move early instead of burning time).
3. **line 362** — Cancels the in-flight motion on stall.
4. **line 369** — driveForward(inches): converts a relative distance into an absolute moveToPoint along the current heading.
5. **line 379** — driveTurn(degrees): relative turn via turnToHeading.

waitUnlessStalled()

- Every move has a 10 s hard cap but we want to leave a move as soon as progress stops
- Every 20 ms it compares pose to the last pose; if position moved under 0.05 in and heading under 0.2 degrees for stallMs, the motion is cancelled
- So the stallMs values in the routine are tuned contact timeouts

Relative moves

- driveForward(in, speed, stallMs): moveToPoint along the current heading
- driveTurn(deg, speed, stallMs): turnToHeading(current + deg)
- driveCurve(...): moveToPose to a point offset in the robot frame (defined, not yet used)

Why relative

- Moves are easy to tune on the real field one at a time
- Trade-off: errors accumulate through the routine, which is the gap the Monte Carlo localiser is meant to close

Source: 4613B/src/main.cpp lines 353-403, documentation/06-autonomous.md section 6.3

Autonomous Code 1/2

src/main.cpp — "Red Preload" autonomous + autonomous() dispatcher

```
397     static void autonRedPreload() {
398 1       chassis.setPose(-48.0f, -48.0f, 0.0f);
399         driveForward(8);
400 2       liftTo(100);
401         intake(100);
402
403         driveForward(-2,100,300);
404         driveTurn(15, 50);
405         driveForward(-25, 100, 600);
406         driveForward(6, 100, 200);
407         driveTurn(-20, 75);
408         driveForward(-13, 100, 300);
409
410 3       dr4b.resetZero();
411
412         driveForward(10,70,500);
413         driveTurn(-43, 60);
414         driveForward(31,90,1000);
415 4       intakeState(true);
416         pros::delay(150);
417         driveForward(3,40,100);
418         driveForward(-2,40,100);
419         driveTurn(112, 70,100);
420         driveForward(18,70,250);
421         driveTurn(-30,100,300);
422         pros::delay(100);
423         intakeState(false);
424
425 5       liftStart(140);
426         driveForward(-20,80,400);
427 6       clawPiston(true);
428 7       liftWait();
429         driveForward(-11,60,200);
```

Steps in this part

- setPose(-48, -48, 0) declares the start; creep forward 8 in; liftTo(100) raises the lift (3 s cap)
- intake(100) releases the ramp and runs intake and claw rollers
- Back 2 in, turn +15, back 25 in at full speed (contact timeout 600 ms), forward 6 in, turn -20, back 13 in
- dr4b.resetZero() re-zeros the lift encoder while the arm is at rest
- Forward 10 in, turn -43, forward 31 in, then intakeState(true) with a 150 ms wait

Source: 4613B/src/main.cpp autonRedPreload(), documentation/06-autonomous.md section 6.4

Autonomous Code 2/2

src/main.cpp — "Red Preload" autonomous + autonomous() dispatcher

```
430         liftTo(70);
431         pros::delay(150);
432         claw.move(-100);
433         pros::delay(200);
434         liftTo(100);
435         driveForward(5, 40,200);
436         clawPiston(false);
437         intakeState(true);
438
439         //sKILLS
440        // driveTurn(40, 50,400);
441        // driveForward(40,30,2000);
442     }
443
444     void autonomous() {
445         ensureAutonTask();
446         autonTaskActive = true;
447 8       switch (autonSelector.getSelectedIndex()) {
448             case 0: autonRedPreload(); break;
449             case 1: autonExamplePath(); break;
450             case 2:     break;
451         }
452
453         intakeState(false);
454         autonTaskActive = false;
455         pros::delay(40);
456         dr4b.release();
457         intakeLeft.move(0);
458         intakeRight.move(0);
459
460         char buf[48];
461         std::snprintf(buf, sizeof(buf), "Auton done: %s", autonSelector.getSelectedName().c_str());
462         setStatus(buf);
463     }
```

1. **line 398** — Start pose (-48, -48), facing north.
2. **line 400** — Raise lift to 100 deg (blocking) then run the intake.
3. **line 410** — Re-zeroes the lift encoder mid-routine after the arm has settled.
4. **line 415** — Starts the background intake/claw logic (stall-stop included).
5. **line 425** — Non-blocking lift raise: lets the robot keep driving while it rises.
6. **line 427** — Extend claw piston via the tracked write-on-change wrapper.
7. **line 428** — Join point: waits for the lift started above.
8. **line 447** — Selector index picks the routine; 0 = Red Preload, 1 = Example Path, 2 = Do Nothing.

Steps in this part

- Nudge in and out (3 in, -2 in), turn +112, forward 18 in, turn -30, stop intake
- liftStart(140) begins raising while driving back 20 in, then clawPiston(true) and liftWait() joins the lift
- Back 11 in, lower to 70, run the claw in reverse for 200 ms to release
- Raise to 100, forward 5 in, retract the piston and resume intake

Notes on this routine

- It is a contact-driven sequence of relative moves; the field meaning of each move belongs to whoever designed it
- The planned 9-step moveToPose route and its 103 point calculation are on the Autonomous Diagram, Logic and Score pages
- A skills extension is present in the file but commented out

Source: 4613B/src/main.cpp lines 405-503

Localiser Task Loop 1/2

src/localization/localization.cpp — the 40 ms Monte Carlo background task

```
144     void Localizer::taskFn() {
145         // Wait for IMU calibration if it is still in progress — BOUNDED. A
146         // disconnected/faulty IMU reports is_calibrating() forever (PROS_ERR is
147         // truthy), which used to park this task here for good with no AMCL at
148         // all. After the bound we run anyway on whatever heading the IMU gives.
149         {
150             const uint32_t t0 = pros::millis();
151 1           while (_imu.is_calibrating() && pros::millis() - t0 < 5000 && _running)
152                 pros::delay(10);
153         }
154
155         uint32_t wake = pros::millis();
156
157         while (_running) {
158             // 0. Service a pending setPose() reseed request, if any. Handled here
159             //    (not in setPose() itself) because only this thread may touch
160             //    _pf / the encoders / _last_* bookkeeping.
161 2           bool do_reseed = false;
162             Pose reseed_pose;
163             {
164                 std::lock_guard<pros::Mutex> guard(_mutex);
165                 if (_pending_reseed) {
166                     do_reseed       = true;
167                     reseed_pose     = _pending_reseed_pose;
168                     _pending_reseed = false;
169                 }
170             }
171             if (do_reseed) {
172                 _fwd_enc.set_position(0);
173                 if (_hor_enc) _hor_enc->set_position(0);
174                 _imu.set_heading(headingDeg360(reseed_pose.theta));
175
176                 _last_fwd_cdeg     = 0;
177                 _last_hor_cdeg     = 0;
178                 _last_heading_rad  = reseed_pose.theta;
179                 _dist_since_update = 0.0f;
180                 _cycles_since_update = 0;
181
182                 _pf.init(reseed_pose);
183
184                 {
185                     std::lock_guard<pros::Mutex> guard(_mutex);
186                     _best_pose  = reseed_pose;
187                     _confidence = 0.0f;
188                 }
189
190 7               pros::Task::delay_until(&wake, TASK_PERIOD_MS);
191                 continue;
192             }
193
194             // 1. Compute odometry delta since last cycle
195 3           OdomDelta delta = computeDelta();
196
197             // 2. Read distance sensors
198             SensorData z = readSensors();
199
```

One cycle, every 40 ms (25 Hz)

- Odometry delta from the tracking wheel and IMU
- Four distance readings, bias-corrected and quality-gated
- PREDICT every cycle: move every particle by the delta plus noise
- UPDATE and RESAMPLE only after 8 in of travel, or when confidence drops below 0.30 (at least 3 cycles apart)
- Publish the weighted mean of the evidence-backed particles

Source: 4613B/src/localization/localization.cpp, documentation/03-monte-carlo-localization.md section 3.2

Localiser Task Loop 2/2

src/localization/localization.cpp — the 40 ms Monte Carlo background task

```
200             // 3. Predict step — odometry is the continuous baseline, runs every cycle.
201             _pf.predict(delta);
202             const float step_in =
203                 std::sqrt(delta.forward * delta.forward + delta.lateral * delta.lateral);
204             _dist_since_update += step_in;
205             _cycles_since_update++;
206
207             // Robot speed (in/s) inflates the measurement σ inside the filter —
208             // readings taken while moving fast are ~30 ms stale.
209             _pf.setSpeed(step_in / (TASK_PERIOD_MS / 1000.0f));
210
211             // 4. Sensor correction — fires only when the robot has travelled far
212             //    enough since the last fix, or confidence has dropped.  Distance
213             //    sensors act as drift compensation on top of odometry.
214             //    The Neff (low-confidence) trigger is additionally rate-limited:
215             //    back-to-back corrections re-use correlated readings and repeated
216             //    resampling on them impoverishes the particle set.
217             //    (Identical gating to the browser simulator.)
218 (4)         const bool do_update =
219                 (_confidence < _mcl_cfg.neff_update_thresh &&
220                  _cycles_since_update >= _mcl_cfg.neff_min_cycles) ||
221                 _dist_since_update >= _mcl_cfg.sensor_update_dist_in;
222
223             float conf = _confidence;
224             if (do_update) {
225 (5)             _pf.update(z);
226                 // Measure BEFORE resample (weights uniform after). A collapsed
227                 // update means "nothing matched" — that is zero confidence, not
228                 // the Neff=1 the artificial uniform reset would report.
229                 conf = _pf.collapsed() ? 0.0f : _pf.confidenceNeff();
230                 _pf.resample(_last_heading_rad);
231                 _dist_since_update   = 0.0f;
232                 _cycles_since_update = 0;
233             }
234
235             // 5. Publish result
236             {
237                 std::lock_guard<pros::Mutex> guard(_mutex);
238 (6)             _best_pose  = _pf.bestEstimate();
239                 _confidence = conf;
240             }
241
242             // Overrun guard: if this cycle took longer than the period,
243             // delay_until() would return without blocking. Always give up the CPU
244             // for at least a few ms so lower-priority tasks (LCD, telemetry) can
245             // never be starved, then re-anchor the schedule from now.
246             if (pros::millis() - wake >= TASK_PERIOD_MS) {
247                 pros::delay(5);
248                 wake = pros::millis();
249             } else {
250                 pros::Task::delay_until(&wake, TASK_PERIOD_MS);
251             }
252         }
253     }
254
255     // ── Odometry delta computation ──────────────────────────────────────
```

(1) **line 151** — Waits (bounded 5 s) for IMU calibration.
(2) **line 161** — Pending setPose() reseed is applied on the task thread only (no races).
(3) **line 195** — 1. odometry delta 2. read sensors 3. predict (every cycle).
(4) **line 218** — Gate: correct only after 8 in of travel, or when Neff < 0.30 (3+ cycles apart).
(5) **line 225** — Sensor correction: update, measure confidence, resample.
(6) **line 238** — Publishes the pose under a mutex for the motion code to read.
(7) **line 190** — Drift-free fixed 40 ms period.

Threading and safety

- Runs on its own task at priority DEFAULT - 1, below opcontrol and autonomous, so an overrun can never starve the Brain
- delay_until gives a drift-free 40 ms period; an overrun guard always sleeps at least 5 ms
- setPose() posts a request that the task applies at the top of its next cycle, so all filter access stays on one thread
- start() is non-blocking: IMU wait is bounded to 5 s so a dead IMU cannot park the task

Source: 4613B/documentation/03-monte-carlo-localization.md section 3.9

Odometry Delta

src/localization/localization.cpp — odometry delta + distance-sensor gating

```
256     OdomDelta Localizer::computeDelta() {
257         // Rotation sensor positions accumulate (wrapping handled by int32 arithmetic)
258         const int32_t fwd_now    = _fwd_enc.get_position(); // centidegrees
259 (1)     const int32_t d_fwd_cdeg = fwd_now - _last_fwd_cdeg;
260         _last_fwd_cdeg = fwd_now;
261
262         // Convert centidegrees → inches of arc
263         const float TWO_PI_OVER_CDEG = TWO_PI / CDEG_PER_REV;
264         float df = static_cast<float>(d_fwd_cdeg) * TWO_PI_OVER_CDEG
265                  * _hw.fwd_wheel_radius_in;
266         float dl = 0.0f; // stays exactly 0 with no horizontal wheel — see below
267
268         if (_hor_enc) {
269             const int32_t hor_now    = _hor_enc->get_position();
270             const int32_t d_hor_cdeg = hor_now - _last_hor_cdeg;
271             _last_hor_cdeg = hor_now;
272             dl = static_cast<float>(d_hor_cdeg) * TWO_PI_OVER_CDEG
273                  * _hw.hor_wheel_radius_in;
274         }
275
276         // Heading from IMU (degrees, CW from zeroed position → radians)
277         const float heading_now_rad =
278 (2)         normalizeAngle(static_cast<float>(_imu.get_heading()) * (PI / 180.0f));
279         float dh = normalizeAngle(heading_now_rad - _last_heading_rad);
280         _last_heading_rad = heading_now_rad;
281
282         // Correct forward displacement for heading arc (standard odom formula).
283         // If the tracking wheel has a lateral offset, it sweeps an arc during rotation.
284         // Skip the lateral correction entirely with no horizontal wheel: otherwise
285         // a nonzero hor_offset_in alone would synthesize a phantom dl out of thin air.
286         //
287         // SIGN: the offsets are calibrated by spinning in place and reading the
288         // SAME encoder (robot_config.hpp §2: offset = D x r / 360000), so in a
289         // pure rotation the wheel measures exactly offset x dh — that term must
290         // be SUBTRACTED to recover the turning centre's motion. Check: pure spin
291         // -> r_fwd == offset -> (r_fwd - offset) == 0 -> df == 0, as it must.
292         // The original `+` doubled the arc instead of cancelling it.
293         if (std::abs(dh) > 1e-6f) {
294 (3)         const float r_fwd = df / dh;          // radius of forward wheel arc
```

Computing the movement

- Heading comes from the IMU, not from the difference of wheel encoders, because wheel difference compounds tiny errors
- Forward distance: delta centidegrees x 2 pi / 36000 x wheel radius
- Arc correction for an offset wheel: chord = 2 (r - offset) sin(dh / 2); a pure spin gives zero forward motion as it must (an earlier version used + and doubled the arc - fixed)

Source: 4613B/src/localization/localization.cpp computeDelta(), documentation/03-monte-carlo-localization.md section 3.3

Sensor Gating

src/localization/localization.cpp — odometry delta + distance-sensor gating

```
295             df = 2.0f * (r_fwd - _hw.fwd_offset_in) * std::sin(dh * 0.5f);
296             if (_hor_enc) {
297                 const float r_hor = dl / dh;       // radius of lateral wheel arc
298                 dl = 2.0f * (r_hor - _hw.hor_offset_in) * std::sin(dh * 0.5f);
299             }
300         }
301
302         return {df, dl, dh};
303     }
304
305     // ── Sensor reading ──────────────────────────────────────────────────
306     // Each raw reading is corrected and quality-gated before the filter sees it:
307     //   • the sensor's calibrated base reading (mount recession + zero bias,
308     //     SensorMount::base_mm) is subtracted
309     //   • beyond max_range_mm (real V5 distance sensor tops out ~2 m) → invalid
310     //   • sensor's own confidence (0–63) below min_distance_confidence → invalid
311     // Invalid readings are reported as 9999, which logLikelihood() skips.
312     SensorData Localizer::readSensors() const {
313 (4)     const auto gated = [this](pros::Distance& d, const SensorMount& m) -> int32_t {
314             const int32_t raw = d.get();
315 (5)         if (raw <= 0 || raw >= 9000) return 9999;   // PROS "no object" / error
316             const int32_t mm = raw - static_cast<int32_t>(m.base_mm);
317             if (mm <= 0) return 9999;
318             if (mm >= static_cast<int32_t>(_mcl_cfg.max_range_mm)) return 9999;
319             const std::int32_t c = d.get_confidence();
320             if (c >= 0 && c < _mcl_cfg.min_distance_confidence) return 9999;
321             return mm;
322         };
323         const SensorMounts& mts = _mcl_cfg.mounts;
324         SensorData s;
325         s.front_mm = gated(_front_dist, mts.front);
326         s.back_mm  = gated(_back_dist,  mts.back);
327         s.left_mm  = gated(_left_dist,  mts.left);
328         s.right_mm = gated(_right_dist, mts.right);
329         return s;
330     }
331
332     } // namespace lillet
```

1. **line 259** — Tracking wheel centidegrees to inches of arc.
2. **line 278** — Heading comes from the IMU, not wheel differences.
3. **line 294** — Arc-to-chord correction removing the tracking wheel offset arc.
4. **line 313** — Each reading is bias-corrected (base_mm) and range/confidence gated.
5. **line 315** — Invalid readings are reported as 9999 and skipped by the filter.

Cleaning readings before the filter sees them

- Subtract the per-sensor calibrated base reading
- Discard anything beyond 1900 mm, where the V5 distance sensor is unreliable
- Discard readings with sensor confidence below 45 out of 63
- Invalid readings (9999) are skipped, never guessed

Source: 4613B/src/localization/localization.cpp readSensors(), documentation/03-monte-carlo-localization.md section 3.8

Motion Model

src/localization/particle_filter.cpp — Monte Carlo STEP 1: predict (Thrun odometry motion model)

```
37      void ParticleFilter::predict(const OdomDelta& delta) {
38          const float abs_f = std::abs(delta.forward);
39          const float abs_l = std::abs(delta.lateral);
40          const float abs_h = std::abs(delta.heading);
41          const float trans = std::sqrt(abs_f * abs_f + abs_l * abs_l);
42
43  (1)     const float sigma_h = std::sqrt(_cfg.alpha1 * abs_h * abs_h +
44                                          _cfg.alpha2 * trans * trans);
45  (2)     const float sigma_t = std::sqrt(_cfg.alpha3 * trans * trans +
46                                          _cfg.alpha4 * abs_h * abs_h);
47          const float sigma_l = std::sqrt(_cfg.alpha3 * abs_l * abs_l +
48                                          _cfg.alpha4 * abs_h * abs_h);
49
50  (3)     for (auto& p : _particles) {
51              const float dh = delta.heading + sampleN(0.0f, sigma_h);
52              const float df = delta.forward + sampleN(0.0f, sigma_t);
53              const float dl = delta.lateral + sampleN(0.0f, sigma_l);
54
55              // Apply in robot-local frame at the particle's own heading.
56  (4)         const float theta_mid = p.pose.theta + dh * 0.5f;
57              const float s = std::sin(theta_mid);
58              const float c = std::cos(theta_mid);
59
60              // CW positive, 0 = north: forward = (sin,cos), right = (cos,-sin)
61  (5)         p.pose.x     += df * s + dl * c;
62              p.pose.y     += df * c - dl * s;
63              p.pose.theta  = normalizeAngle(p.pose.theta + dh);
64          }
65      }
```

1. **line 43** — Heading noise sigma = sqrt(a1*dh^2 + a2*trans^2).
2. **line 45** — Translation noise sigma = sqrt(a3*trans^2 + a4*dh^2).
3. **line 50** — EVERY particle gets its own independent noise sample — that is what spreads the cloud.
4. **line 56** — Mid-point heading = second-order arc approximation.
5. **line 61** — Apply the noisy motion in world frame (CW-positive, 0 = north).

Predict step

- Each particle moves by the odometry delta plus its own Gaussian error, so the cloud spreads like real drift
- sigma_heading = sqrt(a1 dh^2 + a2 trans^2); sigma_trans = sqrt(a3 trans^2 + a4 dh^2)
- Midpoint heading gives a second-order arc

Tuned values

- a1 0.025, a2 0.0025, a3 0.05, a4 0.05
- Worked example: 1 in with no rotation gives sigma = sqrt(0.05) = 0.22 in per step
- Larger alpha = trust odometry less; smaller = tighter cloud

Source: 4613B/src/localization/particle_filter.cpp predict(), documentation/03-monte-carlo-localization.md section 3.4

Sensor Model

src/localization/particle_filter.cpp — Monte Carlo STEP 2: update (sensor likelihood)

```
67      // ── Measurement model ──────────────────────────────────────────────
68      // Each sensor reading is modelled as:
69  (1) //   p(z|x) = z_hit * Gaussian(z; z_exp, σ) + z_rand * Uniform(0, MAX_RANGE)
70      // Readings that appear to be occluded by a dynamic obstacle (measured << expected)
71      // receive a reduced but non-zero likelihood so they don't kill good particles.
72      //
73      // Two-pass log-sum-exp: compute every particle's log-likelihood first, shift
74      // by the max, then exponentiate. The best particle always gets factor 1, so
75      // 32-bit float weights can never underflow no matter how badly the cloud
76      // mismatches — and relative weighting is mathematically unchanged (the common
77      // exp(-maxLL) factor cancels in normalisation).
78      void ParticleFilter::update(const SensorData& z) {
79          const int n = static_cast<int>(_particles.size());
80          _ll_buf.resize(n);
81
82          int n_valid = 0;
83          const auto isValid = [](int32_t v) {
84              return v > 0 && v < static_cast<int32_t>(FieldModel::MAX_RANGE_MM);
85          };
86          if (isValid(z.front_mm)) ++n_valid;
87          if (isValid(z.back_mm))  ++n_valid;
88          if (isValid(z.left_mm))  ++n_valid;
89          if (isValid(z.right_mm)) ++n_valid;
90
91  (2)     float max_ll = -1e30f;
92          for (int i = 0; i < n; ++i) {
93              const float ll = logLikelihood(_particles[i].pose, z);
94              _ll_buf[i] = ll;
95              if (ll > max_ll) max_ll = ll;
96          }
97
98          // Collapse = even the BEST particle scored at ~the uniform floor
99          // (log(z_rand / MAX_RANGE) ≈ -10.5) on every valid sensor: nothing in the
100         // cloud explains the readings. The caller publishes confidence 0.
101 (3)     _collapsed = n_valid > 0 && max_ll < static_cast<float>(n_valid) * -9.5f;
102
103         const float shift = (max_ll > -1e30f) ? max_ll : 0.0f;
104         for (int i = 0; i < n; ++i)
105 (4)         _particles[i].weight *= std::exp(_ll_buf[i] - shift);
106
107         _fresh_from = n;   // everyone has now been weighted by evidence
108 (5)     normalizeWeights();
109     }
110
111     // ── Systematic resampling + diversity injection ────────────────────
112     void ParticleFilter::resample(float heading_hint_rad) {
```

1. **line 69** — Mixture model: Gaussian hit + uniform random floor.
2. **line 91** — Pass 1: compute every particle log-likelihood and the max.
3. **line 101** — Collapse detect: even the best particle sits at the uniform floor, so the filter is lost.
4. **line 105** — Pass 2: log-sum-exp shift so float weights can never underflow.
5. **line 108** — Weights renormalised to sum to 1.

Weighting each particle

- Ray-cast the field from each particle to find what each sensor would read
- Gaussian hit (sigma 80 mm + 1.5 mm per in/s of speed) mixed with a uniform floor (0.75 / 0.25)
- If measured is 300 mm shorter than expected, assume another robot and use mix 0.15 / 0.85

Why it is built this way

- Log-likelihood with the best particle shifted to factor 1 (two-pass log-sum-exp) so 32-bit weights never underflow
- A 40 mm error beats a 300 mm error by about x107 per sensor; four sensors multiply
- Collapse detection publishes confidence 0 when no particle explains the readings

Source: 4613B/src/localization/particle_filter.cpp update(), documentation/03-monte-carlo-localization.md section 3.5

Resampling

src/localization/particle_filter.cpp — Monte Carlo STEP 3: resample, best estimate, confidence

```cpp
// ── Systematic resampling + diversity injection ──────────────────────
void ParticleFilter::resample(float heading_hint_rad) {
    const int N      = static_cast<int>(_particles.size());
    const int n_rand = static_cast<int>(std::round(_cfg.random_inject_frac * N));
    const int n_keep = N - n_rand;

    // Build CDF
    std::vector<float> cdf(N);
    cdf[0] = _particles[0].weight;
    for (int i = 1; i < N; ++i)
        cdf[i] = cdf[i - 1] + _particles[i].weight;

    // Systematic sampling of n_keep particles
    std::vector<Particle> resampled;
    resampled.reserve(N);
    const float step  = cdf[N - 1] / static_cast<float>(n_keep);
    float       pos   = _uniform01(_rng) * step;
    int         j     = 0;
    for (int i = 0; i < n_keep; ++i) {
        while (pos > cdf[j] && j < N - 1) ++j;
        resampled.push_back(_particles[j]);
        resampled.back().weight = 1.0f / N;
        pos += step;
    }

    // Inject random particles for diversity (guards against kidnapping / drift).
    // Full weight so they compete fairly in the NEXT update/resample — that
    // competition is exactly how kidnap recovery works — but bestEstimate()
    // excludes them until then (see _fresh_from): at uniform weight, 10
    // field-wide randoms drag the weighted mean ~5% toward field centre, a
    // constant ~3.4" bias when the robot sits in a corner.
    for (int i = 0; i < n_rand; ++i) {
        Particle rp;
        rp.pose   = randomPose(heading_hint_rad);
        rp.weight = 1.0f / N;
        resampled.push_back(rp);
    }

    _particles  = std::move(resampled);
```

Systematic resampling

- Build the weight CDF, take one random offset, step evenly: heavy particles are copied, light ones vanish
- Only one random number is used, so variance is low; weights reset to 1/N

Random injection

- 10 percent (20 of 200) particles are re-spawned over the field, heading about IMU +/- 0.05 rad
- This is the kidnap / wrong-mirror insurance
- Fresh injections do not vote in the estimate until their first update - including them biased the pose toward the centre by about 3.4 in

Estimate and confidence

- Position: weighted mean; heading: circular mean atan2(sum w sin, sum w cos)
- Confidence: effective sample size Neff / N, measured after update and before resample

Source: 4613B/src/localization/particle_filter.cpp resample(), documentation/03-monte-carlo-localization.md sections 3.6-3.7

Field Model Ray Casting

src/localization/field_model.cpp — ray casting the field (what each sensor SHOULD read)

```cpp
float FieldModel::raycast(float ox, float oy, float dx, float dy) const {
    // Work in inches internally; convert to mm at the end.
    float min_t_in = MAX_RANGE_MM * INCH_PER_MM;

    // Perimeter walls (inner face at ±FIELD_HALF_IN)
    if (dy >  1e-6f) min_t_in = std::min(min_t_in, ( FIELD_HALF_IN - oy) / dy);
    if (dy < -1e-6f) min_t_in = std::min(min_t_in, (-FIELD_HALF_IN - oy) / dy);
    if (dx >  1e-6f) min_t_in = std::min(min_t_in, ( FIELD_HALF_IN - ox) / dx);
    if (dx < -1e-6f) min_t_in = std::min(min_t_in, (-FIELD_HALF_IN - ox) / dx);

    // Additional fixed field elements
    for (const auto& obs : _obstacles) {
        float t = raycastRect(obs, ox, oy, dx, dy);
        if (t > 1e-4f) min_t_in = std::min(min_t_in, t);
    }

    return min_t_in * MM_PER_INCH;
}

// One sensor's expected reading with real mounting geometry.
// nominal_rad: sensor facing relative to robot forward (front 0, right +π/2,
// back π, left -π/2), CW positive. The mount position uses the NOMINAL frame
// (yaw only tilts the beam, it doesn't move the glass).
float FieldModel::castMount(const Pose& p, float nominal_rad, const SensorMount& m,
                            float default_forward_in) const {
    const float fwd = (m.forward_in >= 0.0f) ? m.forward_in : default_forward_in;

    // CW-positive, 0 = north: unit vector at world angle a = (sin a, cos a)
    const float af = p.theta + nominal_rad;          // facing (mount frame)
    const float ux = std::sin(af), uy = std::cos(af);
    const float rx = std::cos(af), ry = -std::sin(af); // 90° CW of facing

    const float ad = af + m.yaw_deg * (PI / 180.0f); // actual beam direction
    return raycast(p.x + fwd * ux + m.lateral_in * rx,
                   p.y + fwd * uy + m.lateral_in * ry,
                   std::sin(ad), std::cos(ad));
}

SensorData FieldModel::expectedReadings(const Pose& p, const SensorMounts& mounts,
                                        float robot_half_in) const {
    SensorData out;
```

The map

- Override field with inner half-width 71 in
- Perimeter walls: 4 analytic planes at +/-71 in
- Goals: 9 boxes of 10 x 10 in; loaders: 4 protrusions of 16.8 x 4 in; toggles: 4 protrusions of 24 x 6 in

Source: 4613B/src/localization/field_model.cpp, documentation/03-monte-carlo-localization.md section 3.8

Ray Casting Method

src/localization/field_model.cpp — ray casting the field (what each sensor SHOULD read)

```
99          out.front_mm = static_cast<int32_t>(castMount(p, 0.0f,        mounts.front, robot_half_in));
100         out.back_mm  = static_cast<int32_t>(castMount(p, PI,          mounts.back,  robot_half_in));
101         out.right_mm = static_cast<int32_t>(castMount(p, PI * 0.5f,   mounts.right, robot_half_in));
102         out.left_mm  = static_cast<int32_t>(castMount(p, -PI * 0.5f,  mounts.left,  robot_half_in));
103         return out;
104     }
105
106     SensorData FieldModel::expectedReadings(const Pose& p, float robot_half_in) const {
107         // Ideal mounts: face centre, perfectly aimed, no bias.
108         return expectedReadings(p, SensorMounts{}, robot_half_in);
109     }
110
111 5   // Slab-method AABB intersection.
112     // Returns distance in inches to the entry face, or -1 if no forward hit.
113     float FieldModel::raycastRect(const Rect& r, float ox, float oy,
114                                   float dx, float dy) {
115         float t_min = -1e18f, t_max = 1e18f;
116
117         if (std::abs(dx) > 1e-9f) {
118             float t1 = (r.x_min - ox) / dx;
119             float t2 = (r.x_max - ox) / dx;
120             if (t1 > t2) std::swap(t1, t2);
121             t_min = std::max(t_min, t1);
122             t_max = std::min(t_max, t2);
123         } else if (ox < r.x_min || ox > r.x_max) {
124             return -1.0f;
125         }
126
127         if (std::abs(dy) > 1e-9f) {
128             float t1 = (r.y_min - oy) / dy;
129             float t2 = (r.y_max - oy) / dy;
130             if (t1 > t2) std::swap(t1, t2);
131             t_min = std::max(t_min, t1);
132             t_max = std::min(t_max, t2);
133         } else if (oy < r.y_min || oy > r.y_max) {
134             return -1.0f;
135         }
136
137         if (t_max < t_min || t_max < 1e-4f) return -1.0f;
138         return (t_min > 1e-4f) ? t_min : t_max;
139     }
```

1. **line 60** — Start from max range.
2. **line 63** — Four perimeter walls solved analytically.
3. **line 70** — Then every fixed obstacle (goals, loaders, toggles).
4. **line 81** — Per-sensor mounting offset/yaw/aim handled here.
5. **line 111** — Slab-method AABB intersection.

Slab method

- Each sensor ray starts at the sensor glass position and is intersected with the walls and every box
- Per axis compute entry and exit parameters; take the latest entry and earliest exit; a hit exists if t_exit >= t_enter
- About 200 particles x 4 rays x 21 surfaces = 16 800 tests per correction, fine at 25 Hz on the V5

Source: 4613B/src/localization/field_model.cpp castMount(), documentation/03-monte-carlo-localization.md section 3.8

Tuned Filter Parameters

include/robot_config.hpp — Monte Carlo tuning values (Auto-Tuned in the simulator)

```
240     inline lillet::MCLConfig makeMclConfig() {
241         lillet::MCLConfig mcl;
242
243         // — Particle cloud —
244 (1)     mcl.num_particles      = 200;    // hypotheses. More = smoother but more CPU
245                                          // (raycast cost is linear in this). 150–300.
246 (2)     mcl.random_inject_frac = 0.10f;  // fraction re-spawned randomly per resample.
247                                          // Too low → can't recover from kidnap;
248                                          // too high → estimate gets noisy.
249         mcl.random_inject_heading_stddev_rad = 0.05f; // injected headings ~ IMU ±3°
250
251         // — Motion model noise (Thrun α) — how much the filter distrusts odometry —
252 (3)     mcl.alpha1 = 0.025f;  // heading error   ∝ rotation
253         mcl.alpha2 = 0.0025f; // heading error   ∝ translation
254         mcl.alpha3 = 0.05f;   // translation err ∝ translation
255         mcl.alpha4 = 0.05f;   // translation err ∝ rotation
256         // Symptoms: cloud too tight & ignores real bumps → raise α3/α4.
257         //           estimate wanders while driving straight → lower α3.
258
259         // — Sensor likelihood model —
260 (4)     mcl.sigma_hit_mm           = 80.0f; // Gaussian σ for wall hits. Too low →
261                                             // overconfident, rejects good readings;
262                                             // too high → barely learns from sensors.
263         mcl.sigma_speed_mm_per_ips = 1.5f;  // extra σ per in/s (reading staleness)
264         mcl.z_hit  = 0.75f;                 // Gaussian mixture weight
265         mcl.z_rand = 0.25f;                 // uniform mixture weight (robustness floor)
266
267         // — Real V5 distance-sensor limits (readings failing these are discarded) —
268 (5)     mcl.max_range_mm            = 1900.0f; // sensor is unreliable near its 2 m limit
269         mcl.min_distance_confidence = 45;      // get_confidence() 0–63; raise if the
270                                                // filter twitches near other robots,
271                                                // lower if too few readings survive
272
273         // — Occlusion gate —
274 (6)     mcl.occlusion_thresh_mm = 300.0f; // reading ≥ this much SHORTER than expected
275                                           // = another robot blocking the sensor →
276                                           // penalised gently instead of nuking weights
277
278         // — Sensor mounting geometry (constants in §4b above) —
279         mcl.robot_half_in = ROBOT_HALF_IN; // default glass distance; wrong value =
280                                            // constant offset on EVERY comparison.
281         mcl.mounts.front = { DIST_FRONT_FWD_IN, DIST_FRONT_LAT_IN,
282                              DIST_FRONT_YAW_DEG, DIST_FRONT_BASE_MM };
283         mcl.mounts.back  = { DIST_BACK_FWD_IN,  DIST_BACK_LAT_IN,
284                              DIST_BACK_YAW_DEG,  DIST_BACK_BASE_MM };
285         mcl.mounts.left  = { DIST_LEFT_FWD_IN,  DIST_LEFT_LAT_IN,
286                              DIST_LEFT_YAW_DEG,  DIST_LEFT_BASE_MM };
287         mcl.mounts.right = { DIST_RIGHT_FWD_IN, DIST_RIGHT_LAT_IN,
288                              DIST_RIGHT_YAW_DEG, DIST_RIGHT_BASE_MM };
289
290         // — Odom-primary correction gating —
291 (7)     mcl.sensor_update_dist_in = 8.0f;  // correct after this many inches of travel
292         mcl.neff_update_thresh    = 0.30f; // …or when confidence drops below this
293         mcl.neff_min_cycles       = 3;     // min cycles (~120 ms) between Neff-forced
294                                            // corrections (fights correlated updates)
295         return mcl;
296     }
```

(1) **line 244** — 200 particles.
(2) **line 246** — 10% random injection.
(3) **line 252** — Thrun alpha1-alpha4 motion-noise terms.
(4) **line 260** — sigma_hit = 80 mm sensor noise model.
(5) **line 268** — 1900 mm usable range of the V5 distance sensor.
(6) **line 274** — 300 mm occlusion gate.
(7) **line 291** — Correct every 8 in or when Neff < 0.30.

Values

- 200 particles; 10 percent random injection; update after 8 in of travel
- sigma_hit 80 mm; z_hit 0.75 / z_rand 0.25; max range 1900 mm; min sensor confidence 45
- Neff threshold 0.30 with at least 3 cycles between corrections

Where they came from

- Found with the simulator Auto Tune (hill climbing, scored on rollouts with encoder noise and the 2 m sensor cap)
- The comment in robot_config.hpp records 0.22 in mean error in simulation (4 July 2026)
- No on-robot accuracy measurement is stored in the repository

Source: 4613B/include/robot_config.hpp makeMclConfig(), documentation/03-monte-carlo-localization.md section 3.10

Code Status and Limits

Part	Status	Evidence in the repository
Driver control, DR4B, claw	Running on the robot	src/main.cpp, dr4b.hpp, claw_controller.hpp
LemLib PID drive and odometry	Running on the robot	main.cpp chassis, controllers
Autonomous routines + selector	Running on the robot	autonRedPreload, autonExamplePath, selector
Monte Carlo localiser (lillet)	Built, simulated, unit tested; not constructed in the current main.cpp	20 simulator tests; 0.22 in mean error in simulation
Simulator and path planner	Used to design routes	Sim.html, sim_tests.mjs

Known behaviours and open items

- Because every autonomous helper is relative to current odometry, errors accumulate through the routine; the localiser is the planned fix
- Two sensor-placement constants (front forward -5.25 in and back forward 0.0 in) are read in a way that may not match the intended 5.25 in and 8 in; this has no effect while the filter is not running and is to be checked before enabling it
- Lift targets are absolute degrees from the zero set in initialize(), so the arm must be folded down at power-up

What would be needed to enable it

- Construct lillet::Localizer and the lillet Chassis in main.cpp
- Re-calibrate the four distance sensors (*_BASE_MM) and correct the two placement constants
- Replace waitUnlessStalled isInMotion() with isMoving()

Source: 4613B/documentation/03 section 3.12, 02 section 2.4, 06 section 6.7, 00-what-it-does.md

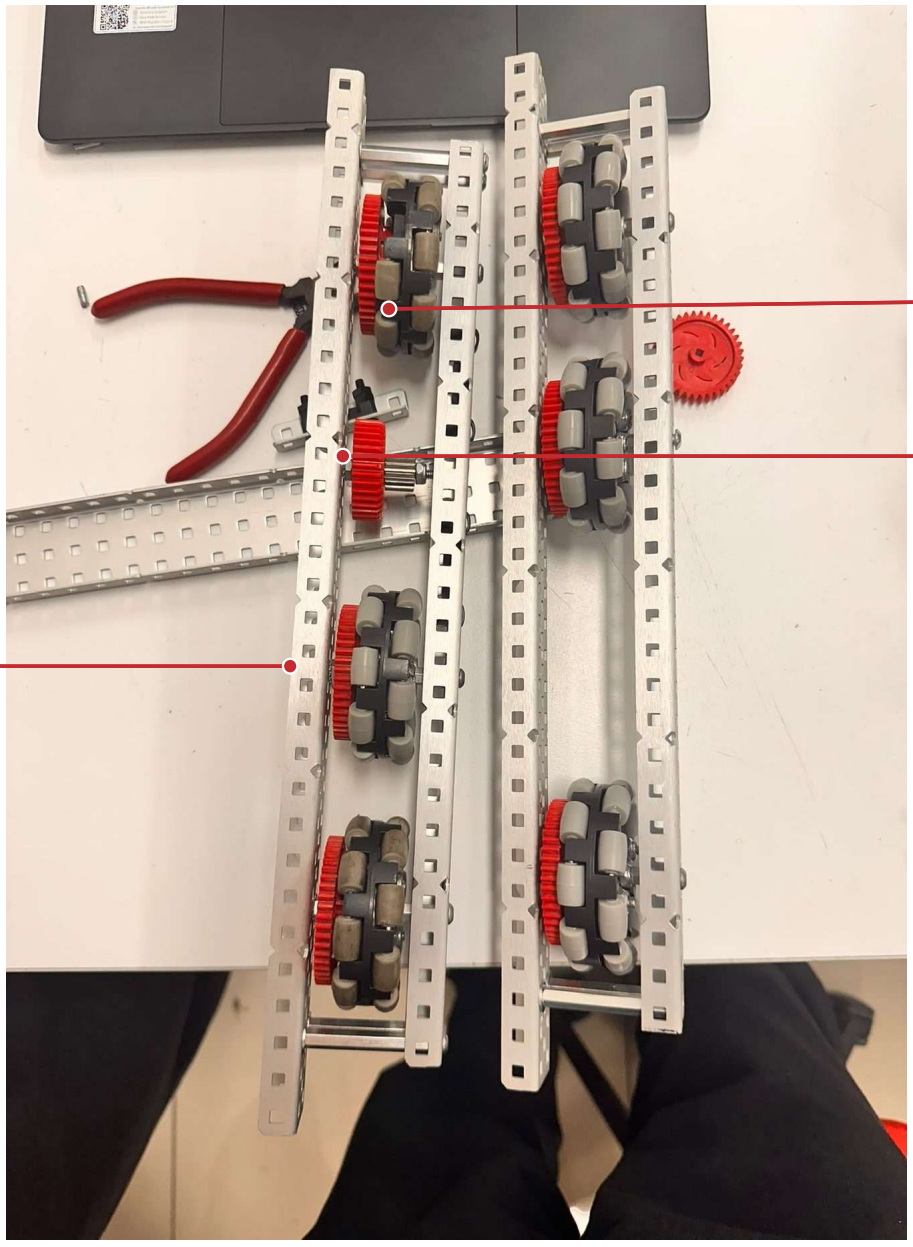
Build Log

Regionals

Drivetrain Build

July 6th 2026

6 July (holidays): built both drive rails, 36:48 gearing + vertical motor layout. Ryan + Josh: build. Brandon: PROS project. Rio: photos + log.



Drivetrain Build

July 8th 2026

8 July: cross-braces + wiring done; first drive. Ryan: fixed wheel rub (spacer). Josh: began DR4B. Brandon: tank control test. Rio: schedule page.



Lessons – T3W2

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 27 Jul	Kick-off of RoboDev task. Re-read Robot Skills rules (RSC1–RSC5) and set team roles. Started Gantt chart. DR4B C-channel prototype assembled to 70%.	Brandon: summarised RSC rules. Ryan: DR4B prototype linkages. Josh: began intake sketch pages. Rio: Gantt chart draft + roles table.
Wed 29 Jul	Robot Skills breakdown: scoring table, Toggle ownership and Midfield rules. Static rubber-band roller intake mounted for first ground test – jammed on pins (8/20).	Rio: scoring summary + worked examples. Josh: mounted static roller, ran first pickup test. Ryan: finished DR4B prototype. Brandon: robot_config.hpp port map.
Fri 31 Jul	Finalised Skills strategy (prioritise yellow Pins, set red Toggles, end in Midfield). DR4B moved through full stroke; checked 50" height limit with tape measure (47").	Rio + Brandon: strategy page. Ryan: DR4B stroke + height check. Josh: analysed why static roller pushed objects into tiles (downforce).

Lessons – T3W3

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 3 Aug	Planned design: top & side schematic drawn with all motors/sensors labelled. Researched vertical vs horizontal rollers (606X, 94999E, Purdue SIGBots wiki).	Josh: planned design drawing + research citations. Ryan: chassis cross-brace check. Rio: chassis research summary. Brandon: driver-control code.
Wed 5 Aug	Built pivoting intake with 2" flex wheels (iteration 2). Pickup improved to 15/20. Driver control with expo curve tested on the field.	Josh: pivot intake build. Ryan: pivot bearing blocks + spacing. Brandon: expo curve + button map. Rio: ran 20-object pickup test, logged results.
Fri 7 Aug	Added ground kicker roller ~1 cm below ramp (iteration 3). Pickup 19/20 Pins, 18/20 Cups. DR4B redesigned in L-channel to save mass.	Josh: kicker roller. Ryan: L-channel DR4B rebuild. Rio: test log + photos. Brandon: drive straight test (96") – found 2.3" drift.

Lessons – T3W4

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 10 Aug	Combined DR4B with chassis. Started transfer roller – first mounted on intake pivot axis; CAD showed pivot height would rise, so redesign planned.	Ryan: DR4B-to-chassis mounting. Josh: transfer roller CAD + pivot analysis. Rio: weighted decision matrix (chassis). Brandon: odometry task (10 ms).
Wed 12 Aug	Transfer roller decoupled from pivot onto elevated section. Chain run added to synchronise intake, kicker and transfer rollers with a banded tensioner.	Josh: chain path + tensioner. Ryan: elevated transfer mounts. Brandon: IMU heading + arc-chord correction coded. Rio: intake decision matrix.
Fri 14 Aug	Transfer alignment test 16/20 → raised roller 6 mm → 19/20. Began best-solution pages (final views, justification).	Josh + Rio: transfer test. Ryan: transfer height change. Brandon: tracking-wheel radius calibration (48" roll test).

Lessons – T3W5

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 17 Aug	Gravity ratchet concept built: sun gear + free planet gear on bent polycarbonate guide. Direction of motor selects lift or intake.	Ryan: ratchet gearbox. Josh: polycarbonate guide bending. Brandon: PID class + anti-windup. Rio: end effector prototype 1 (standard claw) test.
Wed 19 Aug	Ratchet switching test 27/30 correct; re-bent guide for more friction. End effector prototype 2 (powered horizontal roller) built – too wide for transfer.	Ryan + Josh: ratchet guide re-bend. Rio: prototype 2 build + pros/cons. Brandon: lift PID hold tuned (kP 0.4, kD −0.02).
Fri 21 Aug	End effector prototype 3 (two parallel flex rollers, 1 × 5.5 W) built and selected. Week 5 checkpoint feedback received – need more annotation on planned design.	Rio: prototype 3 + effector decision matrix. Josh: annotations added to drawings. Ryan: build plan table. Brandon: checkpoint notes.

Lessons – T3W6

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 24 Aug	Motor integration: 6 drive, 2 ratchet, 1 effector, 1 intake-reverse motor fitted (88 W total). Port map confirmed against config file.	Ryan: motor mounting + gear meshes. Brandon: port map + config. Josh: motor guard plates. Rio: robot components table.
Wed 26 Aug	Sensors fitted: IMU at geometric centre, tracking wheel + rotation sensor, 4 distance sensors F/B/L/R. Wiring diagram drawn.	Brandon + Ryan: wiring diagram + cable routing. Josh: distance sensor brackets. Rio: sensor justification notes.
Fri 28 Aug	Pneumatic tank relocated onto intake (extra down-force). Air-usage model checked (≈120 usable actuations). First full driver test of whole robot.	Ryan: tank relocation. Rio: air-usage calculation + driver test. Josh: tube routing. Brandon: particle filter predict step.

Lessons – T3W7

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 31 Aug	Stand-offs loosening after 50 intake cycles (3 found loose) – added high-strength collars on flat faces. Intake now driven from both sides.	Josh: collars + dual-side drive. Ryan: re-tensioned chain. Rio: loosening test log. Brandon: ray-cast field model coded.
Wed 2 Sep	Testing day: drive speed (96" in 1.9 s), turning accuracy, intake reliability. Results recorded in test tables.	Rio: ran tests + recorded data. Brandon: logged turn error data. Ryan: motor temperature checks. Josh: intake repeat test.
Fri 4 Sep	Fixed turn oscillation at ±180° by locking turn direction (locked-sign turn). DR4B rubber bands re-tensioned so lift holds at mid height.	Brandon: locked-sign turn code. Ryan: rubber-band assist. Rio + Josh: retest turns (10 × 90°).

Lessons – T3W8

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 7 Sep	Autonomous diagram drawn on field template (9 steps). Autonomous logic written as numbered steps + flowchart; score calculated (103 max).	Rio + Brandon: auton diagram + logic. Josh: checked goal heights for each step. Ryan: end effector hard-stop tuning.
Wed 9 Sep	AMCL resampling + Neff trigger running on robot. Localisation test: odometry-only end error 3.1" vs AMCL 0.9" over a 3 m path.	Brandon: resampling + test. Rio: recorded localisation data. Ryan + Josh: GPS strip check (required for Auton Skills, RSC4).
Fri 11 Sep	Week 8 checkpoint feedback. First half of skills auton (steps 1–5) running in simulator; exported to robot.	Brandon: simulator path + export. Rio: checkpoint actions list. Josh: cup-nesting test (clear side). Ryan: fixed wire pinch on DR4B.

Lessons – T3W9

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 14 Sep	Boomerang moveToPose tuned (lead 0.5 → 0.6). Steps 1–5 succeed 3/5 runs on field. Possession test: never held 2 Pins after top-plate stop added.	Brandon: tuning. Josh: top-plate stop on intake. Rio: possession test (20 runs). Ryan: effector roller compression.
Wed 16 Sep	Steps 6–9 added (bottom Toggle, Red Goal B, Tall Goal, Midfield park). First full 60 s run: 63 pts (Pin dropped at Tall Goal).	Brandon: full routine code. Rio: skills run log. Ryan: lift height preset for Tall Goal. Josh: Toggle push plate.
Fri 18 Sep	Effector release time 250 → 350 ms fixed Tall Goal drop. Runs: 78, 83, 83. Driver skills practice (best 112).	Brandon: timing fix. Rio: driver practice + log. Ryan + Josh: field reset + timing.

Lessons – T3W10

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 21 Sep	Battery test: auton scores same at 60 % and 100 % charge. Added time guard so robot always parks in Midfield before 59 s.	Brandon: time guard. Rio: battery test. Ryan: battery strap. Josh: cable tidy.
Wed 23 Sep	Runs: 98, 83, 103 (first full-score run). Testing pages completed for each subsystem with adjustments listed.	Rio: test pages. Brandon: run log. Ryan: checked gear wear. Josh: photos for notebook.
Fri 25 Sep	Presentation planning: split 10-minute talk so each member presents one section. Started transcript.	Ryan: robot design section. Rio: strategy + teamwork. Brandon: programming. Josh: highlights, challenges, wins.

Lessons – T4W1

Lesson	Tasks worked on + progress achieved	Team member contributions
Mon 12 Oct	Post-holiday check: re-calibrated tracking wheel and IMU, re-ran auton (88, 103, 103). Notebook cross-checked against marking criteria.	Brandon: calibration + runs. Rio: criteria checklist. Ryan: inspection (size, 88 W, pneumatics). Josh: notebook layout.
Wed 14 Oct (planned)	Final presentation rehearsal (timed < 10 min). Final testing summary + individual commitment pages completed.	All: rehearsal. Rio: test summary. Brandon: final code snapshot. Ryan + Josh: robot tidy for assessment.
Fri 16 Oct (planned)	Submission day: notebook (Part 1) and presentation + transcript (Part 2) uploaded; robot assessed in class (Part 3).	All members: submission + robot demonstration.

Code Journal 1/5

T3W2 · 27 Jul – robot_config.hpp – one file for every port and constant

Added since last update: first code commit for this task. All ports live in one header so changing wiring never means editing several files.

```
// robot_config.hpp  (T3W2) — single source of truth
namespace cfg {
constexpr int PORT_DRIVE_LF =  1, PORT_DRIVE_LM =  2, PORT_DRIVE_LB =  3;
constexpr int PORT_DRIVE_RF = -4, PORT_DRIVE_RM = -5, PORT_DRIVE_RB = -6; // reversed
constexpr int PORT_RATCHET_A = 7, PORT_RATCHET_B = 8;   // gravity ratchet
constexpr int PORT_EFFECTOR  = 9, PORT_INTAKE_REV = 10;
constexpr int PORT_IMU = 11, PORT_FWD_ENCODER = 12;
constexpr int PORT_DIST_FRONT = 13, PORT_DIST_BACK = 14,
              PORT_DIST_LEFT  = 15, PORT_DIST_RIGHT = 16;
constexpr char  PORT_PISTON   = 'A';                    // 3-wire
constexpr float WHEEL_DIAM_IN = 2.75f;                  // omni
constexpr float DRIVE_RPM     = 450.0f;                 // 600 x 36/48
}
```

T3W3 · 3 Aug – Tank driver control + gravity-ratchet buttons

Added since last update: expo curve so small stick movements are gentle; R1 spins the ratchet one way (intake), L1 the other way (lift). Releasing L1 lets the DR4B lower under gravity.

```
pros::MotorGroup left ({cfg::PORT_DRIVE_LF, cfg::PORT_DRIVE_LM, cfg::PORT_DRIVE_LB});
pros::MotorGroup right({cfg::PORT_DRIVE_RF, cfg::PORT_DRIVE_RM, cfg::PORT_DRIVE_RB});
pros::MotorGroup ratchet({cfg::PORT_RATCHET_A, cfg::PORT_RATCHET_B});

void opcontrol() {
  while (true) {
    chassis.tank(master.get_analog(ANALOG_LEFT_Y),     // expo curve applied
                 master.get_analog(ANALOG_RIGHT_Y));   // inside Chassis
    if      (master.get_digital(DIGITAL_R1)) ratchet.move( 127); // intake
    else if (master.get_digital(DIGITAL_L1)) ratchet.move(-127); // lift up
    else                                     ratchet.move(0);    // gravity lowers
    pros::delay(10);                                     // 100 Hz loop
  }
}
```

Code Journal 2/5

T3W4 · 10 Aug – Odometry step (tracking wheel + IMU)

Added since last update: odometry task. Encoder centidegrees → inches, IMU gives heading, arc→chord correction for movement during turns. Uses our clockwise-from-north frame, so forward = (sin θ, cos θ).

```
void Localizer::odomStep() {                  // every 10 ms (own RTOS task)
  float cdeg  = fwdEnc.get_position();        // centidegrees
  float dFwd  = (cdeg - lastCdeg) * (2*M_PI/36000.0f) * hw.fwdWheelRadius;
  float th    = degToRad(imu.get_heading()); // IMU trusted for heading
  float dTh   = angleWrap(th - lastTheta);
  float chord = (fabsf(dTh) > 1e-4f) ? 2*(dFwd/dTh)*sinf(dTh/2) : dFwd;
  float mid   = lastTheta + dTh/2;            // heading at mid-step
  pose.x += chord * sinf(mid);                // +X right
  pose.y += chord * cosf(mid);                // +Y forward (north)
  pose.theta = th;  lastCdeg = cdeg;  lastTheta = th;
}
```

T3W5 · 17 Aug – PID class + lift hold

Added since last update: reusable PID with integral clamp (anti-windup). Lift hold only ever drives the lift direction, because the ratchet cannot push the lift down — gravity does that.

```
float PID::update(float error) {
  integral   = std::clamp(integral + error*dt, -iMax, iMax); // anti-windup
  float der  = (error - prevError) / dt;  prevError = error;
  return kP*error + kI*integral + kD*der;
}
lillet::PID liftPid(0.4f, 0.0f, -0.02f);    // tuned on robot 19 Aug

void holdLift(float targetDeg) {
  float err = targetDeg - lift.get_position();
  float up  = std::max(liftPid.update(err), 0.0f); // never drive down
  ratchet.move(-up);                              // - = lift direction
}
```

Code Journal 3/5

T3W6 · 24 Aug – Particle filter – predict step

Added since last update: 200-particle cloud. Every cycle the odometry motion is applied to each particle with its own random noise (Thrun motion model, four α constants).

```
void ParticleFilter::predict(float dFwd, float dTh) {
  float sTh = sqrtf(a1*dTh*dTh + a2*dFwd*dFwd);   // heading noise
  float sD  = sqrtf(a3*dFwd*dFwd + a4*dTh*dTh);   // distance noise
  for (auto& p : particles) {                     // N = 200
    float th = p.theta + dTh + gauss(0, sTh);
    float d  = dFwd + gauss(0, sD);
    p.x += d * sinf(th);
    p.y += d * cosf(th);
    p.theta = angleWrap(th);
  }
}
```

T3W7 · 31 Aug – Sensor update – ray-cast, hybrid likelihood, occlusion gate

Added since last update: each particle predicts what the 4 distance sensors should read (ray-cast to walls/goals). Readings >300 mm shorter than expected are treated as blocked by an object and ignored.

```
void ParticleFilter::update(const std::array<float,4>& meas) {
  for (auto& p : particles)
    for (int s = 0; s < 4; ++s) {
      if (meas[s] <= 0 || meas[s] > 2000) continue;   // no valid return
      float exp = field.rayCast(p, mount[s]);         // expected mm
      if (exp - meas[s] > 300) continue;              // occluded -> skip
      float d = meas[s] - exp;
      p.w *= 0.75f*gaussPdf(d, 35.0f) + 0.25f*(1.0f/9000.0f);
    }
  normaliseWeights();
}
```

Code Journal 4/5

T3W8 · 7 Sep – Resampling, confidence trigger, pose read-out

Added since last update: systematic resampling + 5 % random re-spawn; correction fires every 8" of travel or when N_eff < 0.30. Pose = weighted mean with circular mean for heading.

```
float ParticleFilter::neff() const {               // 1 / (N * sum w^2)
  float s = 0; for (auto& p : particles) s += p.w*p.w;
  return 1.0f / (N * s);
}
void ParticleFilter::resample() {
  systematicResample(particles);                    // keep likely poses
  for (int i = 0; i < 10; ++i)                      // 5 % of 200
    particles[rand() % N] = randomParticleNear(estimate(), 24.0f);
}
// Localizer loop
if (distSinceFix >= 8.0f || pf.neff() < 0.30f) {
  pf.update(readDistances());  pf.resample();  distSinceFix = 0;
}
```

T3W9 · 14 Sep – Boomerang moveToPose + locked-sign turn

Added since last update: carrot point behind the target (lead 0.6) so the robot curves in at the right heading; turn direction locked once to stop ±180° oscillation; speed ramps down over last 20".

```
float d  = hypotf(tx - pose.x, ty - pose.y);
float cx = tx - sinf(tTh)*d*lead, cy = ty - cosf(tTh)*d*lead; // carrot
if (d < 6.0f) { cx = tx; cy = ty; }                // near-lock phase
float aErr = angleWrap(atan2f(cx - pose.x, cy - pose.y) - pose.theta);
float lin  = linPid.update(d) * cosf(aErr);
lin = std::min(lin, maxSpeed * std::min(1.0f, d / 20.0f)); // ramp down
float ang  = angPid.update(aErr);
left.move(lin + ang);  right.move(lin - ang);
// turnToHeading(): direction chosen ONCE
if (!started) { dir = angleWrap(target - pose.theta) >= 0 ? 1 : -1; started = true; }
w = std::min({wMax, w + acc*dt, sqrtf(2*acc*remaining)});  // v = sqrt(2as)
```

Code Journal 5/5

T3W10 · 21 Sep – Full 60-second Autonomous Coding Skills routine

Added since last update: complete 9-step routine exported from the simulator (coordinates in inches, field centre = 0,0). Letters a–e match pickup points on the Autonomous Diagram.

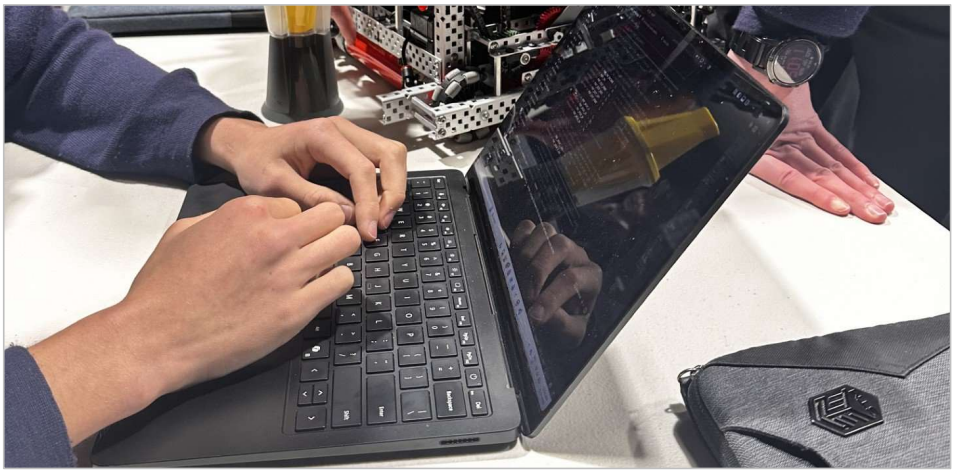
```
void autonSkills() {
  chassis.setPose(-58, 9, 90);
  chassis.moveToPose(-47,-19,180,2500); effector.score();          // 1 preload
  chassis.moveToPose(-66,  0,270,2000); pushToggle();              // 2 Toggle L
  intakePin(-24,-21,2500);                                          // a
  chassis.moveToPose(-44, 21,  0,2500); liftTo(SHORT);  effector.score(); // 3
  intakeCup(-66, 24, 2500);                                         // b
  chassis.moveToPose(-44, 24, 90,2000); liftTo(SHORT_C);effector.score(); // 4
  intakePin(-25, 24, 2000);                                         // c
  chassis.moveToPose(-44, 26,270,2000); liftTo(SHORT_2);effector.score(); // 5
  chassis.moveToPose(  0,-66,180,3000); pushToggle();              // 6 Toggle B
  intakePin(-45,-45,2500);                                          // d
  chassis.moveToPose(-27,-44, 90,2000); liftTo(SHORT);  effector.score(); // 7
  intakePin( 20,-65,3000);                                          // e
  chassis.moveToPose(  3, -4,  0,3000); liftTo(TALL);   effector.score(); // 8
  lowerLift();                                  // 9 stay in Midfield
}
```

T4W1 · 12 Oct – Time guard + tuning changes from testing

Added since last update: the routine checks the clock and abandons a step if fewer than 9 s remain, so it always banks the +8 Midfield points. Constants changed after tests T1, T9 and T13.

```
const uint32_t END = pros::millis() + 59000;          // 1 s safety margin
auto timeLeft = [&]{ return (int)(END - pros::millis()); };
if (timeLeft() < 9000) { goParkMidfield(); return; }   // always bank +8

// robot_config.hpp — changed after testing
constexpr float FWD_WHEEL_RADIUS_IN = 1.003f; // was 1.000 (T1 drift 2.3")
constexpr float BOOMERANG_LEAD      = 0.6f;   // was 0.5   (T13 goal approach)
constexpr int   EFFECTOR_SCORE_MS   = 350;    // was 250   (T9 dropped Pins)
constexpr float MCL_SIGMA_MM        = 35.0f;  // from simulator auto-tune
```



Programming and testing the robot at the bench.

Testing Method

Our testing steps (used for every subsystem)

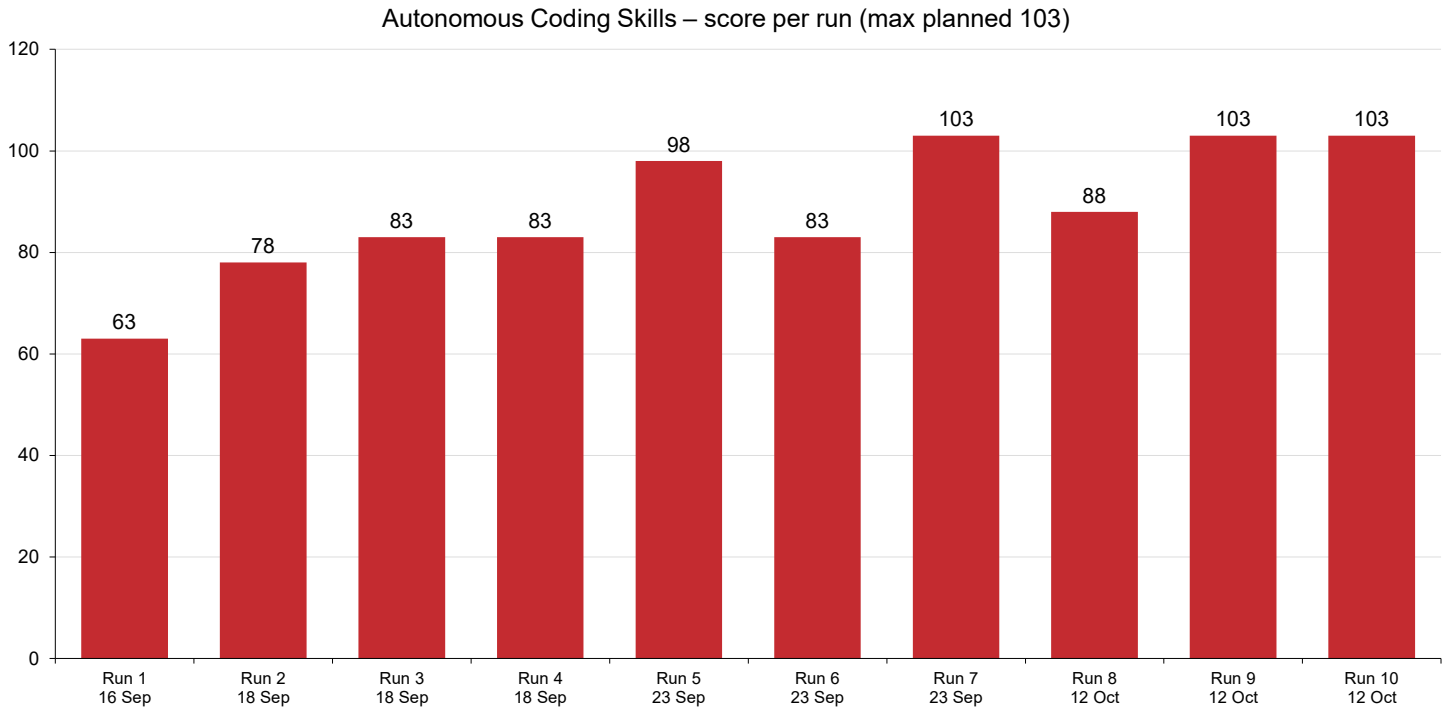
1. **Define** the requirement from the game rules or design brief (e.g. hold max 1 Pin – SG6).
2. **Set a measurable success criterion** (e.g. 0 of 20 runs holding 2 Pins).
3. **Run repeated trials** (5–30) under match-like conditions on a full field; same battery charge where possible.
4. **Record** results in the table immediately; photo/video for failures.
5. **Analyse** the cause of each failure (physics, code or build quality).
6. **Adjust** one variable at a time, then **re-test** and record the new result.
7. **Regression check:** after every change, re-run the full 60 s Skills routine.

Subsystem	Tests
Drivetrain	T1 straight-line accuracy · T2 top speed · T3 turning accuracy
Intake + transfer	T4 pickup reliability · T5 possession rule · T6 transfer alignment
Lift + ratchet	T7 DR4B speed + temperature · T8 ratchet switching
End effector	T9 placing accuracy
Structure + air	T10 stand-off loosening · T12 pneumatic endurance
Software	T11 localisation accuracy · T13 full autonomous skills run log

Test Results

ID	Subsystem	Test procedure	Success criterion	Result	Adjustment made
T1	Drive – straight line	Drive 96" forward 5×; measure sideways drift + distance error	≤ 1" error	Before: 2.3" avg drift. After: 0.6"	Re-measured tracking-wheel radius (48" roll test); 1.000 → 1.003"
T2	Drive – speed	Time 96" run from standstill 5×	≥ 45 in/s avg	1.9 s avg → 50.5 in/s (78 % of 64.8 in/s theoretical, due to acceleration)	None – meets target
T3	Drive – turning	10 × 90° turns; read IMU error	≤ 2° error, no oscillation	Before: oscillated near ±180°. After locked-sign turn: 0.8° avg	Locked turn direction once per turn (code T3W9)
T4	Intake – pickup	20 Pins + 20 Cups in random orientations; count clean pickups	≥ 18/20 each	Static roller 8/20 → pivot 15/20 → +kicker 19/20 Pins, 18/20 Cups	Pivoting flex wheels, then kicker roller 1 cm below ramp
T5	Intake – possession rule (SG6)	Drive through 2-Pin clusters 20×	0 runs holding 2 Pins	Before: 3/20 held 2 Pins. After: 0/20	Added top-plate stop above transfer
T6	Transfer → end effector	20 transfers; check Pin seats against hard-stop	≥ 18/20 aligned	16/20 → 19/20	Raised transfer roller 6 mm; decoupled from pivot
T7	DR4B lift	Floor → Tall Goal height 10×; motor temperature after 5 min cycling	≤ 1.5 s, < 55 °C	1.4 s avg, 48 °C	Re-tensioned rubber-band assist
T8	Gravity ratchet	30 direction reversals; check correct output engages	30/30	27/30 → 30/30	Re-bent polycarbonate guide for more planet-gear friction
T9	End effector – scoring	Place 20 Pins on a Short Goal	≥ 18/20 Placed	Prototype 2: 12/20. Prototype 3: 17/20 → 19/20	Release time 250 → 350 ms; roller compression tuned
T10	Structure – stand-offs	50 intake cycles then inspect fasteners	0 loose	3 loose → 0 loose	High-strength collars on stand-off flats; dual-side intake drive
T11	Localisation	3 m test path 5×; measure end-pose error	≤ 2"	Odometry only 3.1"; odom + AMCL 0.9"	Kept AMCL; sensor σ = 35 mm from simulator auto-tune
T12	Pneumatics	Count actuations from 100 psi until < 35 psi	≥ 100	128 (model predicted ≈120)	None – one tank is enough
T13	Full 60 s Auton Skills	Run full routine (see run log)	≥ 83 pts on 3 consecutive runs	See run log: last 5 runs avg 96 pts	Lead 0.5 → 0.6; time guard; release timing

Skills Run Log



Statistics

Best: 103 pts

Mean (all 10): 88.5 pts

Mean (last 5): 96.0 pts

Runs ≥ 83 pts: 8/10

Failures fixed

Run 1: Pin dropped at Tall Goal → release 350 ms

Runs 3–4: stack missed → boomerang lead 0.6

Run 6: cup step timed out → time guard

Evaluation against our goals

• Target '3 runs in a row ≥ 83' was met (runs 8–10).

• The maximum of 103 points matches the calculated score, which confirms the scoring logic.

• Localisation (AMCL) is the reason scores became consistent: end-pose error fell from 3.1" to 0.9" (T11).

• Remaining weakness: stacking on a Cup (steps 4–5) still fails about 1 in 5 runs. Next iteration: a funnel on the end effector to centre the Cup.

Refine

Iteration 2

Refinement Overview

After the main build and our tests, we went back through the robot subsystem by subsystem. This table links each problem we found to the change we made. The pages that follow explain each one.

Problem found	Change made	Page
Pieces not transferring from intake to 2nd roller	End effector moved closer to the intake	185
Pins entering the intake misaligned	Polycarbonate edges, front ramps, back wedges	186
Front roller and kicker surface speeds did not match	Kicker ≈ 2× intake speed; intake sped up	187
Lift motors heavily loaded	Rubber bands assist the lift upwards	188
Side Toggle had to be flipped	Piston-actuated lever on the C-channel	189
Back Toggle had to be set without a new actuator	Long stand-off on the lift pushes it over	190
Claw end effectors too big / needed hand alignment	Vertical-roller end effector	83

Transfer Alignment

Problem

- Game pieces did not always transfer from the intake into the second roller.

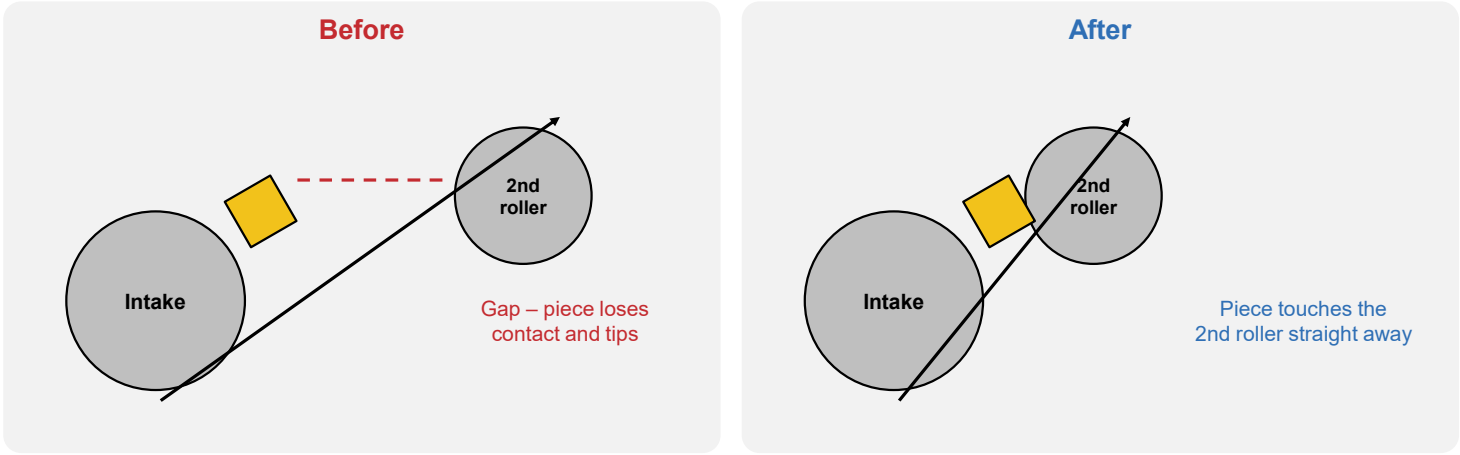
Change

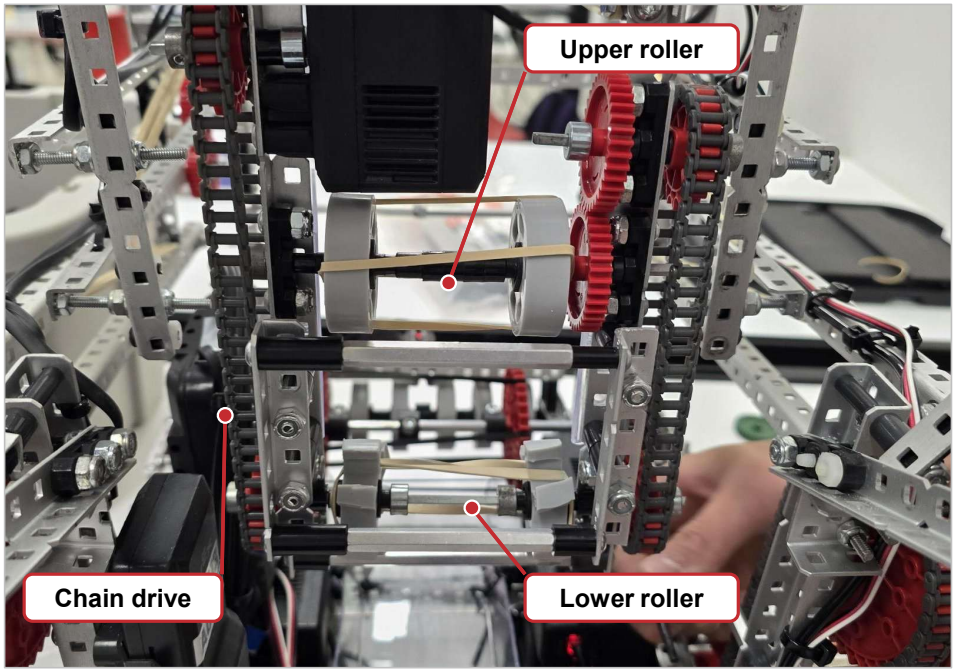
- Moved the end effector closer to the intake.
- Now, as a piece leaves the intake pin it immediately touches the end of the second roller.

Result

- Better alignment and a smoother transfer between the two stages.

Side view of the transfer (diagram)





Why it worked

- There is no unsupported gap, so the piece is always held by a roller.
- Pieces arrive the same way up every time, ready for scoring.

Guides & Ramps

Problem
- Pins could tilt and enter the intake misaligned.
- A standard metal frame around the intake gave pieces edges to catch on.

Change
- Polycarbonate edges around the intake instead of a standard frame.
- Polycarbonate ramps on the front.
- Wedges at the back as extra guides.

Result
- Pins stay straight and game pieces enter smoothly without misaligning.

Top view of the guide layout (diagram)

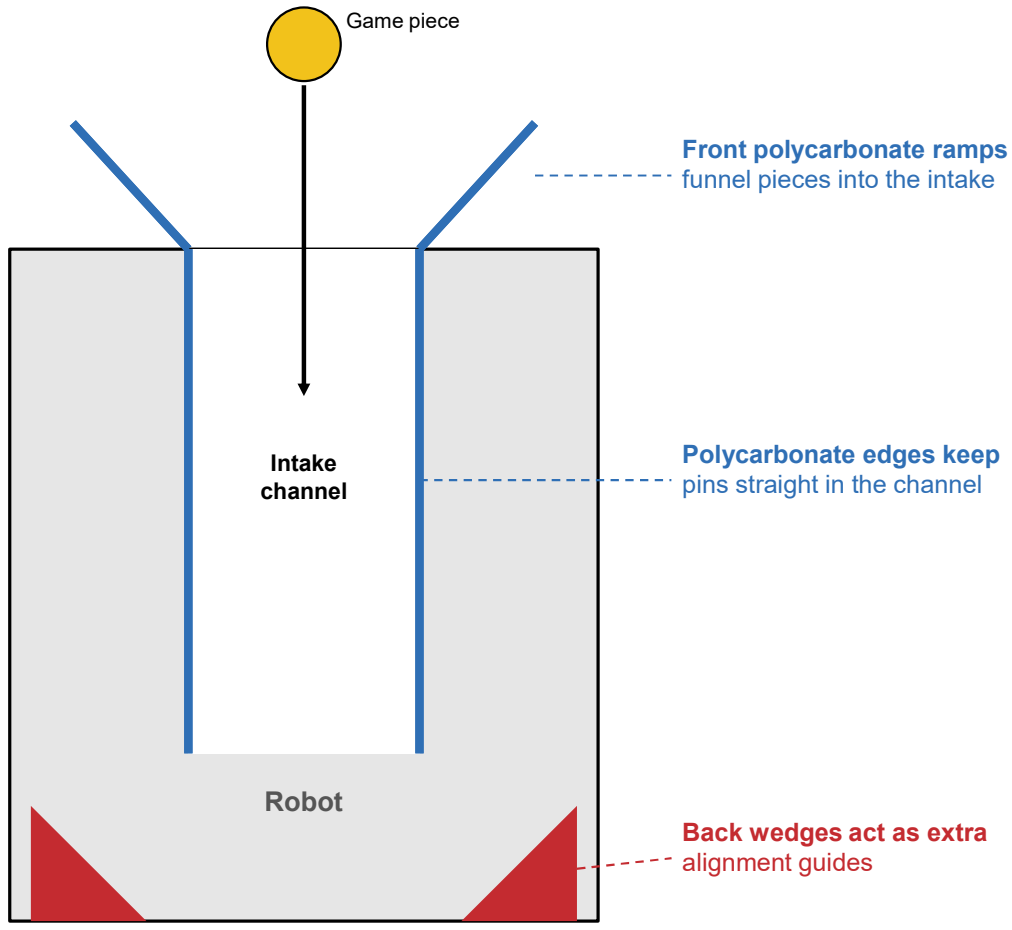


Diagram not to scale. Blue = polycarbonate, red = wedges. Polycarbonate flexes and springs back instead of bending (see Chassis Engineering).

Intake Roller Speed

Problem
- The kicker roller and front intake rollers used different diameter rollers.
- The ~2" front roller moved much faster at its surface than the smaller kicker at the same rpm, so transfers were inconsistent.

Change
- Kicker roller speed increased to about 2× the intake speed so the surface speeds match.
- Overall intake speed also increased for more reliable pickups.

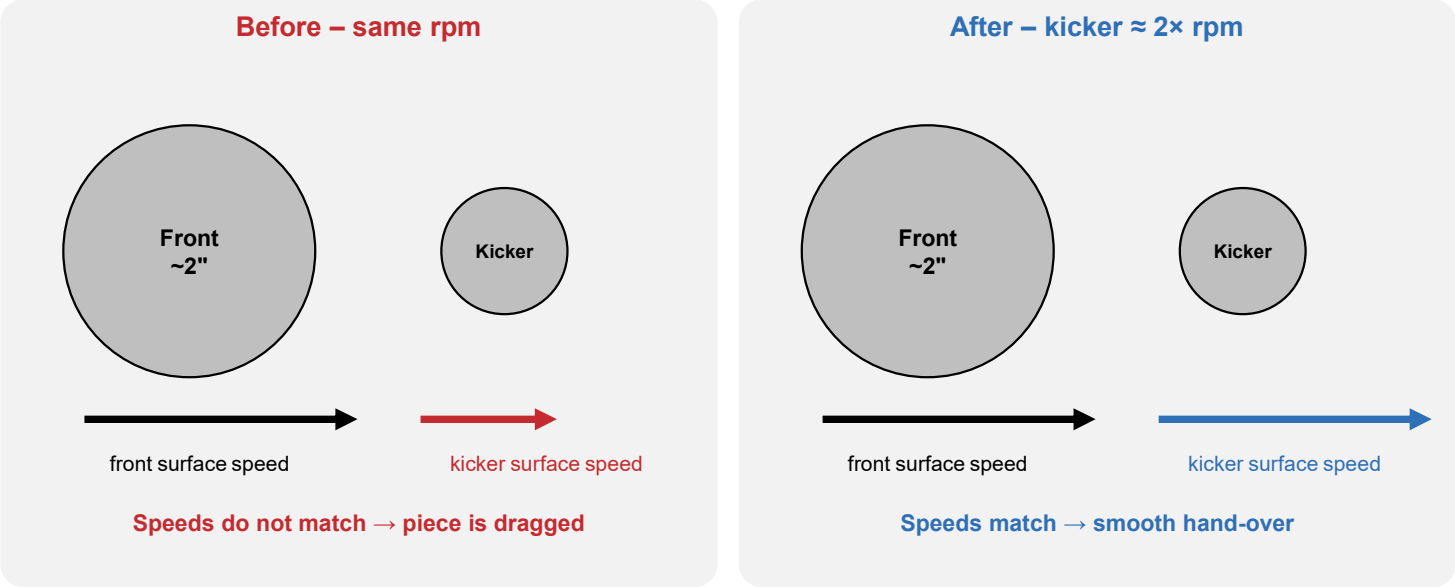
Result
- Transfers became more reliable because both rollers now move the piece at the same speed.

The Maths

Surface speed of a roller: $v = \pi \times d \times N$ (d = diameter, N = rpm)

- At the same rpm, a bigger roller has a faster surface – v is proportional to d.
- For two rollers to move a piece at the same speed: $N_{kicker} \div N_{front} = d_{front} \div d_{kicker}$
- So a kicker about half the front roller's diameter needs about twice the rpm – which is why we set it to ≈ 2×.

Surface speed comparison (diagram)



Arrow lengths show relative surface speed. Only the ~2" front roller size and the ≈ 2× speed change were recorded.

Lift Rubber Bands

Problem

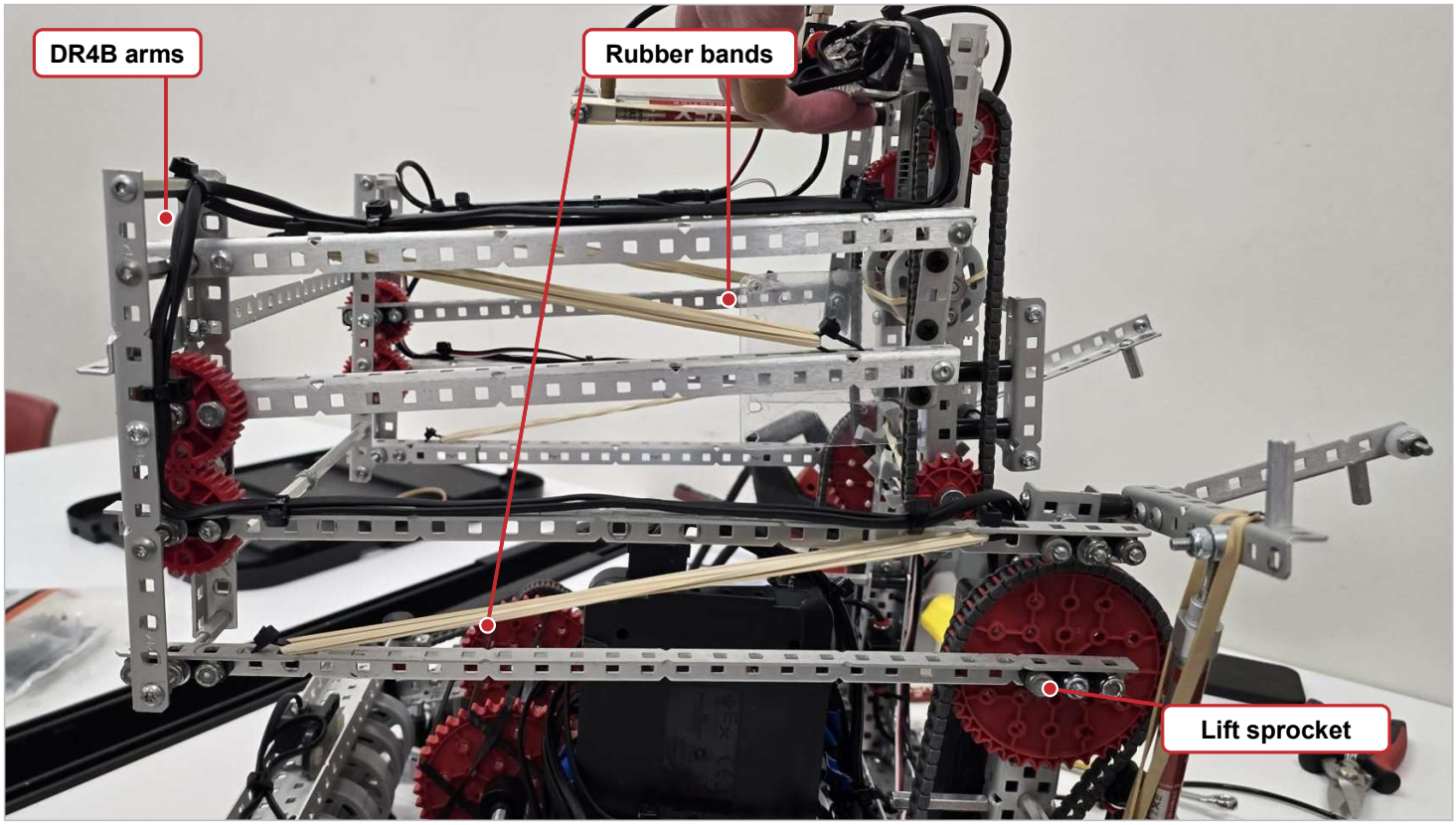
- The lift motors carried the whole weight of the DR4B and end effector on every lift.

Change

- Added rubber bands across the lift that assist it on the way up.

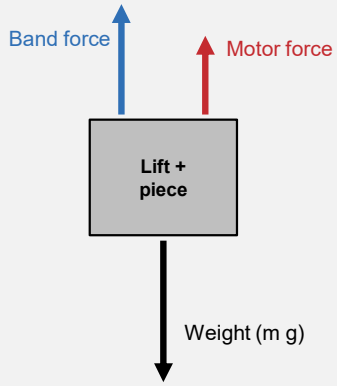
Result

- The motors run with less load and lift performance improved.



Rubber bands stretched across the DR4B links.

Forces on the lift (diagram)



Why it helps

- Motor force needed = weight – band force, so the bands take part of the load.
- Lower load means less current and heat in the motors.
- Our gravity ratchet lowers the lift by gravity, so band tension has to stay balanced with the lift weight.

Side Toggle Mechanism

Problem

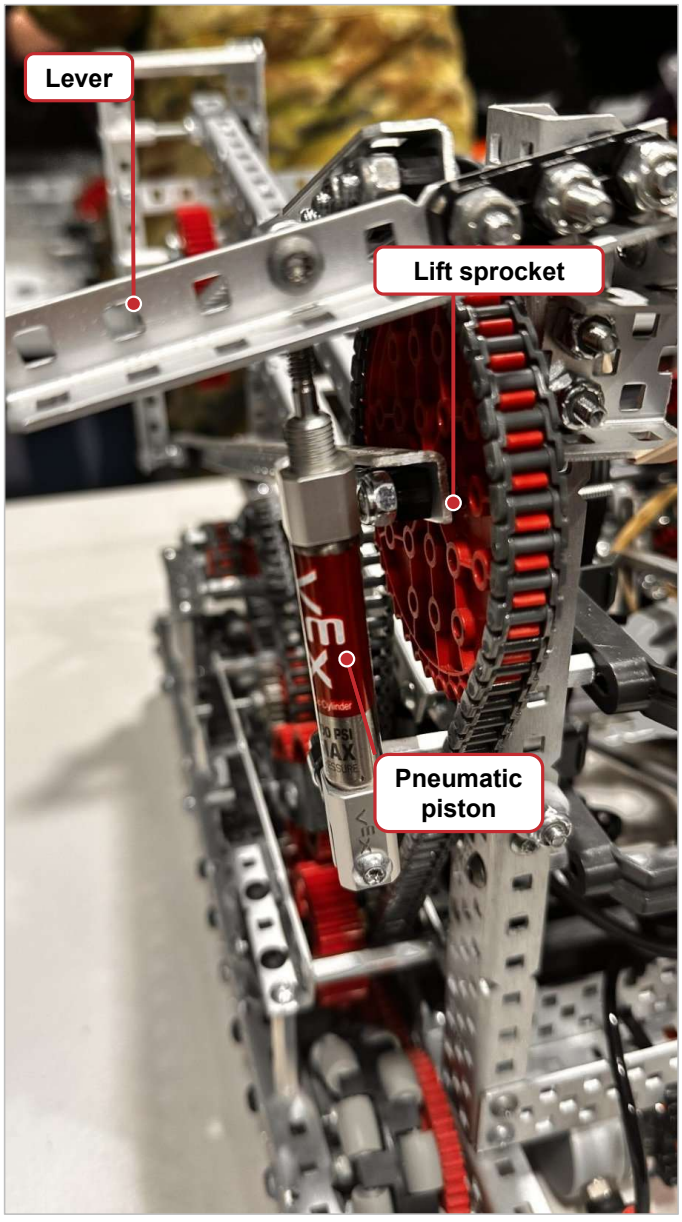
- We needed a way to flip the side Toggle while the robot drives alongside it.

Change

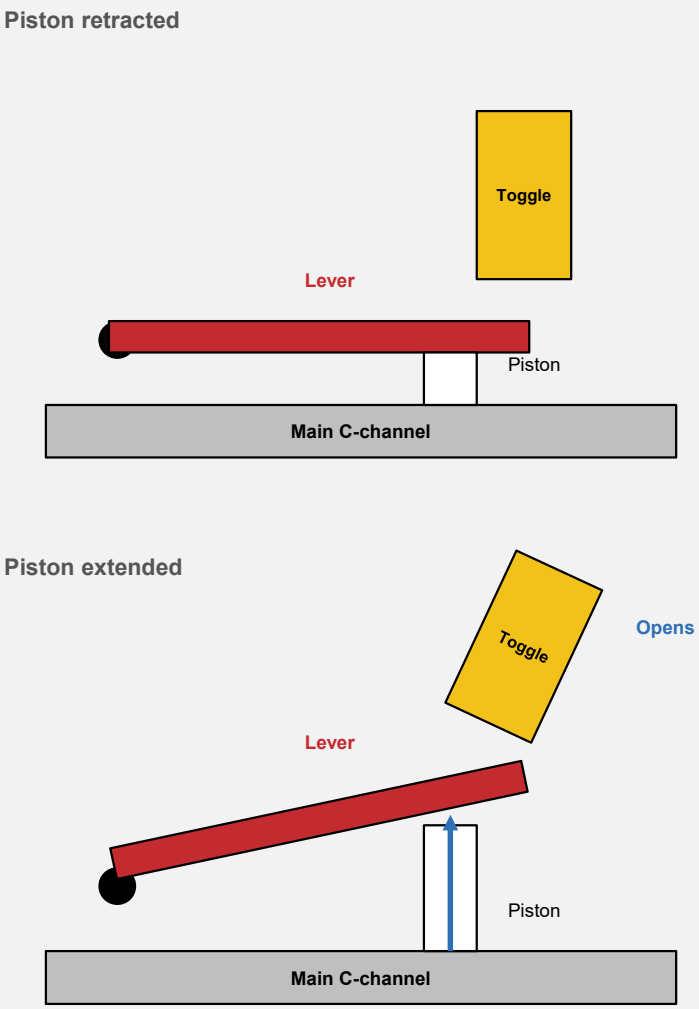
- Added a piston-actuated lever mounted to the main C-channel.
- When activated, the piston pushes the lever up from underneath the Toggle, opening it.

Result

- Tuned the stand-off length until the lever contacted and guided the Toggle through its full range of motion.



How the lever works (side view diagram)



Stand-off length sets where the lever meets the Toggle.

Back Toggle Mechanism

Problem

- The back Toggle also had to be set, but we did not want to spend another motor or more air on it.

Change

- A long stand-off fixed to the lift extends over the top of the bar.
- When the lift moves up, the stand-off contacts the Toggle and pushes it over.

Result

- The Toggle is set automatically by the lift's own movement – no separate actuator.

Lift moving up pushes the Toggle over (side view diagram)

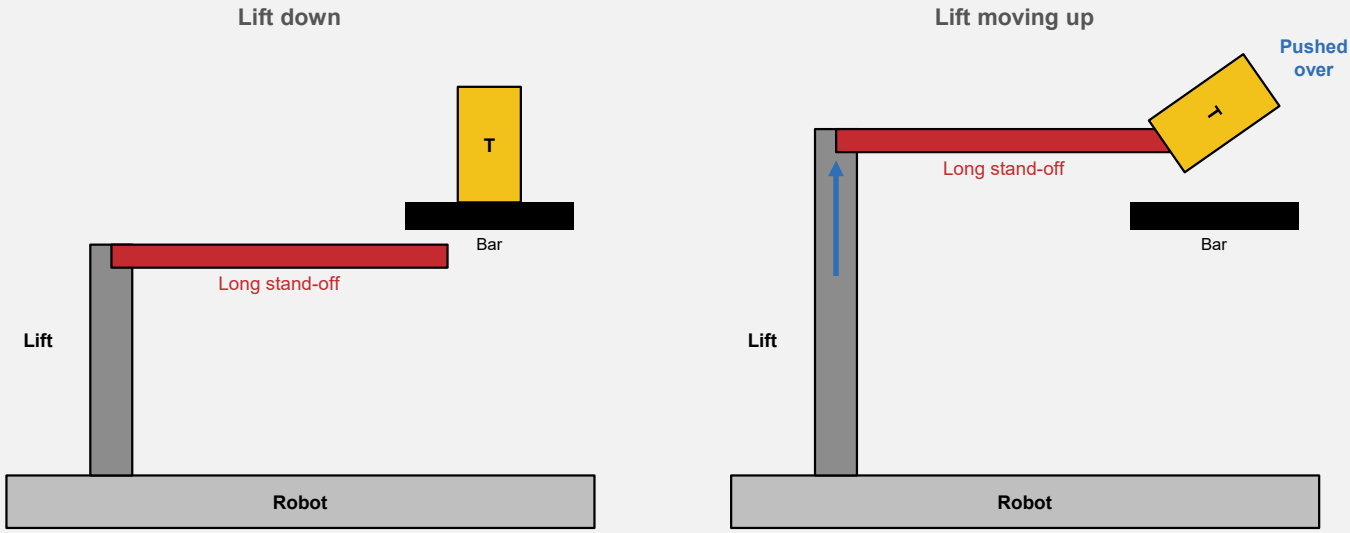
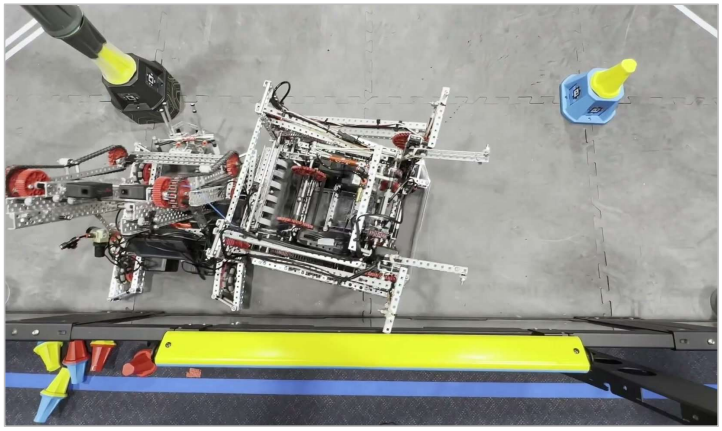
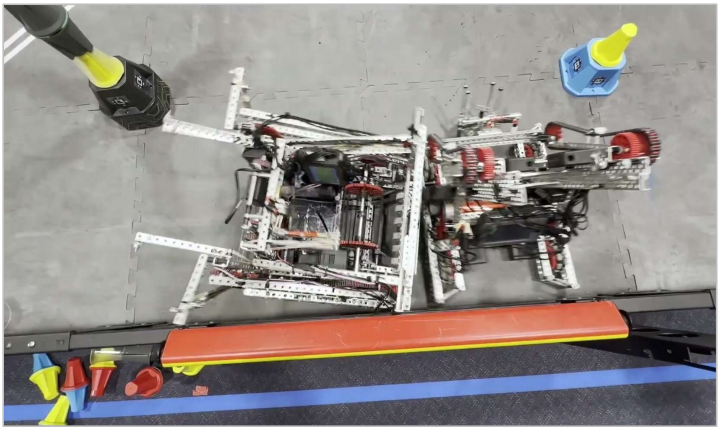


Diagram not to scale. T = Toggle.





Reveal video (1:29 → 1:31): a wall Toggle changes from yellow to red as our robot works beside it.

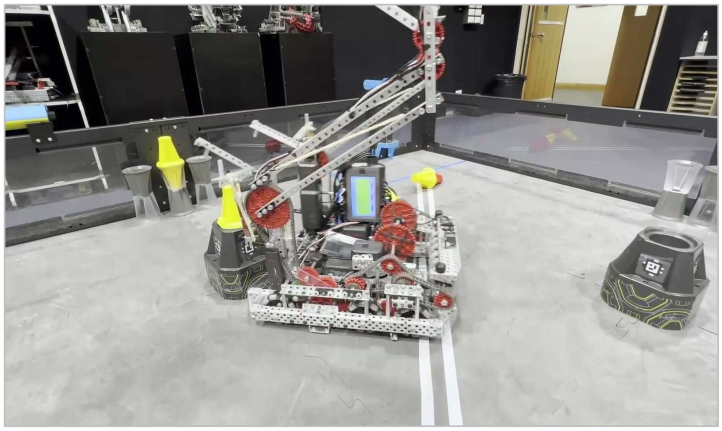
Robot Reveal

Robot name: Switch

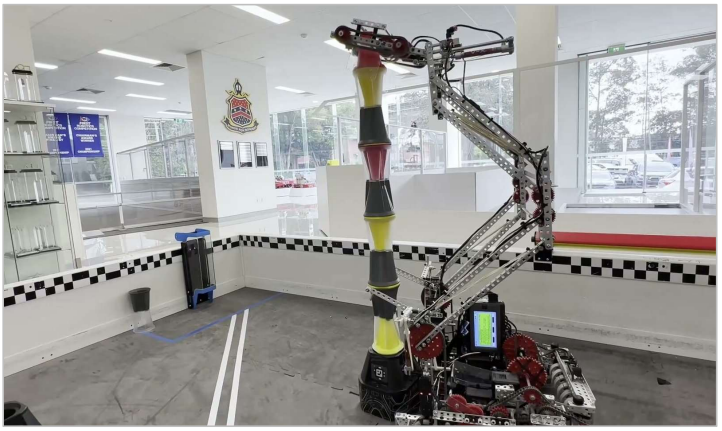
“4613 Redbacks Robot Reveal – Vex Override” was published on YouTube on 24 September 2026 (1 min 39 s). It shows the finished robot working on a full field.

youtube.com/watch?v=qsefnmMplAk

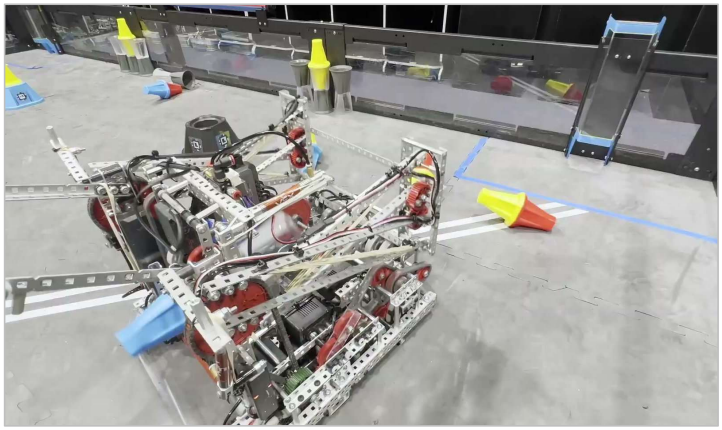
Scan to watch



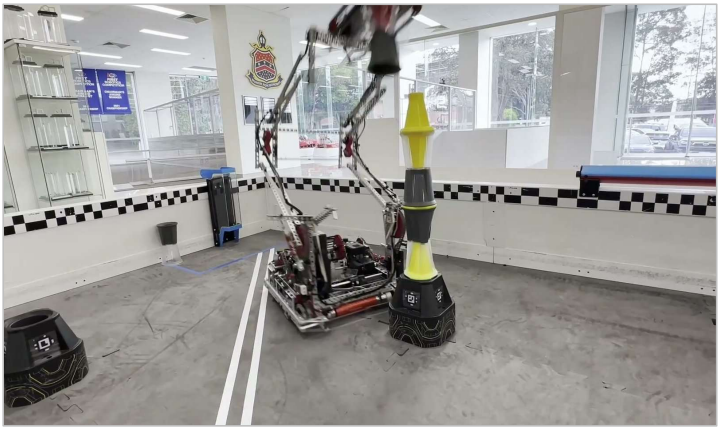
Driving with the lift raised



Stacking Pins and Cups on a tall Goal



Collecting game pieces on the field



DR4B reaching over a tall stack

What the video shows

- 0:18 – 0:58 Driving, collecting and scoring game pieces
- 0:31 and 0:49 Close-ups of the gravity ratchet, guides and distance sensor
- 1:04 – 1:16 The DR4B fully extended, stacking on a tall Goal
- 1:25 – 1:31 Working beside a wall Toggle as it changes to red

Individual Effort 1/2

Brandon – Team Captain, Lead Designer, Programmer, Mechanical

Accountable element (from Team pages 6–9)	Status	Evidence (page numbers)
lillet PROS library (odometry, AMCL, motion control)	Complete	Pages 94–127, 132–161; Code Journal 175–179
Full robot CAD and manufacturing	Complete	Pages 28, 85–86; Build Log 163–164
Path-planning simulator + auton	Complete	Simulator pages 111–114; Autonomous Logic 130
Driver-control + autonomous code	Complete	Robot code pages 132–161; Code Journal 176, 179; Skills Run Log 182
Programming journal (weekly)	Complete	Pages 175–179

Contribution summary: Wrote and tuned the lillet localisation and motion library, designed the robot in CAD and built it, and exported the autonomous routine from the simulator to the robot (Autonomous Score, page 131).

Ryan – Driver, Mechanical

Accountable element (from Team pages 6–9)	Status	Evidence (page numbers)
End effector prototypes 1 and 2	Complete	Pages 78–80, 83; Test T9
Autonomous programming	Complete	Autonomous Logic 130; Code Journal 179; Skills Run Log 182
Driving	Complete	Goals & Strategy 23; Lessons 172–174
Pneumatics and cable management	Complete	Pages 70, 62, 92; Test T12
Transfer	Complete	Pages 58–59, 60, 185; Test T6

Contribution summary: Drove the robot, built the mechanical subsystems and end effector prototypes 1 and 2, and handled the pneumatics and cable routing (Wiring Diagram, page 92).

Individual Effort 2/2

Josh – First Stage Design Lead, Programming Advisor, Lead Notebook

Accountable element (from Team pages 6–9)	Status	Evidence (page numbers)
Intake research, sketches, prototypes 1–4	Complete	Pages 46–62
Transfer roller + chain power system	Complete	Pages 58–59, 60; Test T6
CAD drawings / planned design views	Complete	Pages 28, 85
Intake + chassis decision matrices	Complete	Pages 46–62
Stand-off & dual-side drive reinforcement	Complete	Intake Structure page 61; Test T10

Contribution summary: Led the first-stage (intake) design through its iterations and the transfer redesign, produced the CAD views and decision matrices, and kept the notebook.

Rio – Lead Mechanical, Lead Experimental Design

Accountable element (from Team pages 6–9)	Status	Evidence (page numbers)
End Effector Concept	Complete	Pages 77–84; Test T9
Building half the robot	Complete	Pages 163–164, 165–174
Tuning	Complete	Pages 181–182, 184–191
Intake	Complete	Pages 46–62
4 Bar	Complete	Pages 64–67; Test T7
Construction	Complete	Build Log 163–164; page 86

Contribution summary: Designed the end effector, built half the robot including the intake and 4 bar, and led the tuning and refinement tests.

Sources & Citations

Ideas, images and rules used in this notebook came from the sources below. Images on brainstorming pages are captioned with their team/source.

1. VEX Robotics / REC Foundation (2026). V5RC Override Game Manual v2.1 – Sections 2 (SG, SC rules), 3 (RSC1–RSC5) and Robot rules (R10, R11, R25, R26). content.vexrobotics.com/docs/2026-2027/override/files/Override-2.1.pdf

2. REC Foundation (2026). Official Q&A: V5RC 2026-2027 Override. events.vex.com/V5RC/2026-2027/QA

3. Purdue SIGBots Wiki – Intakes. wiki.purduesigbots.com (horizontal vs vertical roller comparison, Brainstorming: Horizontal vs Vertical Rollers page)

4. VRC Team 606X – Spin Up intake test (video) and Team 94999E intake (Horizontal vs Vertical Rollers page)

5. Team 254 – 2020-21 Change Up robot (fixed-point roller inspiration, Fixed point roller page)

6. Printables: 'Rubber Band Intake Wheel' by RhindleLAK; 'VEX V5 Override Pin' and 'Cup' models by Sebastien Harens (5606A)

7. VRC Team 473Z – pneumatic PTO system (Gravity Ratchet introduction page)

8. FRC Team 2056 (2025) – end effector (End Effector Prototype 2 page)

9. VEX Forum threads: 'Simpler X-Drive Design', 'Upgrading Drivetrain for Maximum Performance', 'Creating a Basic Drive' (drivetrain brainstorming pages)

10. VEX Library – Identifying Location Details Using the GPS Sensor (GPS localisation page)

11. Thrun, S., Burgard, W. & Fox, D. (2005). Probabilistic Robotics. MIT Press (motion model, particle filter)

12. MathWorks – Monte Carlo Localization Algorithm documentation; LemLib documentation (API style for moveToPose)

13. Core Electronics – goBILDA Pinpoint Odometry Computer (IMU + odometry fusion image)

14. 4613P (2026). lillet library docs and path-planning simulator – beel487.github.io/lillet/docs.html and /Sim.html

15. 4613 Redbacks (2026). 4613 Redbacks Robot Reveal – Vex Override [video]. YouTube, 24 Sep 2026. youtube.com/watch?v=qsefnmMplAk (photos on the Robot Reveal, Gravity Ratchet Built, Annotated Robot Views and Back Toggle pages)